DEVOPS ENGINEERING MASTERY

CLOUD ENGINEERING AMAZON WEB SERVICES (AWS)

AWS Architecture, IAM, VPC, EC2, S3, Load Balancing, Auto Scaling, Databases, Containers, Serverless, Security & Production Cloud Operations

VOLUME 11 • FIRST EDITION

```
nahid@technova:~$ aws sts get-caller-identity
✓ Account 111122223333  •  arn:aws:iam::111122223333:user/nahid
nahid@technova:~$ aws ec2 describe-instances --filters Name=tag:Env,Values=prod
✓ 4 instances running  •  2 AZs  •  IMDSv2 enforced
nahid@technova:~$ aws s3 ls s3://technova-app-assets --recursive
```

IAM • VPC • EC2 • S3 • RDS • ECS • Lambda • MERN

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

Cloud Engineering — Amazon Web Services (AWS)

DEVOPS ENGINEERING MASTERY — Volume 11

AWS Architecture, IAM, VPC, EC2, S3, Load Balancing, Auto Scaling, Databases, Containers, Serverless, Security ও Production Cloud Operations

First Edition — Published 9 November 2026.

Hands-on environment: AWS CLI v2, AWS Management Console, Terraform and AWS CloudFormation, with Node.js 20, Docker, and a MERN (MongoDB, Express, React, Node.js) example application called TechNova. All domains, IP addresses, account numbers and credentials in examples are fictional (technova.example, 203.0.113.x, 111122223333, CHANGE-ME-...). Sample terminal outputs are illustrative. AWS service availability, features and console behaviour change over time — official AWS documentation is the authority, and no price is stated as a timeless fact in this book: always check the official AWS Pricing Calculator and pricing pages before running a lab. Resources that can generate charges — NAT Gateway, EKS, EC2, load balancers, managed databases, data transfer — are flagged in every lab, and every lab ends with explicit cleanup steps.

How to use this book. Read the parts in order — each part builds on the previous one. Keep an AWS account (with MFA on the root user and billing alerts switched on) and the AWS CLI beside you, and do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Do not copy a CLI command or a policy document blindly; read the explanation under each field first. Destructive operations are always flagged with a WARNING box and a safe alternative. Never practise on a production account: use a dedicated test account, and never paste a real secret or access key into an example — use the fictional placeholders shown.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test environment, finding the root cause in your own logs and metrics, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk (including cost risk), **PRO TIP** gives professional advice, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Cloud changes should first be practised in an isolated test account, never directly on a production system. The author and publisher accept no liability for data loss, service disruption or unexpected cloud charges caused by running commands without a backup, an approval gate, or a billing alert.

Who this book is for

A learner who knows JavaScript, React, Next.js, Node.js, Express and MongoDB, understands basic Git and Linux, has studied Linux server administration, CI/CD, Docker, Kubernetes, Terraform and Ansible, and

wants to design, build, secure, operate and troubleshoot real cloud systems on AWS — from a first IAM policy to a production multi-AZ MERN platform with load balancing, auto scaling, managed databases, containers, CI/CD, observability, backup and disaster recovery.

Learning outcomes

By the end of this book you will be able to: explain cloud computing models and AWS global infrastructure and design for Availability Zones and Regions; govern AWS accounts with Organizations, SCPs and Control Tower concepts; design least-privilege IAM policies, roles and trust relationships and troubleshoot access failures; use the AWS CLI, SDKs, named profiles and OIDC authentication safely; apply the Shared Responsibility Model and the Well-Architected Framework and investigate an unexpected bill; design VPCs with correct CIDR planning, public and private subnets, NAT and VPC endpoints; secure networks with security groups and NACLs and connect hybrid networks; run hardened EC2 instances with IMDSv2 and Session Manager; operate ALB/NLB and Auto Scaling Groups; engineer S3, EBS, EFS, RDS, Aurora, DynamoDB and ElastiCache for production data; deliver content with Route 53, CloudFront and ACM; run containers on ECS and EKS; build serverless, event-driven systems with Lambda, SQS, SNS and EventBridge; build CI/CD with CodePipeline and GitHub Actions OIDC; provision infrastructure with CloudFormation, CDK and Terraform; monitor with CloudWatch, audit with CloudTrail and Config, secure with KMS, GuardDuty and Security Hub; and design reliable, recoverable, cost-controlled production architectures with runbooks.

সূচিপত্র — Contents

PART I — AWS Foundations & Cloud Architecture

Chapter 01 — Cloud Computing and AWS Global Infrastructure 7

Chapter 02 — AWS Account, Root User and Account Governance 15

Chapter 03 — Identity and Access Management (IAM) 21

Chapter 04 — AWS CLI, SDKs and Credential Engineering 28

Chapter 05 — Shared Responsibility, Well-Architected and Billing 34

PART II — Networking & Compute Engineering

Chapter 06 — Amazon VPC Architecture 41

Chapter 07 — Internet Gateway, NAT and VPC Endpoints 48

Chapter 08 — Network Security and Hybrid Connectivity 54

Chapter 09 — Amazon EC2 Engineering 60

Chapter 10 — Elastic Load Balancing 67

Chapter 11 — Auto Scaling and High Availability 73

PART III — Storage, Database & Application Services

Chapter 12 — Amazon S3 Engineering 82

Chapter 13 — EBS, EFS, FSx and Backup 94

Chapter 14 — RDS and Aurora 103

Chapter 15 — DynamoDB and ElastiCache 111

Chapter 16 — Route 53, CloudFront, ACM and Edge Architecture 120

PART IV — Containers, Serverless & Delivery

Chapter 17 — AWS Container Engineering 130

Chapter 18 — Serverless and Event-Driven Architecture 138

Chapter 19 — AWS CI/CD Engineering 143

Chapter 20 — Infrastructure as Code on AWS 147

PART V — Production AWS Engineering

Chapter 21 — Monitoring, Logging and Auditing 153

Chapter 22 — AWS Security Engineering 157

Chapter 23 — Reliability, Backup and Disaster Recovery 161

Chapter 24 — Production AWS Architecture and FinOps 164

Companion Volumes

Lab Workbook — 24 Guided Labs • 15 Assignments • 3 Major Projects • Capstone

Assessment and Answer Guide — 120 MCQs • Short Answers • Scenarios • Case Studies • Final Exam

Quick Reference Handbook — Glossary (150 Terms) • AWS CLI / IAM / VPC References • Architecture Patterns • Checklists • Decision Trees

Guided labs (24), major projects (3) and the capstone are in the separate *AWS Lab Workbook*. MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Glossary, AWS CLI, IAM and VPC references, architecture patterns, checklists and decision trees are in the separate *Quick Reference Handbook*.

PART I • DEVOPS ENGINEERING MASTERY

AWS Foundations & Cloud Architecture

ক্লাউড কম্পিউটিং, AWS গ্লোবাল ইনফ্রাস্ট্রাকচার, অ্যাকাউন্ট গভর্ন্যান্স, IAM, AWS CLI ও ক্রেডেনশিয়াল, Shared Responsibility, Well-Architected ও বিলিং।

অধ্যায় ১ • PART I — AWS Foundations

Chapter 1 — Cloud Computing and AWS Global Infrastructure

১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) ক্লাউড কম্পিউটিং কী এবং প্রচলিত অন-প্রিমিসেস (on-premises) অবকাঠামোর তুলনায় এটি কোন সমস্যার সমাধান করে, তা প্রথম নীতি (first principles) থেকে ব্যাখ্যা করতে; (২) IaaS, PaaS ও SaaS-এর মধ্যে পার্থক্য করতে এবং কোনো কাজের জন্য কোন মডেলটি উপযুক্ত তা সিদ্ধান্ত নিতে; (৩) পাবলিক, প্রাইভেট ও হাইব্রিড ক্লাউডের তুলনা করতে; (৪) AWS Region, Availability Zone (AZ) ও Edge Location-এর সংজ্ঞা, সম্পর্ক ও ব্যবহার ব্যাখ্যা করতে; (৫) গ্লোবাল সার্ভিস বনাম রিজিওনাল সার্ভিসের পার্থক্য বুঝতে; (৬) ফল্ট ডোমেইন (fault domain) ও ফেইলিওর ডোমেইন (failure domain) ধারণা দিয়ে উচ্চ-উপলভ্যতা (high availability) নকশা করতে; এবং (৭) TechNova-র MERN অ্যাপ্লিকেশনের জন্য একটি মাল্টি-AZ, মাল্টি-রিজিওন আর্কিটেকচারের রূপরেখা আঁকতে।

১.২ পূর্বশর্ত জ্ঞান

এই অধ্যায় পড়ার আগে আপনার কম্পিউটার নেটওয়ার্কের মৌলিক ধারণা থাকা ভালো — IP ঠিকানা, DNS, HTTP/HTTPS, এবং ক্লায়েন্ট-সার্ভার মডেল। সার্ভার বলতে কী বোঝায়, ডেটাবেস কীভাবে কাজ করে, এবং লিনাক্স টার্মিনালে সাধারণ কমান্ড চালানোর অভিজ্ঞতা থাকলে ভালো হয়। AWS সম্পর্কে আগের কোনো অভিজ্ঞতা দরকার নেই। আপনি যদি Volume 1–10 পড়ে এসে থাকেন, তাহলে Linux, নেটওয়ার্কিং, Docker ও CI/CD-র ধারণাগুলো এখানে বারবার কাজে লাগবে — বিশেষ করে লোড ব্যালান্সিং, DNS ও কন্টেইনার অধ্যায়গুলো। কোনো শব্দ অচেনা লাগলে প্রথম ব্যবহারের জায়গাতেই তার বাংলা ব্যাখ্যা দেওয়া আছে।

১.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

আপনি যদি একজন ডেভঅপস ইঞ্জিনিয়ার হতে চান, তাহলে ক্লাউড আপনার কর্মক্ষেত্র। আজকের দিনে প্রায় প্রতিটি আধুনিক অ্যাপ্লিকেশন — ব্যাংকিং অ্যাপ, ই-কমার্স সাইট, স্ট্রিমিং সেবা, সরকারি পোর্টাল — কোনো না কোনো ক্লাউডে চলে। কিন্তু ক্লাউড শুধু "অন্যের কম্পিউটার ভাড়া নেওয়া" নয়। ক্লাউডের আসল শক্তি হলো এটি ব্যর্থতাকে (failure) একটি স্বাভাবিক ঘটনা হিসেবে ধরে নিয়ে নকশা করে। একটি সার্ভার নষ্ট হবে, একটি ডেটা সেন্টারে বিদ্যুৎ যাবে, একটি নেটওয়ার্ক লিংক ছিঁড়বে — ক্লাউড আর্কিটেকচারের কাজ হলো এসব ঘটনা সত্ত্বেও ব্যবহারকারী যেন কিছু টের না পায়, তা নিশ্চিত করা। এই অধ্যায়ে আপনি সেই চিন্তাভাবনার ভিত্তি শিখবেন। Region ও AZ না বুঝে আপনি কখনোই একটি নির্ভরযোগ্য সিস্টেম বানাতে পারবেন না — আপনি হয়তো একটি EC2 ইনস্ট্যান্স চালাতে পারবেন, কিন্তু সেটি একটি মাত্র AZ-তে থাকলে একটি ডেটা সেন্টারের সমস্যাতেই আপনার পুরো সেবা বন্ধ হয়ে যাবে। এই ভুলটি প্রোডাকশনে খুবই ব্যয়বহুল।

১.৪ ক্লাউডের আগে: সমস্যাটি কী ছিল

২০০০-এর দশকের শুরুতে একটি ওয়েবসাইট চালু করতে চাইলে আপনাকে প্রথমে সার্ভার কিনতে হতো — দামি হার্ডওয়্যার, যার ডেলিভারিতে সপ্তাহ লাগত। তারপর ডেটা সেন্টারে জায়গা (rack space) ভাড়া নিতে হতো, বিদ্যুৎ, কুলিং, নেটওয়ার্ক সংযোগের ব্যবস্থা করতে হতো। ট্র্যাফিক বাড়লে নতুন সার্ভার অর্ডার করতে হতো, আবার ট্র্যাফিক কমলে কেনা সার্ভারগুলো অলস বসে থাকত — টাকা নষ্ট। একে বলে **ক্যাপাসিটি প্ল্যানিং** (capacity planning) সমস্যা: ভবিষ্যতের চাহিদা আন্দাজ করে আগে থেকে খরচ করা। আন্দাজ কম হলে সাইট ডাউন, আন্দাজ বেশি হলে টাকা নষ্ট। এর সঙ্গে ছিল হার্ডওয়্যার নষ্ট হওয়া, ডিস্ক ফেইল করা, অপারেটিং সিস্টেম প্যাচ দেওয়া, ফায়ারওয়াল বসানো — সব দায়িত্ব আপনার। একটি ছোট দলের

পক্ষে এগুলো সামলানো প্রায় অসম্ভব ছিল। ক্লাউড এই সমস্যার উত্তরে এসেছে: কম্পিউটিংকে একটি **ইউটিলিটি** (utility) বানিয়ে দেওয়া — বিদ্যুৎ বা পানির মতো, যতটুকু ব্যবহার করবেন ততটুকুর বিল দেবেন, আর অবকাঠামোর ভার অন্য কেউ নেবে।

> **মূল ধারণা:** ক্লাউড কম্পিউটিং হলো ইন্টারনেটের মাধ্যমে চাহিদা অনুযায়ী (on-demand) কম্পিউটিং সম্পদ — সার্ভার, স্টোরেজ, ডেটাবেস, নেটওয়ার্ক — ভাড়া নেওয়ার মডেল, যেখানে আপনি ব্যবহার অনুযায়ী বিল দেন এবং কয়েক মিনিটে সম্পদ বাড়াতে বা কমাতে পারেন। NIST-এর সংজ্ঞায় এর পাঁচটি বৈশিষ্ট্য আছে: on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service।

১.৫ সার্ভিস মডেল: IaaS, PaaS, SaaS

ক্লাউড সেবাকে দায়িত্ব ভাগাভাগির ভিত্তিতে তিন ভাগে ভাগ করা হয়। নিচের টেবিলটি মুখস্থ না করে বুঝুন — প্রতিটি মডেলে "কে কী সামলায়" সেটাই মূল কথা।

মডেল	আপনি সামলান	প্রোভাইডার সামলায়	AWS উদাহরণ	কখন ব্যবহার করবেন
IaaS (Infrastructure as a Service)	অ্যাপ্লিকেশন, ডেটা, রানটাইম, OS, মিডলওয়্যার	ভার্চুয়ালাইজেশন, সার্ভার, স্টোরেজ, নেটওয়ার্ক	EC2, VPC, EBS, S3	পূর্ণ নিয়ন্ত্রণ দরকার হলে; কাস্টম OS বা নেটওয়ার্ক কনফিগারেশন লাগলে
PaaS (Platform as a Service)	অ্যাপ্লিকেশন ও ডেটা	রানটাইম, OS, প্যাচিং, স্কেলিং	Elastic Beanstalk, Amplify, RDS	দ্রুত ডেপ্লয় করতে চাইলে; অবকাঠামো সামলাতে না চাইলে
SaaS (Software as a Service)	শুধু ব্যবহার ও কনফিগারেশন	প্রায় সবকিছু	WorkMail, Chime, QuickSight (ব্যবহারকারী হিসেবে)	তৈরি সফটওয়্যার ব্যবহার করতে চাইলে; নিজে কিছু বানাতে না চাইলে

একটি সহজ উপমা: IaaS হলো খালি জমি ও ইট-বালি ভাড়া নেওয়া — বাড়ি আপনাকেই বানাতে হবে। PaaS হলো তৈরি ফ্ল্যাট ভাড়া নেওয়া — আসবাব সাজিয়ে থাকা শুরু করুন। SaaS হলো হোটেলে থাকা — সবকিছু তৈরি, শুধু ব্যবহার করুন। TechNova-র MERN অ্যাপের ক্ষেত্রে আমরা বেশিরভাগ সময় IaaS ও PaaS মিশিয়ে ব্যবহার করব: EC2 বা ECS-তে অ্যাপ চালানো (IaaS-ঘেঁষা), আর ডেটাবেসের জন্য RDS (PaaS) — কারণ ডেটাবেসের প্যাচিং, ব্যাকআপ ও রেপ্লিকেশন AWS-কে দিলে আমাদের দল ছোট থাকলেও নিরাপদে চলতে পারে।

১.৫.১ কখন কোনটি নয়

IaaS নেবেন না যখন আপনার দলে OS প্যাচিং ও নিরাপত্তা সামলানোর লোক নেই — তখন PaaS নিরাপদ। PaaS নেবেন না যখন অ্যাপের এমন বিশেষ কনফিগারেশন দরকার যা প্ল্যাটফর্ম সমর্থন করে না। SaaS নেবেন না যখন ডেটা কোথায় থাকে, কীভাবে এনক্রিপ্ট হয়, তার ওপর পূর্ণ নিয়ন্ত্রণ দরকার। প্রতিটি সিদ্ধান্তই একটি ট্রেড-অফ (trade-off) — নিয়ন্ত্রণ বনাম সুবিধা।

১.৬ ডেপ্লয়মেন্ট মডেল: পাবলিক, প্রাইভেট, হাইব্রিড

পাবলিক ক্লাউড (public cloud) হলো AWS, Azure, GCP-র মতো ভাগ করা অবকাঠামো — অনেক গ্রাহক একই ফিজিক্যাল হার্ডওয়্যার ভাগ করে, তবে ভার্চুয়ালভাবে সম্পূর্ণ আলাদা। **প্রাইভেট ক্লাউড** হলো একটিমাত্র সংস্থার জন্য আলাদা

অবকাঠামো — নিজের ডেটা সেন্টারে বা প্রোভাইডারের কাছে আলাদা করে। **হাইব্রিড ক্লাউড** হলো দুটির সংমিশ্রণ: সংবেদনশীল ডেটা প্রাইভেটে, সাধারণ ওয়েব ট্র্যাফিক পাবলিকে। বাংলাদেশের অনেক ব্যাংক ও সরকারি সংস্থা নিয়ন্ত্রক কারণে (regulatory) হাইব্রিড পথে যায় — গ্রাহকের সংবেদনশীল তথ্য দেশের ভেতরে রাখা, আর পাবলিক ওয়েবসাইট ক্লাউডে চালানো। TechNova একটি স্টার্টআপ, তাই আমরা পাবলিক ক্লাউড দিয়ে শুরু করব, তবে ডেটা কোথায় থাকে তা সচেতনভাবে সিদ্ধান্ত নেব।

১.৭ AWS গ্লোবাল ইনফ্রাস্ট্রাকচার: Region, AZ, Edge Location

AWS-এর গ্লোবাল অবকাঠামো তিন স্তরে সাজানো। এই তিনটি শব্দ গুলিয়ে ফেললে পুরো আর্কিটেকচারই ভুল হবে, তাই ধীরে ধীরে বুঝুন।

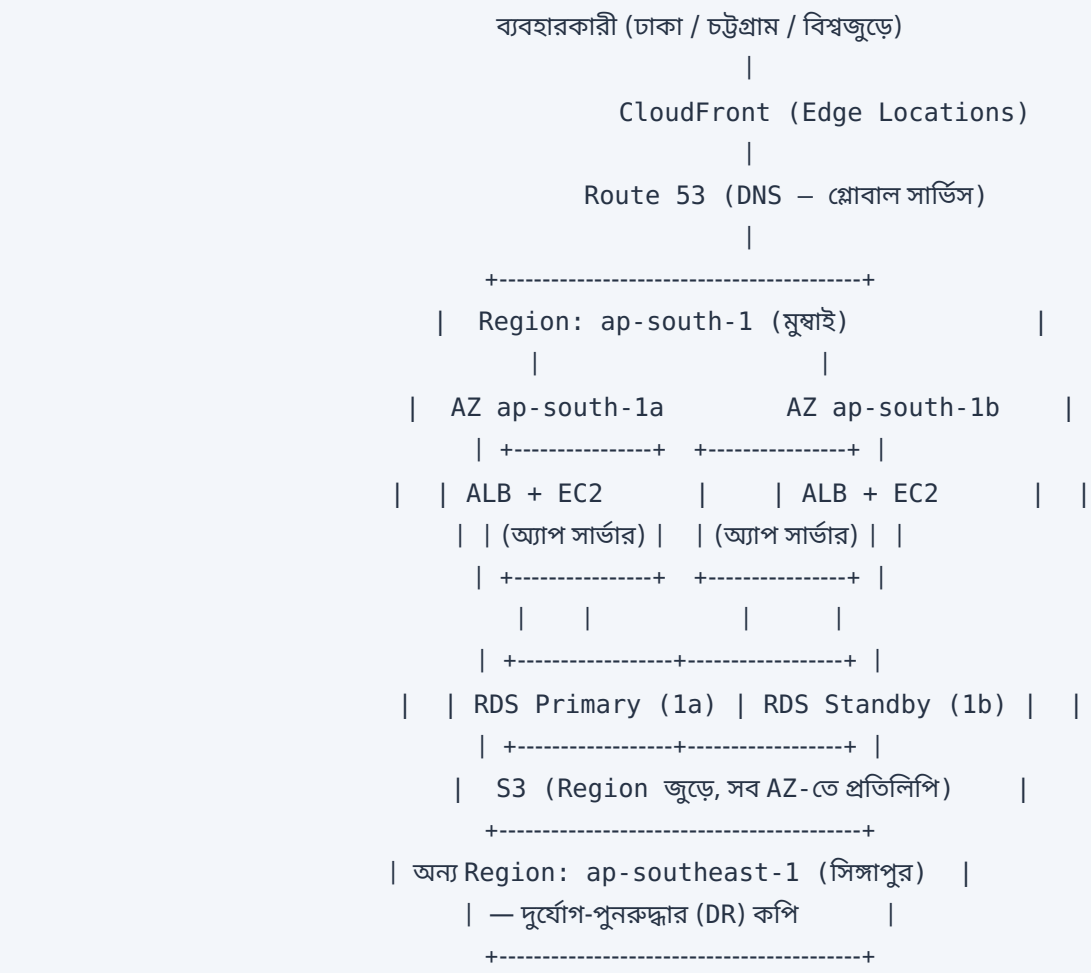
Region (রিজিওন): বিশ্বের একটি ভৌগোলিক এলাকায় অবস্থিত স্বাধীন ক্লাউড অবকাঠামোর সমষ্টি। যেমন `ap-south-1` (মুম্বাই), `ap-southeast-1` (সিঙ্গাপুর), `us-east-1` (N. Virginia)। প্রতিটি Region-এর নিজস্ব নাম, কোড ও এন্ডপয়েন্ট আছে। ডেটা সাধারণত Region ছেড়ে যায় না, যদি না আপনি নিজে সরান।

Availability Zone (AZ): একটি Region-এর ভেতরে এক বা একাধিক ফিজিক্যাল ডেটা সেন্টার, যার নিজস্ব বিদ্যুৎ, কুলিং ও নেটওয়ার্ক আছে। একটি Region-এ সাধারণত কমপক্ষে তিনটি AZ থাকে, যেমন `ap-south-1a`, `ap-south-1b`, `ap-south-1c`। AZ-গুলো একে অপর থেকে কয়েক কিলোমিটার দূরে, কিন্তু কম-লেটেন্সি (low-latency) ফাইবারে সংযুক্ত।

Edge Location: বিশ্বজুড়ে ছড়িয়ে থাকা ছোট পয়েন্ট অব প্রেজেন্স, যেখানে CloudFront (CDN) ও Route 53-এর মতো সেবা ব্যবহারকারীর কাছাকাছি থেকে কনটেন্ট দেয়। Edge Location-এ আপনার অ্যাপ চলে না; এটি ক্যাশিং ও DNS-এর কাজ করে।

কেন এই ভাগ? কারণ ব্যর্থতা (failure) সবসময় একটি সীমানার মধ্যে ঘটে। একটি AZ-তে বিদ্যুৎ গেলে অন্য AZ চলতে থাকে — এটাই **ফল্ট ডোমেইন**-এর ধারণা। একটি Region-এ বড় দুর্যোগ হলে অন্য Region-এর কপি থেকে সেবা চালু করা যায় — এটাই ভৌগোলিক স্থিতিস্থাপকতা (geographic resilience)। আপনার অ্যাপ যদি একটি মাত্র AZ-তে থাকে, তাহলে আপনি আসলে একটি মাত্র বিল্ডিংয়ের ওপর নির্ভর করছেন — ক্লাউডের সুবিধার কিছুই পাচ্ছেন না।

TechNova — মুম্বাই Region-এ মাল্টি-AZ আর্কিটেকচার (সরলীকৃত)



১.৭.১ গ্লোবাল সার্ভিস বনাম রিজিওনাল সার্ভিস

কিছু সার্ভিস **গ্লোবাল** — সেগুলো কোনো একটি Region-এর সঙ্গে বাঁধা নয়: IAM (পরিচয় ও অনুমতি), Route 53 (DNS), CloudFront (CDN), WAF, Organizations। আপনি যেকোনো Region থেকে IAM ব্যবহারকারী বানালে তা সব Region-এ কাজ করে। আবার বেশিরভাগ সার্ভিস **রিজিওনাল**: EC2, RDS, S3 বাকেট, Lambda — এগুলো নির্দিষ্ট Region-এ থাকে। S3 একটি বিশেষ ক্ষেত্র: বাকেট একটি Region-এ তৈরি হয়, কিন্তু AWS S3-র ডেটা সেই Region-এর একাধিক AZ-তে প্রতিলিপি (replicate) করে। নতুনরা প্রায়ই ভুল করে ভাবেন IAM-এর মতো EC2-ও গ্লোবাল — ফলে এক Region-এ বানানো সার্ভার অন্য Region-এ খুঁজে পান না। মনে রাখুন: কনসোলে ওপরে ডানদিকে Region সিলেক্টরটি সবসময় দেখে নিন — আপনি কোন Region-এ কাজ করছেন, সেটাই নির্ধারণ করে আপনার সম্পদ কোথায় তৈরি হবে।

১.৭.২ Region বাছাই: কী দেখে সিদ্ধান্ত নেবেন

বিবেচ্য বিষয়	TechNova-র ক্ষেত্রে	কেন গুরুত্বপূর্ণ
ব্যবহারকারীর কাছাকাছি দূরত্ব (latency)	বাংলাদেশের ব্যবহারকারীর জন্য মুম্বাই (ap-south-1) বা সিঙ্গাপুর (ap-southeast-1) সবচেয়ে কাছে	দূরত্ব বাড়লে পেজ লোড ধীর হয়; CloudFront দিয়ে কিছুটা সামলানো যায়
সার্ভিসের উপলভ্যতা	সব নতুন সার্ভিস সব Region-এ একসঙ্গে আসে না	

		প্রয়োজনীয় সার্ভিস না থাকলে অন্য Region নিতে হয়
ডেটা আবাসন (data residency) ও আইন	গ্রাহকের ডেটা কোন দেশে থাকবে, চুক্তিতে লেখা আছে কি না	কিছু খাতে ডেটা দেশের বাইরে নেওয়া যায় না
খরচ	Region ভেদে দাম আলাদা হতে পারে — AWS Pricing Calculator-এ যাচাই করুন	একই EC2 টাইপের দাম Region অনুযায়ী বদলায়
দুর্যোগ-পুনরুদ্ধার (DR)	প্রাথমিক Region মুম্বাই, DR Region সিঙ্গাপুর	এক Region ডাউন হলে অন্য Region থেকে সেবা চালু করা যায়

TechNova-র সিদ্ধান্ত: প্রাথমিক Region `ap-south-1` (মুম্বাই) — বাংলাদেশের সবচেয়ে কাছের বড় Region, এবং DR Region `ap-southeast-1` (সিঙ্গাপুর)। এই সিদ্ধান্তটি ডকুমেন্টে লিখে রাখা জরুরি, কারণ পরে Region বদলানো মানে ডেটা মাইগ্রেশন — ব্যয়বহুল ও ঝুঁকিপূর্ণ।

১.৮ উচ্চ-উপলভ্যতা, ফল্ট টলারেন্স ও ভৌগোলিক স্থিতিস্থাপকতা

তিনটি শব্দ প্রায়ই এক করে ফেলা হয়, কিন্তু এদের অর্থ আলাদা। **উচ্চ-উপলভ্যতা (high availability, HA)** মানে সিস্টেম বেশিরভাগ সময় চালু থাকা — সাধারণত 99.9% (বছরে ~৮ ঘণ্টা ৪৬ মিনিট ডাউনটাইম) বা 99.99% (বছরে ~৫৩ মিনিট) লক্ষ্য ধরা হয়। **ফল্ট টলারেন্স (fault tolerance)** মানে কোনো উপাদান নষ্ট হলেও সেবা এক মুহূর্তের জন্যও না থামা — যেমন একটি EC2 নষ্ট হলে লোড ব্যালান্সার সঙ্গে সঙ্গে ট্র্যাফিক অন্য ইনস্ট্যান্সে পাঠিয়ে দেয়। **ভৌগোলিক স্থিতিস্থাপকতা (geographic resilience)** মানে একটি পুরো Region-এর দুর্যোগেও সেবা টিকে থাকা — এর জন্য মাল্টি-রিজিওন নকশা লাগে, যা ব্যয়বহুল ও জটিল। প্রতিটি স্তরের খরচ ও জটিলতা বাড়ে: সিঙ্গেল-AZ সস্তা কিন্তু ঝুঁকিপূর্ণ, মাল্টি-AZ মাঝারি খরচে ভালো HA দেয়, মাল্টি-রিজিওন সবচেয়ে নিরাপদ কিন্তু ডেটা সিঙ্ক, DNS ফেইলওভার ও খরচের হিসাব জটিল করে তোলে। TechNova শুরুতে মাল্টি-AZ নকশা নেবে এবং DR-এর জন্য সিঙ্গাপুরে নিয়মিত ব্যাকআপ রাখবে — পূর্ণ সক্রিয়-সক্রিয় (active-active) মাল্টি-রিজিওন পরে, যখন ব্যবসা তা দাবি করবে।

১.৮.১ ফেইলিওর ডোমেইন দিয়ে ভাবা

নকশা করার সময় প্রতিটি উপাদানকে জিজ্ঞেস করুন: "এটি নষ্ট হলে কী কী একসঙ্গে নষ্ট হবে?" একটি AZ একটি ফেইলিওর ডোমেইন। একটি Region একটি বড় ফেইলিওর ডোমেইন। একটি মাত্র ডেটাবেস সার্ভার একটি ফেইলিওর ডোমেইন — তাই RDS Multi-AZ ব্যবহার করা হয়, যেখানে প্রাইমারি নষ্ট হলে স্ট্যান্ডবাই স্বয়ংক্রিয়ভাবে প্রাইমারি হয়। একটি মাত্র লোড ব্যালান্সারও ঝুঁকি — তাই Application Load Balancer (ALB) নিজেই মাল্টি-AZ সেবা হিসেবে চলে। নকশার নিয়ম: কোনো একক উপাদানের ওপর যেন পুরো সেবা নির্ভর না করে — একে বলে **সিঙ্গেল পয়েন্ট অব ফেইলিওর (SPOF)** এড়ানো।

> **WARNING:** একটি মাত্র AZ-তে প্রোডাকশন ডেটাবেস চালাবেন না। ডেভেলপমেন্টে সিঙ্গেল-AZ ঠিক আছে, কিন্তু প্রোডাকশনে RDS Multi-AZ, S3-র মতো মাল্টি-AZ সেবা, এবং কমপক্ষে দুটি AZ-তে অ্যাপ সার্ভার রাখুন। একটি AZ-র বিভ্রাটে আপনার গ্রাহক যেন কিছু টের না পায়।

১.৯ AWS-এ হাতেকলমে: Region ও AZ দেখা

কনসোলে: ওপরে ডানদিকে Region নামের পাশে ক্লিক করে `Asia Pacific (Mumbai) ap-south-1` বেছে নিন। তারপর EC2 ড্যাশবোর্ডে গেলে দেখবেন আপনার সম্পদ শুধু এই Region-এর। CLI-তে একই কাজ:

```
aws configure list
aws ec2 describe-availability-zones \
  --region ap-south-1 \
  --query "AvailabilityZones[].ZoneName" --output table
aws ec2 describe-regions --query "Regions[].RegionName" --output table
```

প্রথম কমান্ডটি দেখায় আপনার বর্তমান কনফিগারেশন — কোন Region, কোন প্রোফাইল। দ্বিতীয়টি মুম্বাই Region-এর সব AZ-র নাম তালিকাভুক্ত করে। তৃতীয়টি সব Region-এর তালিকা দেয়। মনে রাখবেন: `describe-regions` চালাতে IAM অনুমতি লাগে (অধ্যায় ৩ দেখুন), আর প্রতিটি কমান্ডের বিল হয় না — কিন্তু এরপর আপনি যে সম্পদ তৈরি করবেন, তার বিল হবে। তাই পরীক্ষার পর সম্পদ মুছে ফেলুন।

১.৯.১ CloudShell দিয়ে প্রথম পরীক্ষা

AWS Management Console-এর ওপরে CloudShell আইকনে ক্লিক করলে ব্রাউজারেই একটি টার্মিনাল খুলে যায় — এতে AWS CLI v2 আগে থেকে ইনস্টল করা থাকে এবং আপনার বর্তমান লগইনের ক্রেডেনশিয়াল স্বয়ংক্রিয়ভাবে ব্যবহার হয়। নতুনদের জন্য এটি নিরাপদ শুরু: নিজের ল্যাপটপে ক্রেডেনশিয়াল ফাইল বসাতে হয় না। তবে CloudShell-এর স্টোরেজ সীমিত (প্রতি Region-এ 1 GB হোম ডিরেক্টরি) এবং এটি দীর্ঘমেয়াদি কাজের জায়গা নয়। বিস্তারিত অধ্যায় ৪-এ।

১.১০ প্রোডাকশন দৃশ্যকল্প: TechNova MERN অ্যাপ

TechNova একটি MERN অ্যাপ বানাচ্ছে — MongoDB-র বদলে আমরা এখানে PostgreSQL ব্যবহার করব (RDS-এর জন্য উপযুক্ত), Express.js ব্যাকএন্ড, React ফ্রন্টএন্ড, Node.js রানটাইম। ব্যবহারকারী ঢাকা ও চট্টগ্রামে, কিছু প্রবাসী ব্যবহারকারীও আছেন। নকশা: React অ্যাপ S3-তে হোস্ট করে CloudFront দিয়ে দেওয়া হবে (Edge Location থেকে দ্রুত লোড), API চলবে ECS-তে দুটি AZ-জুড়ে, ডেটাবেস RDS PostgreSQL Multi-AZ, ফাইল আপলোড S3-তে। Route 53 দিয়ে `app.technova.example` ডোমেইনটি ALB-তে নির্দেশ করা হবে। এই নকশায় একটি AZ ডাউন হলেও ALB অন্য AZ-র কন্টেইনারে ট্র্যাফিক পাঠাবে; ডেটাবেসের প্রাইমারি নষ্ট হলে স্ট্যান্ডবাই দায়িত্ব নেবে; আর স্ট্যাটিক ফাইল CloudFront ক্যাশ থেকে আসবে। পরের অধ্যায়গুলোতে আমরা এই নকশার প্রতিটি টুকরো বাস্তবায়ন করব — IAM, নেটওয়ার্ক, কম্পিউট, ডেটাবেস, স্টোরেজ।

> **বাস্তব অভিজ্ঞতা:** একটি স্টার্টআপ একবার তাদের পুরো অ্যাপ একটি মাত্র AZ-তে চালিয়েছিল কারণ "তাড়াতাড়ি লঞ্চ করতে হবে"। একদিন সকালে সেই AZ-তে নেটওয়ার্ক সমস্যা হলো — সাইট চার ঘণ্টা ডাউন। গ্রাহক হারানো ও ক্ষতিপূরণের খরচ, মাল্টি-AZ-এর অতিরিক্ত খরচের চেয়ে বহু গুণ বেশি ছিল। HA কোনো বিলাসিতা নয়; এটি ব্যবসার বীমা।

১.১১ নিরাপত্তা বিবেচনা

Region বাছাই নিজেই একটি নিরাপত্তা সিদ্ধান্ত: ডেটা কোন দেশের আইনের আওতায় থাকবে? সংবেদনশীল ডেটা এমন Region-এ রাখবেন না যেখানে আপনার সংস্থার চুক্তি বা নিয়ম অনুমতি দেয় না। Edge Location-এ ক্যাশ করা কনটেন্টে ব্যক্তিগত ডেটা রাখবেন না — CloudFront ক্যাশে শুধু পাবলিক স্ট্যাটিক ফাইল রাখুন। আর মনে রাখবেন, গ্লোবাল সার্ভিস (যেমন IAM) একটি Region-এর বিভ্রাটেও কাজ করে, কারণ সেগুলো আলাদাভাবে নকশা করা — তবে IAM-এর পরিবর্তন (যেমন নতুন নীতি) সব Region-এ ছড়াতে কিছু সময় লাগতে পারে (eventual consistency)।

১.১২ খরচ বিবেচনা

মাল্টি-AZ মানে প্রায় দ্বিগুণ কম্পিউট খরচ — দুটি AZ-তে সার্ভার চালাতে হয়। AZ-গুলোর মধ্যে ডেটা স্থানান্তর (cross-AZ data transfer) বিলযোগ্য হতে পারে — AWS Pricing Calculator-এ যাচাই করুন। মাল্টি-রিজিওন আরও ব্যয়বহুল: রিজিওনগুলোর মধ্যে ডেটা প্রতিলিপি ও স্থানান্তরের বিল যোগ হয়। তাই HA-র স্তর ব্যবসার প্রয়োজনের সঙ্গে মেলান: ডেভ পরিবেশে সিঙ্গেল-AZ, স্টেজিংয়ে মাল্টি-AZ, প্রোডাকশনে মাল্টি-AZ + DR ব্যাকআপ। Free Tier-এর সুবিধা নিন পরীক্ষার জন্য, কিন্তু মনে রাখবেন Free Tier-এর সীমা ও মেয়াদ পরিবর্তিত হয় — সবসময় অফিসিয়াল পেজে যাচাই করুন। অপ্রয়োজনীয় সম্পদ চালু রেখে যাবেন না; অধ্যায় ৫-এ বিল পর্যবেক্ষণ শিখবেন।

১.১৩ সাধারণ ভুল

(১) ভুল Region-এ সম্পদ তৈরি করা এবং পরে খুঁজে না পাওয়া — কনসোলের Region সিলেক্টর সবসময় দেখুন। (২) একটি মাত্র AZ-তে প্রোডাকশন চালানো। (৩) AZ-কে "ডেটা সেন্টার" ভেবে নেওয়া — একটি AZ-তে একাধিক ডেটা সেন্টার থাকতে পারে। (৪) Edge Location-কে কম্পিউটের জায়গা ভাবা — সেখানে অ্যাপ চলে না। (৫) S3 বাকেটকে গ্লোবাল ভাবা — বাকেট রিজিওনাল, নাম গ্লোবালি অনন্য হতে হয়। (৬) Region বদলালে সবকিছু স্বয়ংক্রিয়ভাবে সরে যাবে ভাবা — সম্পদ মাইগ্রেট করতে হয়। (৭) HA-র জন্য শুধু লোড ব্যালান্সার যথেষ্ট ভাবা — ডেটাবেস ও স্টোরেজ স্তরেও HA লাগে।

১.১৪ সমস্যা সমাধান (Troubleshooting)

> **সমস্যা:** কনসোলে EC2 ইনস্ট্যান্স দেখা যাচ্ছে না। **কারণ ও সমাধান:** সম্ভবত ভুল Region-এ আছেন। ওপরে ডানদিকে Region বদলে দেখুন। CLI-তে `--region` ফ্ল্যাগ বা `AWS_DEFAULT_REGION` যাচাই করুন।

সমস্যা: একটি AZ-র ইনস্ট্যান্সে পৌঁছানো যাচ্ছে না, অন্যটি ঠিক আছে। **সমাধান:** এটি AZ-স্তরের সমস্যা হতে পারে — AWS Health Dashboard ও Service Health দেখুন; ALB-র টার্গেট হেলথ যাচাই করুন; আক্রান্ত AZ-র ইনস্ট্যান্স সরিয়ে নেওয়ার পরিকল্পনা করুন। **সমস্যা:** CloudFront থেকে পুরনো কনটেন্ট আসছে। **সমাধান:** Edge ক্যাশে পুরনো কপি আছে — ইনভ্যালিডেশন (invalidation) করুন বা ক্যাশ TTL কমান। **সমস্যা:** নতুন সার্ভিস আপনার Region-এ নেই। **সমাধান:** সার্ভিসটি কোন Region-এ আছে তা অফিসিয়াল ডকুমেন্টেশনে দেখুন; প্রয়োজনে কাছের Region ব্যবহার করুন।

১.১৫ অধ্যায়ের সারাংশ

ক্লাউড এসেছে ক্যাপাসিটি প্ল্যানিং, হার্ডওয়্যার রক্ষণাবেক্ষণ ও ধীরগতির সংস্থানের সমস্যা সমাধান করতে। IaaS/PaaS/SaaS হলো দায়িত্ব ভাগাভাগির তিনটি স্তর; পাবলিক/প্রাইভেট/হাইব্রিড হলো ডেপ্লয়মেন্টের তিনটি ধরন। AWS-এর গ্লোবাল অবকাঠামো Region, AZ ও Edge Location দিয়ে গঠিত — Region ভৌগোলিক এলাকা, AZ স্বাধীন ডেটা সেন্টার গুচ্ছ, Edge Location ক্যাশিং ও DNS-এর পয়েন্ট। উচ্চ-উপলভ্যতা মানে ব্যর্থতাকে ধরে নিয়ে নকশা করা: মাল্টি-AZ দিয়ে AZ-স্তরের ব্যর্থতা, মাল্টি-রিজিওন দিয়ে Region-স্তরের ব্যর্থতা সামলানো। TechNova মুম্বাইকে প্রাথমিক ও সিঙ্গাপুরকে DR Region হিসেবে বেছে মাল্টি-AZ নকশা দিয়ে শুরু করছে।

১.১৬ জ্ঞান যাচাই

১. ক্লাউড কম্পিউটিংয়ের আগে ক্যাপাসিটি প্ল্যানিং কেন সমস্যা ছিল? ২. IaaS, PaaS ও SaaS-এর মধ্যে দায়িত্বের পার্থক্য উদাহরণসহ ব্যাখ্যা করুন। ৩. Region ও Availability Zone-এর সংজ্ঞা ও সম্পর্ক লিখুন। ৪. Edge Location-এ কী চলে আর কী চলে না? ৫. গ্লোবাল সার্ভিস ও রিজিওনাল সার্ভিসের দুটি করে উদাহরণ দিন। ৬. ফল্ট ডোমেইন ও সিঙ্গেল পয়েন্ট অব ফেইলিওর কী? ৭. উচ্চ-উপলভ্যতা, ফল্ট টলারেন্স ও ভৌগোলিক স্থিতিস্থাপকতার পার্থক্য কী? ৮. 99.9% উপলভ্যতা

মানে বছরে কতক্ষণ ডাউনটাইম? ৯. TechNova কেন মুম্বাইকে প্রাথমিক Region হিসেবে বেছে নিল? ১০. একটি অ্যাপ শুধু একটি AZ-তে থাকলে কী ঝুঁকি থাকে?

১.১৭ ব্যবহারিক অনুশীলন

অনুশীলন ১: CloudShell খুলে `aws ec2 describe-availability-zones --region ap-south-1` ও `aws ec2 describe-regions` চালান; ফলাফল লিখে রাখুন এবং কোন Region-গুলো আপনার ব্যবহারকারীর সবচেয়ে কাছে তা নোট করুন। **অনুশীলন ২:** কাগজে TechNova MERN অ্যাপের মাল্টি-AZ আর্কিটেকচার আঁকুন — ALB, দুটি AZ-তে অ্যাপ সার্ভার, RDS Multi-AZ, S3 ও CloudFront দেখান; প্রতিটি উপাদান নষ্ট হলে কী হবে তা পাশে লিখুন। **অনুশীলন ৩:** AWS Pricing Calculator খুলে (দাম না লিখে) একটি মাল্টি-AZ বনাম সিঙ্গেল-AZ নকশার খরচের উপাদানগুলো তালিকাভুক্ত করুন — কোন কোন খাতে মাল্টি-AZ বেশি খরচ করে? **অনুশীলন ৪:** আপনার নিজের একটি ছোট প্রজেক্টের জন্য Region বাছাইয়ের যুক্তি এক পৃষ্ঠায় লিখুন — latency, সার্ভিস উপলভ্যতা, ডেটা আবাসন ও খরচের দিক থেকে।

অধ্যায় ২ • PART I — AWS Foundations

Chapter 2 — AWS Account, Root User and Account Governance

২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) AWS অ্যাকাউন্ট কী এবং রুট ব্যবহারকারী (root user) কেন সবচেয়ে শক্তিশালী ও সবচেয়ে ঝুঁকিপূর্ণ পরিচয়, তা ব্যাখ্যা করতে; (২) রুট ব্যবহারকারীকে সুরক্ষিত করতে MFA চালু করা, শক্তিশালী পাসওয়ার্ড ব্যবহার এবং দৈনন্দিন কাজে রুট ব্যবহার না করার নিয়ম প্রয়োগ করতে; (৩) IAM Identity Center-এর ধারণা ও ব্যবহার ব্যাখ্যা করতে; (৪) মাল্টি-অ্যাকাউন্ট কৌশল — কেন একটি অ্যাকাউন্টে সবকিছু রাখা বিপজ্জনক — যুক্তি দিয়ে বলতে; (৫) AWS Organizations, Organizational Unit (OU) ও Service Control Policy (SCP)-র কাজ ব্যাখ্যা করতে; (৬) Control Tower-এর ধারণা বুঝতে; এবং (৭) TechNova-র জন্য dev/staging/production অ্যাকাউন্ট পৃথকীকরণের নকশা তৈরি করতে।

২.২ পূর্বশর্ত জ্ঞান

অধ্যায় ১-এর Region ও গ্লোবাল সার্ভিসের ধারণা জানা থাকতে হবে — বিশেষ করে IAM যে গ্লোবাল সার্ভিস, তা মনে রাখুন। ইমেইল, পাসওয়ার্ড ও MFA (multi-factor authentication) সম্পর্কে সাধারণ ধারণা থাকা ভালো। সংস্থায় (organization) দল, পরিবেশ (environment) ও বিলিংয়ের ধারণা থাকলে সুবিধা হবে। IAM-এর বিস্তারিত অধ্যায় ৩-এ আসছে; এখানে আমরা অ্যাকাউন্ট স্তরের শাসন (governance) নিয়ে কথা বলব।

২.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

প্রায় প্রতিটি বড় AWS দুর্ঘটনার পেছনে অ্যাকাউন্ট-স্তরের ভুল থাকে: রুট ব্যবহারকারীর ক্রেডেনশিয়াল ফাঁস হয়ে যাওয়া, একটি অ্যাকাউন্টে dev ও production একসঙ্গে থাকায় পরীক্ষার কমান্ডে প্রোডাকশন ডেটাবেস মুছে যাওয়া, বা কোনো কর্মী চলে যাওয়ার পরও তার অ্যাক্সেস থেকে যাওয়া। AWS-এ অ্যাকাউন্ট হলো নিরাপত্তা ও বিলিংয়ের সীমানা (boundary) — একটি অ্যাকাউন্টের ভেতরে ভুল হলে তার ক্ষতি সেই অ্যাকাউন্টেই সীমাবদ্ধ থাকে, যদি আপনি অ্যাকাউন্ট আলাদা করে থাকেন। রুট ব্যবহারকারী হলো সেই অ্যাকাউন্টের "চাবির মালিক" — তার হাতে সবকিছু করার ক্ষমতা, এবং সেই ক্ষমতা কেড়ে নেওয়ার কোনো উপায় নেই, শুধু সুরক্ষিত করা যায়। এই অধ্যায়ের নিয়মগুলো না মানলে পরের অধ্যায়ের IAM নীতিগুলোও আপনাকে বাঁচাতে পারবে না।

২.৪ AWS অ্যাকাউন্ট কী

AWS অ্যাকাউন্ট হলো আপনার AWS সম্পদ, বিলিং ও নিরাপত্তার মূল ধারক (container)। অ্যাকাউন্ট তৈরি হয় একটি ইমেইল ঠিকানা, পাসওয়ার্ড ও পেমেন্ট পদ্ধতি দিয়ে। প্রতিটি অ্যাকাউন্টের একটি অনন্য ১২-সংখ্যার অ্যাকাউন্ট ID থাকে — TechNova-র ক্ষেত্রে আমরা উদাহরণ হিসেবে `111122223333` ব্যবহার করছি। অ্যাকাউন্টের ভেতরে আপনি Region, VPC, EC2, S3 বাকেট — সবকিছু তৈরি করেন। গুরুত্বপূর্ণ বিষয়: অ্যাকাউন্টই হলো **বিলিংয়ের সীমানা** (বিল অ্যাকাউন্ট অনুযায়ী আসে) এবং **নিরাপত্তার সীমানা** (এক অ্যাকাউন্টের IAM ব্যবহারকারী অন্য অ্যাকাউন্টের সম্পদে ঢুকতে পারে না, যদি না স্পষ্টভাবে অনুমতি দেওয়া হয়)। তাই "একটি অ্যাকাউন্টে সবকিছু" রাখা মানে একটি ভুলে সবকিছু হারানোর ঝুঁকি নেওয়া।

> **মূল ধারণা:** AWS-এ অ্যাকাউন্ট = পরিচয় + বিলিং + নিরাপত্তা সীমানা। রুট ব্যবহারকারী (root user) হলো অ্যাকাউন্ট তৈরির সময়কার ইমেইল-ভিত্তিক পরিচয়, যার অনুমতি সীমাহীন ও অপরিবর্তনীয়। তাকে শুধু জরুরি কাজে ব্যবহার করুন — দৈনন্দিন কাজে নয়।

২.৫ রুট ব্যবহারকারী: সবচেয়ে শক্তিশালী, সবচেয়ে ঝুঁকিপূর্ণ

রুট ব্যবহারকারীর ক্ষমতা IAM দিয়ে সীমিত করা যায় না — কোনো নীতি (policy) তার ওপর প্রযোজ্য নয়। সে বিলিং দেখতে পারে, অ্যাকাউন্ট বন্ধ করতে পারে, সব IAM ব্যবহারকারী মুছে ফেলতে পারে, সব সম্পদ মুছে ফেলতে পারে। এই ক্ষমতা যেমন জরুরি (যেমন অ্যাকাউন্ট পুনরুদ্ধারে), তেমনই ভয়ংকর — রুটের পাসওয়ার্ড ফাঁস হলে আক্রমণকারী পুরো অ্যাকাউন্ট দখল করতে পারে, বিটকয়েন মাইনিংয়ের জন্য শত শত সার্ভার চালাতে পারে, এবং বিলের অঙ্ক আকাশছোঁয়া হতে পারে। বাস্তবে এমন ঘটনা বহুবার ঘটেছে: GitHub-এ ভুল করে রুটের অ্যাক্সেস কী আপলোড, ফিশিং ইমেইলে পাসওয়ার্ড দেওয়া, পুরনো কর্মীর কাছে রুটের তথ্য থেকে যাওয়া।

২.৫.১ রুট সুরক্ষার নিয়ম (অবশ্যপালনীয়)

(১) রুটের জন্য **শক্তিশালী, অনন্য পাসওয়ার্ড** ব্যবহার করুন — পাসওয়ার্ড ম্যানেজারে রাখুন, কোথাও লিখে রাখবেন না। (২) **MFA চালু করুন** — হার্ডওয়্যার কী বা অথেনটিকেটর অ্যাপ (যেমন virtual MFA) ব্যবহার করুন; SMS MFA-র চেয়ে অ্যাপ-ভিত্তিক MFA নিরাপদ। (৩) রুটের **অ্যাক্সেস কী তৈরি করবেন না** — দৈনন্দিন CLI কাজের জন্য IAM ব্যবহারকারী বা IAM Identity Center ব্যবহার করুন। (৪) রুট দিয়ে **দৈনন্দিন কাজ করবেন না** — সাইন ইন করুন শুধু তখনই, যখন শুধু রুটই করতে পারে এমন কাজ দরকার (যেমন অ্যাকাউন্টের ইমেইল বদলানো, Support প্ল্যান বদলানো, অ্যাকাউন্ট বন্ধ করা)। (৫) রুটের লগইনের জন্য **আলাদা ইমেইল** ব্যবহার করুন, যা দৈনন্দিন কাজে ব্যবহৃত হয় না। (৬) CloudTrail-এ রুটের কার্যকলাপ পর্যবেক্ষণ করুন — রুট সাইন ইন হলে অ্যালার্ট আসা উচিত।

```
# রুটের MFA অবস্থা যাচাই (IAM ব্যবহারকারী দিয়ে চালান)
aws iam get-account-summary \
  --query "SummaryMap.AccountMFAEnabled"
# আউটপুট 1 হলে রুটে MFA চালু আছে; 0 হলে এখনই চালু করুন
```

কনসোলে: IAM → Dashboard → "Root user has MFA" দেখুন। MFA চালু করতে: ওপরে ডানদিকে অ্যাকাউন্ট নাম → Security credentials → Multi-factor authentication (MFA) → Assign MFA device → Authenticator app বেছে QR কোড স্ক্যান করুন। MFA ডিভাইসের ব্যাকআপ কোড বা বিকল্প ডিভাইস নিরাপদে সংরক্ষণ করুন — MFA ডিভাইস হারালে অ্যাকাউন্ট পুনরুদ্ধার দীর্ঘ ও কষ্টকর হয়।

> **WARNING:** রুট ব্যবহারকারীর অ্যাক্সেস কী কখনো তৈরি করবেন না, এবং তৈরি থাকলে এখনই মুছে ফেলুন। অ্যাক্সেস কী ফাঁস হলে আক্রমণকারী পাসওয়ার্ড বা MFA ছাড়াই API দিয়ে অ্যাকাউন্ট চালাতে পারে। IAM → Security credentials → Access keys-এ রুটের কী আছে কি না যাচাই করুন।

২.৬ IAM Identity Center: মানুষের জন্য সঠিক পথ

আগে এর নাম ছিল AWS SSO। **IAM Identity Center** হলো সংস্থার কর্মীদের জন্য কেন্দ্রীয় লগইন ব্যবস্থা — একজন কর্মী একবার লগইন করে একাধিক AWS অ্যাকাউন্টে ঢুকতে পারে, প্রতিটি অ্যাকাউন্টে তার ভূমিকা (role) অনুযায়ী অনুমতি নিয়ে। এটি IAM ব্যবহারকারী (দীর্ঘমেয়াদি পাসওয়ার্ড ও অ্যাক্সেস কী) বানানোর চেয়ে অনেক নিরাপদ: কর্মী চলে গেলে এক জায়গা থেকে তার অ্যাক্সেস বন্ধ করা যায়, প্রতিটি লগইনে MFA বাধ্যতামূলক করা যায়, এবং অস্থায়ী ক্রেডেনশিয়াল ব্যবহার

হয়। TechNova-র নিয়ম: মানুষের জন্য Identity Center, মেশিন/অ্যাপ্লিকেশনের জন্য IAM রোল — মানুষের জন্য দীর্ঘমেয়াদি অ্যাক্সেস কী নয়। Identity Center-এ **permission set** দিয়ে বলা হয় কে কোন অ্যাকাউন্টে কী করতে পারবে — যেমন "Developer" permission set dev অ্যাকাউন্টে EC2/RDS চালাতে পারে, কিন্তু production অ্যাকাউন্টে শুধু পড়তে (read-only) পারে।

২.৭ মাল্টি-অ্যাকাউন্ট কৌশল: কেন একটি অ্যাকাউন্ট যথেষ্ট নয়

একটি মাত্র অ্যাকাউন্টে dev, staging ও production রাখলে: (১) একটি ভুল কমান্ডে প্রোডাকশন মুছে যেতে পারে — `aws s3 rm --recursive` ভুল বাকেটে চালালে আর ফেরত নেই; (২) বিল আলাদা করা কঠিন — কোন পরিবেশ কত খরচ করছে বোঝা যায় না; (৩) নিরাপত্তার সীমানা নেই — dev-এর একজন ডেভেলপার প্রোডাকশন ডেটা দেখতে পারে; (৪) সার্ভিস কোটা (quota) ভাগ হয়ে যায় — dev-এর পরীক্ষায় কোটা শেষ হলে production-এ নতুন সার্ভার চালু হবে না; (৫) অডিট কঠিন — কে কী করেছে, কোন পরিবেশে, তা আলাদা করা যায় না। সমাধান: প্রতিটি পরিবেশের জন্য আলাদা অ্যাকাউন্ট, সঙ্গে একটি ব্যবস্থাপনা (management) অ্যাকাউন্ট, একটি লগ আর্কাইভ অ্যাকাউন্ট ও একটি নিরাপত্তা (security) অ্যাকাউন্ট।

অ্যাকাউন্ট	উদ্দেশ্য	কে ঢুকতে পারে	TechNova উদাহরণ
Management (ব্যবস্থাপনা)	Organizations, বিলিং, SCP	শুধু অ্যাডমিন (২ জন)	technova-mgmt (111122223333)
Development	পরীক্ষা, শেখা, ভাঙা	সব ডেভেলপার	technova-dev
Staging	প্রোডাকশনের মতো পরীক্ষা	ডেভেলপার + QA (সীমিত)	technova-staging
Production	গ্রাহকের সেবা	শুধু সিনিয়র + অন-কল (সীমিত)	technova-prod
Log Archive	CloudTrail ও Config লগ সংরক্ষণ	শুধু নিরাপত্তা দল	technova-logs
Security / Audit	নিরাপত্তা পর্যবেক্ষণ (GuardDuty, Security Hub)	শুধু নিরাপত্তা দল	technova-security

২.৮ AWS Organizations ও Organizational Unit (OU)

AWS Organizations হলো একাধিক অ্যাকাউন্টকে একসঙ্গে পরিচালনার সেবা। একটি management অ্যাকাউন্টের অধীনে member অ্যাকাউন্টগুলোকে **Organizational Unit (OU)** নামে গোষ্ঠীতে সাজানো যায় — যেমন "Workloads" OU-তে dev/staging/prod, "Security" OU-তে logs/security অ্যাকাউন্ট। Organizations দিয়ে একত্রিত বিলিং (consolidated billing) করা যায় — সব অ্যাকাউন্টের বিল এক জায়গায় আসে, আর ভলিউম ডিসকাউন্ট ভাগাভাগি হয়। তবে মনে রাখবেন: management অ্যাকাউন্টে কোনো workload চালাবেন না — সেটি শুধু ব্যবস্থাপনার জন্য।

২.৯ Service Control Policy (SCP): সংস্থার নিরাপত্তা-বেড়া

SCP হলো Organizations-এর নীতি, যা একটি OU বা অ্যাকাউন্টে সর্বোচ্চ কী করা যাবে তা বেঁধে দেয় — এটি অনুমতি দেয় না, শুধু সীমা টানে। উদাহরণ: "কেউ `ap-south-1` ও `ap-southeast-1` ছাড়া অন্য Region-এ সম্পদ তৈরি করতে পারবে না" — এমন SCP থাকলে একজন ডেভেলপার ভুল করে `us-east-1`-এ দামি সার্ভার চালাতে পারবে না, এমনকি তার IAM

নীতিতে অনুমতি থাকলেও। SCP IAM নীতির ওপরে একটি অতিরিক্ত ফিল্টার: চূড়ান্ত অনুমতি = SCP-র অনুমোদিত অংশ ∩ IAM নীতির অনুমোদিত অংশ। SCP management অ্যাকাউন্টের রুট ব্যবহারকারীর ওপর প্রযোজ্য নয় — তাই management অ্যাকাউন্টকে আরও কঠোরভাবে সুরক্ষিত করুন। একটি সাধারণ SCP উদাহরণ:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyOutsideApprovedRegions",
    "Effect": "Deny",
    "Action": "*",
    "Resource": "*",
    "Condition": {
      "StringNotEquals": {
        "aws:RequestedRegion": ["ap-south-1", "ap-southeast-1"]
      }
    }
  }]
}
```

এই নীতিটি পড়ুন: যেকোনো কাজ (`Action: *`), যেকোনো সম্পদে, যদি অনুরোধ করা Region অনুমোদিত দুটির বাইরে হয়, তাহলে **Deny**। কিছু গ্লোবাল সার্ভিস (যেমন IAM) Region-শর্তে আটকে যেতে পারে — তাই SCP চালুর আগে পরীক্ষামূলক OU-তে যাচাই করুন।

২.১০ Control Tower: মাল্টি-অ্যাকাউন্টের প্রস্তুত কাঠামো

AWS Control Tower হলো Organizations-এর ওপরে তৈরি একটি সেবা, যা নতুন অ্যাকাউন্ট তৈরির সময় স্বয়ংক্রিয়ভাবে নিরাপত্তা নিয়ন্ত্রণ (guardrail) বসিয়ে দেয় — যেমন CloudTrail সব অ্যাকাউন্টে চালু, লগ কেন্দ্রীয় অ্যাকাউন্টে যাওয়া, রুটের MFA বাধ্যতামূলক। ছোট দলের জন্য Control Tower দ্রুত শুরু করার ভালো উপায়, কারণ নিজে হাতে সব guardrail বানাতে অনেক সময় লাগে। তবে Control Tower-এর নিজস্ব কাঠামো ও খরচ আছে; খুব ছোট দল (২–৩ জন) হাতে Organizations + SCP দিয়েও শুরু করতে পারে, পরে Control Tower-এ যেতে পারে। সিদ্ধান্তটি দলের আকার ও বৃদ্ধির গতির ওপর নির্ভর করে।

২.১১ প্রোডাকশন দৃশ্যকল্প: TechNova-র অ্যাকাউন্ট নকশা

TechNova ছয়টি অ্যাকাউন্ট দিয়ে শুরু করছে (উপরের টেবিল দেখুন)। প্রতিটি অ্যাকাউন্টের রুটে MFA চালু, রুটের অ্যাক্সেস কী নেই, রুটের লগইন শুধু জরুরি কাজে। মানুষ Identity Center দিয়ে ঢোকে; production অ্যাকাউন্টে ডেভেলপারদের শুধু read-only অনুমতি, পরিবর্তন করতে হলে অন-কল ইঞ্জিনিয়ারের অনুমোদন লাগে। SCP দিয়ে Region সীমাবদ্ধ (`ap-south-1`, `ap-southeast-1`) এবং production OU-তে EC2-র বড় ইনস্ট্যান্স টাইপ নিষিদ্ধ। CloudTrail সব অ্যাকাউন্টে চালু, লগ যায় Log Archive অ্যাকাউন্টের S3 বাকেটে, যেখানে কেউ মুছতে পারে না (Object Lock)। এই নকশার ফলে একজন ডেভেলপার dev-এ যা খুশি ভাঙতে পারে, কিন্তু production নিরাপদ থাকে।

TechNova — AWS Organizations কাঠামো

```
Management অ্যাকাউন্ট (111122223333)
                    |
          +---------+---------+
          |                   |
   OU: Workloads          OU: Security
  |-- technova-dev       |-- technova-logs
  |-- technova-staging   |-- technova-security
  |-- technova-prod
                    |
   SCP: শুধু ap-south-1 ও ap-southeast-1
   SCP: production OU-তে বড় ইনস্ট্যান্স নিষিদ্ধ
Identity Center: এক লগইন, অ্যাকাউন্ট অনুযায়ী ভূমিকা
```

২.১২ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: রুটের MFA, Identity Center-এ MFA বাধ্যতামূলক, CloudTrail অ্যালার্ট, অ্যাকাউন্ট পৃথকীকরণ — এগুলো কোনো "পরে করব" কাজ নয়; অ্যাকাউন্ট তৈরির দিনেই করতে হবে। খরচ: প্রতিটি অ্যাকাউন্টে আলাদা বিলিং হলেও Organizations-এর একত্রিত বিলিংয়ে মোট বিল এক জায়গায় দেখা যায়; Cost Explorer-এ অ্যাকাউন্ট অনুযায়ী ফিল্টার করে কোন পরিবেশ কত খরচ করছে তা জানা যায় (অধ্যায় ৫)। অতিরিক্ত অ্যাকাউন্টের জন্য AWS অতিরিক্ত চার্জ নেয় না — অ্যাকাউন্ট নিজে বিনামূল্যে, বিল হয় সম্পদের জন্য।

২.১৩ সাধারণ ভুল ও সমস্যা সমাধান

ভুল: (১) রুট দিয়ে দৈনন্দিন কাজ করা; (২) রুটের অ্যাক্সেস কী তৈরি করা; (৩) সব পরিবেশ এক অ্যাকাউন্টে রাখা; (৪) SCP পরীক্ষা না করে production OU-তে প্রয়োগ করা — এতে বৈধ কাজও বন্ধ হয়ে যেতে পারে; (৫) management অ্যাকাউন্টে workload চালানো; (৬) কর্মী চলে যাওয়ার পর Identity Center থেকে অ্যাক্সেস না সরানো। সমস্যা: Identity Center-এ লগইন হচ্ছে না — MFA ডিভাইসের সময় (clock) ঠিক আছে কি না দেখুন; SCP-র কারণে কাজ বন্ধ — CloudTrail-এ `AccessDenied` ইভেন্টে কোন SCP বাধা দিচ্ছে তা দেখুন; নতুন অ্যাকাউন্টে বিলিং দেখা যাচ্ছে না — একত্রিত বিলিংয়ে member অ্যাকাউন্টের বিল management অ্যাকাউন্টে দেখা যায়।

> **PRO TIP:** অ্যাকাউন্ট তৈরির দিনেই একটি চেকলিস্ট চালান: রুট MFA ✓, রুট অ্যাক্সেস কী নেই ✓, CloudTrail চালু ✓, বিলিং অ্যালার্ট চালু ✓, Identity Center সেটআপ ✓। এই পাঁচটি কাজ এক ঘণ্টার — কিন্তু না করলে পরে পস্তাতে হয়।

২.১৪ অধ্যায়ের সারাংশ

AWS অ্যাকাউন্ট হলো নিরাপত্তা ও বিলিংয়ের সীমানা। রুট ব্যবহারকারী সীমাহীন ক্ষমতার অধিকারী — তাকে MFA দিয়ে সুরক্ষিত করুন, অ্যাক্সেস কী দেবেন না, দৈনন্দিন কাজে ব্যবহার করবেন না। মানুষের জন্য IAM Identity Center, মেশিনের জন্য IAM রোল। মাল্টি-অ্যাকাউন্ট কৌশল — dev/staging/production আলাদা অ্যাকাউন্টে — ভুলের ক্ষতি সীমাবদ্ধ

রাখে। Organizations দিয়ে অ্যাকাউন্ট গুছিয়ে, OU ও SCP দিয়ে সীমা বেঁধে, প্রয়োজনে Control Tower দিয়ে প্রস্তুত কাঠামো নিন।

২.১৫ জ্ঞান যাচাই

১. রুট ব্যবহারকারীর ক্ষমতা IAM দিয়ে সীমিত করা যায় না কেন? ২. রুট সুরক্ষার পাঁচটি নিয়ম লিখুন। ৩. IAM Identity Center কেন IAM ব্যবহারকারীর চেয়ে নিরাপদ? ৪. একটি অ্যাকাউন্টে dev ও production রাখার তিনটি ঝুঁকি কী? ৫. Organizations-এ OU-র কাজ কী? ৬. SCP কী দেয় আর কী দেয় না? ৭. SCP ও IAM নীতির মধ্যে চূড়ান্ত অনুমতি কীভাবে নির্ধারিত হয়? ৮. Control Tower কখন ব্যবহার করবেন, কখন হাতে Organizations যথেষ্ট? ৯. Log Archive অ্যাকাউন্ট কেন আলাদা রাখা হয়? ১০. একজন কর্মী চলে গেলে তার অ্যাক্সেস কীভাবে বন্ধ করবেন?

২.১৬ ব্যবহারিক অনুশীলন

অনুশীলন ১: নিজের অ্যাকাউন্টে IAM ড্যাশবোর্ডে গিয়ে রুটের MFA অবস্থা ও রুটের অ্যাক্সেস কী আছে কি না যাচাই করুন; ফলাফল লিখুন। **অনুশীলন ২:** কাগজে TechNova-র ছয়-অ্যাকাউন্ট কাঠামো আঁকুন এবং প্রতিটি অ্যাকাউন্টে কে ঢুকতে পারবে তা লিখুন। **অনুশীলন ৩:** উপরের Region-সীমাবদ্ধ SCP-টি পড়ে ব্যাখ্যা করুন — কোন কাজ বন্ধ হবে, কোনটি হবে না; একটি গ্লোবাল সার্ভিসের উদাহরণ দিয়ে বলুন কেন SCP পরীক্ষা করে প্রয়োগ করতে হয়। **অনুশীলন ৪:** আপনার দলের জন্য একটি অ্যাকাউন্ট-নামকরণ নিয়ম (naming convention) লিখুন — যেমন `technova-dev`, `technova-prod` — এবং প্রতিটি অ্যাকাউন্টের মালিক (owner) কে হবেন তা নির্ধারণ করুন।

অধ্যায় ৩ • PART I — AWS Foundations

Chapter 3 — Identity and Access Management (IAM)

৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) IAM ব্যবহারকারী (user), গোষ্ঠী (group) ও ভূমিকা (role)-র মধ্যে পার্থক্য করতে এবং কখন কোনটি ব্যবহার করবেন তা বলতে; (২) পরিচয়-ভিত্তিক (identity-based) ও সম্পদ-ভিত্তিক (resource-based) নীতির পার্থক্য ব্যাখ্যা করতে; (৩) নীতি মূল্যায়নের যুক্তি (policy evaluation logic) — explicit deny, allow, implicit deny — ধাপে ধাপে প্রয়োগ করতে; (৪) সর্বনিম্ন অনুমতি (least privilege) নীতি অনুযায়ী নীতি লিখতে; (৫) ট্রাস্ট নীতি (trust policy) ও STS অস্থায়ী ক্রেডেনশিয়াল ব্যাখ্যা করতে; (৬) MFA, permission boundary ও ক্রস-অ্যাকাউন্ট অ্যাক্সেস বাস্তবায়ন করতে; এবং (৭) `AccessDenied` সমস্যার ধাপে ধাপে সমস্যা সমাধান করতে।

৩.২ পূর্বশর্ত জ্ঞান

অধ্যায় ২-এর অ্যাকাউন্ট, রুট ও Identity Center-এর ধারণা জানা থাকতে হবে। JSON-এর মৌলিক গঠন (অবজেক্ট, অ্যারে, কী-ভ্যালু) জানা ভালো — IAM নীতি JSON-এ লেখা হয়। AWS CLI-র মৌলিক ব্যবহার (অধ্যায় ৪-এর সঙ্গে পাশাপাশি পড়তে পারেন) থাকলে উদাহরণগুলো চালিয়ে দেখতে পারবেন।

৩.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

IAM হলো AWS নিরাপত্তার হৃদপিণ্ড। AWS-এ প্রায় প্রতিটি কাজ — EC2 চালু করা, S3-তে ফাইল রাখা, RDS-এ সংযোগ করা — একটি IAM সিদ্ধান্তের মধ্য দিয়ে যায়: "এই পরিচয় (identity) কি এই কাজটি এই সম্পদে করতে পারে?" IAM ভুল বুঝলে দুটি বিপদ: হয় আপনি প্রয়োজনের চেয়ে বেশি অনুমতি দিয়ে দেবেন (একটি ফাঁস হওয়া কী দিয়ে আক্রমণকারী সবকিছু মুছে ফেলবে), নয়তো প্রয়োজনের চেয়ে কম দিয়ে কাজ আটকে দেবেন (ডেভেলপাররা কাজ করতে পারবে না)। সবচেয়ে সাধারণ বাস্তব ভুল হলো `AdministratorAccess` নীতি সবাইকে দিয়ে দেওয়া — এটি দ্রুত, কিন্তু একটি ফাঁস হওয়া ক্রেডেনশিয়ালেই পুরো অ্যাকাউন্ট বিপদে পড়ে। এই অধ্যায়ে আপনি শিখবেন কীভাবে সঠিক মাপের অনুমতি দিতে হয় এবং কেন একটি অনুরোধ প্রত্যাখ্যাত হলো তা বের করতে হয়।

৩.৪ IAM-এর মূল উপাদান: User, Group, Role

IAM User: একজন মানুষ বা অ্যাপ্লিকেশনের জন্য দীর্ঘমেয়াদি পরিচয় — পাসওয়ার্ড (কনসোল লগইনের জন্য) বা অ্যাক্সেস কী (CLI/API-র জন্য) থাকে। ব্যবহারকারী তৈরি হয় একটি অ্যাকাউন্টের ভেতরে।

IAM Group: ব্যবহারকারীদের গোষ্ঠী — নীতি গোষ্ঠীতে লাগালে গোষ্ঠীর সব সদস্য সেই অনুমতি পায়। ব্যক্তিকে নয়, গোষ্ঠীকে অনুমতি দিন — এতে কর্মী যোগদান/প্রস্থান সহজ হয়।

IAM Role: অস্থায়ী অনুমতির ধারক — কোনো ব্যবহারকারীর সঙ্গে স্থায়ীভাবে বাঁধা নয়। EC2 ইনস্ট্যান্স, Lambda ফাংশন, বা অন্য অ্যাকাউন্টের ব্যবহারকারী একটি রোল "ধারণ" (assume) করে অস্থায়ী ক্রেডেনশিয়াল পায়। রোলের সঙ্গে একটি ট্রাস্ট নীতি থাকে — কে এই রোল ধারণ করতে পারবে।

নিয়ম: মানুষের জন্য Identity Center (অধ্যায় ২) বা IAM User, দলগত অনুমতির জন্য Group, আর মেশিন/সেবা ও অস্থায়ী অ্যাক্সেসের জন্য Role। EC2-কে S3 পড়ার অনুমতি দিতে EC2-তে অ্যাক্সেস কী রাখবেন না — EC2-তে একটি IAM Role লাগান; ইনস্ট্যান্সটি স্বয়ংক্রিয়ভাবে অস্থায়ী ক্রেডেনশিয়াল পাবে। অ্যাক্সেস কী কোডে বা সার্ভারে রাখা সবচেয়ে সাধারণ ও বিপজ্জনক ভুলগুলোর একটি।

৩.৪.১ কনসোলে ব্যবহারকারী ও গোষ্ঠী তৈরি

কনসোলে: IAM → Users → Create user → নাম দিন (যেমন `nahid-dev`) → "Provide user access to the AWS Management Console" বেছে পাসওয়ার্ড দিন → পরের ধাপে গোষ্ঠীতে যোগ করুন। গোষ্ঠী তৈরি: IAM → User groups → Create group → নাম (যেমন `Developers`) → নীতি লাগান। CLI-তে:

```
aws iam create-user --user-name nahid-dev
aws iam create-group --group-name Developers
aws iam add-user-to-group \
  --user-name nahid-dev --group-name Developers
aws iam list-users --query "Users[].UserName"
```

প্রতিটি কমান্ডের অর্থ: প্রথমটি ব্যবহারকারী তৈরি করে, দ্বিতীয়টি গোষ্ঠী, তৃতীয়টি ব্যবহারকারীকে গোষ্ঠীতে যোগ করে, চতুর্থটি তালিকা দেখায়। ব্যবহারকারী তৈরি করলেই সে কিছু করতে পারে না — অনুমতি আসে নীতি থেকে।

৩.৫ নীতি (Policy): অনুমতির ভাষা

IAM নীতি হলো JSON ডকুমেন্ট, যা বলে কে (বা কী) কোন কাজ (Action) কোন সম্পদে (Resource) কোন শর্তে (Condition) করতে পারে। একটি নীতির মূল ক্ষেত্রগুলো:

ক্ষেত্র	অর্থ	উদাহরণ
`Version`	নীতির ভাষার সংস্করণ	`2012-10-17` (সবসময় এটিই ব্যবহার করুন)
`Statement`	নিয়মের তালিকা	এক বা একাধিক Statement
`Effect`	`Allow` বা `Deny`	`Allow`
`Action`	কোন API কাজ	`s3:GetObject`, `ec2:StartInstances`
`Resource`	কোন সম্পদ (ARN দিয়ে)	`arn:aws:s3:::technova-app/*`
`Condition`	অতিরিক্ত শর্ত	শুধু নির্দিষ্ট IP থেকে; শুধু MFA থাকলে
`Principal`	কার জন্য (শুধু resource-based ও trust নীতিতে)	`arn:aws:iam::111122223333:role/AppRole`

ARN (Amazon Resource Name) হলো প্রতিটি AWS সম্পদের অনন্য ঠিকানা — যেমন `arn:aws:s3:::technova-app` একটি বাকেট, `arn:aws:iam::111122223333:user/nahid-dev` একজন ব্যবহারকারী। নীতিতে সম্পদ চেনাতে ARN ব্যবহার করুন; `*` মানে "সবকিছু" — শুধু তখনই ব্যবহার করুন যখন সত্যিই সবকিছু বোঝাতে চান।

৩.৫.১ পরিচয়-ভিত্তিক বনাম সম্পদ-ভিত্তিক নীতি

পরিচয়-ভিত্তিক নীতি (identity-based policy) ব্যবহারকারী, গোষ্ঠী বা রোলের সঙ্গে লাগানো হয় — "এই পরিচয় কী করতে পারে"। **সম্পদ-ভিত্তিক নীতি (resource-based policy)** সম্পদের সঙ্গে লাগানো হয় — "এই সম্পদে কে কী করতে পারে" — যেমন S3 বাকেট নীতি, SQS কিউ নীতি। S3 একটি বিশেষ ক্ষেত্র: একটি বাকেটে দুই ধরনের নীতিই কাজ করতে

পারে। ক্রস-অ্যাকাউন্ট অ্যাক্সেসে প্রায়ই দুটিই লাগে: অন্য অ্যাকাউন্টের পরিচয়ের নীতিতে অনুমতি থাকতে হবে, আবার আপনার বাকেটের নীতিতেও সেই পরিচয়কে অনুমতি দিতে হবে। একটি মাত্র দিক দিলে কাজ হবে না — এটি নতুনদের সবচেয়ে সাধারণ বিভ্রান্তির জায়গা।

৩.৬ নীতি মূল্যায়নের যুক্তি (Policy Evaluation Logic)

যখন কেউ কোনো কাজের অনুরোধ করে, AWS এই ধাপগুলো অনুসরণ করে। এই ফ্লোচার্টটি মুখস্থ করুন — প্রতিটি `AccessDenied` সমস্যার উত্তর এখানেই আছে।

নীতি মূল্যায়ন ফ্লোচার্ট (সরলীকৃত)

```
অনুরোধ আসে (কে, কী কাজ, কোন সম্পদ)
                  |
কোনো নীতিতে Explicit Deny আছে?
            /       \
          হ্যাঁ       না
           |          |
      DENY (চূড়ান্ত)   কোনো নীতিতে Allow আছে?
                       /      \
                     হ্যাঁ      না
                      |         |
                SCP / boundary    Implicit Deny
                অনুমতি দেয়?     (চূড়ান্ত DENY)
                      /    \
                    হ্যাঁ    না
                     |       |
                  ALLOW     Implicit Deny
                            (চূড়ান্ত DENY)

নিয়ম: Explicit Deny সবকিছুর ওপরে।
Allow না থাকলে স্বয়ংক্রিয়ভাবে Deny (implicit deny)।
SCP, permission boundary ও session নীতি — সবাই
Allow-এর ওপর অতিরিক্ত ফিল্টার হিসেবে কাজ করে।
```

তিনটি সিদ্ধান্ত মনে রাখুন: (১) **Explicit Deny** — কোনো নীতিতে স্পষ্টভাবে `Deny` থাকলে, যত `Allow`-ই থাকুক, ফলাফল Deny। (২) **Allow** — কোনো নীতিতে Allow থাকলে এবং কোনো Deny বা সীমা না থাকলে অনুমতি মেলে। (৩) **Implicit Deny** — কোনো Allow না থাকলে স্বয়ংক্রিয়ভাবে Deny; নতুন ব্যবহারকারীর কোনো অনুমতিই থাকে না — এটাই ডিফল্ট। SCP (অধ্যায় ২), permission boundary ও session policy — এরা Allow-কে আরও ছোট করে, কখনো বড় করে না। অর্থাৎ চূড়ান্ত অনুমতি হলো সব স্তরের Allow-এর ছেদ (intersection)।

> **WARNING:** `Action: *` ও `Resource: *` একসঙ্গে দিয়ে `Allow` করা মানে অ্যাকাউন্টের সবকিছু করার অনুমতি দেওয়া — `AdministratorAccess` নীতির সমতুল্য। পরীক্ষার জন্য সাময়িকভাবে ঠিক আছে, কিন্তু প্রোডাকশনে বা স্থায়ীভাবে কখনো নয়।

৩.৭ পূর্ণ কাজের উদাহরণ: TechNova ডেভেলপার নীতি (লাইন-বাই-লাইন)

নিচের নীতিটি TechNova-র `Developers` গোষ্ঠীর জন্য — ডেভেলপাররা `technova-app` বাকেটের ফাইল পড়তে ও লিখতে পারে, EC2 ইনস্ট্যান্স দেখতে পারে কিন্তু বন্ধ করতে পারে না, এবং শুধু MFA দিয়ে লগইন করলে তবেই এসব কাজ করতে পারে।

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadWriteAppBucket",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::technova-app/*"
    },
    {
      "Sid": "ListBucketsAndDescribeEC2",
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "ec2:Describe*"],
      "Resource": "*"
    },
    {
      "Sid": "DenyWithoutMFA",
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "BoolIfExists": {
          "aws:MultiFactorAuthPresent": "false"
        }
      }
    }
  ]
}
```

লাইন-বাই-লাইন ব্যাখ্যা: `Version: 2012-10-17` — নীতির ভাষার সংস্করণ, সবসময় এটিই লিখুন। প্রথম Statement (`Sid: ReadWriteAppBucket`) — `Allow`, কাজ `s3:GetObject` (ফাইল পড়া) ও `s3:PutObject` (ফাইল লেখা), সম্পদ শুধু `technova-app` বাকেটের ভেতরের অবজেক্ট (`/*` মানে বাকেটের ভেতরের সবকিছু, বাকেট নিজে নয়)। লক্ষ করুন `s3:DeleteObject` নেই — ডেভেলপাররা ফাইল মুছতে পারবে না; এটি ইচ্ছাকৃত, least privilege। দ্বিতীয় Statement — `s3:ListBucket` (বাকেটের তালিকা দেখা) ও `ec2:Describe*` (EC2-র সব দেখার কাজ) অনুমোদন, `Resource: *` কারণ List ও Describe কাজে নির্দিষ্ট সম্পদ ARN সমর্থিত নয় — AWS ডকুমেন্টেশনে প্রতিটি Action-এর জন্য Resource সমর্থন যাচাই করুন। তৃতীয় Statement — `DenyWithoutMFA`: `Effect: Deny`, সব কাজ, সব সম্পদ, শর্ত `aws:MultiFactorAuthPresent` false হলে — অর্থাৎ MFA ছাড়া লগইন করলে এই নীতির সব Allow অকার্যকর হয়ে যাবে, কারণ Explicit Deny সবকিছুর ওপরে। `BoolIfExists` ব্যবহার করা হয়েছে যাতে MFA তথ্য না থাকলেও (যেমন কিছু API কল) শর্তটি সঠিকভাবে মূল্যায়িত হয়। এই নীতিটি IAM Policy Simulator-এ পরীক্ষা করুন: নীতি

লাগানোর আগে সিমুলেটরে `nahid-dev` হিসেবে `s3:GetObject` চালিয়ে দেখুন Allow আসে কি না, আর MFA ছাড়া Deny আসে কি না।

৩.৮ সর্বনিম্ন অনুমতি (Least Privilege) বাস্তবে

Least privilege মানে: কাজটি করতে যতটুকু অনুমতি দরকার, ঠিক ততটুকুই দিন — তার বেশি নয়, তার কমও নয়। বাস্তবে এটি একটি চলমান প্রক্রিয়া: শুরুতে কিছুটা ব্যাপক অনুমতি দিয়ে, CloudTrail ও IAM Access Analyzer-এর **Access Advisor** দেখে কোন অনুমতি আসলে ব্যবহৃত হয়নি তা বের করে, সেগুলো কেটে ছোট করুন। AWS-এর **Access Analyzer** নীতি যাচাই করে বলে দেয় কোন নীতিতে অতিরিক্ত অনুমতি আছে। আরেকটি কৌশল: নতুন নীতি প্রথমে পরীক্ষামূলক গোষ্ঠীতে লাগান, এক সপ্তাহ পর্যবেক্ষণ করুন, তারপর সবার জন্য চালু করুন।

৩.৯ ট্রাস্ট নীতি, STS ও অস্থায়ী ক্রেডেনশিয়াল

রোলের দুটি নীতি থাকে: **ট্রাস্ট নীতি (trust policy)** — কে এই রোল ধারণ করতে পারবে; এবং **অনুমতি নীতি (permission policy)** — রোল ধারণ করলে কী করা যাবে। EC2-র জন্য ট্রাস্ট নীতিতে `Principal: ec2.amazonaws.com` থাকে; ক্রস-অ্যাকাউন্টের ক্ষেত্রে অন্য অ্যাকাউন্টের ব্যবহারকারীর ARN থাকে। **STS** (Security Token Service) হলো অস্থায়ী ক্রেডেনশিয়াল দেওয়ার সেবা — `AssumeRole` কল করলে ১ ঘণ্টা (সর্বোচ্চ ১২ ঘণ্টা) মেয়াদের অ্যাক্সেস কী, সিক্রেট কী ও সেশন টোকেন পাওয়া যায়। অস্থায়ী ক্রেডেনশিয়াল ফাঁস হলেও ক্ষতি সীমিত, কারণ মেয়াদ শেষ হলে তা অকার্যকর হয়ে যায়। CLI-তে:

```
aws sts assume-role \
  --role-arn arn:aws:iam::111122223333:role/ReadOnlyCrossAccount \
  --role-session-name nahid-audit
aws sts get-caller-identity
```

প্রথম কমান্ডটি অন্য অ্যাকাউন্টের রোল ধারণ করে অস্থায়ী ক্রেডেনশিয়াল দেয়; দ্বিতীয়টি দেখায় আপনি বর্তমানে কোন পরিচয়ে কাজ করছেন — সমস্যা সমাধানের সময় এই কমান্ডটি প্রথমে চালান।

৩.৯.১ Permission Boundary: অনুমতির সর্বোচ্চ সীমা

Permission boundary হলো একটি পরিচয়ের ওপর বসানো সর্বোচ্চ অনুমতির সীমা — এটি নিজে অনুমতি দেয় না, শুধু বলে "এর বেশি কখনো নয়"। যেমন একজন টিম লিডকে ব্যবহারকারী তৈরি করার অনুমতি দিলেন, কিন্তু boundary দিয়ে বেঁধে দিলেন যে সে শুধু `Developers` গোষ্ঠীর নীতি লাগাতে পারবে, `AdministratorAccess` নয়। এটি SCP-র মতো, কিন্তু একটি অ্যাকাউন্টের ভেতরে, পরিচয়-স্তরে কাজ করে। বড় দলে প্রতিনিধিত্ব (delegation) নিরাপদ করতে boundary অপরিহার্য।

৩.৯.২ ক্রস-অ্যাকাউন্ট অ্যাক্সেস

TechNova-র dev অ্যাকাউন্টের একজন ডেভেলপারকে prod অ্যাকাউন্টের একটি S3 বাকেট পড়তে দিতে হলে দুটি কাজ লাগে: (১) prod অ্যাকাউন্টে একটি রোল, যার ট্রাস্ট নীতিতে dev অ্যাকাউন্টের ব্যবহারকারীকে অনুমতি দেওয়া; (২) সেই রোলের অনুমতি নীতিতে বাকেট পড়ার Allow; এবং প্রায়ই (৩) বাকেটের resource-based নীতিতেও সেই রোলকে Allow। শুধু একদিক করলে `AccessDenied` আসবে — এবং সেটাই সঠিক আচরণ। ক্রস-অ্যাকাউন্ট অ্যাক্সেসের সময় সর্বদা বাহ্যিক ID (external ID) ব্যবহার করুন, যাতে অন্য কেউ আপনার রোল ধারণ করতে না পারে (confused deputy সমস্যা)।

৩.১০ MFA, পাসওয়ার্ড নীতি ও অ্যাক্সেস কী পরিচালনা

MFA চালু করা শুধু রুটের জন্য নয় — প্রতিটি IAM ব্যবহারকারীর জন্য MFA বাধ্যতামূলক করুন (উপরের `DenyWithoutMFA` নীতিটি দিয়ে)। পাসওয়ার্ড নীতি: ন্যূনতম দৈর্ঘ্য ১৪ অক্ষর, জটিলতা চালু, পুরনো পাসওয়ার্ড পুনরায় ব্যবহার নিষিদ্ধ, এবং নিয়মিত পরিবর্তনের চেয়ে শক্তিশালী অনন্য পাসওয়ার্ড বেশি গুরুত্বপূর্ণ। অ্যাক্সেস কী: ৯০ দিনের বেশি পুরনো কী ঘুরিয়ে (rotate) নতুন করুন; অব্যবহৃত কী নিষ্ক্রিয় (deactivate) করুন; প্রতিটি ব্যবহারকারীর সর্বোচ্চ দুটি কী থাকতে পারে। IAM Credential Report ডাউনলোড করে দেখুন কোন কী কবে ব্যবহৃত হয়েছে — ৯০ দিন ধরে অব্যবহৃত কী মুছে ফেলুন।

```
# ক্রেডেনশিয়াল রিপোর্ট তৈরি ও দেখা
aws iam generate-credential-report
aws iam get-credential-report --query Content --output text | base64 -d | head -20
# একটি নির্দিষ্ট ব্যবহারকারীর অ্যাক্সেস কী তালিকা
aws iam list-access-keys --user-name nahid-dev
```

৩.১১ প্রোডাকশন দৃশ্যকল্প: TechNova-র IAM নকশা

TechNova-র গোষ্ঠী: `Developers` (dev অ্যাকাউন্টে EC2/RDS/S3 সীমিত অনুমতি, MFA বাধ্যতামূলক), `ReadOnly` (সব অ্যাকাউন্টে শুধু দেখার অনুমতি), `OpsOnCall` (production-এ সীমিত পরিবর্তনের অনুমতি, permission boundary সহ)। মানুষ Identity Center দিয়ে ঢোকে; অ্যাপ্লিকেশন EC2/Lambda রোল ব্যবহার করে; CI/CD (GitHub Actions) OIDC দিয়ে অস্থায়ী ক্রেডেনশিয়াল পায় (অধ্যায় ৪)। কোনো দীর্ঘমেয়াদি অ্যাক্সেস কী কোডে, Git-এ বা সার্ভারে রাখা হয় না। প্রতিটি গোষ্ঠীর নীতি Policy Simulator-এ পরীক্ষা করা হয় এবং Access Analyzer দিয়ে নিয়মিত পর্যালোচনা করা হয়।

৩.১২ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: IAM নিজে বিনামূল্যে — এর জন্য আলাদা বিল হয় না, কিন্তু ভুল IAM নীতির খরচ হতে পারে আকাশছোঁয়া (আক্রমণকারী দামি সম্পদ চালালে)। MFA, least privilege, নিয়মিত Access Advisor পর্যালোচনা — এগুলো বিনামূল্যের নিয়ন্ত্রণ, কিন্তু সবচেয়ে কার্যকর। খরচ: IAM-এর কোনো সরাসরি খরচ নেই; তবে Identity Center-এর কিছু উন্নত বৈশিষ্ট্য ও Access Analyzer-এর কিছু ব্যবহারে খরচ হতে পারে — AWS Pricing Calculator-এ যাচাই করুন।

৩.১৩ সাধারণ ভুল

(১) সবাইকে `AdministratorAccess` দেওয়া। (২) নীতিতে `Resource: *` অপ্রয়োজনে ব্যবহার করা। (৩) `Allow` থাকলেই কাজ হবে ভাবা — SCP/boundary-র কথা ভুলে যাওয়া। (৪) resource-based নীতি ছাড়া ক্রস-অ্যাকাউন্ট S3 অ্যাক্সেস চেষ্টা করা। (৫) অ্যাক্সেস কী Git-এ কমিট করা। (৬) ব্যবহারকারীকে সরাসরি নীতি লাগানো, গোষ্ঠীকে নয় — এতে পরে পরিচালনা কঠিন হয়। (৭) `Deny`-র প্রাধান্য না বোঝা — একটি Deny সব Allow-কে হারিয়ে দেয়।

৩.১৪ সমস্যা সমাধান (Troubleshooting)

সমস্যা: `AccessDenied` আসছে, কিন্তু নীতিতে Allow আছে। **ধাপ:** (১) `aws sts get-caller-identity` দিয়ে সঠিক পরিচয়ে আছেন কি না দেখুন। (২) Policy Simulator-এ একই অনুরোধ পরীক্ষা করুন। (৩) CloudTrail-এ ইভেন্টটি দেখুন — কোন নীতি Deny করেছে, তাতে `errorCode` ও `userIdentity` থাকে। (৪) SCP, permission boundary ও session policy যাচাই করুন। (৫) resource-based নীতি (S3 বাকেট নীতি) যাচাই করুন। (৬) শর্ত (Condition) — যেমন MFA বা IP শর্ত — মিলছে কি না দেখুন।

সমস্যা: `AssumeRole` ব্যর্থ। **সমাধান:** ট্রাস্ট নীতিতে আপনার ARN আছে কি না, বাহ্যিক ID ঠিক আছে কি না, এবং রোলের নাম/অ্যাকাউন্ট ID ঠিক আছে কি না যাচাই করুন। **সমস্যা:** নতুন নীতি লাগানোর পরও কাজ হচ্ছে না। **সমাধান:** IAM পরিবর্তন ছড়াতে (propagate) কিছু সময় লাগতে পারে (eventual consistency) — কয়েক মিনিট অপেক্ষা করুন, তারপর আবার চেষ্টা করুন।

৩.১৫ অধ্যায়ের সারাংশ

IAM-এ User হলো দীর্ঘমেয়াদি পরিচয়, Group হলো পরিচালনার একক, Role হলো অস্থায়ী অনুমতির ধারক। নীতি JSON-এ লেখা হয় — Effect, Action, Resource, Condition দিয়ে। মূল্যায়নের নিয়ম: Explicit Deny সবকিছুর ওপরে, Allow না থাকলে Implicit Deny, আর SCP/boundary হলো অতিরিক্ত ফিল্টার। Least privilege, MFA, STS অস্থায়ী ক্রেডেনশিয়াল ও permission boundary — এই চারটি অভ্যাসই IAM নিরাপত্তার মূল। ক্রস-অ্যাকাউন্ট অ্যাক্সেসে পরিচয়-ভিত্তিক ও সম্পদ-ভিত্তিক — দুই দিকেই অনুমতি লাগে।

৩.১৬ জ্ঞান যাচাই

১. User, Group ও Role-র পার্থক্য ও ব্যবহার লিখুন। ২. পরিচয়-ভিত্তিক ও সম্পদ-ভিত্তিক নীতির পার্থক্য কী? ৩. Explicit Deny, Allow ও Implicit Deny-র সম্পর্ক ফ্লোচার্টসহ ব্যাখ্যা করুন। ৪. ARN কী এবং নীতিতে কেন দরকার? ৫. Least privilege বাস্তবে কীভাবে প্রয়োগ করবেন? ৬. ট্রাস্ট নীতি ও অনুমতি নীতির পার্থক্য কী? ৭. STS অস্থায়ী ক্রেডেনশিয়াল কেন দীর্ঘমেয়াদি অ্যাক্সেস কী-র চেয়ে নিরাপদ? ৮. Permission boundary কী এবং কখন ব্যবহার করবেন? ৯. ক্রস-অ্যাকাউন্ট S3 অ্যাক্সেসে দুই দিকে অনুমতি কেন লাগে? ১০. `AccessDenied` পেলে প্রথম পাঁচটি যাচাইয়ের ধাপ কী?

৩.১৭ ব্যবহারিক অনুশীলন

অনুশীলন ১: উপরের TechNova ডেভেলপার নীতিটি নিজের অ্যাকাউন্টে (dev পরিবেশে) তৈরি করুন, একটি পরীক্ষামূলক গোষ্ঠীতে লাগান, এবং Policy Simulator-এ MFA ছাড়া ও MFA সহ `s3:GetObject` পরীক্ষা করুন। **অনুশীলন ২:** একটি EC2-র জন্য IAM Role তৈরি করুন যার ট্রাস্ট নীতিতে `ec2.amazonaws.com` আছে এবং অনুমতি নীতিতে শুধু একটি নির্দিষ্ট S3 বাকেট পড়ার অনুমতি আছে; `aws sts get-caller-identity` দিয়ে যাচাই করুন। **অনুশীলন ৩:** ইচ্ছাকৃতভাবে একটি ভুল নীতি তৈরি করুন (যেমন ভুল ARN) এবং `AccessDenied` পাওয়ার পর CloudTrail-এ ইভেন্টটি খুঁজে বের করুন — কোন ক্ষেত্রটি ভুল ছিল তা লিখুন। **অনুশীলন ৪:** আপনার দলের জন্য তিনটি গোষ্ঠী (Developers, ReadOnly, OpsOnCall) ও তাদের নীতির রূপরেখা এক পৃষ্ঠায় লিখুন।

অধ্যায় ৪ • PART I — AWS Foundations

Chapter 4 — AWS CLI, SDKs and Credential Engineering

৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Ubuntu 24.04-এ AWS CLI v2 ইনস্টল, আপডেট ও যাচাই করতে; (২) `aws configure`, named profile ও `config` / `credentials` ফাইলের কাজ ব্যাখ্যা করতে; (৩) CloudShell ব্যবহার করতে এবং এর সীমা বলতে; (৪) SDK-র ধারণা ও Node.js 20-এ AWS SDK for JavaScript v3-এর মৌলিক ব্যবহার দেখাতে; (৫) পরিবেশ চলক (environment variable), credential provider chain ও অস্থায়ী ক্রেডেনশিয়ালের ক্রম ব্যাখ্যা করতে; (৬) নিরাপদ অটোমেশন — OIDC ও GitHub Actions দিয়ে কী-বিহীন (keyless) প্রমাণীকরণ — বাস্তবায়নের রূপরেখা দিতে; এবং (৭) ক্রেডেনশিয়াল ফাঁসের বাস্তব দৃশ্যকল্প চিনতে ও ফাঁস হলে ধাপে ধাপে কী করতে হবে তা বলতে।

৪.২ পূর্বশর্ত জ্ঞান

লিনাক্স টার্মিনালের মৌলিক ব্যবহার (ফাইল দেখা, পরিবেশ চলক, `PATH`), JSON-এর ধারণা, এবং অধ্যায় ৩-এর IAM User/Role ও STS-র ধারণা জানা থাকতে হবে। Node.js ও npm-এর মৌলিক অভিজ্ঞতা থাকলে SDK অংশটি সহজ হবে। Git-এর মৌলিক ব্যবহার জানা থাকলে ক্রেডেনশিয়াল ফাঁসের অংশটি আরও স্পষ্ট হবে।

৪.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

কনসোলে ক্লিক করে আপনি একটি সার্ভার চালাতে পারেন, কিন্তু একশোটি সার্ভার, দশটি পরিবেশ, বা প্রতিদিনের ডেপ্লয় — এগুলো কনসোলে করা যায় না। পেশাদার DevOps কাজের মূল হলো অটোমেশন, আর অটোমেশনের মূল হলো CLI ও SDK। কিন্তু CLI-র সঙ্গে আসে সবচেয়ে বিপজ্জনক জিনিসটি: ক্রেডেনশিয়াল। একটি অ্যাক্সেস কী ভুল জায়গায় রাখলে — GitHub-এ, Slack-এ, স্ক্রিনশটে — কয়েক মিনিটের মধ্যে আক্রমণকারীর বট তা খুঁজে পেয়ে আপনার অ্যাকাউন্টে দামি সম্পদ চালাতে পারে। এই অধ্যায়ে আপনি শিখবেন কীভাবে CLI-কে শক্তিশালী হাতিয়ার হিসেবে ব্যবহার করবেন, আবার ক্রেডেনশিয়ালকে এমনভাবে সামলাবেন যেন তা কখনো ফাঁস না হয়।

৪.৪ AWS CLI v2 ইনস্টলেশন (Ubuntu 24.04 LTS)

AWS CLI v2 হলো AWS সেবাগুলোকে টার্মিনাল থেকে চালানোর অফিসিয়াল কমান্ড-লাইন টুল। Ubuntu 24.04-এ প্রস্তাবিত পদ্ধতি হলো অফিসিয়াল ইনস্টলার ব্যবহার করা — apt-এর পুরনো সংস্করণ নয়।

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" \
  -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version
# প্রত্যাশিত আউটপুট: aws-cli/2.x.x Python/3.x.x ...
```

প্রতিটি ধাপের অর্থ: প্রথমে অফিসিয়াল জিপ ডাউনলোড, তারপর আনজিপ, তারপর ইনস্টলার চালানো (এটি `/usr/local/bin/aws`-এ বসে), শেষে সংস্করণ যাচাই। `aws --version`-এ v2 দেখা গেলে ইনস্টল সফল। আপডেট করতে একই ইনস্টলার আবার চালান — `sudo ./aws/install --update`। CLI v1 আর ব্যবহার করবেন না — v2-তে SSO লগইন, উন্নত আউটপুট ফরম্যাট ও নিরাপত্তা আছে।

> **PRO TIP:** CLI ইনস্টলের পর `aws configure list` চালিয়ে দেখুন কী কনফিগার করা আছে। কিছুই না থাকলে সেটাই স্বাভাবিক — পরের অংশে আপনি নিরাপদে কনফিগার করবেন।

৪.৫ কনফিগারেশন: Named Profile, config ও credentials ফাইল

AWS CLI দুটি ফাইলে কনফিগারেশন রাখে: `~/.aws/credentials` (গোপন তথ্য — অ্যাক্সেস কী) এবং `~/.aws/config` (অ-গোপন তথ্য — Region, আউটপুট ফরম্যাট)। **Named profile** হলো একাধিক অ্যাকাউন্ট/পরিবেশের জন্য আলাদা নামে কনফিগারেশন রাখার উপায় — যেমন `technova-dev` ও `technova-prod`। প্রতিটি কমান্ডে `--profile` দিয়ে বলুন কোন প্রোফাইল ব্যবহার করবেন; না বললে `default` প্রোফাইল ব্যবহৃত হয়।

```
aws configure --profile technova-dev
# AWS Access Key ID [None]: CHANGE-ME-ACCESS-KEY
# AWS Secret Access Key [None]: CHANGE-ME-SECRET
# Default region name [None]: ap-south-1
# Default output format [None]: json

aws configure list-profiles
aws sts get-caller-identity --profile technova-dev
aws s3 ls --profile technova-dev
```

সতর্কতা: `aws configure` ইন্টার‍্যাক্টিভভাবে কী চাইবে — টার্মিনালে টাইপ করা সিক্রেট কী স্ক্রিনে দেখা যেতে পারে, তাই নিরাপদ জায়গায় বসে করুন। আরও নিরাপদ পদ্ধতি হলো `aws configure sso` (Identity Center দিয়ে লগইন) — এতে দীর্ঘমেয়াদি কী সংরক্ষণ করতে হয় না। প্রোফাইল ফাইলের উদাহরণ (`~/.aws/config`):

```
[profile technova-dev]
region = ap-south-1
output = json
[profile technova-prod]
region = ap-south-1
output = json
```

মনে রাখবেন: `~/.aws/credentials` ফাইলের অনুমতি (permission) শুধু আপনার ব্যবহারকারীর জন্য থাকা উচিত (`chmod 600 ~/.aws/credentials`)। এই ফাইল কখনো Git-এ কমিট করবেন না, ব্যাকআপে তুলবেন না, বা অন্য কারও সঙ্গে ভাগ করবেন না।

৪.৫.১ পরিবেশ চলক (Environment Variables)

CLI পরিবেশ চলক থেকেও ক্রেডেনশিয়াল নিতে পারে — CI/CD-তে এটিই সাধারণ পদ্ধতি:

```
export AWS_ACCESS_KEY_ID=CHANGE-ME-ACCESS-KEY
export AWS_SECRET_ACCESS_KEY=CHANGE-ME-SECRET
export AWS_SESSION_TOKEN=CHANGE-ME-SESSION-TOKEN
export AWS_DEFAULT_REGION=ap-south-1
aws sts get-caller-identity
```

তবে টার্মিনালে `export` করে কী রাখলে তা শেলের ইতিহাসে (history) থেকে যেতে পারে — তাই CI/CD-তে সিক্রেট ম্যানেজমেন্ট সেবা (যেমন GitHub Secrets) ব্যবহার করুন, আর স্থানীয়ভাবে প্রোফাইল বা SSO ব্যবহার করুন।

৪.৬ Credential Provider Chain: CLI কীভাবে ক্রেডেনশিয়াল খোঁজে

AWS SDK ও CLI ক্রেডেনশিয়াল খোঁজে একটি নির্দিষ্ট ক্রমে — একে বলে **credential provider chain**। ক্রমটি মনে রাখুন, কারণ "কোন ক্রেডেনশিয়াল ব্যবহার হচ্ছে" বুঝতে না পারলে সমস্যা সমাধান অসম্ভব:

১. কমান্ড-লাইনে দেওয়া অপশন (যেমন `--profile`)। ২. পরিবেশ চলক (`AWS_ACCESS_KEY_ID` ইত্যাদি)। ৩. `~/.aws/credentials` ফাইলের প্রোফাইল। ৪. `~/.aws/config`-এর SSO বা `credential_process`। ৫. কন্টেইনার ক্রেডেনশিয়াল (ECS task role)। ৬. ইনস্ট্যান্স মেটাডেটা (EC2 instance role — IMDS)। প্রথম যেখানে ক্রেডেনশিয়াল পাওয়া যায়, সেটাই ব্যবহৃত হয়; পরেরগুলো দেখা হয় না। তাই পরিবেশ চলকে পুরনো কী থাকলে প্রোফাইলের নতুন কী কাজ করবে না — এই ভুলে অনেক সময় নষ্ট হয়। যাচাই করুন: `aws sts get-caller-identity` দিয়ে দেখুন কোন অ্যাকাউন্ট ও পরিচয়ে আছেন; `aws configure list` দিয়ে দেখুন কোন উৎস থেকে ক্রেডেনশিয়াল আসছে।

Credential Provider Chain (যে ক্রমে খোঁজা হয়)

1. --profile / কমান্ড-লাইন অপশন
2. পরিবেশ চলক (AWS_ACCESS_KEY_ID ...)
3. ~/.aws/credentials (প্রোফাইল)
4. ~/.aws/config (SSO / credential_process)
5. ECS task role (কন্টেইনার)
6. EC2 instance metadata — IMDS (instance role)

|

প্রথম মিলেই থেমে যায় — পরের উৎস দেখা হয় না।
যাচাই: aws sts get-caller-identity

৪.৭ CloudShell: ব্রাউজারে টার্মিনাল

CloudShell হলো কনসোলের ভেতরে তৈরি টার্মিনাল — AWS CLI v2, Python, Node.js আগে থেকে ইনস্টল করা থাকে, এবং আপনার কনসোল লগইনের ক্রেডেনশিয়াল স্বয়ংক্রিয়ভাবে ব্যবহৃত হয়। শুরু করার জন্য দারুণ: নিজের ল্যাপটপে কী বসাতে হয় না। সীমা: হোম ডিরেক্টরি প্রতি Region-এ 1 GB, ২০ মিনিট নিষ্ক্রিয় থাকলে সেশন বন্ধ হয়, এবং এটি দীর্ঘমেয়াদি স্ক্রিপ্ট চালানোর জায়গা নয়। CloudShell-এ লেখা স্ক্রিপ্ট পরে নিজের ল্যাপটপে বা CI/CD-তে নিয়ে যান।

৪.৮ SDK ধারণা ও Node.js-এ AWS SDK for JavaScript v3

SDK (Software Development Kit) হলো প্রোগ্রামিং ভাষা থেকে AWS API ডাকার লাইব্রেরি। CLI যেখানে মানুষের জন্য, SDK সেখানে অ্যাপ্লিকেশনের জন্য। Node.js 20-এ AWS SDK for JavaScript v3 ব্যবহার করা হয় — v3 মডুলার (প্রতিটি সেবার জন্য আলাদা প্যাকেজ), তাই শুধু প্রয়োজনীয় প্যাকেজ ইনস্টল করুন:

```
npm init -y
npm install @aws-sdk/client-s3 @aws-sdk/client-sts
node --version  # Node.js 20 যাচাই করুন
```

```
// list-buckets.mjs — S3 বাকেট তালিকা (SDK v3)
import { S3Client, ListBucketsCommand } from "@aws-sdk/client-s3";
const client = new S3Client({ region: "ap-south-1" });
```

```
const out = await client.send(new ListBucketsCommand({}));
console.log(out.Buckets.map(b => b.Name));
```

লক্ষ করুন: কোডে কোনো অ্যাক্সেস কী লেখা নেই — SDK স্বয়ংক্রিয়ভাবে credential provider chain থেকে ক্রেডেনশিয়াল নেয় (পরিবেশ চলক, প্রোফাইল, বা EC2/Lambda রোল)। EC2 বা Lambda-তে চলা কোডে রোল ব্যবহার করুন; স্থানীয়ভাবে প্রোফাইল বা SSO ব্যবহার করুন। কোডে কী হার্ডকোড করা সবচেয়ে বড় ভুল — সেই কোড Git-এ গেলে কী ফাঁস হয়ে যায়। SDK v2 (পুরনো `aws-sdk` প্যাকেজ) আর নতুন প্রজেক্টে ব্যবহার করবেন না — v3-ই বর্তমান মান।

৪.৮.১ CLI-র দরকারি ব্যবহার: আউটপুট, ফিল্টার, পেজিনেশন

```
aws ec2 describe-instances --profile technova-dev \
  --query "Reservations[].Instances[].InstanceId" --output table
aws s3 ls s3://technova-app --profile technova-dev --human-readable
aws logs tail /aws/lambda/technova-api --follow --profile technova-dev
```

`--query` (JMESPath) দিয়ে বড় JSON থেকে শুধু প্রয়োজনীয় ক্ষেত্র নিন; `--output table` বা `text` দিয়ে পড়ার মতো করুন; `--human-readable` দিয়ে ফাইলের আকার মানুষের পড়ার মতো করুন। বড় তালিকায় `--max-items` ও `--starting-token` দিয়ে পেজিনেশন করুন। CLI-র প্রতিটি কমান্ডের সাহায্য পেতে `aws <service> <command> help` চালান।

৪.৯ নিরাপদ অটোমেশন: OIDC ও GitHub Actions

CI/CD-তে দীর্ঘমেয়াদি অ্যাক্সেস কী রাখা বিপজ্জনক — কী ফাঁস হলে, ঘোরানো (rotate) না হলে, বা কর্মী চলে গেলে ঝুঁকি থাকে। আধুনিক পদ্ধতি হলো **OIDC** (OpenID Connect) ফেডারেশন: GitHub Actions একটি স্বল্পমেয়াদি OIDC টোকেন দেয়, AWS সেই টোকেন যাচাই করে একটি IAM Role-র অস্থায়ী ক্রেডেনশিয়াল দেয় — কোনো স্থায়ী কী সংরক্ষণ করতে হয় না। ধাপ: (১) AWS-এ একটি OIDC identity provider তৈরি করুন (`token.actions.githubusercontent.com`); (২) একটি IAM Role তৈরি করুন, যার ট্রাস্ট নীতিতে শুধু আপনার GitHub রিপোজিটরি (`repo:technova/app:ref:refs/heads/main`) অনুমোদিত; (৩) GitHub Actions ওয়ার্কফ্লোতে `aws-actions/configure-aws-credentials` অ্যাকশন দিয়ে রোল ধারণ করুন। ট্রাস্ট নীতিতে শাখা (branch) ও রিপোজিটরি সীমাবদ্ধ করুন — যেকোনো রিপোজিটরি থেকে রোল ধারণ করা গেলে তা নিরাপত্তা ফাঁক।

```
# GitHub Actions ওয়ার্কফ্লো (সরলীকৃত অংশ)
# - uses: aws-actions/configure-aws-credentials@v4
#   with:
#     role-to-assume: arn:aws:iam::111122223333:role/GitHubDeploy
#     aws-region: ap-south-1
```

একই নীতি অন্য CI সিস্টেমেও (GitLab, Jenkins OIDC) প্রযোজ্য। নিয়ম: মেশিনের জন্য রোল + OIDC, মানুষের জন্য Identity Center/SSO, দীর্ঘমেয়াদি কী শুধু তখনই যখন আর কোনো উপায় নেই — এবং তখনও নিয়মিত ঘোরান।

৪.১০ ক্রেডেনশিয়াল নিরাপত্তা ব্যর্থতা: বাস্তব দৃশ্যকল্প ও করণীয়

দৃশ্যকল্প ১ — GitHub-এ কী কমিট: একজন ডেভেলপার `.env` ফাইল ভুল করে কমিট করলেন। কয়েক মিনিটের মধ্যে স্বয়ংক্রিয় বট কীটি খুঁজে পেয়ে `us-east-1` -এ দামি GPU ইনস্ট্যান্স চালাতে শুরু করল। বিল কয়েক হাজার ডলার ছাড়াল। **দৃশ্যকল্প ২ — স্ক্রিনশটে কী:** Slack-এ পাঠানো টার্মিনাল স্ক্রিনশটে অ্যাক্সেস কী দেখা যাচ্ছিল। **দৃশ্যকল্প ৩ — পুরনো কর্মীর কী:** চলে যাওয়া কর্মীর কী নিষ্ক্রিয় করা হয়নি; কয়েক মাস পর তা ব্যবহার করে ডেটা চুরি হলো।

ফাঁস হলে এখনই যা করবেন (ধাপে ধাপে): (১) আতঙ্কিত না হয়ে প্রথমে কীটি **নিষ্ক্রিয় (deactivate)** করুন — IAM → Users → Security credentials → Make inactive; মুছে ফেলার আগে নিষ্ক্রিয় করুন, যাতে বৈধ কাজ কী ভাঙছে তা বোঝা যায়। (২) CloudTrail-এ দেখুন কীটি দিয়ে কী কী করা হয়েছে — কোন সম্পদ তৈরি হয়েছে, কোন ডেটা পড়া হয়েছে। (৩) অজানা সম্পদ মুছে ফেলুন, অজানা IAM ব্যবহারকারী/রোল মুছুন। (৪) নতুন কী তৈরি করুন (প্রয়োজনে) এবং পুরনো কী মুছে ফেলুন। (৫) AWS Support-কে জানান (প্রয়োজনে) এবং বিল পর্যবেক্ষণ করুন। (৬) মূল কারণ ঠিক করুন: `.gitignore`-এ `.env`, `~/.aws/` যোগ করুন, pre-commit হুক বা git-secrets ব্যবহার করুন, এবং দলকে প্রশিক্ষণ দিন। প্রতিরোধই সেরা চিকিৎসা: কী Git-এ যাওয়ার আগেই আটকান।

> **WARNING:** `~/.aws/credentials`, `.env`, `*.pem` ফাইল কখনো Git-এ কমিট করবেন না। নতুন রিপোজিটরিতে প্রথম কমিটের আগেই `.gitignore` তৈরি করুন। একটি কী একবার পাবলিক Git-এ গেলে তা চিরকালের জন্য ফাঁস বলে ধরে নিন — শুধু মুছে ফেললেই হবে না, কীটি নিষ্ক্রিয় করে নতুন কী নিতে হবে, কারণ Git ইতিহাসে তা থেকে যায়।

৪.১১ প্রোডাকশন দৃশ্যকল্প: TechNova-র CLI ও CI/CD কাঠামো

TechNova-র ডেভেলপাররা স্থানীয়ভাবে `technova-dev` প্রোফাইল + SSO ব্যবহার করে; production প্রোফাইল শুধু অন-কল ইঞ্জিনিয়ারদের কাছে, এবং production কমান্ড চালানোর আগে `aws sts get-caller-identity` দিয়ে যাচাই করা বাধ্যতামূলক। ডেপ্লয় হয় GitHub Actions + OIDC দিয়ে — কোনো দীর্ঘমেয়াদি কী CI-তে নেই। অ্যাপ কোড SDK v3 ব্যবহার করে, ক্রেডেনশিয়াল চেইন থেকে রোলের অস্থায়ী ক্রেডেনশিয়াল নেয়। প্রতিটি রিপোজিটরিতে `.gitignore`-এ `.env`, `*.pem`, `.aws/` আছে এবং pre-commit হুকে গোপন তথ্য স্ক্যান করা হয়।

৪.১২ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: CLI/SDK নিজে বিনামূল্যে, কিন্তু ভুল ক্রেডেনশিয়ালের খরচ বিশাল। IMDSv2 ব্যবহার করুন (EC2 মেটাডেটায় টোকেন-ভিত্তিক সুরক্ষা), IMDSv1 বন্ধ করুন। খরচ: CLI দিয়ে তৈরি প্রতিটি সম্পদের বিল হয় — পরীক্ষার পর `aws ec2 terminate-instances`, `aws s3 rb` দিয়ে মুছে ফেলুন। CloudShell বিনামূল্যে (সীমার মধ্যে), কিন্তু CloudShell থেকে তৈরি সম্পদের বিল আলাদা।

৪.১৩ সাধারণ ভুল ও সমস্যা সমাধান

ভুল: (১) ভুল প্রোফাইলে production কমান্ড চালানো — প্রতিটি production কমান্ডের আগে পরিচয় যাচাই করুন। (২) পরিবেশ চলকে পুরনো কী রেখে প্রোফাইলের নতুন কী কাজ করছে না ভাবা — chain-এর ক্রম মনে রাখুন। (৩) কোডে কী হার্ডকোড করা। (৪) `aws configure`-এ ভুল Region দেওয়া। সমস্যা: `Unable to locate credentials` — প্রোফাইল নাম, পরিবেশ চলক, বা SSO লগইন যাচাই করুন; `ExpiredToken` — SSO পুনরায় লগইন করুন (`aws sso login --profile technova-dev`) বা অস্থায়ী ক্রেডেনশিয়াল নবায়ন করুন; `AccessDenied` — অধ্যায় ৩-এর ধাপগুলো অনুসরণ করুন।

> **সমস্যা:** `aws sts get-caller-identity` ভুল অ্যাকাউন্ট দেখাচ্ছে। **সমাধান:** `aws configure list` চালান — কোন উৎস থেকে ক্রেডেনশিয়াল আসছে দেখুন; পরিবেশ চলক (`env | grep AWS`) খালি করুন বা সঠিক প্রোফাইল দিয়ে `--profile` ব্যবহার করুন।

৪.১৪ অধ্যায়ের সারাংশ

AWS CLI v2 হলো অটোমেশনের মূল হাতিয়ার; named profile দিয়ে একাধিক অ্যাকাউন্ট সামলান; CloudShell দিয়ে নিরাপদে শুরু করুন। SDK অ্যাপ্লিকেশন থেকে AWS API ডাকে — কোডে কখনো কী লিখবেন না, credential provider chain থেকে নিন। Chain-এর ক্রম: কমান্ড-লাইন, পরিবেশ চলক, credentials ফাইল, SSO, কন্টেইনার রোল, EC2 instance role। CI/CD-তে OIDC দিয়ে কী-বিহীন প্রমাণীকরণ করুন। ক্রেডেনশিয়াল ফাঁস হলে প্রথমে নিষ্ক্রিয় করুন, তারপর CloudTrail-এ তদন্ত করুন, তারপর মূল কারণ ঠিক করুন।

৪.১৫ জ্ঞান যাচাই

১. AWS CLI v2 ইনস্টল ও যাচাইয়ের ধাপ লিখুন। ২. Named profile কী এবং কেন দরকার? ৩. `~/.aws/credentials` ও `~/.aws/config`-এর পার্থক্য কী? ৪. Credential provider chain-এর ক্রম লিখুন। ৫. CloudShell-এর সুবিধা ও সীমা কী? ৬. SDK v3 কেন v2-র চেয়ে ভালো? ৭. OIDC দিয়ে GitHub Actions কীভাবে কী ছাড়াই AWS-এ ঢোকে? ৮. একটি কী GitHub-এ ফাঁস হলে প্রথম পাঁচটি পদক্ষেপ কী? ৯. `ExpiredToken` ত্রুটির কারণ ও সমাধান কী? ১০. EC2-তে অ্যাপ চালাতে অ্যাক্সেস কী না রোল — কোনটি এবং কেন?

৪.১৬ ব্যবহারিক অনুশীলন

অনুশীলন ১: Ubuntu 24.04-এ AWS CLI v2 ইনস্টল করুন, `technova-dev` প্রোফাইল তৈরি করুন, এবং `aws sts get-caller-identity --profile technova-dev` চালিয়ে পরিচয় যাচাই করুন। **অনুশীলন ২:** Node.js 20-এ SDK v3 দিয়ে একটি স্ক্রিপ্ট লিখুন যা S3 বাকেটের তালিকা দেখায় — কোডে কোনো কী না লিখে প্রোফাইল থেকে ক্রেডেনশিয়াল নিন। **অনুশীলন ৩:** একটি পরীক্ষামূলক Git রিপোজিটরিতে `.gitignore` তৈরি করে `.env` ও `*.pem` বাদ দিন; ভুল করে `.env` কমিট করার চেষ্টা করে দেখুন তা বাদ পড়ে কি না। **অনুশীলন ৪:** ইচ্ছাকৃতভাবে পরিবেশ চলকে ভুল কী বসিয়ে `aws sts get-caller-identity` চালান, তারপর চলক খালি করে প্রোফাইল দিয়ে আবার চালান — chain-এর ক্রমটি নিজে পরীক্ষা করে লিখুন।

অধ্যায় ৫ • PART I — AWS Foundations

Chapter 5 — Shared Responsibility, Well-Architected and Billing

৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Shared Responsibility Model ব্যাখ্যা করতে — AWS কী সামলায়, আপনি কী সামলান, এবং সেবা অনুযায়ী সীমানা কীভাবে বদলায়; (২) Well-Architected Framework-এর ছয়টি স্তম্ভ (pillar) ব্যাখ্যা করতে ও প্রতিটি স্তম্ভের নকশা নীতি প্রয়োগ করতে; (৩) অন-ডিমান্ড, রিজার্ভড ও সেভিংস প্ল্যানের ধারণা ও পার্থক্য বলতে; (৪) AWS Budgets, Cost Explorer ও বিলিং অ্যালার্ট সেটআপ করতে; (৫) Free Tier-এর সুবিধা ও সীমা ব্যাখ্যা করতে; (৬) cost allocation tag দিয়ে খরচ ভাগ করতে; এবং (৭) অপ্রত্যাশিত বিলের ধাপে ধাপে তদন্ত করতে।

৫.২ পূর্বশর্ত জ্ঞান

অধ্যায় ১–৪-এর ধারণা — Region/AZ, অ্যাকাউন্ট, IAM, CLI — জানা থাকতে হবে। বিলিংয়ের জন্য অ্যাকাউন্টে বিল দেখার অনুমতি লাগে (রুট বা বিলিং-অ্যাডমিন)। হিসাবের মৌলিক ধারণা (ঘণ্টা, GB, অনুরোধের সংখ্যা) থাকলে মূল্য-মডেল বোঝা সহজ হবে। কোনো নির্দিষ্ট দাম মুখস্থ করার দরকার নেই — দাম পরিবর্তিত হয়, তাই আমরা শুধু ধারণা ও যাচাইয়ের পদ্ধতি শেখাব।

৫.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

দুটি প্রশ্ন প্রতিটি AWS ইঞ্জিনিয়ারকে তাড়া করে: "এই সেবাটি নিরাপদ তো?" এবং "বিল কত আসবে?" প্রথম প্রশ্নের উত্তর Shared Responsibility Model-এ — AWS ডেটা সেন্টার নিরাপদ রাখে, কিন্তু আপনার ডেটা ও অ্যাক্সেস নিয়ন্ত্রণ আপনার দায়িত্ব; এটি না বুঝলে আপনি ভাববেন AWS সব নিরাপত্তা সামলাচ্ছে, অথচ আপনার S3 বাকেট পাবলিক হয়ে আছে। দ্বিতীয় প্রশ্নের উত্তর বিলিং সচেতনতায় — AWS-এ বিল আসে ব্যবহার অনুযায়ী, এবং একটি ভুলে যাওয়া সম্পদ, একটি অসীম লুপে চলা Lambda, বা একটি ভুল Region-এ চালু হওয়া সার্ভার — সবই বিল বাড়ায়। Well-Architected Framework এই দুটি প্রশ্নের সঙ্গে নির্ভরযোগ্যতা, কর্মক্ষমতা ও পরিচালনাকে একসঙ্গে বেঁধে দেয় — এটি ভালো আর্কিটেকচারের চেকলিস্ট।

৫.৪ Shared Responsibility Model: কে কী সামলায়

মডেলটি দুই ভাগে ভাগ করা: **"Security OF the Cloud"** — AWS-এর দায়িত্ব: ফিজিক্যাল ডেটা সেন্টার, হার্ডওয়্যার, নেটওয়ার্ক অবকাঠামো, ভার্চুয়ালাইজেশন স্তর। **"Security IN the Cloud"** — আপনার দায়িত্ব: আপনার ডেটা, অ্যাপ্লিকেশন, IAM নীতি, নেটওয়ার্ক কনফিগারেশন (security group, NACL), OS প্যাচিং (EC2-তে), এনক্রিপশন। সীমানা সেবা অনুযায়ী বদলায়:

সেবা	AWS সামলায়	আপনি সামলান
EC2 (IaaS)	হার্ডওয়্যার, ভার্চুয়ালাইজেশন, ফিজিক্যাল নেটওয়ার্ক	OS প্যাচিং, অ্যাপ, ডেটা, security group, IAM
RDS (PaaS)	OS প্যাচিং, ডেটাবেস ইঞ্জিন প্যাচিং, ব্যাকআপ অবকাঠামো	ডেটা, অ্যাক্সেস নিয়ন্ত্রণ, এনক্রিপশন সেটিং, প্যারামিটার

S3	অবকাঠামো, প্রতিলিপি, স্থায়িত্ব (durability)	বাকেট নীতি, পাবলিক অ্যাক্সেস, এনক্রিপশন, ডেটা
Lambda	রানটাইম, অবকাঠামো, স্কেলিং	কোড, ডেটা, IAM রোল, পরিবেশ চলক

সহজ নিয়ম: আপনি যত ওপরে যাবেন (IaaS → PaaS → SaaS), AWS তত বেশি সামলাবে — কিন্তু ডেটা ও অ্যাক্সেস নিয়ন্ত্রণের দায়িত্ব কখনোই AWS-এর নয়। একটি S3 বাকেট পাবলিক হয়ে গেলে তা আপনার ভুল, AWS-এর নয়। একটি EC2-তে পুরনো OS-এর দুর্বলতা থেকে আক্রমণ হলে তা আপনার দায়িত্ব, কারণ OS প্যাচিং আপনার কাজ।

> **মূল ধারণা:** AWS নিরাপদ ক্লাউড দেয়; আপনি সেই ক্লাউডে নিরাপদ সিস্টেম বানান। "Security OF the Cloud" = AWS; "Security IN the Cloud" = আপনি। সেবা যত পরিচালিত (managed) হবে, AWS-এর অংশ তত বড় হবে — কিন্তু ডেটা ও পরিচয় নিয়ন্ত্রণ সবসময় আপনার।

৫.৫ Well-Architected Framework: ছয়টি স্তম্ভ

AWS Well-Architected Framework হলো ভালো ক্লাউড আর্কিটেকচারের ছয়টি স্তম্ভের ওপর দাঁড়ানো নির্দেশিকা। প্রতিটি স্তম্ভের নকশা নীতি আছে; একসঙ্গে এগুলো একটি চেকলিস্ট হিসেবে ব্যবহার করুন — নতুন আর্কিটেকচার বানানোর আগে ও নিয়মিত পর্যালোচনায়।

৫.৫.১ Operational Excellence (পরিচালনাগত উৎকর্ষ)

সিস্টেম চালানো ও উন্নত করার ক্ষমতা: অটোমেশন, ডকুমেন্টেশন, ছোট পরিবর্তন ঘন ঘন করা, ব্যর্থতা থেকে শেখা। TechNova-র ক্ষেত্রে: ডেপ্লয় স্ক্রিপ্ট দিয়ে করা, হাতে ক্লিক করে নয়; প্রতিটি পরিবর্তনের রেকর্ড (Git) রাখা; ঘটনার পর blameless পর্যালোচনা (post-mortem) করা। নীতি: "কাজ হিসেবে অপারেশন করুন" — কোড দিয়ে অবকাঠামো বানান (IaC), হাতে নয়।

৫.৫.২ Security (নিরাপত্তা)

ডেটা ও সিস্টেম রক্ষা করা: শক্তিশালী পরিচয় ব্যবস্থা (MFA, least privilege), সব স্তরে ট্রেসেবিলিটি (CloudTrail), সব ডেটা এনক্রিপ্ট করা (বিশ্রামে ও চলাচলে), নিরাপত্তা ইভেন্ট অটোমেশন। নীতি: কিছুই বিশ্বাস করবেন না, সবকিছু যাচাই করুন; সর্বনিম্ন অনুমতি দিন; নিরাপত্তা প্রতিটি স্তরে বসান। অধ্যায় ২–৪-এর IAM, MFA ও ক্রেডেনশিয়াল নিয়মগুলো এই স্তম্ভের বাস্তবায়ন।

৫.৫.৩ Reliability (নির্ভরযোগ্যতা)

ব্যর্থতা থেকে পুনরুদ্ধার ও চাহিদা মেটানোর ক্ষমতা: স্বয়ংক্রিয় পুনরুদ্ধার, মাল্টি-AZ নকশা, ব্যাকআপ ও DR পরীক্ষা। নীতি: ব্যর্থতা স্বয়ংক্রিয়ভাবে পুনরুদ্ধার করুন; অনুভূমিকভাবে (horizontally) স্কেল করুন; ধারণক্ষমতা আন্দাজ করা বন্ধ করুন (ক্লাউডে চাহিদা অনুযায়ী নিন); পরিবর্তন অটোমেশন দিয়ে করুন। অধ্যায় ১-এর মাল্টি-AZ ও ফল্ট ডোমেইন আলোচনা এই স্তম্ভের ভিত্তি। TechNova প্রতি ত্রৈমাসিকে একটি AZ বিভ্রাটের অনুশীলন (game day) করবে — পরিকল্পিতভাবে একটি AZ বন্ধ করে দেখবে সেবা টিকে থাকে কি না।

৫.৫.৪ Performance Efficiency (কর্মক্ষমতা দক্ষতা)

সম্পদ দক্ষভাবে ব্যবহার করে চাহিদা মেটানো: সঠিক সম্পদ বাছা (right-sizing), নতুন সেবা পরীক্ষা করা, যান্ত্রিকভাবে স্কেল করা। নীতি: সর্বাধুনিক প্রযুক্তি ব্যবহার করুন; কয়েক মিনিটে বিশ্বজুড়ে পৌঁছান (CloudFront, মাল্টি-রিজিওন); সার্ভারবিহীন (serverless) সেবা ব্যবহার করুন যেখানে উপযুক্ত; পরীক্ষা করে সিদ্ধান্ত নিন, অনুমান করে নয়। একটি ছোট EC2-তে অ্যাপ ধীর হলে প্রথমে মাপুন (CloudWatch মেট্রিক) — CPU, মেমরি, নাকি ডেটাবেস বটলনেক — তারপর বড় ইনস্ট্যান্স নিন।

৫.৫.৫ Cost Optimization (খরচ অপ্টিমাইজেশন)

সবচেয়ে কম খরচে ব্যবসার লক্ষ্য মেটানো: ব্যবহার অনুযায়ী বিল দেওয়া, অপ্রয়োজনীয় সম্পদ বন্ধ করা, সঠিক মূল্য-মডেল বাছা। নীতি: খরচের সচেতনতা তৈরি করুন; ব্যবহার অনুযায়ী বিল দিন; পরিমাপ করুন; অপ্রয়োজনীয় খরচ বন্ধ করুন। এই অধ্যায়ের বাকি অংশ এই স্তম্ভের বাস্তবায়ন।

৫.৫.৬ Sustainability (টেকসইতা)

পরিবেশগত প্রভাব কমানো: সম্পদের ব্যবহার সর্বোচ্চ করা, দক্ষ হার্ডওয়্যার বাছা, অপ্রয়োজনীয় কাজ কমানো। ক্লাউডে এটি প্রায়শই খরচ অপ্টিমাইজেশনের সঙ্গে মিলে যায় — অলস সার্ভার বন্ধ করা যেমন বিল কমায়, তেমন কার্বন ফুটপ্রিন্টও কমায়। TechNova-র জন্য ব্যবহারিক নিয়ম: dev পরিবেশ রাতে ও ছুটির দিনে বন্ধ রাখা, সঠিক আকারের ইনস্ট্যান্স ব্যবহার করা।

Well-Architected — ছয় স্তম্ভ এক নজরে

```
Operational Excellence — অটোমেশন, ছোট পরিবর্তন, শেখা
Security               — পরিচয়, এনক্রিপশন, ট্রেসেবিলিটি
Reliability            — মাল্টি-AZ, স্বয়ংক্রিয় পুনরুদ্ধার, DR
Performance Efficiency — সঠিক আকার, পরিমাপ, স্কেলিং
Cost Optimization      — ব্যবহার অনুযায়ী বিল, অপচয় বন্ধ
Sustainability         — দক্ষ ব্যবহার, অপ্রয়োজনীয় কাজ কমানো
                         |
সব স্তম্ভ একসঙ্গে — কোনো একটিকে উপেক্ষা করলে
আর্কিটেকচার দুর্বল হয়।
```

৫.৬ মূল্য-মডেল: On-Demand, Reserved, Savings Plans

AWS-এ মূলত তিনভাবে কম্পিউটের বিল দেওয়া যায় — কোনো নির্দিষ্ট দাম এখানে লেখা হচ্ছে না, কারণ দাম পরিবর্তিত হয়; AWS Pricing Calculator-এ যাচাই করুন। **On-Demand**: যতক্ষণ ব্যবহার করবেন ততক্ষণের বিল, কোনো অঙ্গীকার নেই — পরীক্ষা ও পরিবর্তনশীল কাজের জন্য উপযুক্ত। **Reserved Instances / Capacity Reservation**: ১ বা ৩ বছরের অঙ্গীকারে ছাড় — স্থির, অনুমানযোগ্য কাজের জন্য (যেমন সবসময় চলা প্রোডাকশন সার্ভার)। **Savings Plans**: প্রতি ঘণ্টায় নির্দিষ্ট ব্যবহারের অঙ্গীকারে ছাড় — Reserved-এর চেয়ে নমনীয়, কারণ ইনস্ট্যান্স টাইপ বদলানো যায়। এছাড়া **Spot Instances**: অব্যবহৃত ক্ষমতা ছাড়ে — ব্যাঘাত সহ্য করতে পারে এমন কাজের জন্য (ব্যাচ প্রসেসিং, পরীক্ষা), তবে AWS যেকোনো সময় তা ফেরত নিতে পারে। TechNova-র নিয়ম: dev/staging On-Demand বা Spot, production-এর স্থির অংশ Savings Plans দিয়ে, আর হঠাৎ বাড়তি চাহিদা On-Demand দিয়ে।

মডেল	অঙ্গীকার	নমনীয়তা	কখন ব্যবহার করবেন
On-Demand	নেই	সর্বোচ্চ	পরীক্ষা, পরিবর্তনশীল ট্র্যাফিক, স্বল্পমেয়াদি কাজ
Reserved Instances	১–৩ বছর	কম (নির্দিষ্ট টাইপ/Region)	স্থির প্রোডাকশন কাজ, দীর্ঘমেয়াদি
Savings Plans	১–৩ বছর (প্রতি ঘণ্টার ব্যবহার)	মাঝারি	স্থির ব্যবহার, কিন্তু টাইপ বদলাতে পারে

Spot	নেই	কম (যেকোনো সময় বন্ধ হতে পারে)	ব্যাঘাত-সহনশীল ব্যাচ কাজ

৫.৭ AWS Budgets, Cost Explorer ও বিলিং অ্যালার্ট

AWS Budgets দিয়ে বলুন "আমার মাসিক খরচ X ডলারের বেশি হলে আমাকে জানাও" — আসল খরচ বা পূর্বাভাস (forecast) সীমা ছাড়ালে ইমেইল বা SNS নোটিফিকেশন আসে। **Cost Explorer** দিয়ে দেখুন খরচ কোথায় যাচ্ছে — সেবা, Region, অ্যাকাউন্ট বা ট্যাগ অনুযায়ী। **বিলিং অ্যালার্ট** (CloudWatch billing alarm) দিয়ে আনুমানিক বিল সীমা ছাড়ালে অ্যালার্ট পান — এটি অ্যাকাউন্ট তৈরির দিনেই চালু করুন।

```
# বিলিং অ্যালার্টের জন্য (us-east-1 Region-এ, কারণ বিলিং মেট্রিক সেখানে থাকে)
aws budgets create-budget --account-id 111122223333 \
  --budget file://budget.json
# budget.json-এ BudgetName, BudgetLimit ও TimeUnit থাকে —
# ফাইলটি তৈরি করার আগে AWS ডকুমেন্টেশনে ফরম্যাট যাচাই করুন।
```

কনসোলে: Billing and Cost Management → Budgets → Create budget → Monthly cost budget → সীমা ও ইমেইল দিন। Cost Explorer-এ: Billing → Cost Explorer → Group by Service — দেখুন কোন সেবা সবচেয়ে বেশি খরচ করছে। প্রথম মাসে প্রতিদিন Cost Explorer দেখুন — অভ্যাসটি পরে বড় বিল থেকে বাঁচাবে।

> **PRO TIP:** অ্যাকাউন্ট তৈরির দিনেই তিনটি কাজ করুন: (১) একটি মাসিক বাজেট তৈরি করুন, (২) বিলিং অ্যালার্ট চালু করুন, (৩) Cost Explorer চালু করুন। এগুলো বিনামূল্যে (সীমার মধ্যে) এবং একটি অপ্রত্যাশিত বিলের ক্ষেত্রে ঘণ্টার মধ্যে জানতে পারবেন, মাস শেষে নয়।

৫.৮ Free Tier: সুবিধা ও সীমা

AWS Free Tier নতুন অ্যাকাউন্টকে সীমিত পরিমাণে বিনামূল্যে সেবা ব্যবহার করতে দেয় — যেমন প্রতি মাসে নির্দিষ্ট ঘণ্টা EC2, নির্দিষ্ট GB S3 স্টোরেজ, নির্দিষ্ট সংখ্যক Lambda অনুরোধ। তিনটি ধরন আছে: ১২ মাসের বিনামূল্যে (কিছু সেবা), সবসময়ের বিনামূল্যে (কিছু সীমার মধ্যে), এবং স্বল্পমেয়াদি ট্রায়াল। **সতর্কতা:** Free Tier-এর সুবিধা, সীমা ও মেয়াদ পরিবর্তিত হয় — সবসময় অফিসিয়াল AWS Free Tier পেজে যাচাই করুন। সীমা ছাড়ালেই বিল শুরু হয় — যেমন Free Tier-এর EC2 ঘণ্টা শেষ হলে পরের ঘণ্টা থেকেই On-Demand বিল আসে। তাই Free Tier-এ পরীক্ষা করলেও Budgets ও বিলিং অ্যালার্ট চালু রাখুন, এবং পরীক্ষার পর সম্পদ মুছে ফেলুন।

৫.৯ Cost Allocation Tag: খরচের হিসাব ভাগ করা

ট্যাগ (tag) হলো সম্পদের সঙ্গে লাগানো কী-ভ্যালু লেবেল — যেমন `Environment=production`, `Project=technova-app`, `Owner=nahid`। **Cost allocation tag** চালু করলে Cost Explorer-এ ট্যাগ অনুযায়ী খরচ দেখা যায় — কোন প্রজেক্ট, কোন পরিবেশ, কোন দল কত খরচ করছে। নিয়ম: প্রতিটি সম্পদে অন্তত `Environment` ও `Project` ট্যাগ বাধ্যতামূলক করুন (SCP বা AWS Config নিয়ম দিয়ে); ট্যাগ ছাড়া সম্পদ খুঁজে বের করে ট্যাগ করুন বা মুছে ফেলুন। TechNova-র ট্যাগ নিয়ম: `Environment` (dev/staging/prod), `Project` (technova-app), `Owner` (দায়িত্বপ্রাপ্ত ব্যক্তি), `CostCenter` (বিলিং ভাগের জন্য)।

৫.১০ অপ্রত্যাশিত বিলের তদন্ত (ধাপে ধাপে)

বিল প্রত্যাশার চেয়ে বেশি এলে আতঙ্কিত হবেন না — ধাপে ধাপে তদন্ত করুন: **ধাপ ১:** Billing ড্যাশবোর্ডে গিয়ে মোট বিল ও পূর্বাভাস দেখুন। **ধাপ ২:** Cost Explorer-এ Group by Service — কোন সেবা হঠাৎ বেড়েছে? **ধাপ ৩:** সেই সেবায় Group by Region — কোন Region-এ খরচ? ভুল Region-এ সম্পদ চালু আছে কি না দেখুন। **ধাপ ৪:** Group by Usage Type — কোন ধরনের ব্যবহার (যেমন ডেটা স্থানান্তর, স্টোরেজ, কম্পিউট ঘণ্টা) বেড়েছে? **ধাপ ৫:** Cost and Usage Report (CUR) বা CloudTrail দেখে কোন সম্পদ, কখন, কে তৈরি করেছে তা বের করুন। **ধাপ ৬:** অপ্রয়োজনীয় সম্পদ বন্ধ/মুছে ফেলুন — EC2 বন্ধ করা, EBS ভলিউম মুছে ফেলা, S3-র পুরনো সংস্করণ পরিষ্কার করা, NAT Gateway-র মতো সবসময় চলা সেবা যাচাই করা। **ধাপ ৭:** পুনরাবৃত্তি রোধ করুন — বাজেট, অ্যালার্ট, ট্যাগ নিয়ম ও নিয়মিত পর্যালোচনা চালু করুন। সাধারণ অপরাধী: ভুলে যাওয়া EC2, অসংযুক্ত EBS ভলিউম, NAT Gateway, cross-Region ডেটা স্থানান্তর, CloudWatch Logs-এর অসীম ধারণ (retention), এবং S3-তে অপ্রয়োজনীয় সংস্করণ।

> **WARNING:** বিলযোগ্য সম্পদ (billable resource) তৈরি করার আগে সবসময় ভাবুন — এটি কতক্ষণ চলবে, কে বন্ধ করবে, এবং বিল কে দেখবে? পরীক্ষার সম্পদে `Environment=experiment` ট্যাগ ও একটি মেয়াদোত্তীর্ণের তারিখ লিখুন; পরীক্ষা শেষে মুছে ফেলুন। একটি ভুলে যাওয়া NAT Gateway বা বড় EC2 মাস শেষে বড় বিল আনতে পারে — AWS Pricing Calculator-এ যাচাই করুন।

৫.১১ প্রোডাকশন দৃশ্যকল্প: TechNova-র খরচ শাসন

TechNova প্রতি মাসের শুরুতে বাজেট পর্যালোচনা করে: dev অ্যাকাউন্টের বাজেট ছোট, production-এর বাজেট ব্যবসার আয়ের সঙ্গে মিলিয়ে। প্রতিটি সম্পদে ট্যাগ বাধ্যতামূলক; ট্যাগ ছাড়া সম্পদ সাপ্তাহিক রিপোর্টে আসে। dev পরিবেশ রাত ১০টা থেকে সকাল ৭টা বন্ধ থাকে (অটোমেটেড শিডিউল)। Cost Explorer-এ প্রতি সপ্তাহে একজন ইঞ্জিনিয়ার "খরচের মালিক" (cost owner) হিসেবে দেখে কোন সেবা বেড়েছে। Well-Architected পর্যালোচনা প্রতি ত্রৈমাসিকে হয় — ছয়টি স্তম্ভের প্রতিটিতে একটি করে উন্নতির কাজ বেছে নেওয়া হয়।

৫.১২ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা: বিলিং তথ্য সংবেদনশীল — শুধু প্রয়োজনীয় ব্যক্তিকে বিল দেখার অনুমতি দিন (IAM-এ বিলিং অনুমতি আলাদা)। MFA ছাড়া বিলিং পরিবর্তন করা যাবে না — এমন নীতি রাখুন। খরচ: Cost Explorer, Budgets ও বিলিং অ্যালার্টের মৌলিক ব্যবহার বিনামূল্যে (সীমার মধ্যে); তবে বিস্তারিত CUR ও কিছু উন্নত বৈশিষ্ট্যে খরচ হতে পারে — AWS Pricing Calculator-এ যাচাই করুন। সবচেয়ে বড় খরচের নিয়ন্ত্রণ হলো অভ্যাস: পরীক্ষার পর মুছে ফেলা, সঠিক আকার বাছা, ও নিয়মিত পর্যালোচনা।

৫.১৩ সাধারণ ভুল

(১) "AWS সব নিরাপত্তা সামলায়" ভাবা — S3 বাকেট পাবলিক রেখে দেওয়া। (২) Free Tier সীমাহীন ভাবা — সীমা ছাড়ালেই বিল। (৩) বাজেট ও অ্যালার্ট না বসানো — মাস শেষে বিল দেখে চমকে যাওয়া। (৪) ট্যাগ না দেওয়া — পরে খরচের হিসাব মেলাতে না পারা। (৫) Reserved/Savings Plan কেনার আগে ব্যবহারের ধরন না বোঝা — অঙ্গীকার করে পরে ব্যবহার না করা। (৬) Well-Architected-কে একবারের কাজ ভাবা — এটি নিয়মিত পর্যালোচনার চেকলিস্ট। (৭) দাম মুখস্থ করা — দাম পরিবর্তিত হয়, সবসময় Pricing Calculator-এ যাচাই করুন।

৫.১৪ সমস্যা সমাধান (Troubleshooting)

সমস্যা: Cost Explorer-এ খরচ দেখা যাচ্ছে না। **সমাধান:** Cost Explorer চালু হতে ২৪ ঘণ্টা লাগতে পারে; বিলিং অনুমতি আছে কি না যাচাই করুন। **সমস্যা:** বাজেট অ্যালার্ট আসছে না। **সমাধান:** বাজেটের ইমেইল ঠিক আছে কি না, SNS টপিকের অনুমতি আছে কি না দেখুন। **সমস্যা:** ট্যাগ অনুযায়ী খরচ দেখা যাচ্ছে না। **সমাধান:** cost allocation tag চালু করতে হবে (Billing → Cost allocation tags) এবং চালুর পর ২৪ ঘণ্টা অপেক্ষা করতে হবে। **সমস্যা:** বিল হঠাৎ বেড়েছে। **সমাধান:** উপরের ৫.১০-এর ধাপগুলো অনুসরণ করুন — প্রথমে সেবা, তারপর Region, তারপর Usage Type দিয়ে সংকুচিত করুন।

৫.১৫ অধ্যায়ের সারাংশ

Shared Responsibility Model বলে AWS অবকাঠামো সামলায়, আপনি ডেটা ও অ্যাক্সেস সামলান — সীমানা সেবা অনুযায়ী বদলায়। Well-Architected Framework-এর ছয়টি স্তম্ভ — Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability — ভালো আর্কিটেকচারের চেকলিস্ট। মূল্য-মডেল: On-Demand (নমনীয়), Reserved/Savings Plans (অঙ্গীকারে ছাড়), Spot (ব্যাঘাত-সহনশীল)। Budgets, Cost Explorer ও বিলিং অ্যালার্ট অ্যাকাউন্ট তৈরির দিনেই চালু করুন। Free Tier সীমিত ও পরিবর্তনশীল — সবসময় অফিসিয়াল পেজে যাচাই করুন। ট্যাগ দিয়ে খরচ ভাগ করুন, আর অপ্রত্যাশিত বিলের তদন্ত ধাপে ধাপে করুন: সেবা → Region → Usage Type → সম্পদ।

৫.১৬ জ্ঞান যাচাই

১. "Security OF the Cloud" ও "Security IN the Cloud"-এর পার্থক্য উদাহরণসহ লিখুন। ২. EC2, RDS ও S3-তে আপনার দায়িত্ব কীভাবে আলাদা? ৩. Well-Architected-এর ছয়টি স্তম্ভের নাম ও প্রতিটির একটি নীতি লিখুন। ৪. On-Demand, Reserved ও Savings Plans-এর পার্থক্য কী? ৫. Spot Instance কখন ব্যবহার করবেন, কখন নয়? ৬. AWS Budgets ও বিলিং অ্যালার্টের পার্থক্য কী? ৭. Cost Explorer দিয়ে কী কী জানা যায়? ৮. Free Tier সম্পর্কে তিনটি সতর্কতা লিখুন। ৯. Cost allocation tag কীভাবে খরচের হিসাব সহজ করে? ১০. অপ্রত্যাশিত বিলের তদন্তের সাতটি ধাপ লিখুন।

৫.১৭ ব্যবহারিক অনুশীলন

অনুশীলন ১: নিজের অ্যাকাউন্টে একটি মাসিক বাজেট তৈরি করুন (সীমা আপনার খরচ অনুযায়ী) এবং একটি বিলিং অ্যালার্ট চালু করুন; সেটআপের স্ক্রিনশট বা নোট রাখুন। **অনুশীলন ২:** Cost Explorer খুলে Group by Service ও Group by Region দিয়ে গত মাসের খরচ দেখুন; সবচেয়ে বড় তিনটি খরচের খাত লিখুন। **অনুশীলন ৩:** TechNova-র জন্য একটি ট্যাগ নিয়ম লিখুন — কোন ট্যাগ বাধ্যতামূলক, কোন মান (value) অনুমোদিত, এবং ট্যাগ ছাড়া সম্পদ কীভাবে ধরা হবে। **অনুশীলন ৪:** একটি কাল্পনিক অপ্রত্যাশিত বিলের দৃশ্য লিখুন (যেমন NAT Gateway ভুলে চালু) এবং ৫.১০-এর ধাপগুলো দিয়ে তদন্তের পরিকল্পনা লিখুন — কোন সেবা, কোন Region, কোন Usage Type দেখবেন।

PART II • DEVOPS ENGINEERING MASTERY

Networking & Compute Engineering

VPC আর্কিটেকচার, Internet Gateway ও NAT, নেটওয়ার্ক সিকিউরিটি ও হাইব্রিড কানেক্টিভিটি, EC2, Elastic Load Balancing ও Auto Scaling।

অধ্যায় ৬ • PART II — Networking & Compute

Chapter 6 — Amazon VPC Architecture

৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) ব্যাখ্যা করতে কেন AWS-এ প্রতিটি workload-এর নিচে একটি Virtual Private Cloud (VPC) থাকে; (২) CIDR নোটেশন দিয়ে সাবনেট হিসাব করতে এবং একটি `10.0.0.0/16` VPC-কে পাবলিক ও প্রাইভেট সাবনেটে ভাগ করতে; (৩) Availability Zone (AZ) জুড়ে multi-AZ আর্কিটেকচার ডিজাইন করতে; (৪) route table, Internet Gateway ও NAT Gateway-এর সম্পর্ক বুঝতে; (৫) default VPC বনাম custom VPC-এর পার্থক্য বলতে; এবং (৬) AWS CLI দিয়ে একটি সম্পূর্ণ production-grade VPC তৈরি করতে।

৬.২ পূর্বশর্ত ঝালাই (Prerequisite Refresher)

এই অধ্যায়ে ঢোকার আগে তিনটি ধারণা ঝালিয়ে নিন। প্রথম, **IP address** — প্রতিটি ডিভাইসের নেটওয়ার্ক ঠিকানা, যেমন `10.0.1.23`। দ্বিতীয়, **CIDR** — `10.0.0.0/16` মানে প্রথম ১৬ বিট নেটওয়ার্ক অংশ, বাকি ১৬ বিট হোস্ট অংশ। তৃতীয়, **subnet** — একটি বড় নেটওয়ার্ককে ছোট ছোট ভাগে ভাগ করা, যাতে ট্র্যাফিক আলাদা করে নিয়ন্ত্রণ করা যায়। Volume 9-এ আপনি শিখেছেন on-premises নেটওয়ার্কে VLAN ও firewall দিয়ে কীভাবে segmentation করা হয়; VPC হলো সেই একই ধারণার cloud সংস্করণ — তবে এখানে segmentation-টি software-defined, অর্থাৎ API কল দিয়েই তৈরি, পরিবর্তন ও মুছে ফেলা যায়।

৬.৩ কেন VPC শিখতে হবে — আগে কী সমস্যা ছিল?

ক্লাউডের আগে, একটি কোম্পানিকে সার্ভার চালাতে হলে নিজের ডেটা সেন্টারে র‍্যাক, সুইচ, রাউটার, ফায়ারওয়াল কিনতে হতো। নেটওয়ার্ক বদলাতে চাইলে কেবল লাগাতে হতো, সুইচ কনফিগার করতে হতো — সপ্তাহ লেগে যেত। আর সবচেয়ে বড় সমস্যা ছিল **isolation**: একই শেয়ার্ড নেটওয়ার্কে থাকা অন্য গ্রাহকের সার্ভার থেকে আপনার সার্ভারকে আলাদা রাখা ছিল কঠিন ও ব্যয়বহুল। AWS প্রথমে EC2-Classic চালু করেছিল — সেখানে সব গ্রাহকের instance একটাই ফ্ল্যাট নেটওয়ার্কে থাকত। সেটি দ্রুত, কিন্তু নিরাপত্তা ও নিয়ন্ত্রণের দিক থেকে সীমিত ছিল।

২০০৯ সালে VPC আসে এই সমস্যার সমাধান হিসেবে। VPC মানে **Virtual Private Cloud** — AWS-এর শেয়ার্ড infrastructure-এর ভেতরে আপনার নিজের একটি logically isolated নেটওয়ার্ক। আপনি নিজের IP range বাছেন, নিজের সাবনেট বানান, নিজের route table লেখেন, নিজের firewall নিয়ম দেন। বাইরের দুনিয়া থেকে কেউ আপনার private সাবনেটের সার্ভারে পৌঁছাতে পারে না — যদি না আপনি স্পষ্টভাবে রাস্তা খুলে দেন। TechNova-র ক্ষেত্রে ভাবুন: তাদের ওয়েব সার্ভার ইন্টারনেটে থাকবে, কিন্তু ডেটাবেস থাকবে এমন এক সাবনেটে যেখানে ইন্টারনেট থেকে সরাসরি কোনো রাস্তাই নেই। এটাই VPC-এর মূল প্রতিশ্রুতি।

> **CONCEPT:** VPC একটি "ভার্চুয়াল ডেটা সেন্টার"। হার্ডওয়্যার নেই, কেবল নেই — আছে শুধু কনফিগারেশন। আপনি যা কনফিগার করেন, AWS-এর software-defined networking (SDN) সেটিকে বাস্তবে রূপ দেয়। VPC মুছে ফেললে পুরো নেটওয়ার্কটাই উধাও হয়ে যায়, কোনো কেবল খুলতে হয় না।

৬.৪ VPC কীভাবে কাজ করে — প্রথম নীতি থেকে

VPC-এর ভেতরে প্রতিটি resource — EC2 instance, RDS ডেটাবেস, Load Balancer — একটি **Elastic Network Interface (ENI)**-এর মাধ্যমে যুক্ত হয়। ENI হলো একটি ভার্চুয়াল নেটওয়ার্ক কার্ড: এর একটি private IP, একটি MAC address, এবং এক বা একাধিক security group থাকে। আপনি যখন instance চালু করেন, AWS আপনার বাছা সাবনেট থেকে একটি private IP দেয় এবং ENI-টি সেই সাবনেটে attach করে।

ট্র্যাফিকের পথ নির্ধারণ করে **route table**। প্রতিটি সাবনেটের সঙ্গে একটি route table যুক্ত থাকে। Route table-এ থাকে destination CIDR ও target — যেমন `0.0.0.0/0 → igw-xxxx` মানে "সব ইন্টারনেট ট্র্যাফিক Internet Gateway-এ পাঠাও"। কোনো route না থাকলে প্যাকেট কোথাও যায় না — এটাই VPC-এর default-deny প্রকৃতি। অর্থাৎ নিরাপত্তা এখানে "খুলে দেওয়া" নয়, "বন্ধ রেখে প্রয়োজনমতো খোলা"।

একটি VPC সবসময় একটি **region**-এর ভেতরে থাকে, কিন্তু একাধিক **Availability Zone**-এ ছড়িয়ে থাকে। AZ হলো একই region-এর ভেতরে আলাদা আলাদা ডেটা সেন্টার — আলাদা বিদ্যুৎ, আলাদা নেটওয়ার্ক, আলাদা ভবন। একটি AZ-তে আগুন লাগলেও অন্য AZ চলতে থাকে। তাই production আর্কিটেকচারে অন্তত দুটি AZ ব্যবহার করা বাধ্যতামূলক প্রায়।

> **Availability Zone (AZ):** একটি AWS region-এর ভেতরে এক বা একাধিক physically আলাদা ডেটা সেন্টারের সমষ্টি। AZ-গুলোর মধ্যে latency খুব কম (এক অঙ্কের মিলিসেকেন্ড), কিন্তু failure domain আলাদা। ap-south-1-এ তিনটি AZ আছে: ap-south-1a, ap-south-1b, ap-south-1c।

৬.৫ CIDR ও সাবনেট হিসাব — হাতে-কলমে

CIDR বোঝা VPC ডিজাইনের সবচেয়ে গুরুত্বপূর্ণ গাণিতিক দক্ষতা। `10.0.0.0/16`-এর মানে: মোট ৩২ বিটের মধ্যে ১৬ বিট নেটওয়ার্ক, বাকি ১৬ বিট হোস্ট। মোট ঠিকানা = 2^{16} = ৬৫,৫৩৬টি। কিন্তু AWS প্রতিটি সাবনেটে ৫টি IP সংরক্ষণ করে: নেটওয়ার্ক ঠিকানা (.0), VPC router (.1), DNS server (.2), ভবিষ্যতের জন্য (.3), এবং broadcast ঠিকানা (.255)। তাই usable IP = মোট − ৫।

৬.৫.১ উদাহরণ ১: 10.0.0.0/16-কে /20 সাবনেটে ভাগ করা

/16 থেকে /20-এ যেতে ৪ বিট ধার নিতে হয়: 2^4 = ১৬টি /20 সাবনেট পাওয়া যায়। প্রতিটি /20-তে 2^{12} = ৪,০৯৬টি ঠিকানা, usable ৪,০৯১টি। সাবনেটগুলো হয়: `10.0.0.0/20`, `10.0.16.0/20`, `10.0.32.0/20`, `10.0.48.0/20` — এভাবে ১৬ ধাপে `10.0.240.0/20` পর্যন্ত। /20 বড় সাবনেট — EKS নোড বা বড় Auto Scaling Group-এর জন্য ভালো, কারণ প্রতিটি pod/node-এর IP লাগে।

৬.৫.২ উদাহরণ ২: /24 সাবনেট — সবচেয়ে সাধারণ পছন্দ

/24 মানে ২৫৬টি ঠিকানা, usable ২৫১টি। `10.0.0.0/16` থেকে ২৫৬টি /24 সাবনেট বানানো যায়। TechNova-র ডিজাইনে আমরা /24 ব্যবহার করব — হিসাব সহজ, ভবিষ্যতে বাড়ানো যায়, এবং প্রতিটি AZ-তে পাবলিক ও প্রাইভেট সাবনেট আলাদা করে দেওয়া যায়। একটি /24 সাবনেটে ২৫১টি usable IP মানে প্রায় ২৫০টি instance — ছোট-মাঝারি workload-এর জন্য যথেষ্ট। তবে মনে রাখবেন: Load Balancer, NAT Gateway, VPC endpoint — এগুলোও সাবনেটের IP খায়। তাই সাবনেট খুব ছোট (/28) বানাবেন না; একদিন IP ফুরিয়ে গেলে নতুন instance চালু হবে না, আর সাবনেটের CIDR পরে বদলানো যায় না — VPC-তে নতুন CIDR যোগ করা যায়, কিন্তু সাবনেটের CIDR বদলানো যায় না।

CIDR	মোট IP	Usable IP (AWS)	সাধারণ ব্যবহার

/16	65,536	65,531	পুরো VPC
/20	4,096	4,091	বড় workload, EKS
/24	256	251	সাধারণ সাবনেট (সুপারিশকৃত)
/28	16	11	খুব ছোট; এড়িয়ে চলুন

> **WARNING:** VPC-এর CIDR বাছার সময় আপনার on-premises নেটওয়ার্ক বা অন্য VPC-এর CIDR-এর সঙ্গে overlap করবেন না। Overlap হলে VPC Peering বা VPN কাজ করবে না — রাউটার বুঝতে পারবে না প্যাকেট কোন দিকে যাবে। TechNova ব্যবহার করে `10.0.0.0/16` (RFC 1918 private range) — এটি ইন্টারনেটে রাউটেবল নয়, তাই নিরাপদ।

৬.৬ পাবলিক বনাম প্রাইভেট সাবনেট

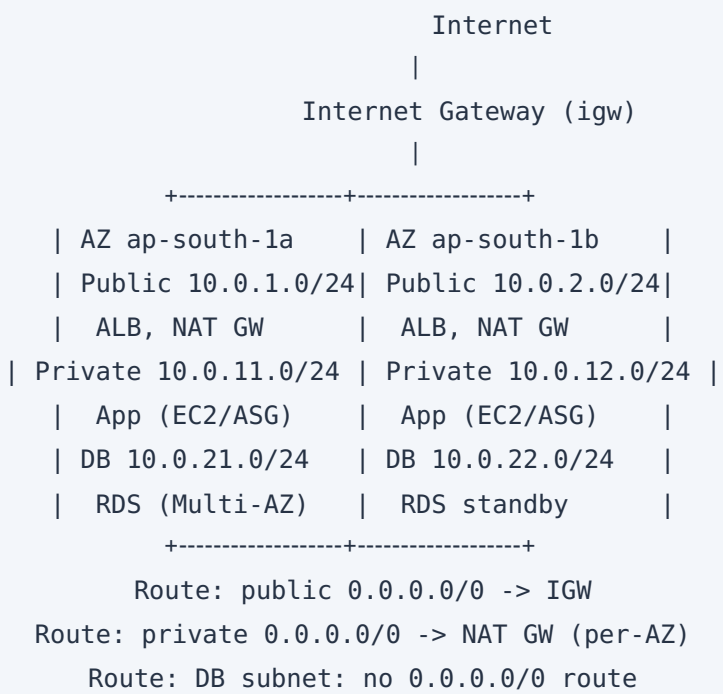
সাবনেট "পাবলিক" না "প্রাইভেট" — এটি নির্ধারণ করে route table, সাবনেটের নাম নয়। যে সাবনেটের route table-এ `0.0.0.0/0 → Internet Gateway` আছে, সেটি পাবলিক সাবনেট। যে সাবনেটে সেই route নেই, সেটি প্রাইভেট — সেখানকার instance-এর public IP থাকলেও ইন্টারনেটে পৌঁছাতে পারবে না। সাধারণ নিয়ম: **Load Balancer ও NAT Gateway পাবলিক সাবনেটে; অ্যাপ সার্ভার ও ডেটাবেস প্রাইভেট সাবনেটে।** ডেটাবেস সাবনেটে তো NAT Gateway-এর route-ও দেবেন না — ডেটাবেসের ইন্টারনেটে যাওয়ার কোনো দরকারই নেই; শুধু অ্যাপ সার্ভার থেকে আসা ট্র্যাফিকই যথেষ্ট।

একটি সাধারণ ভুল ধারণা: "private সাবনেটের instance ইন্টারনেট থেকে software update ডাউনলোড করতে পারবে না" — ঠিক, যদি NAT Gateway না থাকে। তাই প্রাইভেট অ্যাপ সাবনেটের route table-এ `0.0.0.0/0 → NAT Gateway` দেওয়া হয়। NAT Gateway পাবলিক সাবনেটে বসে, নিজের Elastic IP দিয়ে outbound ট্র্যাফিক পাঠায়, আর reply ফিরিয়ে আনে — কিন্তু inbound connection শুরু করতে দেয় না। এটাই একমুখী দরজা।

৬.৭ Default VPC বনাম Custom VPC

প্রতিটি region-এ AWS একটি **default VPC** বানিয়ে দেয়: `172.31.0.0/16`, প্রতিটি AZ-তে একটি /20 পাবলিক সাবনেট, Internet Gateway সংযুক্ত, সব instance-এ public IP। এটি শেখার জন্য ভালো, production-এর জন্য নয় — কারণ CIDR আপনার নিয়ন্ত্রণে নেই, সাবনেট design আপনার মতো নয়, আর সবকিছুই পাবলিক। **Custom VPC**-তে আপনি CIDR, সাবনেট, route, DNS সেটিংস সব নিজে বাছেন। এই বইয়ের সব উদাহরণ custom VPC দিয়ে — TechNova-র VPC-ই আমাদের reference architecture।

TechNova Production VPC — ap-south-1 (10.0.0.0/16)



৬.৮ AWS CLI দিয়ে TechNova VPC তৈরি

Console-এ VPC Wizard দিয়ে এই কাজটি কয়েক ক্লিকে করা যায় (VPC Console → Create VPC → "VPC and more")। কিন্তু CLI শিখলে আপনি IaC-এর দিকে এক ধাপ এগিয়ে যান। নিচের কমান্ডগুলো ক্রমানুসারে চালান — প্রতিটি কমান্ডের output থেকে পরের কমান্ডের ID নিতে হয়।

```
# 1. VPC তৈরি
aws ec2 create-vpc --cidr-block 10.0.0.0/16 \
  --tag-specifications \
  'ResourceType=vpc,Tags=[{Key=Name,Value=technova-prod}]'
# Output থেকে VpcId নিন, যেমন vpc-0abc1234def56789

# 2. DNS hostnames চালু (EC2-এর public DNS নামের জন্য)
aws ec2 modify-vpc-attribute --vpc-id vpc-0abc1234def56789 \
  --enable-dns-hostnames '{"Value":true}'

# 3. Internet Gateway তৈরি ও attach
aws ec2 create-internet-gateway \
  --tag-specifications \
  'ResourceType=internet-gateway,Tags=[{Key=Name,Value=technova-igw}]'
aws ec2 attach-internet-gateway \
  --vpc-id vpc-0abc1234def56789 \
  --internet-gateway-id igw-0abc1234def56789
```

```
# 4. পাবলিক সাবনেট (দুই AZ-তে)
aws ec2 create-subnet --vpc-id vpc-0abc1234def56789 \
  --cidr-block 10.0.1.0/24 --availability-zone ap-south-1a \
  --tag-specifications \
  'ResourceType=subnet,Tags=[{Key=Name,Value=technova-pub-1a}]'
```

```
aws ec2 create-subnet --vpc-id vpc-0abc1234def56789 \
  --cidr-block 10.0.2.0/24 --availability-zone ap-south-1b \
  --tag-specifications \
  'ResourceType=subnet,Tags=[{Key=Name,Value=technova-pub-1b}]'

# 5. প্রাইভেট অ্যাপ সাবনেট
aws ec2 create-subnet --vpc-id vpc-0abc1234def56789 \
  --cidr-block 10.0.11.0/24 --availability-zone ap-south-1a \
  --tag-specifications \
  'ResourceType=subnet,Tags=[{Key=Name,Value=technova-app-1a}]'
aws ec2 create-subnet --vpc-id vpc-0abc1234def56789 \
  --cidr-block 10.0.12.0/24 --availability-zone ap-south-1b \
  --tag-specifications \
  'ResourceType=subnet,Tags=[{Key=Name,Value=technova-app-1b}]'

# 6. Route table: পাবলিক
aws ec2 create-route-table --vpc-id vpc-0abc1234def56789 \
  --tag-specifications \
  'ResourceType=route-table,Tags=[{Key=Name,Value=technova-rt-pub}]'
aws ec2 create-route --route-table-id rtb-0abc1234def56789 \
  --destination-cidr-block 0.0.0.0/0 \
  --gateway-id igw-0abc1234def56789
aws ec2 associate-route-table \
  --route-table-id rtb-0abc1234def56789 \
  --subnet-id subnet-0abc1234def56789
```

লক্ষ্য করুন: প্রতিটি `create-*` কমান্ড একটি ID ফেরত দেয়। বাস্তব কাজে এই ID-গুলো shell variable-এ ধরে রাখুন বা Terraform/CloudFormation ব্যবহার করুন — হাতে ID কপি-পেস্ট করা ভুলের উৎস। এই বইয়ের পরের volume-গুলোতে আমরা এই একই VPC Terraform দিয়ে বানাব।

৬.৯ Multi-AZ আর্কিটেকচার — কেন দুটি AZ যথেষ্ট নয়, তিনটি ভালো

দুটি AZ দিয়ে শুরু করা যায়, তবে তিনটি AZ ব্যবহার করলে quorum-based সিস্টেম (যেমন EKS control plane, RDS Multi-AZ DB cluster) ভালো কাজ করে। প্রতিটি AZ-তে পাবলিক + প্রাইভেট সাবনেট রাখুন, NAT Gateway প্রতিটি AZ-তে আলাদা রাখুন — নইলে একটি AZ ডাউন হলে অন্য AZ-র private instance-গুলো ইন্টারনেট হারাবে (cross-AZ NAT কাজ করে, কিন্তু সেটি single point of failure এবং cross-AZ data transfer খরচ বাড়ায়)। একইভাবে Application Load Balancer-কে অন্তত দুটি AZ-তে সাবনেট দিন — ALB নিজেই multi-AZ সার্ভিস।

৬.১০ নিরাপত্তা, খরচ ও সাধারণ ভুল

নিরাপত্তা: VPC-এর default security group সব inbound বন্ধ রাখে — ভালো। কিন্তু default NACL সবকিছু খোলা রাখে — সেটি বুঝে ব্যবহার করুন। VPC Flow Logs চালু করুন (অধ্যায় ৮-এ বিস্তারিত)। **খরচ:** VPC নিজে বিনামূল্যে, কিন্তু NAT Gateway প্রতি ঘণ্টায় চার্জ হয় ও প্রতি GB data-processing চার্জ হয় — তিনটি AZ-তে তিনটি NAT Gateway মানে তিনগুণ hourly চার্জ; খরচের হিসাব AWS Pricing Calculator-এ যাচাই করুন। **সাধারণ ভুল:** (১) সব সাবনেটকে পাবলিক বানানো; (২) একটাই AZ ব্যবহার করা; (৩) /28-এর মতো ছোট সাবনেট; (৪) CIDR overlap করে VPC Peering করতে চাওয়া; (৫) route table-এ ভুল AZ-র NAT Gateway দেওয়া।

৬.১১ সমস্যা সমাধান (Troubleshooting)

Instance-এ ইন্টারনেট নেই? চেকলিস্ট: (১) সাবনেট কি পাবলিক — route table-এ IGW route আছে কি? (২) instance-এর public IP বা Elastic IP আছে কি? (৩) Security Group-এ outbound 0.0.0.0/0 খোলা কি? (৪) NACL outbound নিয়ম কি block করছে? (৫) private সাবনেট হলে NAT Gateway-এর route আছে কি, NAT Gateway কি "available" অবস্থায়? VPC Reachability Analyzer দিয়ে দুটি endpoint-এর মধ্যে পথ পরীক্ষা করুন — এটি ঠিক কোথায় প্যাকেট আটকাচ্ছে দেখিয়ে দেয়।

৬.১২ VPC-এর ভেতরের জীবন — DNS, DHCP ও IP পরিচালনা

VPC তৈরি করলেই নেটওয়ার্ক শেষ নয় — ভেতরের সার্ভিসগুলো একে অপরকে খুঁজে পায় কীভাবে, সেটিও ডিজাইনের অংশ। প্রতিটি VPC-তে AWS একটি **DNS resolver** চালায় — VPC-র CIDR-এর .2 ঠিকানায় (যেমন `10.0.0.2`)। এই resolver দুটি কাজ করে: (১) public DNS নাম resolve করা (যেমন `amazonaws.com`); (২) private hosted zone-এর নাম resolve করা (অধ্যায় ৮)। VPC-র `enableDnsSupport` ও `enableDnsHostnames` — দুটি attribute-ই চালু রাখুন, নইলে instance-এর public DNS নাম তৈরি হবে না এবং private DNS নাম কাজ করবে না।

DHCP Options Set নির্ধারণ করে instance-গুলো কোন DNS server ও কোন domain name ব্যবহার করবে। Default DHCP options set AWS-এর DNS ব্যবহার করে; কাস্টম DHCP options set দিয়ে আপনি নিজের DNS server (যেমন on-premises DNS) বা নিজের domain name (`technova.example`) দিতে পারেন। Hybrid পরিবেশে এটি জরুরি — on-premises-এর `corp.technova.example` নাম VPC-র ভেতর থেকে resolve করতে হলে DHCP options-এ সেই DNS server দিতে হয়। মনে রাখবেন: DHCP options set VPC-র সঙ্গে যুক্ত থাকে, সাবনেটের সঙ্গে নয়; বদলালে নতুন instance-গুলো নতুন সেটিংস পায়, চলমান instance-গুলো পায় না (reboot বা DHCP renew লাগে)।

IP পরিচালনা (IPAM): কোম্পানিতে VPC-র সংখ্যা বাড়লে CIDR পরিকল্পনা হাতে করা কঠিন হয়ে যায় — কোন VPC-তে কোন range, কোথায় overlap। **Amazon VPC IP Address Manager (IPAM)** CIDR বরাদ্দ, ট্র্যাকিং ও overlap শনাক্তকরণ স্বয়ংক্রিয় করে। ছোট TechNova-তে হাতে করা যায়, কিন্তু দশটির বেশি VPC হলে IPAM ব্যবহার করুন। আর একটি গুরুত্বপূর্ণ সীমা: প্রতিটি VPC-তে সর্বোচ্চ ৫টি CIDR (primary + ৪টি secondary) যোগ করা যায় — তাই প্রথম CIDR-টি বড় করে বাছুন (/16), ছোট CIDR (/24) দিয়ে শুরু করবেন না — পরে বাড়ানো কঠিন হবে।

VPC-র ব্যর্থতা কেমন দেখায়: VPC নিজে একটি regional সার্ভিস — পুরো VPC ডাউন হওয়া প্রায় অসম্ভব। ব্যর্থতা আসে উপাদানে: একটি AZ ডাউন হলে সেই AZ-র সাবনেটের instance-গুলো যায়; NAT Gateway ডাউন হলে সেই AZ-র outbound যায়; route table-এর ভুলে পুরো সাবনেট বিচ্ছিন্ন হয়। তাই ডিজাইনের মূলনীতি: প্রতিটি AZ-তে সবকিছুর কপি রাখুন, কোনো single AZ-র ওপর নির্ভর করবেন না, আর পরিবর্তনের আগে route table-এর backup নিন (Infrastructure as Code-ই সেই backup)।

কখন VPC ব্যবহার করবেন না: শুধু একটি Lambda function বা একটি S3 bucket চালাতে চাইলে VPC দরকার নেই — Lambda default-এ VPC-র বাইরে চলে, S3 public সার্ভিস। VPC দরকার হয় যখন আপনার workload-এর private নেটওয়ার্কিং, নির্দিষ্ট IP range, বা on-premises সংযোগ দরকার। অপ্রয়োজনে VPC জটিলতা বাড়াবেন না — প্রতিটি VPC মানে route table, Security Group, Flow Logs — পরিচালনার ভার।

৬.১৩ সারাংশ

VPC হলো AWS-এর ভেতরে আপনার নিজের isolated নেটওয়ার্ক। CIDR দিয়ে ঠিকানা পরিকল্পনা করুন, পাবলিক সাবনেটে শুধু Load Balancer ও NAT Gateway রাখুন, অ্যাপ ও ডেটাবেস রাখুন প্রাইভেট সাবনেটে, অন্তত দুটি AZ ব্যবহার

করুন, এবং route table-ই হলো সত্য — সাবনেটের নাম নয়। Default VPC দিয়ে শিখুন, production-এ custom VPC ব্যবহার করুন।

৬.১৪ পর্যালোচনা প্রশ্ন

১. VPC কী এবং EC2-Classic থেকে এটি কীভাবে আলাদা? ২. 10.0.0.0/16 থেকে কয়টি /24 সাবনেট বানানো যায় এবং প্রতিটিতে কয়টি usable IP থাকে? ৩. AWS প্রতিটি সাবনেটে কোন ৫টি IP সংরক্ষণ করে? ৪. একটি সাবনেটকে "পাবলিক" করে কী — নাম, না route table? ৫. NAT Gateway কেন পাবলিক সাবনেটে রাখতে হয়? ৬. একটি AZ ডাউন হলে single NAT Gateway ডিজাইনে কী হয়? ৭. VPC Peering-এ CIDR overlap করলে কী সমস্যা হয়? ৮. Default VPC কেন production-এর জন্য উপযুক্ত নয়? ৯. Route table-এ longest-prefix match কীভাবে কাজ করে? ১০. তিনটি AZ ব্যবহারের সুবিধা কী?

৬.১৫ অনুশীলনী

অনুশীলন ১: 172.16.0.0/16 VPC-কে চারটি /24 সাবনেটে ভাগ করুন (২টি পাবলিক, ২টি প্রাইভেট, ২টি AZ) এবং প্রতিটি সাবনেটের usable IP লিখুন। **অনুশীলন ২:** CLI দিয়ে TechNova VPC-টি নিজের অ্যাকাউন্টে বানান, তারপর একটি EC2 instance পাবলিক সাবনেটে ও একটি প্রাইভেট সাবনেটে চালু করে connectivity পরীক্ষা করুন। **অনুশীলন ৩:** ইচ্ছা করে প্রাইভেট সাবনেটের route table থেকে NAT route মুছে ফেলুন, instance থেকে `curl` ব্যর্থ হতে দেখুন, তারপর route ফিরিয়ে এনে ঠিক করুন — এবং প্রতিটি ধাপে VPC Flow Logs-এ কী দেখা যায় লিখুন।

অধ্যায় ৭ • PART II — Networking & Compute

Chapter 7 — Internet Gateway, NAT and VPC Endpoints

৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Internet Gateway (IGW), NAT Gateway, NAT instance ও Egress-only Internet Gateway-এর পার্থক্য ব্যাখ্যা করতে; (২) private সাবনেট থেকে নিরাপদ outbound ইন্টারনেট অ্যাক্সেস ডিজাইন করতে; (৩) gateway endpoint (S3, DynamoDB) ও interface endpoint (PrivateLink) দিয়ে ইন্টারনেট ছাড়াই AWS সার্ভিসে পৌঁছাতে; (৪) NAT Gateway-এর খরচের হিসাব বুঝতে ও খরচ কমানোর কৌশল বলতে; এবং (৫) endpoint policy দিয়ে least-privilege অ্যাক্সেস লিখতে।

৭.২ পূর্বশর্ত ঝালাই

অধ্যায় ৬ থেকে মনে করুন: পাবলিক সাবনেটের route table-এ `0.0.0.0/0 → IGW` থাকে; প্রাইভেট সাবনেটে থাকে না। Route table-ই নির্ধারণ করে প্যাকেট কোথায় যাবে। আর মনে রাখুন: VPC-এর private IP (`10.0.x.x`) ইন্টারনেটে রাউটেবল নয় — ইন্টারনেটে যেতে হলে private IP-কে একটি public IP-তে অনুবাদ (NAT) করতে হয়। এই অধ্যায়ের সবকিছুই এই একটি প্রশ্নের উত্তর: "ভেতরের সার্ভার বাইরে যাবে কীভাবে, আর বাইরের দুনিয়া ভেতরে আসবে কীভাবে — নিরাপদে?"

৭.৩ কেন এই গেটওয়েগুলো দরকার — আগে কী সমস্যা ছিল?

একটি private সাবনেটের সার্ভারকে ধরুন: তাকে OS patch ডাউনলোড করতে হবে, `apt update` চালাতে হবে, তৃতীয় পক্ষের API-তে কল করতে হবে — সবই ইন্টারনেটের দিকে outbound ট্র্যাফিক। কিন্তু সরাসরি ইন্টারনেটে খুলে দিলে সেই সার্ভার inbound আক্রমণেরও লক্ষ্য হয়ে যায়। আবার ডেটাবেস সার্ভারের তো ইন্টারনেটে যাওয়ার দরকারই নেই — তবু অ্যাপ সার্ভারকে S3 থেকে ফাইল আনতে হয়, DynamoDB-তে লিখতে হয়। প্রতিটি প্রয়োজনে আলাদা দরজা না বানিয়ে AWS তিন ধরনের দরজা দিয়েছে: IGW (দুমুখো, পাবলিকের জন্য), NAT Gateway (একমুখী outbound, প্রাইভেটের জন্য), আর VPC Endpoint (ইন্টারনেট না ছুঁয়ে AWS সার্ভিসের জন্য)। প্রতিটি দরজার নিজস্ব উদ্দেশ্য, নিজস্ব খরচ, নিজস্ব ব্যর্থতার ধরন।

৭.৪ Internet Gateway (IGW) — প্রথম নীতি থেকে

Internet Gateway হলো VPC ও ইন্টারনেটের মধ্যে horizontally scaled, redundant, managed দরজা। এটি একটি VPC-তে একটাই থাকে। IGW দুটি কাজ করে: (১) VPC-এর private IP ও public IP/Elastic IP-এর মধ্যে NAT অনুবাদ করে; (২) route table-এর `0.0.0.0/0` route অনুযায়ী ট্র্যাফিক ইন্টারনেটে পাঠায়। গুরুত্বপূর্ণ: IGW নিজে কোনো bandwidth সীমা আরোপ করে না — সীমা নির্ধারণ করে instance-এর network performance। IGW ব্যর্থ হলে কী হয়? AWS-এর ভাষায় IGW "highly available" — একটি AZ ডাউন হলেও IGW চলে, কারণ এটি region-level সার্ভিস, কোনো single AZ-র ওপর নির্ভরশীল নয়। কিন্তু ভুল route table থাকলে IGW থাকা সত্ত্বেও ট্র্যাফিক যাবে না — route-ই সত্য।

IGW-এর সঙ্গে সম্পর্কিত আরেকটি ধারণা: **Elastic IP (EIP)** — একটি static public IPv4 ঠিকানা যা আপনি কোনো instance বা NAT Gateway-এর সঙ্গে যুক্ত করতে পারেন। Instance বন্ধ করলেও EIP আপনার অ্যাকাউন্টে থাকে (তবে ব্যবহার না করলে চার্জ হয় — AWS Pricing Calculator-এ যাচাই করুন)। IPv6-এর ক্ষেত্রে IGW-এর ভাই হলো **Egress-**

only Internet Gateway (EIGW) — শুধু IPv6 outbound ট্র্যাফিকের জন্য, inbound শুরু করতে দেয় না। IPv6-এ NAT-এর প্রয়োজন নেই (প্রতিটি ঠিকানাই globally unique), তাই EIGW শুধু "বাইরে যাওয়া যাবে, ভেতরে আসা যাবে না" — এই নীতিটি প্রয়োগ করে।

NAT (Network Address Translation): একটি private IP ঠিকানাকে public IP ঠিকানায় বদলে দেওয়ার প্রক্রিয়া। ইন্টারনেট শুধু public IP বোঝে; আপনার `10.0.11.25` সার্ভারের প্যাকেট ইন্টারনেটে যাওয়ার আগে NAT Gateway সেটিকে নিজের public IP দিয়ে প্রতিস্থাপন করে, আর উত্তর এলে আবার সঠিক private IP-তে ফিরিয়ে দেয়। এই ম্যাপিং টেবিলই NAT-এর হৃদয়।

৭.৫ NAT Gateway বনাম NAT Instance

NAT Gateway হলো AWS-managed সার্ভিস: আপনি শুধু তৈরি করেন, AWS চালায়, patch করে, scale করে। এটি একটি AZ-র ভেতরে redundant, ৪৫ Gbps পর্যন্ত burst করতে পারে, আর আপনাকে কিছু maintain করতে হয় না। **NAT instance** হলো পুরনো পদ্ধতি — আপনি নিজে একটি EC2 instance-এ NAT software চালান (community AMI)। সস্তা মনে হলেও এর দায় আপনার: instance ডাউন হলে NAT ডাউন, bandwidth instance type-এর ওপর নির্ভরশীল, patching ও monitoring আপনার। নতুন ডিজাইনে NAT instance প্রায় কখনোই সুপারিশ করা হয় না — শুধু খুব কম ট্র্যাফিকের dev পরিবেশে বা বিশেষ port-forwarding প্রয়োজনে বিবেচনা করা যেতে পারে।

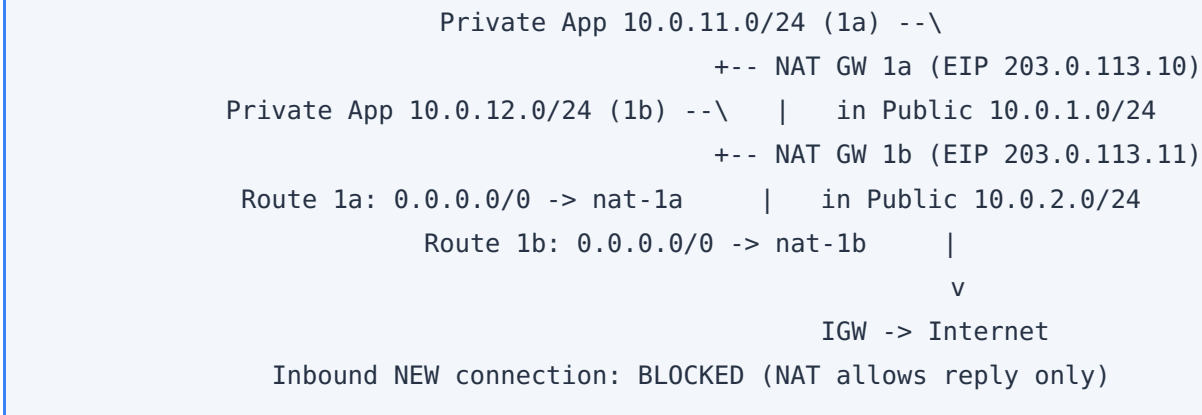
বৈশিষ্ট্য	NAT Gateway (managed)	NAT Instance (self-managed)
Availability	AZ-র ভেতরে redundant	Single instance — SPOF
Bandwidth	45 Gbps পর্যন্ত burst	Instance type-নির্ভর
Maintenance	AWS করে	আপনি করেন
খরচ	Hourly + প্রতি GB processing	EC2 খরচ + EIP
সুপারিশ	Production-এ এটিই ব্যবহার করুন	শুধু বিশেষ ক্ষেত্রে

WARNING: NAT Gateway প্রায়ই AWS বিলের সবচেয়ে বড় চমক। প্রতিটি NAT Gateway-এর hourly চার্জ আছে, তার ওপর দিয়ে যাওয়া প্রতি GB ডেটার data-processing চার্জ আছে, আর cross-AZ ট্র্যাফিকের আলাদা চার্জ আছে। তিনটি AZ-তে তিনটি NAT Gateway মানে তিনগুণ hourly চার্জ। সঠিক অঙ্ক AWS Pricing Calculator-এ যাচাই করুন — কখনো অনুমান দিয়ে বাজেট করবেন না। বিলে "NAT Gateway" লাইনটি প্রতি মাসে পরীক্ষা করুন।

৭.৬ NAT Gateway ডিজাইন — প্রতি AZ-তে একটি

সঠিক ডিজাইন: প্রতিটি AZ-র পাবলিক সাবনেটে একটি NAT Gateway, আর সেই AZ-র প্রাইভেট সাবনেটের route table-এ `0.0.0.0/0 → সেই AZ-র NAT Gateway`। এতে একটি AZ ডাউন হলে অন্য AZ-র ট্র্যাফিক চলতে থাকে। ভুল ডিজাইন: একটি NAT Gateway শেয়ার করা — সস্তা, কিন্তু AZ ডাউন হলে সব প্রাইভেট সাবনেট ইন্টারনেট হারায়, আর cross-AZ data transfer চার্জ বাড়ে। Dev পরিবেশে একটিমাত্র NAT Gateway গ্রহণযোগ্য হতে পারে; production-এ নয়।

NAT Gateway — Per-AZ Design (TechNova)




```
# NAT Gateway তৈরি (প্রতি AZ-তে একটি)
# প্রথমে Elastic IP বরাদ্দ করুন
aws ec2 allocate-address --domain vpc \
  --tag-specifications \
  'ResourceType=elastic-ip,Tags=[{Key=Name,Value=technova-nat-1a}]'
# Output-এর AllocationId নিন, যেমন eipalloc-0abc1234

# NAT Gateway তৈরি — পাবলিক সাবনেটে
aws ec2 create-nat-gateway \
  --subnet-id subnet-0abc1234def56789 \
  --allocation-id eipalloc-0abc1234def56789 \
  --tag-specifications \
  'ResourceType=natgateway,Tags=[{Key=Name,Value=technova-nat-1a}]'

# তৈরি হতে 2-5 মিনিট লাগে; অবস্থা দেখুন
aws ec2 describe-nat-gateways \
  --nat-gateway-ids nat-0abc1234def56789 \
  --query 'NatGateways[0].State'

# প্রাইভেট route table-এ NAT route যোগ করুন
aws ec2 create-route --route-table-id rtb-0abc1234def56789 \
  --destination-cidr-block 0.0.0.0/0 \
  --nat-gateway-id nat-0abc1234def56789
```

৭.৭ VPC Endpoints — ইন্টারনেট না ছুঁয়ে AWS সার্ভিস

এবার ভাবুন: আপনার প্রাইভেট অ্যাপ সার্ভারকে S3 থেকে একটি ফাইল আনতে হবে। NAT Gateway দিয়ে গেলে ট্র্যাফিক ইন্টারনেট ঘুরে আসে — ধীর, খরচসাপেক্ষ, আর আক্রমণের সুযোগ বেশি। **VPC Endpoint** এই সমস্যার সমাধান: VPC-র ভেতর থেকেই সরাসরি AWS সার্ভিসে যাওয়ার private রাস্তা। দুই ধরনের endpoint আছে।

৭.৭.১ Gateway Endpoint — S3 ও DynamoDB

Gateway endpoint শুধু S3 ও DynamoDB-র জন্য। এটি route table-এ একটি prefix-list route যোগ করে — যেমন S3-র জন্য `pl-xxxx → vpce-xxxx`। কোনো ENI লাগে না, কোনো hourly চার্জ নেই (বিনামূল্যে)। TechNova-র প্রতিটি প্রাইভেট route table-এ S3 gateway endpoint থাকা উচিত — এতে S3 ট্র্যাফিক NAT Gateway এড়িয়ে যায়, NAT বিল কমে, latency কমে।

৭.৭.২ Interface Endpoint — AWS PrivateLink

Interface endpoint অন্য সব AWS সার্ভিসের জন্য (EC2 API, SSM, ECR, Secrets Manager, CloudWatch Logs ইত্যাদি) এবং তৃতীয় পক্ষের SaaS-এর জন্য। এটি আপনার সাবনেটে একটি ENI তৈরি করে, যার একটি private IP থাকে — যেমন `10.0.11.50`। আপনার সার্ভার `ssm.ap-south-1.amazonaws.com`-কে DNS-এর মাধ্যমে সেই private IP-তে resolve করে। Interface endpoint-এর hourly চার্জ ও প্রতি GB চার্জ আছে (AWS Pricing Calculator-এ যাচাই করুন), তাই শুধু প্রয়োজনীয় সার্ভিসের endpoint বানান। Private DNS চালু রাখুন, নইলে endpoint-এর DNS নাম কাজ করবে না।

```
# S3 Gateway Endpoint (বিনামূল্যে, route table-এ যোগ হয়)
aws ec2 create-vpc-endpoint --vpc-id vpc-0abc1234def56789 \
  --service-name com.amazonaws.ap-south-1.s3 \
  --route-table-ids rtb-0abc1234def56789 \
  --tag-specifications \
  'ResourceType=vpc-endpoint,Tags=[{Key=Name,Value=technova-s3}]'

# SSM Interface Endpoint (Session Manager-এর জন্য জরুরি)
aws ec2 create-vpc-endpoint --vpc-id vpc-0abc1234def56789 \
  --service-name com.amazonaws.ap-south-1.ssm \
  --vpc-endpoint-type Interface \
  --subnet-ids subnet-0abc1234def56789 subnet-0abc1234def56789 \
  --security-group-ids sg-0abc1234def56789 \
  --private-dns-enabled
```

> **PRO TIP:** Endpoint policy দিয়ে least privilege প্রয়োগ করুন। S3 gateway endpoint-এ এমন policy দিন যাতে শুধু TechNova-র bucket (`arn:aws:s3:::technova-app-data`) থেকেই পড়া যায় — অন্য কোনো bucket নয়। এতে কোনো instance compromise হলেও আক্রমণকারী আপনার endpoint দিয়ে অন্য bucket-এ পৌঁছাতে পারবে না।

৭.৮ খরচ কমানোর কৌশল

NAT Gateway-এর বিল কমাতে: (১) S3 ও DynamoDB-র জন্য gateway endpoint ব্যবহার করুন — এই ট্র্যাফিক NAT এড়িয়ে যাবে; (২) ECR, SSM, CloudWatch-এর জন্য interface endpoint বিবেচনা করুন — ছোট ট্র্যাফিকে NAT-এর চেয়ে সস্তা হতে পারে, হিসাব করে দেখুন; (৩) dev পরিবেশে একটিমাত্র NAT Gateway বা NAT instance ব্যবহার করুন; (৪) বড় ডাউনলোড (Docker image, OS patch) ক্যাশ করুন — প্রতিবার NAT দিয়ে টানা মানে প্রতিবার data-processing চার্জ; (৫) VPC Flow Logs দিয়ে দেখুন কোন instance সবচেয়ে বেশি NAT ট্র্যাফিক তৈরি করছে — অনেক সময় একটি misconfigured লগ shipper বা metrics agent-ই বিলের মূল কারণ হয়।

৭.৯ নিরাপত্তা বিবেচনা ও সাধারণ ভুল

নিরাপত্তা: NAT Gateway নিজে কোনো security group নেয় না — এটি managed। Interface endpoint-এর security group-এ শুধু প্রয়োজনীয় port (সাধারণত 443) ও শুধু VPC-র CIDR থেকে allow করুন। Endpoint policy-তে `Principal` সীমিত করুন। **সাধারণ ভুল:** (১) NAT Gateway প্রাইভেট সাবনেটে বানানো — এটি কাজ করবে না, NAT Gateway-কে পাবলিক সাবনেটে থাকতে হয় ও তার EIP লাগে; (২) route table-এ NAT route দেওয়ার আগে NAT Gateway "available" হওয়ার অপেক্ষা না করা; (৩) interface endpoint-এর Private DNS বন্ধ রাখা — তখন DNS public IP-তে resolve হয় ও ট্র্যাফিক ইন্টারনেট ঘুরে যায়; (৪) S3 endpoint থাকা সত্ত্বেও bucket policy-তে VPC endpoint condition না দেওয়া।

৭.১০ সমস্যা সমাধান

Private instance থেকে S3-তে পৌঁছানো যাচ্ছে না? চেকলিস্ট: (১) route table-এ S3 prefix-list route আছে কি? (২) endpoint policy কি bucket-কে allow করে? (৩) bucket policy কি VPC endpoint-কে deny করছে? (৪) Security Group outbound 443 খোলা কি? NAT দিয়ে ইন্টারনেট যাচ্ছে না? (১) NAT Gateway state "available" কি? (২) route table-এ সঠিক AZ-র NAT Gateway দেওয়া আছে কি? (৩) NAT Gateway-এর EIP associate হয়েছে কি? (৪) instance-এর Security Group outbound খোলা কি? CloudWatch-এ NAT Gateway-এর `ErrorPortAllocation` metric দেখুন — port exhaustion হলে নতুন connection ব্যর্থ হয়; তখন NAT Gateway-এর সংখ্যা বাড়াতে বা connection pooling করতে হয়।

৭.১১ VPC Endpoint-এর নিরাপত্তা মডেল ও PrivateLink-এর গভীরে

Interface endpoint শুধু একটি private রাস্তা নয় — এটি একটি নিরাপত্তা সীমানাও। প্রতিটি interface endpoint-এর নিজস্ব **Security Group** থাকে: শুধু প্রয়োজনীয় port (সাধারণত HTTPS 443) ও শুধু VPC-র CIDR থেকে allow করুন। আর **endpoint policy** হলো একটি IAM resource policy — এতে লিখুন কোন principal কোন action কোন resource-এ করতে পারবে। উদাহরণ: ECR endpoint-এর policy-তে শুধু TechNova-র repository থেকে image pull-এর অনুমতি দিন; SSM endpoint-এর policy-তে শুধু নির্দিষ্ট role-কে অনুমতি দিন। Endpoint policy না দিলে default policy সবকিছু allow করে — অর্থাৎ endpoint থাকা সত্ত্বেও যেকোনো instance যেকোনো AWS resource-এ পৌঁছাতে পারে, যদি তার IAM role অনুমতি দেয়। Least privilege-এর জন্য endpoint policy ও IAM policy — দুটোই সীমিত করুন।

PrivateLink-এর তৃতীয় পক্ষের দিক: Interface endpoint শুধু AWS সার্ভিসের জন্য নয় — তৃতীয় পক্ষের SaaS (যেমন Datadog, Snowflake, MongoDB Atlas) AWS PrivateLink দিয়ে তাদের সার্ভিস আপনার VPC-তে প্রকাশ করতে পারে। তখন আপনার সার্ভার ইন্টারনেট না ছুঁয়ে সেই SaaS-এ পৌঁছায় — latency কমে, ডেটা ইন্টারনেটে বেরোয় না। SaaS-এর endpoint service-এর নাম দিয়ে আপনার VPC-তে endpoint বানান — তবে মনে রাখবেন, তৃতীয় পক্ষের endpoint-এরও hourly + GB চার্জ আছে।

Gateway endpoint-এর সীমা: Gateway endpoint শুধু S3 ও DynamoDB-র জন্য, আর শুধু একই region-এর resource-এ কাজ করে। অন্য region-এর S3 bucket-এ যেতে হলে NAT Gateway বা interface endpoint (S3 interface endpoint) দরকার। আর gateway endpoint route table-ভিত্তিক — endpoint বানানোর সময় যে route table দেননি, সেই সাবনেট থেকে S3-তে যাওয়া যাবে না। নতুন সাবনেট বানালে তার route table-এ gateway endpoint-এর route যোগ করতে ভুলবেন না — এটিই সবচেয়ে সাধারণ ভুল।

কখন endpoint ব্যবহার করবেন না: Public সাবনেটের instance-এর জন্য endpoint দরকার নেই — তার IGW route আছেই। খুব কম ট্র্যাফিকের dev পরিবেশে প্রতিটি সার্ভিসের interface endpoint বানানো NAT Gateway-এর চেয়ে দামি হতে পারে — হিসাব করে দেখুন: interface endpoint-এর hourly চার্জ × সার্ভিসের সংখ্যা বনাম NAT Gateway-এর data-processing চার্জ। Production-এ নিরাপত্তা সাধারণত খরচের চেয়ে গুরুত্বপূর্ণ — private connectivity-ই সঠিক পছন্দ।

৭.১২ সারাংশ

IGW পাবলিক সাবনেটের দুমুখো দরজা; NAT Gateway প্রাইভেট সাবনেটের একমুখী outbound দরজা; EIGW IPv6-এর একমুখী দরজা; VPC Endpoint ইন্টারনেট না ছুঁয়ে AWS সার্ভিসে যাওয়ার private রাস্তা। Production-এ প্রতি AZ-তে NAT Gateway, S3/DynamoDB-র জন্য gateway endpoint, আর প্রয়োজনীয় সার্ভিসের জন্য interface endpoint — এই তিনটি মিলিয়ে ডিজাইন করুন, আর NAT বিলের দিকে নজর রাখুন।

৭.১৩ পর্যালোচনা প্রশ্ন

১. IGW ও NAT Gateway-এর মূল পার্থক্য কী? ২. NAT Gateway কেন পাবলিক সাবনেটে থাকতে হয়? ৩. Egress-only Internet Gateway কখন ব্যবহার করবেন? ৪. Gateway endpoint ও interface endpoint-এর পার্থক্য কী? ৫. NAT Gateway-এর খরচের দুটি উপাদান কী কী? ৬. S3 gateway endpoint থাকলে NAT বিল কীভাবে কমে? ৭. Interface endpoint-এর Private DNS বন্ধ থাকলে কী হয়? ৮. একটি AZ ডাউন হলে single NAT Gateway ডিজাইনে কী হয়? ৯. Endpoint policy দিয়ে কীভাবে least privilege প্রয়োগ করবেন? ১০. NAT port exhaustion-এর লক্ষণ কী?

৭.১৪ অনুশীলনী

অনুশীলন ১: TechNova VPC-তে একটি NAT Gateway বানিয়ে প্রাইভেট instance থেকে `curl https://example.com` চালান; তারপর NAT route মুছে আবার চেষ্টা করুন — পার্থক্য লিখুন। **অনুশীলন ২:** S3 gateway endpoint যোগ করে `aws s3 ls`-এর ট্র্যাফিক NAT এড়িয়ে যাচ্ছে কি না VPC Flow Logs দিয়ে যাচাই করুন। **অনুশীলন ৩:** SSM interface endpoint বানিয়ে NAT Gateway ছাড়াই Session Manager-এ সংযোগ করুন — প্রমাণ করুন ইন্টারনেট ছাড়া ব্যবস্থাপনা সম্ভব।

অধ্যায় ৮ • PART II — Networking & Compute

Chapter 8 — Network Security and Hybrid Connectivity

৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Security Group ও Network ACL-এর পার্থক্য ব্যাখ্যা করতে এবং stateful বনাম stateless filtering-এর তুলনা করতে; (২) Route 53 দিয়ে DNS রেকর্ড, health check ও routing policy ডিজাইন করতে; (৩) VPC Peering, Transit Gateway, Site-to-Site VPN ও Direct Connect-এর মধ্যে সঠিক hybrid connectivity বাছতে; (৪) VPC Flow Logs ও Reachability Analyzer দিয়ে নেটওয়ার্ক সমস্যার মূল কারণ খুঁজে বের করতে; এবং (৫) defense-in-depth নীতিতে স্তরে স্তরে নিরাপত্তা সাজাতে।

৮.২ পূর্বশর্ত ঝালাই

অধ্যায় ৬-৭ থেকে মনে করুন: route table প্যাকেটের পথ নির্ধারণ করে; IGW ও NAT Gateway দরজার কাজ করে। এবার প্রশ্ন হলো: দরজা থাকলেই কি যে কেউ ঢুকতে পারবে? না — দরজায় পাহারা দরকার। AWS-এ পাহারা দুই স্তরে: **Security Group** (instance-level, stateful) ও **Network ACL** (subnet-level, stateless)। আর মনে রাখুন OSI মডেলের কথা: Security Group মূলত transport layer (port/protocol) ও network layer (IP) — এ কাজ করে। DNS-এর মূল ধারণাও ঝালিয়ে নিন: মানুষ নাম মনে রাখে (`app.technova.example`), কম্পিউটার IP বোঝে — DNS নামকে IP-তে বদলায়।

৮.৩ কেন defense-in-depth — আগে কী সমস্যা ছিল?

On-premises যুগে নিরাপত্তা মানে ছিল একটি বড় perimeter firewall — "ভেতরে সবাই বিশ্বস্ত, বাইরে সবাই শত্রু"। একবার আক্রমণকারী ভেতরে ঢুকে গেলে আর কোনো বাধা থাকত না — lateral movement ছিল সহজ। Cloud-এ এই মডেল ভেঙে গেছে: workload ছড়িয়ে আছে, IP বদলায়, instance তৈরি-মুছে যায়। তাই AWS-এর নীতি **defense-in-depth**: প্রতিটি স্তরে আলাদা নিয়ন্ত্রণ — subnet-এ NACL, instance-এ Security Group, অ্যাপে IAM, ডেটায় encryption। একটি স্তর ভেঙে গেলেও পরের স্তর আটকায়। TechNova-র ডেটাবেসের কথা ভাবুন: NACL শুধু অ্যাপ সাবনেট থেকে 5432 port খোলে, Security Group শুধু অ্যাপের Security Group থেকে allow করে, আর IAM শুধু অ্যাপের role-কে ডেটাবেসে সংযোগের অনুমতি দেয়। তিনটি স্তর, তিনটি ভিন্ন ব্যর্থতার ধরন।

৮.৪ Security Groups — প্রথম নীতি থেকে

Security Group হলো একটি virtual firewall যা ENI-level-এ কাজ করে। এর মূল বৈশিষ্ট্য: (১) **stateful** — outbound request-এর reply স্বয়ংক্রিয়ভাবে allow হয়, return traffic-এর জন্য আলাদা নিয়ম লাগে না; (২) **allow-only** — deny নিয়ম লেখা যায় না, যা allow করা হয়নি তা-ই deny; (৩) **default deny** — নতুন Security Group-এ সব inbound বন্ধ থাকে; (৪) নিয়ম লেখা যায় IP CIDR দিয়ে বা অন্য Security Group-এর reference দিয়ে — দ্বিতীয়টি বেশি শক্তিশালী, কারণ IP বদলালেও reference ঠিক থাকে।

উদাহরণ: TechNova-র অ্যাপ সার্ভারের Security Group-এ inbound নিয়ম হবে `ALB Security Group থেকে port 8080` — কোনো IP নয়, ALB-র Security Group-এর ID। এতে ALB-র IP বদলালেও নিয়ম ভাঙে না। আর ডেটাবেসের Security Group-এ inbound হবে `অ্যাপ Security Group থেকে port 5432` এই chaining-ই cloud-native নিরাপত্তার হৃদয়। মনে রাখবেন: Security Group-এর নিয়ম পরিবর্তন সঙ্গে সঙ্গে কার্যকর হয় — কোনো restart লাগে না।

```
# অ্যাপ সার্ভারের Security Group তৈরি
aws ec2 create-security-group \
  --group-name technova-app-sg \
  --description "TechNova app servers" \
  --vpc-id vpc-0abc1234def56789
# Output থেকে GroupId নিন, যেমন sg-0abc1234def56789

# ALB-র Security Group থেকে 8080 allow করুন
aws ec2 authorize-security-group-ingress \
  --group-id sg-0abc1234def56789 \
  --protocol tcp --port 8080 \
  --source-group sg-0abc1234def56789

# Outbound: সবকিছু allow (default) — প্রয়োজনে সীমিত করুন
aws ec2 describe-security-groups \
  --group-ids sg-0abc1234def56789
```

৮.৫ Network ACLs — Subnet-Level Stateless Filtering

Network ACL (NACL) সাবনেট-level-এ কাজ করে এবং **stateless** — return traffic-এর জন্য আলাদা outbound নিয়ম লাগে। NACL-এ নিয়মের নম্বর থাকে (১০০, ২০০...); সবচেয়ে ছোট নম্বর প্রথমে মূল্যায়ন হয়। NACL-এ allow ও deny — দুটোই লেখা যায়। Default NACL সবকিছু allow করে; custom NACL সবকিছু deny করে শুরু হয়। কখন NACL ব্যবহার করবেন? যখন একটি পুরো সাবনেটকে একসঙ্গে block করতে চান — যেমন একটি পরিচিত malicious IP range থেকে সব ট্র্যাফিক deny করা, বা একটি সাবনেট থেকে নির্দিষ্ট port-এর outbound বন্ধ করা। দৈনন্দিন কাজে Security Group-ই যথেষ্ট; NACL হলো দ্বিতীয় স্তরের ঢাল।

বৈশিষ্ট্য	Security Group (stateful)	Network ACL (stateless)
স্তর	Instance / ENI	Subnet
State	Stateful — reply স্বয়ংক্রিয়	Stateless — reply-র নিয়ম আলাদা
নিয়ম	শুধু Allow	Allow ও Deny
মূল্যায়ন	সব নিয়ম একসঙ্গে	নম্বর অনুযায়ী ক্রমান্বয়ে
Default	সব inbound deny	Custom NACL: সব deny
সাধারণ ব্যবহার	প্রতিদিনের firewall	Subnet-wide block, দ্বিতীয় স্তর

TROUBLE: "Security Group খোলা, তবু সংযোগ হচ্ছে না" — এর সবচেয়ে সাধারণ কারণ NACL। মনে করুন: আপনি inbound 443 allow করেছেন Security Group-এ, কিন্তু NACL-এর outbound-এ ephemeral port (1024–65535) বন্ধ আছে — তাহলে reply প্যাকেট ফিরতে পারবে না। Stateful বনাম stateless-এর এই পার্থক্যটি মনে রাখুন: NACL-এ সবসময় return traffic-এর কথা ভাবুন।

৮.৬ Route 53 — DNS Fundamentals

Route 53 AWS-এর DNS সার্ভিস — নামটি এসেছে DNS port 53 থেকে। এটি দুটি কাজ করে: (১) **domain registration** ও (২) **DNS routing**। একটি **hosted zone** হলো একটি ডোমেইনের DNS রেকর্ডের সংগ্রহ। Public hosted zone ইন্টারনেটের জন্য; **private hosted zone** শুধু আপনার VPC-র ভেতরে কাজ করে — যেমন `db.internal.technova.example` শুধু VPC-র ভেতর থেকে resolve হয়, বাইরে থেকে নয়। এটি service discovery-র সহজ উপায়।

গুরুত্বপূর্ণ রেকর্ড: **A record** নাম → IPv4; **AAAA** নাম → IPv6; **CNAME** নাম → অন্য নাম; **Alias record** Route 53-এর বিশেষত্ব — ALB বা CloudFront-এর মতো AWS resource-এর দিকে সরাসরি নির্দেশ করে, কোনো IP ছাড়াই, আর বিনামূল্যে (query চার্জ ছাড়া)। ALB-র IP বদলায় — তাই ALB-র জন্য কখনো A record-এ IP লিখবেন না; Alias record ব্যবহার করুন। **Routing policy**-গুলো: Simple (একটাই উত্তর), Weighted (ট্র্যাফিক ভাগ), Latency-based (সবচেয়ে কাছের region), Failover (primary ডাউন হলে secondary), Geolocation (ব্যবহারকারীর দেশ অনুযায়ী)। Health check-এর সঙ্গে Failover মিলিয়ে Route 53 নিজেই একটি global load balancer-এর মতো কাজ করতে পারে।

```
# Private hosted zone তৈরি (VPC-র ভেতরে)
aws route53 create-hosted-zone \
  --name internal.technova.example \
  --vpc VPCRegion=ap-south-1,VPCId=vpc-0abc1234def56789 \
  --caller-reference technova-internal-001

# ALB-র দিকে Alias record (public hosted zone-এ)
# Change batch JSON ফাইলে লিখে change-resource-record-sets চালান
aws route53 change-resource-record-sets \
  --hosted-zone-id Z0abc1234def56789 \
  --change-batch file://alias-change.json
```

৮.৭ Hybrid Connectivity — VPC Peering, Transit Gateway, VPN, Direct Connect

একটি VPC একা থাকে না — কোম্পানির অন্য VPC, on-premises ডেটা সেন্টার, বা তৃতীয় পক্ষের নেটওয়ার্কের সঙ্গে যুক্ত হতে হয়। চারটি পথ আছে, প্রতিটির নিজস্ব জায়গা।

৮.৭.১ VPC Peering

দুটি VPC-র মধ্যে private সংযোগ। Non-transitive — A peer B, B peer C হলে A C-তে যেতে পারে না। CIDR overlap করা যাবে না। ছোট, দুটি VPC-র সংযোগের জন্য সহজ ও সস্তা। কিন্তু VPC-র সংখ্যা বাড়লে peering mesh জটিল হয়ে যায় (nটি VPC-তে $n(n-1)/2$টি peering লাগে)।

৮.৭.২ Transit Gateway

একটি central hub — অনেক VPC ও on-premises নেটওয়ার্ক একটাই Transit Gateway-এ যুক্ত হয়। Transitive routing সম্ভব, route table দিয়ে নিয়ন্ত্রণ করা যায় কোন VPC কোন VPC-তে যেতে পারবে। অনেক VPC থাকলে এটাই সঠিক পছন্দ। খরচ: hourly চার্জ + প্রতি GB data-processing চার্জ — AWS Pricing Calculator-এ যাচাই করুন।

৮.৭.৩ Site-to-Site VPN

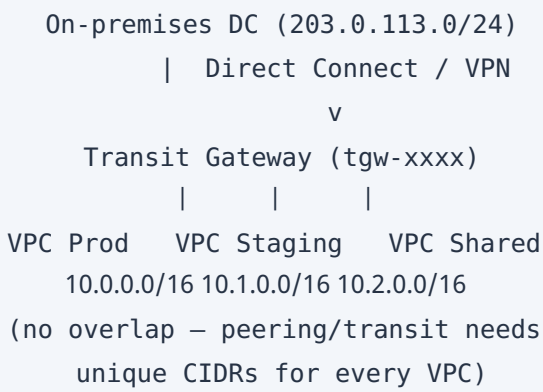
ইন্টারনেটের ওপর দিয়ে encrypted IPsec tunnel — on-premises থেকে VPC-তে। দ্রুত সেটআপ করা যায়, কিন্তু bandwidth ইন্টারনেট-নির্ভর ও পরিবর্তনশীল। দুটি tunnel থাকে redundancy-র জন্য।

৮.৭.৪ Direct Connect

AWS ডেটা সেন্টারে সরাসরি dedicated private লাইন — স্থিতিশীল latency, বেশি bandwidth, ইন্টারনেট এড়িয়ে যাওয়া। সেটআপে সপ্তাহ লাগে, খরচ বেশি, কিন্তু production hybrid-এর জন্য সবচেয়ে নির্ভরযোগ্য। Direct Connect Gateway দিয়ে একাধিক region-এ যুক্ত হওয়া যায়।

পদ্ধতি	কখন ব্যবহার করবেন	সীমা
VPC Peering	দুটি VPC-র সরল সংযোগ	Non-transitive, overlap নিষেধ
Transit Gateway	অনেক VPC + on-prem hub	Hourly + GB চার্জ
Site-to-Site VPN	দ্রুত hybrid, কম bandwidth	ইন্টারনেট-নির্ভর latency
Direct Connect	স্থিতিশীল, বেশি bandwidth	সেটআপ সময় ও খরচ বেশি

TechNova Hybrid — Transit Gateway Hub



৮.৮ নেটওয়ার্ক ট্রাবলশুটিং — Flow Logs ও Reachability Analyzer

VPC Flow Logs আপনার VPC-র নেটওয়ার্ক ট্র্যাফিকের রেকর্ড — কোন IP থেকে কোন IP-তে, কোন port-এ, কত bytes, accept না reject। Flow Logs CloudWatch Logs বা S3-তে যায়। এটি দিয়ে আপনি দেখতে পারেন কোন Security Group বা NACL প্যাকেট reject করছে। মনে রাখবেন: Flow Logs-এ প্যাকেটের content থাকে না, শুধু metadata। **Reachability Analyzer** দুটি endpoint-এর মধ্যে পথ বিশ্লেষণ করে — source থেকে destination পর্যন্ত প্রতিটি hop (route table, Security Group, NACL, IGW, NAT) পরীক্ষা করে বলে দেয় পথটি reachable কি না, আর না হলে ঠিক কোন component বাধা দিচ্ছে। এটি "কেন সংযোগ হচ্ছে না" প্রশ্নের সবচেয়ে দ্রুত উত্তর।

```
# VPC Flow Logs চালু (CloudWatch Logs-এ)
aws ec2 create-flow-logs \
  --resource-type VPC \
  --resource-ids vpc-0abc1234def56789 \
  --traffic-type ALL \
```

```
  --log-destination-type cloud-watch-logs \
  --log-group-name technova-vpc-flow-logs \
  --deliver-logs-permission-arn \
  arn:aws:iam::111122223333:role/technova-flow-logs-role

# Reachability Analyzer: পথ পরীক্ষা
aws ec2 create-network-insights-path \
  --source subnet-0abc1234def56789 \
  --destination 203.0.113.10 \
  --protocol tcp --destination-port 443
```

৮.৯ নিরাপত্তা, খরচ ও সাধারণ ভুল

নিরাপত্তা: কখনো `0.0.0.0/0` থেকে SSH (22) বা RDP (3389) খুলবেন না — Session Manager ব্যবহার করুন (অধ্যায় ৯)। Security Group-এ "Anywhere" নিয়ম নিয়মিত audit করুন (AWS Config বা Security Hub দিয়ে)। Flow Logs সব VPC-তে চালু রাখুন — incident-এর পরে লগ না থাকলে তদন্ত অসম্ভব। **খরচ:** Transit Gateway ও Direct Connect-এর hourly + data চার্জ আছে; Flow Logs-এর CloudWatch ingestion চার্জ আছে — বেশি ট্র্যাফিকের VPC-তে S3 destination সস্তা হতে পারে; Route 53-এর hosted zone ও query চার্জ আছে — AWS Pricing Calculator-এ যাচাই করুন। **সাধারণ ভুল:** (১) Security Group-এ IP দিয়ে নিয়ম লেখা, SG reference নয়; (২) NACL-এ return port ভুলে যাওয়া; (৩) ALB-র জন্য A record-এ IP লেখা; (৪) VPC Peering-এ transitive routing আশা করা; (৫) Flow Logs ছাড়া production চালানো।

৮.১০ Production Scenario — TechNova-র Incident

একদিন সকালে TechNova-র অ্যাপ সার্ভার থেকে ডেটাবেসে সংযোগ বন্ধ হয়ে গেল। চেকলিস্ট অনুযায়ী: (১) Reachability Analyzer চালানো হলো — পথটি "not reachable", বাধা Security Group; (২) দেখা গেল কেউ ডেটাবেসের Security Group-এ অ্যাপ SG-র reference মুছে IP দিয়ে বসিয়েছিল, আর Auto Scaling-এ নতুন instance-এর IP বদলে গিয়েছিল; (৩) SG reference ফিরিয়ে দেওয়া হলো, সংযোগ ফিরল। শিক্ষা: SG-তে কখনো IP লিখবেন না — reference ব্যবহার করুন, আর পরিবর্তন CloudTrail-এ audit করুন।

৮.১১ DNS নিরাপত্তা, Resolver ও Network Firewall — গভীরে

DNS শুধু নাম resolve করার সার্ভিস নয় — এটি আক্রমণেরও একটি পথ। **DNS exfiltration**-এ আক্রমণকারী DNS query-র ভেতরে ডেটা লুকিয়ে বাইরে পাঠায়; **DNS tunneling**-এ DNS-কে covert channel হিসেবে ব্যবহার করে। প্রতিরোধ: **Route 53 Resolver DNS Firewall** দিয়ে শুধু অনুমোদিত ডোমেইন resolve করতে দিন — যেমন TechNova-র instance শুধু `*.amazonaws.com` ও অনুমোদিত API ডোমেইন resolve করতে পারবে, অন্য কোনো ডোমেইন নয়। DNS query log CloudWatch-এ পাঠিয়ে অস্বাভাবিক query pattern (যেমন হঠাৎ হাজার হাজার unique subdomain) শনাক্ত করুন — GuardDuty DNS-ভিত্তিক threat detection-ও করে।

Route 53 Resolver endpoint: Hybrid পরিবেশে on-premises DNS ও VPC DNS-এর মধ্যে সংযোগ দরকার। **Inbound endpoint** দিয়ে on-premises থেকে VPC-র private hosted zone resolve করা যায়; **outbound endpoint** দিয়ে VPC থেকে on-premises DNS-এ query পাঠানো যায়। Forwarding rule-এ লিখুন কোন ডোমেইন (`corp.technova.example`) কোন DNS server-এ যাবে। এটি ছাড়া hybrid DNS কাজ করে না — আর DNS না কাজ করলে কোনো সংযোগই কাজ করে না, কারণ সবকিছু নাম দিয়ে শুরু হয়।

AWS Network Firewall: Security Group ও NACL Layer 4-এ কাজ করে; Network Firewall Layer 7-পর্যন্ত stateful inspection করে — domain-based filtering, intrusion prevention (Suricata-compatible rule), TLS inspection। VPC-র central inspection point হিসেবে Transit Gateway-এর সঙ্গে মিলিয়ে ব্যবহার করুন: সব VPC-র outbound ট্র্যাফিক Network Firewall-এর মধ্যে দিয়ে যাবে। খরচ বেশি (hourly + GB) — শুধু তখনই ব্যবহার করুন যখন compliance বা deep inspection দরকার; সাধারণ ওয়েব অ্যাপে Security Group + WAF যথেষ্ট।

নেটওয়ার্ক নিরাপত্তার চেকলিস্ট (TechNova): (১) কোনো Security Group-এ 0.0.0.0/0 থেকে 22/3389 নেই; (২) Flow Logs সব VPC-তে চালু; (৩) GuardDuty চালু — এটি VPC Flow Logs, DNS log ও CloudTrail বিশ্লেষণ করে threat শনাক্ত করে; (৪) Security Group পরিবর্তন CloudTrail-এ audit হয়; (৫) প্রতি মাসে Security Hub-এর finding পর্যালোচনা করা হয়। নিরাপত্তা একটি প্রকল্প নয় — এটি একটি অভ্যাস।

৮.১২ সারাংশ

Security Group stateful, instance-level, allow-only; NACL stateless, subnet-level, allow+deny। Route 53 শুধু DNS নয় — health check ও failover-সহ global routing। দুটি VPC-তে Peering, অনেক VPC-তে Transit Gateway, দ্রুত hybrid-এ VPN, স্থিতিশীল hybrid-এ Direct Connect। আর সমস্যা হলে Flow Logs দেখুন, Reachability Analyzer চালান — অনুমান নয়, প্রমাণ দিয়ে তদন্ত করুন।

৮.১৩ পর্যালোচনা প্রশ্ন

১. Stateful ও stateless filtering-এর পার্থক্য কী? ২. Security Group-এ deny নিয়ম লেখা যায় কি? কেন? ৩. NACL-এ return traffic-এর জন্য কী করতে হয়? ৪. ALB-র জন্য Alias record কেন ব্যবহার করবেন? ৫. Private hosted zone কখন ব্যবহার করবেন? ৬. VPC Peering কেন non-transitive? ৭. Transit Gateway কখন Peering-এর চেয়ে ভালো? ৮. VPN ও Direct Connect-এর মূল পার্থক্য কী? ৯. VPC Flow Logs-এ কী থাকে, কী থাকে না? ১০. Reachability Analyzer কীভাবে কাজ করে?

৮.১৪ অনুশীলনী

অনুশীলন ১: একটি অ্যাপ SG ও ডেটাবেস SG বানিয়ে SG reference দিয়ে 5432 খুলুন; তারপর IP দিয়ে নিয়ম বদলে Auto Scaling-এ IP বদলের সমস্যাটি নিজে দেখুন। **অনুশীলন ২:** একটি private hosted zone বানিয়ে `db.internal.technova.example` নামটি VPC-র ভেতর থেকে resolve করুন। **অনুশীলন ৩:** ইচ্ছা করে NACL-এ outbound ephemeral port বন্ধ করে অ্যাপের সংযোগ ভাঙুন, Flow Logs-এ reject দেখুন, Reachability Analyzer দিয়ে বাধাটি শনাক্ত করুন।

অধ্যায় ৯ • PART II — Networking & Compute

Chapter 9 — Amazon EC2 Engineering

৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) EC2 instance family ও type বাছাই করতে workload অনুযায়ী; (২) AMI, key pair, EBS, User Data ও instance metadata-র কাজ ব্যাখ্যা করতে; (৩) IMDSv2 কেন SSRF আক্রমণ থেকে রক্ষা করে তা বলতে; (৪) Session Manager দিয়ে SSH port না খুলে instance পরিচালনা করতে; (৫) hardened User Data লিখতে; এবং (৬) production-এ নিরাপদ EC2 কনফিগারেশন ডিজাইন করতে।

৯.২ পূর্বশর্ত ঝালাই

অধ্যায় ৬-৮ থেকে মনে করুন: instance একটি সাবনেটে বসে, ENI-এর মাধ্যমে নেটওয়ার্ক পায়, Security Group তার firewall। এবার instance-এর ভেতরে ঢুকি। মনে রাখুন Linux-এর মূল ধারণা: SSH দিয়ে remote shell, `cloud-init` দিয়ে প্রথম boot-এ script চালানো, আর block storage (হার্ড ডিস্ক) বনাম instance store (অস্থায়ী ডিস্ক)-এর পার্থক্য। Volume 9-এ আপনি on-premises সার্ভার provisioning শিখেছেন — EC2 সেই একই সার্ভার, কিন্তু API কলে সেকেন্ডে তৈরি হয়, আর আপনি শুধু ভাড়া দেন যতক্ষণ চালান।

৯.৩ কেন EC2 — আগে কী সমস্যা ছিল?

২০০৬-এর আগে সার্ভার মানে ছিল hardware কেনা: অর্ডার দিতে সপ্তাহ, ডেলিভারি, র‍্যাকে বসানো, OS ইনস্টল — মোট ছয় থেকে আট সপ্তাহ। চাহিদা বাড়লে সার্ভার নেই; চাহিদা কমলে সার্ভার পড়ে থাকে, টাকা নষ্ট। EC2 (Elastic Compute Cloud) এই সমস্যার সমাধান: virtual server API কলে চালু হয়, মিনিটে বন্ধ হয়, আর আপনি শুধু চলমান সময়ের ভাড়া দেন। "Elastic" শব্দটি এখানেই: চাহিদা অনুযায়ী বাড়ানো-কমানো যায় — অধ্যায় ১১-এর Auto Scaling সেই elasticity-কে স্বয়ংক্রিয় করে। TechNova-র ক্ষেত্রে ভাবুন: সকালে ট্র্যাফিক কম, দুপুরে বেশি — EC2-তে আপনি দুপুরে instance বাড়ান, রাতে কমান; hardware কিনলে এই নমনীয়তা অসম্ভব।

৯.৪ EC2 কীভাবে কাজ করে — প্রথম নীতি থেকে

EC2 instance হলো একটি virtual machine যা AWS-এর physical host-এ hypervisor (Nitro system)-এর ওপর চলে। Nitro হলো AWS-এর নিজস্ব virtualization — নেটওয়ার্ক, storage ও security-র কাজ dedicated hardware card-এ সরিয়ে দেওয়া হয়েছে, ফলে প্রায় bare-metal performance পাওয়া যায়। প্রতিটি instance-এর থাকে: **vCPU** (virtual CPU), **memory**, **network performance**, আর **EBS volume** (persistent block storage) অথবা instance store (অস্থায়ী, host-এর local SSD)। Instance বন্ধ করলে EBS volume-এর ডেটা থেকে যায়; instance store-এর ডেটা মুছে যায়। তাই গুরুত্বপূর্ণ ডেটা কখনো instance store-এ রাখবেন না।

Instance চালু হওয়ার ক্রম: (১) আপনি AMI (Amazon Machine Image) বাছেন — OS + pre-installed software-এর template; (২) instance type বাছেন — vCPU ও memory; (৩) key pair, Security Group, সাবনেট, IAM role দেন; (৪) User Data দেন — প্রথম boot-এ চালানোর script; (৫) AWS host-এ instance তৈরি করে, EBS attach করে, User Data চালিয়ে instance "running" করে। বন্ধ করলে: stop (EBS থাকে, instance ফিরে আসে), terminate (সব মুছে যায়, যদি DeleteOnTermination সেট থাকে)।

AMI (Amazon Machine Image): একটি instance চালু করার template — OS, application, configuration-এর snapshot। AWS নিজে Amazon Linux, Ubuntu, Windows AMI দেয়; আপনি নিজের AMI বানাতে পারেন (golden image) — এতে instance দ্রুত তৈরি হয়, কারণ User Data-তে সব ইনস্টল করতে হয় না। AMI region-specific — এক region-এর AMI অন্য region-এ কপি করতে হয়।

৯.৫ Instance Family ও Type বাছাই

Instance type-এর নাম একটি কোড: যেমন `t3.large` — `t` family, `3` generation, `large` size। প্রধান family-গুলো: **General purpose (T, M)** — ভারসাম্যপূর্ণ, বেশিরভাগ অ্যাপের জন্য; **Compute optimized (C)** — বেশি CPU, batch/API সার্ভারের জন্য; **Memory optimized (R, X)** — বেশি RAM, ডেটাবেস/ক্যাশের জন্য; **Storage optimized (I, D)** — দ্রুত local SSD, বড় ডেটা workload-এর জন্য; **Accelerated computing (P, G)** — GPU, ML/AI-এর জন্য। **Burstable (T family)** — CPU credit জমিয়ে প্রয়োজনে burst করে; কম ট্র্যাফিকের অ্যাপে সস্তা, কিন্তু credit ফুরিয়ে গেলে CPU throttled হয় — production-এ T instance ব্যবহারের আগে CPU credit metric দেখুন।

Family	বৈশিষ্ট্য	TechNova-র ব্যবহার
T3/T4g (burstable)	কম খরচ, CPU credit	Dev, ছোট অ্যাপ
M7i/M6i (general)	ভারসাম্যপূর্ণ	অ্যাপ সার্ভার (default)
C7i (compute)	বেশি CPU	API, batch job
R7i (memory)	বেশি RAM	ক্যাশ, ডেটাবেস
G5 (GPU)	GPU	ML inference

সঠিক size বাছাই: প্রথমে ছোট দিয়ে শুরু করুন, CloudWatch-এ CPU ও memory দেখুন, তারপর right-size করুন। Over-provisioning মানে টাকা নষ্ট; under-provisioning মানে ধীর অ্যাপ। AWS Compute Optimizer বিনামূল্যে পরামর্শ দেয় — সেটি দেখুন। আর মনে রাখবেন: instance type-এর দাম region ও purchasing option (On-Demand, Reserved, Spot) অনুযায়ী বদলায় — AWS Pricing Calculator-এ যাচাই করুন।

৯.৬ EBS — Persistent Block Storage

EBS (Elastic Block Store) হলো network-attached block storage — instance-এর "হার্ড ডিস্ক"। প্রধান volume type: **gp3** (general purpose, IOPS ও throughput আলাদা করে বাছা যায় — বেশিরভাগ কাজের default), **io2** (বেশি IOPS, latency-sensitive ডেটাবেস), **st1/sc1** (সস্তা HDD, throughput workload, infrequent access)। EBS volume একটি AZ-র ভেতরে থাকে — instance অন্য AZ-তে গেলে volume যাবে না; snapshot নিয়ে অন্য AZ-তে restore করতে হয়। **Snapshot** হলো EBS-এর point-in-time backup, S3-তে থাকে, incremental হয় — প্রথম snapshot পুরো, পরেরগুলো শুধু বদল। Production-এ প্রতিদিন snapshot নিন, আর snapshot-এর lifecycle policy (DLM) দিয়ে স্বয়ংক্রিয় করুন।

৯.৭ Key Pair, User Data ও Instance Metadata

Key pair হলো SSH-এর জন্য public/private key — private key (.pem) শুধু আপনার কাছে থাকে, public key instance-এ থাকে। Private key হারিয়ে গেলে instance-এ SSH করা যায় না (তবে Session Manager দিয়ে উদ্ধার করা যায় — নিচে দেখুন)। Key pair কখনো Git-এ বা Slack-এ শেয়ার করবেন না।

User Data হলো প্রথম boot-এ চালানো script (cloud-init)। এতে OS update, package install, অ্যাপ deploy — সব করা যায়। কিন্তু মনে রাখবেন: User Data plain text-এ instance metadata থেকে পড়া যায় — এতে কখনো password বা secret লিখবেন না; secret-এর জন্য Secrets Manager বা Parameter Store ব্যবহার করুন। User Data শুধু প্রথম boot-এ চলে (default); আবার চালাতে চাইলে cloud-init-এর configuration বদলাতে হয়।

৯.৭.১ IMDS ও IMDSv2 — কেন এটি গুরুত্বপূর্ণ

Instance Metadata Service (IMDS) হলো `http://169.254.169.254` ঠিকানায় চলা একটি local সার্ভিস — instance নিজের সম্পর্কে তথ্য জানতে পারে: instance ID, AZ, IAM role-এর temporary credential। IMDSv1-এ শুধু GET request-ই যথেষ্ট ছিল — আর এটাই ছিল SSRF (Server-Side Request Forgery) আক্রমণের দরজা: আক্রমণকারী অ্যাপে একটি URL injection করে `169.254.169.254`-এ request পাঠাত, আর IAM credential চুরি করত। **IMDSv2** এই দরজা বন্ধ করে: প্রথমে PUT request দিয়ে একটি session token নিতে হয় (TTL সহ), তারপর সেই token header-সহ GET করতে হয়। SSRF আক্রমণে সাধারণত custom header পাঠানো যায় না — তাই token ছাড়া metadata পাওয়া যায় না। Production-এ IMDSv2-only বাধ্যতামূলক করুন (launch template-এ `HttpTokens: required`)।

```
# IMDSv2 দিয়ে token নিয়ে metadata পড়া
TOKEN=$(curl -X PUT \
  "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id
curl -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/iam/info

# IMDSv2-only বাধ্যতামূলক করা (launch template-এ)
# MetadataOptions: HttpTokens=required, HttpPutResponseHopLimit=1
```

> **WARNING:** IMDSv1 চালু থাকা মানে SSRF-এর দরজা খোলা। ২০১৯-এর Capital One breach-এর মূল কারণ ছিল SSRF দিয়ে IMDS থেকে credential চুরি। সব নতুন instance-এ IMDSv2-only চালু করুন, আর পুরনো instance-গুলো audit করুন। Hop limit 1 রাখুন — container-এর ভেতর থেকে metadata-তে পৌঁছানো কঠিন হয়।

৯.৮ SSH বনাম Session Manager — Session Manager-ই পছন্দ করুন

ঐতিহ্যগতভাবে instance-এ ঢুকতে SSH (port 22) খুলতে হতো — আর সেটাই ছিল আক্রমণের সবচেয়ে বড় লক্ষ্য। **AWS Systems Manager Session Manager** এই সমস্যার সমাধান: কোনো inbound port খোলা লাগে না, কোনো key pair লাগে না, কোনো bastion host লাগে না। Instance-এ SSM Agent চলে, outbound HTTPS (443) দিয়ে SSM সার্ভিসের সঙ্গে যুক্ত হয়, আর আপনি AWS Console বা CLI থেকে shell পান। সব session CloudTrail-এ log হয়, প্রয়োজনে session log S3-তে সংরক্ষণ করা যায়। শর্ত: instance-এ SSM Agent থাকতে হবে (Amazon Linux ও Ubuntu AMI-তে

আগে থেকে থাকে), instance-এর IAM role-এ `AmazonSSMManagedInstanceCore` policy থাকতে হবে, আর instance-কে SSM endpoint-এ পৌঁছাতে হবে — হয় NAT Gateway দিয়ে, নয় VPC endpoint দিয়ে (অধ্যায় ৭)।

```
# Session Manager দিয়ে সংযোগ (কোনো SSH নয়)
aws ssm start-session --target i-0abc1234def56789

# IAM role-এ SSM policy যুক্ত করুন
aws iam attach-role-policy --role-name technova-app-role \
  --policy-arn arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore

# Session Manager পছন্দ করার কারণ:
# - Port 22 ইন্টারনেটে খোলা নেই (আক্রমণের সুযোগ কম)
# - Key pair পরিচালনা করতে হয় না
# - প্রতিটি session audit log-এ থাকে
```

৯.৯ Hardened User Data — সম্পূর্ণ উদাহরণ

নিচে TechNova-র অ্যাপ সার্ভারের hardened User Data। লক্ষ্য করুন: OS update, SSM Agent নিশ্চিত করা, IMDSv2 hop limit, fail2ban নয় (SSH বন্ধ), CloudWatch agent, আর অ্যাপ deploy — কিন্তু কোনো secret plain text-এ নেই; secret আসে SSM Parameter Store থেকে, IAM role-এর অনুমতিতে।

```
#!/bin/bash
set -euo pipefail
exec > /var/log/user-data.log 2>&1

# 1. OS update ও প্রয়োজনীয় package
dnf update -y
dnf install -y amazon-cloudwatch-agent

# 2. Node.js 20 ইনস্টল (TechNova stack)
dnf install -y nodejs20

# 3. অ্যাপ ব্যবহারকারী তৈরি (root-এ অ্যাপ চালাবেন না)
useradd -m -s /bin/bash technova || true

# 4. অ্যাপ কোড নিন (S3 থেকে, IAM role-এর অনুমতিতে)
aws s3 cp s3://technova-app-data/app-v1.tar.gz /opt/app.tar.gz
mkdir -p /opt/technova-app
tar -xzf /opt/app.tar.gz -C /opt/technova-app
chown -R technova:technova /opt/technova-app

# 5. Secret Parameter Store থেকে নিন (কখনো User Data-তে নয়)
DB_PASS=$(aws ssm get-parameter \
  --name /technova/prod/db-password \
  --with-decryption \
  --query 'Parameter.Value' --output text)
echo "DB_PASS=$DB_PASS" > /opt/technova-app/.env
chmod 600 /opt/technova-app/.env
chown technova:technova /opt/technova-app/.env

# 6. systemd service হিসেবে অ্যাপ চালান
cat > /etc/systemd/system/technova-app.service << 'EOF'
[Unit]
Description=TechNova App
After=network.target
```

```
[Service]
User=technova
WorkingDirectory=/opt/technova-app
ExecStart=/usr/bin/node server.js
Restart=always
RestartSec=5
[Install]
WantedBy=multi-user.target
EOF
systemctl daemon-reload
systemctl enable technova-app
systemctl start technova-app

# 7. CloudWatch agent চালু (log ও metric পাঠাতে)
systemctl enable amazon-cloudwatch-agent
systemctl start amazon-cloudwatch-agent
echo "User Data complete"
```

PRO TIP: User Data-তে `set -euo pipefail` ও log ফাইল ব্যবহার করুন। User Data ব্যর্থ হলে instance "running" দেখালেও অ্যাপ চলবে না — `/var/log/cloud-init-output.log` দেখুন। আর বড় deploy-এর জন্য User Data-র বদলে golden AMI ব্যবহার করুন — boot দ্রুত হয়, ব্যর্থতার সুযোগ কমে।

৯.১০ Placement, Tenancy ও Lifecycle

Placement group তিন ধরনের: Cluster (একই AZ-তে কাছাকাছি, কম latency — HPC-র জন্য), Spread (আলাদা hardware-এ, failure isolation), Partition (আলাদা partition-এ, বড় distributed সিস্টেম)। **Tenancy**: Shared (default — অন্য গ্রাহকের সঙ্গে একই host), Dedicated instance (আপনার instance আলাদা host-এ, কিন্তু host শেয়ার হতে পারে), Dedicated Host (পুরো physical host আপনার — compliance-এর জন্য)। Dedicated-এর খরচ বেশি — শুধু প্রয়োজনে ব্যবহার করুন। **Lifecycle**: pending → running → stopping/stopped → terminated। Stop করলে EBS থাকে, IP বদলাতে পারে (Elastic IP না থাকলে); terminate করলে ফিরে পাওয়া যায় না — termination protection চালু রাখুন production instance-এ।

৯.১১ নিরাপত্তা, খরচ ও সাধারণ ভুল

নিরাপত্তা: IMDSv2-only বাধ্যতামূলক করুন; SSH port বন্ধ রেখে Session Manager ব্যবহার করুন; instance-এ IAM role দিন, access key কখনো instance-এ রাখবেন না; EBS encryption চালু করুন (KMS); termination protection চালু রাখুন; OS patch নিয়মিত করুন (SSM Patch Manager)। **খরচ:** On-Demand দিয়ে শুরু করুন, স্থিতিশীল workload-এ Reserved/Savings Plan, সহনশীল workload-এ Spot (৭০% পর্যন্ত সস্তা হতে পারে — AWS Pricing Calculator-এ যাচাই করুন); বন্ধ থাকা instance-এর EBS volume-ও চার্জ হয় — অপ্রয়োজনীয় volume ও snapshot মুছুন। **সাধারণ ভুল:** (১) User Data-তে secret লেখা; (২) IMDSv1 চালু রাখা; (৩) 0.0.0.0/0 থেকে SSH খোলা; (৪) instance store-এ গুরুত্বপূর্ণ ডেটা রাখা; (৫) T instance-এর CPU credit ফুরিয়ে যাওয়া না দেখা; (৬) termination protection না দেওয়া।

৯.১২ সমস্যা সমাধান

Instance-এ সংযোগ হচ্ছে না? (১) Session Manager: SSM Agent চলছে কি? IAM role-এ SSM policy আছে কি? VPC endpoint বা NAT route আছে কি? (২) SSH: Security Group-এ 22 খোলা কি? Key pair ঠিক কি? (৩) Status check ব্যর্থ: system status check ব্যর্থ হলে host-এর সমস্যা — instance stop/start করুন (নতুন host-এ যাবে); instance status check ব্যর্থ হলে OS-এর সমস্যা — console log দেখুন (GetConsoleOutput)। অ্যাপ চলছে না? User Data log (`/var/log/cloud-init-output.log`) দেখুন; systemd status দেখুন (`systemctl status technova-app`)।

৯.১৩ EC2 কেনার পদ্ধতি ও পর্যবেক্ষণ — গভীরে

একই instance type-এর দাম কেনার পদ্ধতিতে বদলে যায় — এটিই EC2-র অর্থনীতির হৃদয়। **On-Demand**: কোনো প্রতিশ্রুতি ছাড়া, প্রতি সেকেন্ডে ভাড়া — নমনীয়, কিন্তু সবচেয়ে দামি। **Reserved Instance / Savings Plan**: এক বা তিন বছরের প্রতিশ্রুতিতে ছাড় — স্থিতিশীল workload-এ (যেমন TechNova-র সবসময় চলা ২টি অ্যাপ সার্ভার) এটিই সঠিক। Savings Plan instance family-তে নমনীয়, Reserved Instance নির্দিষ্ট type-এ আটকে থাকে। **Spot Instance**: AWS-এর অব্যবহৃত capacity নিলামে — অনেক সস্তা, কিন্তু AWS দুই মিনিটের নোটিশে ফিরিয়ে নিতে পারে। Spot শুধু সহনশীল workload-এ (batch job, CI runner, stateless অ্যাপ ASG-তে) ব্যবহার করুন — ডেটাবেসে কখনো নয়। **Dedicated Host**: পুরো physical host — শুধু compliance বা license-এর প্রয়োজনে।

সঠিক কৌশল: baseline (সবসময় চলা) workload Savings Plan-এ, পরিবর্তনশীল অংশ On-Demand-এ, সহনশীল batch Spot-এ। এই মিশ্রণকেই **mixed instance policy** বলে — ASG-তে একসঙ্গে On-Demand ও Spot চালানো যায়, যেমন ২টি On-Demand base + বাকি Spot। Spot interruption হলে ASG On-Demand দিয়ে capacity পূরণ করে।

পর্যবেক্ষণ (Monitoring): CloudWatch-এ EC2-র মূল metric-গুলো: CPUUtilization, StatusCheckFailed, NetworkIn/Out, DiskRead/WriteOps, EBS VolumeRead/WriteLatency। মনে রাখবেন: CloudWatch-এ memory metric default-এ আসে না — CloudWatch Agent ইনস্টল করে `mem_used_percent` পাঠাতে হয় (অধ্যায় ৯-এর User Data-তে agent ইনস্টল করা হয়েছে এই কারণেই)। Alarm বসান: StatusCheckFailed > 0 হলে, CPU > ৮০% টানা ১৫ মিনিট হলে, EBS latency বাড়লে। আর **EC2 Instance Connect** মনে রাখুন — browser থেকে SSH করার AWS-এর উপায়, key পরিচালনা ছাড়া; তবে Session Manager-ই প্রথম পছন্দ, কারণ তাতে port খুলতে হয় না।

EC2 ব্যর্থ হলে কী হয়: Host hardware ব্যর্থ হলে instance-এর system status check ব্যর্থ হয় — সমাধান stop/start (নতুন host-এ যায়)। OS crash হলে instance status check ব্যর্থ হয় — console log দেখে উদ্ধার করুন। EBS volume-এর সমস্যা হলে I/O latency বাড়ে — snapshot থেকে নতুন volume বানিয়ে attach করুন। আর সবচেয়ে গুরুত্বপূর্ণ: instance-কে **cattle, not pet** হিসেবে দেখুন — অসুস্থ instance-কে সারানোর চেষ্টা না করে terminate করে নতুনটি আনুন; ASG সেই দর্শনকেই স্বয়ংক্রিয় করে (অধ্যায় ১১)।

৯.১৪ সারাংশ

EC2 হলো API-চালিত virtual server: AMI থেকে template, instance type থেকে আকার, EBS থেকে persistent disk, User Data থেকে প্রথম boot-এর কনফিগারেশন। নিরাপত্তার মূল কথা তিনটি: IMDSv2-only, SSH বন্ধ করে Session Manager, আর secret কখনো User Data-তে নয়। সঠিক instance family বাছুন, right-size করুন, আর termination protection চালু রাখুন।

৯.১৫ পর্যালোচনা প্রশ্ন

১. AMI কী এবং golden image-এর সুবিধা কী? ২. EBS ও instance store-এর পার্থক্য কী? ৩. IMDSv2 SSRF থেকে কীভাবে রক্ষা করে? ৪. Session Manager SSH-এর চেয়ে কেন নিরাপদ? ৫. T instance-এর CPU credit কী? ৬. gp3 ও io2 volume-এর পার্থক্য কী? ৭. Placement group-এর তিনটি ধরন কী কী? ৮. Dedicated Host কখন দরকার হয়? ৯. User Data-তে secret রাখা কেন বিপজ্জনক? ১০. System status check বনাম instance status check-এর পার্থক্য কী?

৯.১৬ অনুশীলনী

অনুশীলন ১: IMDSv2-only launch template দিয়ে একটি instance চালু করুন; IMDSv1 curl ব্যর্থ হতে দেখুন, IMDSv2 token দিয়ে সফল হতে দেখুন। **অনুশীলন ২:** উপরের hardened User Data দিয়ে একটি অ্যাপ সার্ভার চালু করুন, Session Manager দিয়ে ঢুকে `systemctl status technova-app` পরীক্ষা করুন। **অনুশীলন ৩:** একটি instance-এর EBS snapshot নিয়ে অন্য AZ-তে নতুন volume বানিয়ে ডেটা ফিরে পান — disaster recovery-র মহড়া করুন।

অধ্যায় ১০ • PART II — Networking & Compute

Chapter 10 — Elastic Load Balancing

১০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) load balancing-এর মূল নীতি ব্যাখ্যা করতে; (২) Application Load Balancer (ALB), Network Load Balancer (NLB) ও Gateway Load Balancer (GWLB)-এর মধ্যে সঠিকটি বাছতে; (৩) listener, target group ও health check কনফিগার করতে; (৪) TLS termination ও path/host-based routing ডিজাইন করতে; (৫) unhealthy target-এর সমস্যার মূল কারণ খুঁজে বের করতে; এবং (৬) TechNova-র multi-AZ load balancing আর্কিটেকচার তৈরি করতে।

১০.২ পূর্বশর্ত ঝালাই

অধ্যায় ৬ ও ৯ থেকে মনে করুন: instance প্রাইভেট সাবনেটে থাকে, Security Group তার firewall। এবার প্রশ্ন: ব্যবহারকারী ইন্টারনেট থেকে অ্যাপে পৌঁছাবে কীভাবে, যদি অ্যাপ সার্ভার প্রাইভেট সাবনেটে থাকে? উত্তর: একটি public entry point দরকার — যেটি ইন্টারনেট থেকে ট্র্যাফিক নিয়ে প্রাইভেট instance-গুলোতে ভাগ করে দেয়। সেটিই Load Balancer। মনে রাখুন DNS-এর কথা (অধ্যায় ৮): ব্যবহারকারী `app.technova.example` টাইপ করে — Route 53 Alias record সেটিকে Load Balancer-এর DNS নামে পাঠায়। আর TLS/SSL-এর মূল ধারণা: HTTPS-এ ট্র্যাফিক encrypted থাকে, certificate দিয়ে পরিচয় যাচাই হয়।

১০.৩ কেন Load Balancer — আগে কী সমস্যা ছিল?

একটি মাত্র সার্ভারে অ্যাপ চালালে তিনটি সমস্যা: (১) সার্ভার ডাউন হলে অ্যাপ ডাউন — single point of failure; (২) ট্র্যাফিক বাড়লে সার্ভার overload হয় — scale করার উপায় নেই; (৩) deploy-এর সময় অ্যাপ বন্ধ রাখতে হয়। Load Balancer এই তিনটি সমস্যার একসঙ্গে সমাধান: ট্র্যাফিক অনেক instance-এর মধ্যে ভাগ করে দেয়, অসুস্থ instance-কে সরিয়ে দেয়, আর নতুন instance যোগ করা যায় অ্যাপ বন্ধ না করেই। Load Balancer নিজেও managed ও multi-AZ — এটি ব্যর্থ হওয়ার সম্ভাবনা খুব কম। TechNova-র ক্ষেত্রে ভাবুন: দুটি AZ-তে চারটি অ্যাপ সার্ভার, সামনে একটি ALB — একটি AZ ডাউন হলেও ALB অন্য AZ-র সার্ভারে ট্র্যাফিক পাঠায়, ব্যবহারকারী কিছুই টের পায় না।

> **CONCEPT:** Load Balancer একটি reverse proxy — ক্লায়েন্ট Load Balancer-কে দেখে, আসল সার্ভারকে নয়। ক্লায়েন্টের কাছে মনে হয় একটি সার্ভারই উত্তর দিচ্ছে; ভেতরে অনেক সার্ভার কাজ করছে। এই abstraction-ই high availability ও scalability-র ভিত্তি।

১০.৪ Load Balancing কীভাবে কাজ করে — প্রথম নীতি থেকে

Load Balancer-এর কাজ তিন ধাপে: (১) **Listener** একটি port ও protocol-এ ট্র্যাফিক শোনে (যেমন HTTPS 443); (২) **Rule** অনুযায়ী ঠিক করে কোন **Target Group**-এ পাঠাবে (path বা host দেখে); (৩) Target Group-এর **healthy target**-গুলোর মধ্যে ট্র্যাফিক ভাগ করে দেয় (round-robin বা least-outstanding-requests অ্যালগরিদমে)। **Health check** নিয়মিত প্রতিটি target-কে পরীক্ষা করে — যেমন প্রতি ৩০ সেকেন্ডে `GET /health`-এ 200 response আসে কি না। পরপর কয়েকবার ব্যর্থ হলে target-কে unhealthy ঘোষণা করে ট্র্যাফিক পাঠানো বন্ধ করে দেয়; আবার সুস্থ হলে ফিরিয়ে আনে। এই health check-ই Load Balancer-কে "elastic" করে — অসুস্থ সার্ভার নিজে থেকেই সরে যায়।

১০.৫ ALB বনাম NLB বনাম GWLB — সিদ্ধান্ত টেবিল

AWS-এর তিনটি Load Balancer তিনটি আলাদা স্তরে কাজ করে। ভুলটি বাছলে হয় খরচ বাড়বে, নয় ফিচার পাবেন না।

বৈশিষ্ট্য	ALB (Application)	NLB (Network)	GWLB (Gateway)
OSI স্তর	Layer 7 (HTTP/HTTPS)	Layer 4 (TCP/UDP)	Layer 3 (IP)
Routing	Path/host/header ভিত্তিক	IP/port ভিত্তিক	Third-party appliance-এ
TLS termination	হ্যাঁ (ACM certificate)	হ্যাঁ (সীমিত)	না
Latency	মাঝারি	খুব কম (ultra-low)	কম
Static IP	না (DNS নাম)	হ্যাঁ (Elastic IP)	না
সাধারণ ব্যবহার	ওয়েব অ্যাপ, API, microservice	TCP, gaming, IoT, static IP	Firewall/IDS appliance chain

TechNova-র ওয়েব অ্যাপের জন্য **ALB**-ই সঠিক — কারণ path-based routing (`/api/*` → API target group, `/*` → web target group) ও TLS termination দরকার। NLB দরকার হতো যদি static IP চাইত (যেমন কোনো গ্রাহকের firewall-এ IP whitelist করতে হলে) বা ultra-low latency TCP দরকার হতো। GWLB শুধু তখনই, যখন তৃতীয় পক্ষের firewall/IDS appliance-এর মধ্যে দিয়ে ট্র্যাফিক পাঠাতে চান।

TechNova ALB Architecture — Multi-AZ

```
                Users (Internet)
                       |
          Route 53: app.technova.example
                  |  (Alias record)
                       v
      Application Load Balancer (public subnets)
          10.0.1.0/24 (1a)   10.0.2.0/24 (1b)
             |  Listener: HTTPS 443 (ACM cert)
                |  Rules: /api/* -> api-tg
                  |         /* -> web-tg
                    +----------+----------+
                  v                       v
          Target Group web      Target Group api
        EC2 10.0.11.25 (1a)  EC2 10.0.11.30 (1a)
        EC2 10.0.12.25 (1b)  EC2 10.0.12.30 (1b)
          Health check: GET /health every 30s
```

১০.৬ Listener, Target Group ও Health Check — বিস্তারিত

Listener নির্ধারণ করে কোন port ও protocol-এ শুনবে এবং কী করবে। HTTPS listener-এ ACM certificate যুক্ত করুন — TLS termination ALB-তেই হয়, ফলে backend instance-গুলোতে certificate পরিচালনা করতে হয় না (ALB থেকে

instance-এ ট্র্যাফিক HTTP-তে যেতে পারে, তবে নিরাপত্তা বাড়াতে চাইলে backend-এও HTTPS করতে পারেন — তখন ALB backend-এর certificate যাচাই করে)। HTTP listener-এ redirect rule দিন: সব HTTP ট্র্যাফিক HTTPS-এ redirect হবে — এতে ব্যবহারকারী ভুল করে HTTP-তে এলেও নিরাপদে HTTPS-এ যায়।

Target Group হলো target-দের (EC2 instance, IP, Lambda function, ALB) সমষ্টি ও health check-এর সেটিংস। Target type: instance (EC2 instance ID দিয়ে), IP (IP ঠিকানা দিয়ে — container ও on-premises-এর জন্য), Lambda, ALB। **Health check**-এর সেটিংস: protocol, path (`/health` — একটি হালকা endpoint যা ডেটাবেস সংযোগও পরীক্ষা করে, কিন্তু ভারী কাজ করে না), interval (৩০ সেকেন্ড), timeout (৫ সেকেন্ড), healthy threshold (পরপর ৩ বার সফল), unhealthy threshold (পরপর ৩ বার ব্যর্থ)। মনে রাখবেন: health check endpoint-টি অ্যাপের সত্যিকারের স্বাস্থ্য জানাতে হবে — শুধু 200 ফেরত দেওয়া একটি static পেজ যথেষ্ট নয়; ডেটাবেস ডাউন থাকলে `/health`-ও 500 ফেরত দেবে, নইলে ALB অসুস্থ সার্ভারে ট্র্যাফিক পাঠাতে থাকবে।

```
# Target Group তৈরি
aws elbv2 create-target-group \
  --name technova-web-tg \
  --protocol HTTP --port 8080 \
  --vpc-id vpc-0abc1234def56789 \
  --health-check-path /health \
  --health-check-interval-seconds 30 \
  --healthy-threshold-count 3 \
  --unhealthy-threshold-count 3 \
  --target-type instance

# Target register করুন
aws elbv2 register-targets \
  --target-group-arn arn:aws:elasticloadbalancing:ap-south-1:111122223333:targetgroup/
technova-web-tg/abc123 \
  --targets Id=i-0abc1234def56789 Id=i-0abc1234def56789

# ALB তৈরি (দুটি AZ-র পাবলিক সাবনেটে)
aws elbv2 create-load-balancer \
  --name technova-alb \
  --subnets subnet-0abc1234def56789 subnet-0abc1234def56789 \
  --security-groups sg-0abc1234def56789 \
  --scheme internet-facing --type application

# HTTPS Listener (ACM certificate-সহ)
aws elbv2 create-listener \
  --load-balancer-arn arn:aws:elasticloadbalancing:ap-south-1:111122223333:loadbalancer/app/
technova-alb/abc123 \
  --protocol HTTPS --port 443 \
  --certificates CertificateArn=arn:aws:acm:ap-south-1:111122223333:certificate/abc123 \
  --default-actions Type=forward,TargetGroupArn=arn:aws:elasticloadbalancing:ap-
south-1:111122223333:targetgroup/technova-web-tg/abc123
```

১০.৭ Path-Based ও Host-Based Routing

ALB-এর সবচেয়ে শক্তিশালী ফিচার: একটি ALB দিয়ে অনেক সার্ভিস চালানো। Listener rule-এ শর্ত দিন: `Host: api.technova.example` → API target group; `Path: /api/*` → API; `Path: /static/*` → S3 বা অন্য target; header বা query string দেখেও routing করা যায়। Rule-এর priority নম্বর দিয়ে ক্রম নির্ধারণ করুন — সবচেয়ে নির্দিষ্ট

rule আগে, default rule শেষে। Microservices আর্কিটেকচারে এটি অপরিহার্য: একটি ALB, অনেক target group, প্রতিটি microservice আলাদা target group-এ।

> **PRO TIP:** ALB-এর access log S3-তে চালু করুন। প্রতিটি request-এর latency, status code, target IP — সব লগে থাকে। ধীর API খুঁজতে বা আক্রমণ তদন্ত করতে এই লগ অমূল্য। Access log-এর S3 bucket-এ lifecycle policy দিন, নইলে লগের বিল বাড়তে থাকবে।

১০.৮ TLS Termination ও ACM

ACM (AWS Certificate Manager) বিনামূল্যে public TLS certificate দেয় (AWS resource-এ ব্যবহার করলে)। Certificate তৈরি করে ALB-র listener-এ যুক্ত করুন — renewal স্বয়ংক্রিয়। TLS termination মানে: ক্লায়েন্ট থেকে ALB পর্যন্ত encrypted, ALB decrypt করে backend-এ পাঠায়। Backend-এ HTTP ব্যবহার করলে internal ট্র্যাফিক unencrypted থাকে — VPC-র ভেতরে এটি সাধারণত গ্রহণযোগ্য, তবে কঠোর compliance (যেমন PCI DSS) থাকলে backend-এও HTTPS করুন। TLS policy বাছুন আধুনিকটি (যেমন ELBSecurityPolicy-TLS13-1-2-2021-06) — পুরনো TLS 1.0/1.1 বন্ধ রাখুন।

১০.৯ নিরাপত্তা, খরচ ও সাধারণ ভুল

নিরাপত্তা: ALB-র Security Group-এ শুধু 443 (ও redirect-এর জন্য 80) খুলুন; backend instance-এর Security Group-এ শুধু ALB-র Security Group থেকে allow করুন — কখনো সরাসরি ইন্টারনেট থেকে backend-এ যাওয়া যাবে না; WAF (Web Application Firewall) ALB-এর সামনে যুক্ত করুন SQL injection ও XSS থেকে রক্ষার জন্য। **খরচ:** ALB-এর hourly চার্জ আছে + LCU (Load Balancer Capacity Unit) চার্জ — LCU নির্ধারণ হয় নতুন connection, active connection, processed bytes ও rule evaluation দিয়ে; খালি ALB-ও hourly চার্জ হয় — ব্যবহার না করলে মুছে ফেলুন; সঠিক অঙ্ক AWS Pricing Calculator-এ যাচাই করুন। **সাধারণ ভুল:** (১) ALB-কে একটি AZ-র সাবনেটে দেওয়া; (২) health check path-এ ভারী endpoint দেওয়া; (৩) Security Group-এ ALB-র IP দিয়ে নিয়ম লেখা (IP বদলায় — SG reference ব্যবহার করুন); (৪) HTTP থেকে HTTPS redirect না দেওয়া; (৫) target group-এ ভুল port দেওয়া।

১০.১০ Troubleshooting — Unhealthy Target

Target unhealthy দেখালে ধাপে ধাপে পরীক্ষা করুন: (১) instance চলছে কি? CloudWatch-এ CPU/memory দেখুন; (২) অ্যাপ চলছে কি? Session Manager দিয়ে ঢুকে `curl localhost:8080/health` চালান — instance-এর ভেতর থেকে কাজ করলে সমস্যা নেটওয়ার্কে, না করলে অ্যাপে; (৩) Security Group: ALB-র SG থেকে instance-এর SG-তে 8080 খোলা কি? (৪) NACL: ALB সাবনেট ও অ্যাপ সাবনেটের NACL-এ return port খোলা কি? (৫) Health check path ঠিক কি? অ্যাপের log-এ health check request আসছে কি? (৬) Target group-এর port ঠিক কি? (৭) ALB-র access log-এ কী error দেখা যাচ্ছে? মনে রাখবেন: health check ব্যর্থতার সবচেয়ে সাধারণ কারণ Security Group — ALB থেকে instance-এ যাওয়ার পথ বন্ধ।

১০.১১ Production Scenario — TechNova-র Deploy

TechNova নতুন version deploy করছে: ALB-র সামনে পুরনো ও নতুন — দুটি target group। প্রথমে নতুন target group-এ একটি instance, health check সফল হলে ধীরে ধীরে ট্র্যাফিকের ভাগ বাড়ানো (weighted target group)। কোনো সমস্যা হলে সঙ্গে সঙ্গে পুরনো target group-এ ফিরে যাওয়া — rollback-এর জন্য deploy বন্ধ করতে হয় না। এটাই blue/

green deployment-এর মূল নীতি, আর ALB-ই সেটিকে সম্ভব করে। অধ্যায় ১১-এর Auto Scaling-এর সঙ্গে মিলিয়ে এই deploy সম্পূর্ণ স্বয়ংক্রিয় করা যায়।

১০.১২ ALB-র পর্যবেক্ষণ, WAF ও উন্নত বৈশিষ্ট্য

ALB চালু করলেই কাজ শেষ নয় — তাকে পর্যবেক্ষণ করতে হয়। CloudWatch-এ ALB-র মূল metric-গুলো: **RequestCount** (কত request), **TargetResponseTime** (backend কত দ্রুত উত্তর দেয়), **HTTPCode_Target_5XX_Count** (backend-এর error), **HTTPCode_ELB_5XX_Count** (ALB নিজের error), **HealthyHostCount / UnHealthyHostCount** (কত target সুস্থ), **ActiveConnectionCount**। Alarm বসান: 5XX বাড়লে, healthy host ১-এর নিচে নামলে, response time হঠাৎ বাড়লে। **Access log** S3-তে চালু করুন — প্রতিটি request-এর বিস্তারিত (client IP, request path, status, latency, target) লগে থাকে; Athena দিয়ে সেই লগ query করে ধীর endpoint বা আক্রমণের pattern খুঁজুন।

AWS WAF ALB-র সামনে বসিয়ে SQL injection, cross-site scripting (XSS), ও পরিচিত bad bot থেকে রক্ষা করুন। WAF-এ managed rule group (AWSManagedRulesCommonRuleSet) চালু করা কয়েক ক্লিকের কাজ — আর rate-based rule দিয়ে এক IP থেকে অতিরিক্ত request block করুন। মনে রাখবেন: WAF Layer 7-এ কাজ করে — NLB-তে WAF বসে না, ALB-তেই বসে। **Sticky session (session affinity)**: ALB cookie দিয়ে একই ব্যবহারকারীকে একই target-এ পাঠাতে পারে — তবে আধুনিক stateless অ্যাপে এটি এড়িয়ে চলুন; session Redis বা DynamoDB-তে রাখুন, instance-এর memory-তে নয় — নইলে instance মারা গেলে session-ও মরে।

ALB ব্যর্থ হলে কী হয়: ALB নিজে multi-AZ managed সার্ভিস — একটি AZ ডাউন হলেও ALB চলে। ব্যর্থতা আসে অন্যভাবে: সব target unhealthy হলে ALB 503 ফেরত দেয় (তখন সমস্যা backend-এ); Security Group ভুল হলে ALB target-এ পৌঁছাতে পারে না; certificate expire হলে HTTPS ব্যর্থ হয় (ACM auto-renewal চালু রাখুন)। আর একটি সূক্ষ্ম বিষয়: ALB-র DNS নামের IP বদলায় — কখনো ALB-র IP hardcode করবেন না, সবসময় DNS নাম ব্যবহার করুন। **কখন ALB ব্যবহার করবেন না:** শুধু TCP passthrough দরকার হলে NLB; খুব কম ট্র্যাফিকের dev পরিবেশে ALB-র hourly চার্জ বাঁচাতে সরাসরি instance বা CloudFront + Lambda বিবেচনা করুন — তবে নিরাপত্তা ও availability-র বিনিময়ে খরচ বাঁচাবেন না।

১০.১৩ সারাংশ

Load Balancer হলো আপনার অ্যাপের public মুখ ও high availability-র ভিত্তি। ওয়েব অ্যাপের জন্য ALB, TCP/static IP-র জন্য NLB, appliance chain-এর জন্য GWLB। Listener শোনে, rule ঠিক করে, target group স্বাস্থ্য পরীক্ষা করে, আর healthy target-এ ট্র্যাফিক ভাগ করে দেয়। TLS ALB-তেই terminate করুন, health check endpoint সত্যিকারের স্বাস্থ্য জানাক, আর backend-এ শুধু ALB-কেই ঢুকতে দিন।

১০.১৪ পর্যালোচনা প্রশ্ন

১. Load Balancer ছাড়া একটি সার্ভারের কী কী সমস্যা হয়? ২. ALB ও NLB-এর মূল পার্থক্য কী? ৩. Listener, target group ও health check-এর সম্পর্ক কী? ৪. GWLB কখন ব্যবহার করবেন? ৫. Health check endpoint কেমন হওয়া উচিত? ৬. TLS termination-এর সুবিধা কী? ৭. Path-based routing দিয়ে কীভাবে microservice চালাবেন? ৮. Target unhealthy হলে প্রথমে কী পরীক্ষা করবেন? ৯. ALB-র Security Group-এ কী কী port খোলা উচিত? ১০. LCU কী এবং ALB-র খরচ কীভাবে নির্ধারণ হয়?

১০.১৫ অনুশীলনী

অনুশীলন ১: একটি ALB বানিয়ে দুটি instance target group-এ যুক্ত করুন; একটি instance-এর অ্যাপ বন্ধ করে দেখুন ALB কীভাবে তাকে সরিয়ে দেয়। **অনুশীলন ২:** Path-based routing করুন: `/api/*` → API target group, `/*` → web target group — curl দিয়ে যাচাই করুন। **অনুশীলন ৩:** ইচ্ছা করে target group-এর port ভুল দিন, unhealthy দেখুন, তারপর ALB access log ও Reachability Analyzer দিয়ে মূল কারণ খুঁজে বের করুন।

অধ্যায় ১১ • PART II — Networking & Compute

Chapter 11 — Auto Scaling and High Availability

১১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Auto Scaling Group (ASG) ও launch template দিয়ে self-healing infrastructure ডিজাইন করতে; (২) target tracking, step scaling ও scheduled scaling-এর মধ্যে সঠিক policy বাছতে; (৩) health check ও instance replacement-এর কাজ ব্যাখ্যা করতে; (৪) multi-AZ deployment ও capacity planning করতে; (৫) scaling ব্যর্থতার পরিস্থিতি (throttling, capacity, flapping) শনাক্ত ও সমাধান করতে; এবং (৬) TechNova-র production ASG তৈরি করতে।

১১.২ পূর্বশর্ত ঝালাই

অধ্যায় ৯-১০ থেকে মনে করুন: EC2 instance চালু করা যায় AMI ও User Data দিয়ে; ALB health check করে অসুস্থ target সরিয়ে দেয়। এবার প্রশ্ন: অসুস্থ instance সরিয়ে দিলে তার জায়গায় নতুন instance কে আনবে? ট্র্যাফিক বাড়লে নতুন instance কে চালু করবে? রাতে ট্র্যাফিক কমলে instance কে বন্ধ করবে? উত্তর: **Auto Scaling Group** — এটি instance-এর সংখ্যা নিজে থেকেই বজায় রাখে ও বদলায়। মনে রাখুন CloudWatch metric-এর ধারণা: CPU utilization, request count — এগুলোই scaling-এর সিদ্ধান্তের ভিত্তি।

১১.৩ কেন Auto Scaling — আগে কী সমস্যা ছিল?

হাতে instance চালু করা মানে: ট্র্যাফিক বাড়ার খবর পেয়ে কেউ console-এ ঢুকে instance চালু করবে — ততক্ষণে ব্যবহারকারী ধীর অ্যাপ দেখে চলে গেছে। আর ট্র্যাফিক কমলে instance চলতে থাকে — টাকা নষ্ট হয়। সবচেয়ে খারাপ: রাত ৩টায় একটি instance crash করলে কেউ জানতেও পারে না, সকালে অ্যাপ ডাউন। Auto Scaling এই তিনটি সমস্যার সমাধান: (১) **self-healing** — অসুস্থ instance নিজে থেকে বদলে যায়; (২) **elasticity** — চাহিদা অনুযায়ী instance বাড়ে-কমে; (৩) **automation** — মানুষের হস্তক্ষেপ ছাড়াই, ২৪ ঘণ্টা। TechNova-র ক্ষেত্রে ভাবুন: দুপুরে ট্র্যাফিক তিনগুণ হলে ASG নিজে থেকে instance ২ থেকে ৬ করে; রাতে আবার ২-এ নামিয়ে আনে — বিল কমে, অ্যাপ দ্রুত থাকে, কেউ রাত জাগে না।

> **CONCEPT:** Auto Scaling Group-এর মূল ধারণা "desired capacity" — আপনি বলেন "আমার ২টি healthy instance চাই", ASG সেটি বজায় রাখে। একটি instance মারা গেলে ASG নতুনটি চালু করে; ট্র্যাফিক বাড়লে desired capacity বাড়ায়; কমলে কমায়। আপনি instance গণনা করেন না — আপনি লক্ষ্য বলেন, ASG সেই লক্ষ্য পূরণ করে।

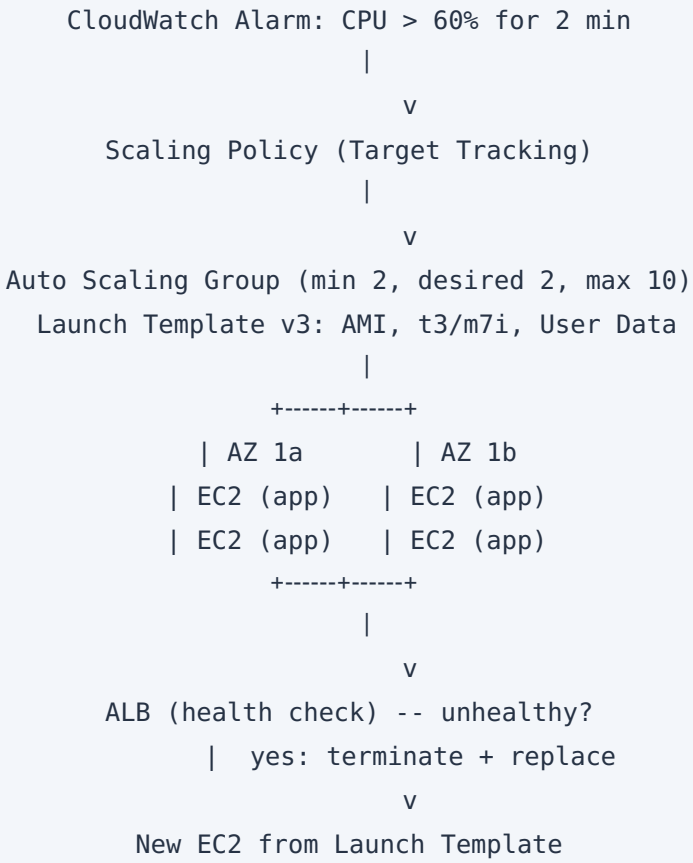
১১.৪ ASG কীভাবে কাজ করে — প্রথম নীতি থেকে

ASG-এর তিনটি সংখ্যা: **Minimum** (সবচেয়ে কম instance — যেমন ২), **Desired** (লক্ষ্য — যেমন ২), **Maximum** (সবচেয়ে বেশি — যেমন ১০)। ASG সবসময় desired সংখ্যক healthy instance বজায় রাখার চেষ্টা করে। নতুন instance চালু করতে ASG ব্যবহার করে **Launch Template** — AMI, instance type, key pair, Security Group, IAM role, User Data, IMDSv2 সেটিংস — সবকিছুর blueprint। Launch template-এর version থাকে — নতুন version দিয়ে rolling update করা যায়।

Health check দুই ধরনের: **EC2 status check** (instance-এর system/instance check) ও **ELB health check** (ALB-র health check — অ্যাপ-level)। ELB health check চালু করলে ALB যাকে unhealthy বলে, ASG তাকে terminate করে নতুন instance আনে। **Health check grace period** হলো নতুন instance-কে সময় দেওয়া — যেমন ৩০০ সেকেন্ড — যাতে User Data শেষ করে অ্যাপ চালু হতে পারে; grace period-এর মধ্যে health check ব্যর্থ হলেও ASG instance-কে মারবে না। এই grace period খুব ছোট দিলে instance বারবার মারা যাবে (অ্যাপ চালু হওয়ার আগেই unhealthy); খুব বড় দিলে সত্যিকারের অসুস্থ instance দেরিতে বদলাবে।

Instance terminate হওয়ার সময় **lifecycle hook** দিয়ে কাজ করা যায় — যেমন terminate-এর আগে log সংগ্রহ করা বা deregistration-এর জন্য অপেক্ষা করা। আর **termination policy** নির্ধারণ করে কোন instance আগে মারা যাবে — সাধারণত সবচেয়ে পুরনো launch template-এর instance বা সবচেয়ে বেশি instance থাকা AZ-র instance (AZ balance রাখতে)।

TechNova Auto Scaling — Multi-AZ Self-Healing



১১.৫ Launch Template — সম্পূর্ণ উদাহরণ

Launch template-ই ASG-এর "recipe"। এতে সবকিছু থাকতে হবে: AMI ID, instance type, Security Group, IAM role, User Data, IMDSv2-only, EBS encryption, termination protection-এর মতো সেটিংস। Console-এ EC2 → Launch Templates → Create, অথবা CLI দিয়ে।

```
# Launch Template তৈরি (IMDSv2-only সহ)
aws ec2 create-launch-template \
  --launch-template-name technova-app-lt \
```

```
  --version-description "v1 hardened" \
  --launch-template-data '{
    "ImageId": "ami-0abc1234def56789",
    "InstanceType": "t3.medium",
    "KeyName": "technova-key",
    "IamInstanceProfile": {
      "Arn": "arn:aws:iam::111122223333:instance-profile/technova-app-role"
    },
    "SecurityGroupIds": ["sg-0abc1234def56789"],
    "MetadataOptions": {
      "HttpTokens": "required",
      "HttpPutResponseHopLimit": 1,
      "HttpEndpoint": "enabled"
    },
    "BlockDeviceMappings": [{
      "DeviceName": "/dev/xvda",
      "Ebs": {
        "VolumeSize": 30,
        "VolumeType": "gp3",
        "Encrypted": true,
        "DeleteOnTermination": true
      }
    }],
    "TagSpecifications": [{
      "ResourceType": "instance",
      "Tags": [{ "Key": "Name", "Value": "technova-app" }]
    }]
  }'

# User Data আলাদাভাবে যোগ করতে চাইলে:
# UserData ফিল্ডে base64-encoded script দিন
# (অধ্যায় ৯-এর hardened User Data ব্যবহার করুন)
```

```
# Auto Scaling Group তৈরি
aws autoscaling create-auto-scaling-group \
  --auto-scaling-group-name technova-app-asg \
  --launch-template \
  'LaunchTemplateName=technova-app-lt,Version=$Latest' \
  --min-size 2 --max-size 10 --desired-capacity 2 \
  --vpc-zone-identifier \
  "subnet-0abc1234def56789,subnet-0abc1234def56789" \
  --target-group-arns \
  arn:aws:elasticloadbalancing:ap-south-1:111122223333:targetgroup/technova-web-tg/abc123 \
  --health-check-type ELB \
  --health-check-grace-period 300 \
  --tags Key=Name,Value=technova-app,PropagateAtLaunch=true

# ASG-এর অবস্থা দেখুন
aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names technova-app-asg \
  --query 'AutoScalingGroups[0].Instances'
```

১১.৬ Scaling Policy — কোনটি কখন

তিন ধরনের scaling policy আছে, প্রতিটির নিজস্ব ব্যবহার।

১১.৬.১ Target Tracking Scaling — সবচেয়ে সহজ ও সুপারিশকৃত

আপনি একটি লক্ষ্য বলেন — যেমন "CPU utilization ৬০% রাখো" বা "ALB-র প্রতি target-এ ১,০০০ request রাখো" — ASG নিজে থেকে instance বাড়িয়ে-কমিয়ে সেই লক্ষ্য ধরে রাখে। বেশিরভাগ workload-এর জন্য এটিই যথেষ্ট। Target tracking-এর সুবিধা: আপনাকে threshold নিয়ে ভাবতে হয় না; ASG metric দেখে নিজে সিদ্ধান্ত নেয়।

১১.৬.২ Step Scaling ও Simple Scaling

CloudWatch alarm-এর ভিত্তিতে ধাপে ধাপে scaling — যেমন CPU ৭০% ছাড়ালে +২ instance, ৯০% ছাড়ালে +৪ instance। Step scaling-এ আপনি ধাপগুলো নিজে নির্ধারণ করেন। Simple scaling পুরনো পদ্ধতি — একটি alarm-এ একটি action; এখন step scaling-ই পছন্দ করুন।

১১.৬.৩ Scheduled Scaling

সময় অনুযায়ী scaling — যেমন প্রতি সকাল ৯টায় desired capacity ২ থেকে ৬ করা, রাত ১০টায় আবার ২-এ নামানো। TechNova-র মতো predictable ট্র্যাফিক pattern থাকলে scheduled scaling খুব কার্যকর — ট্র্যাফিক আসার আগেই instance প্রস্তুত থাকে, cold start-এর অপেক্ষা করতে হয় না। Target tracking-এর সঙ্গে মিলিয়ে ব্যবহার করুন: scheduled scaling baseline দেয়, target tracking অপ্রত্যাশিত spike সামলায়।

```
# Target Tracking Policy: CPU 60% লক্ষ্য
aws autoscaling put-scaling-policy \
  --auto-scaling-group-name technova-app-asg \
  --policy-name technova-cpu-target \
  --policy-type TargetTrackingScaling \
  --target-tracking-configuration '{
    "PredefinedMetricSpecification": {
      "PredefinedMetricType": "ASGAverageCPUUtilization"
    },
    "TargetValue": 60.0
  }'

# Scheduled Scaling: সকাল ৯টায় capacity বাড়ান
aws autoscaling put-scheduled-update-group-action \
  --auto-scaling-group-name technova-app-asg \
  --scheduled-action-name morning-scale-up \
  --recurrence "0 9 * * *" \
  --min-size 4 --desired-capacity 4 --max-size 10

# Predictive Scaling (ঐচ্ছিক): ML দিয়ে আগাম অনুমান
# ইতিহাস দেখে ASG আগেই instance বাড়ায় — দীর্ঘমেয়াদি
# pattern থাকলে কার্যকর, তবে প্রথমে target tracking
# দিয়ে শুরু করুন
```

WARNING: Scaling policy-তে cooldown ও warm-up সময় বুঝুন। **Cooldown** হলো একটি scaling action-এর পর পরের action-এর আগে অপেক্ষা — নইলে ASG বারবার instance বাড়াতে-কমাতে থাকবে (flapping)। **Instance warm-up** হলো নতুন instance-কে metric-এ গোনার আগে অপেক্ষা — User Data শেষ হতে সময় লাগে; warm-up ছাড়া নতুন instance-এর CPU ০% দেখে ASG ভুল করে instance কমিয়ে দিতে পারে। Target tracking-এ default warm-up ৩০০ সেকেন্ড — আপনার অ্যাপের boot সময় অনুযায়ী বদলান।

১১.৭ Multi-AZ Deployment ও Capacity Planning

ASG-কে অন্তত দুটি AZ-র সাবনেট দিন — ASG instance-গুলো AZ-গুলোর মধ্যে ভারসাম্য রাখার চেষ্টা করে। একটি AZ ডাউন হলে ASG অন্য AZ-তে instance চালু করে desired capacity ফিরিয়ে আনে। **Capacity planning**-এর প্রশ্ন: minimum কত? — একটি AZ ডাউন হলেও যেন অ্যাপ চলে, তাই minimum অন্তত ২ (প্রতি AZ-তে ১টি)। Maximum কত? — খরচের সীমা ও AWS service quota দেখে ঠিক করুন; হঠাৎ maximum ১০০ দিলে একটি bug-এর কারণে বিল আকাশ ছুঁতে পারে। **AZ rebalancing** চালু রাখুন — ASG AZ-র ভারসাম্য নষ্ট হলে নিজে থেকে instance সরিয়ে ভারসাম্য ফেরায়। আর মনে রাখুন: ASG-এর instance-গুলো ALB target group-এ স্বয়ংক্রিয়ভাবে register হয় — হাতে কিছু করতে হয় না।

পরিস্থিতি	সুপারিশকৃত সেটিং	কেন
Production web app	Min 2, Max 10, 2 AZ	একটি AZ ডাউন হলেও চলে
Dev পরিবেশ	Min 0, Max 2, 1 AZ	রাতে 0 করে খরচ বাঁচান
Batch job	Min 0, scheduled scale	শুধু কাজের সময় চলে
Spiky traffic	Target tracking + scheduled	Baseline + spike দুটোই সামলায়

১১.৮ Scaling ব্যর্থতার পরিস্থিতি — Troubleshooting

১. Instance চালু হচ্ছে না (capacity): AZ-তে instance type-এর capacity নেই — অন্য AZ বা instance type চেষ্টা করুন; অথবা On-Demand Capacity Reservation ব্যবহার করুন। ASG-এর activity history-তে error দেখুন। **২. Instance চালু হয়েই মারা যাচ্ছে:** User Data ব্যর্থ হচ্ছে বা health check grace period খুব ছোট — cloud-init log দেখুন, grace period বাড়ান। **৩. Flapping (বারবার বাড়া-কমা):** cooldown খুব ছোট বা target value খুব aggressive — cooldown বাড়ান, target value বাস্তবসম্মত করুন। **৪. Scaling হচ্ছে না:** CloudWatch alarm ঠিকমতো trigger হচ্ছে কি? IAM role-এ autoscaling অনুমতি আছে কি? Maximum capacity-তে পৌঁছে গেছে কি? **৫. AZ imbalance:** একটি AZ-তে সব instance — termination policy ও AZ rebalancing পরীক্ষা করুন। **৬. Spot interruption:** Spot instance ব্যবহার করলে interruption হতে পারে — ASG-কে mixed instance policy দিন (On-Demand base + Spot), যাতে interruption-এও capacity থাকে।

> **TROUBLE:** "ASG instance চালু করছে, কিন্তু ALB-তে healthy হচ্ছে না" — সবচেয়ে সাধারণ কারণ তিনটি: (১) User Data-তে অ্যাপ চালু হতে grace period-এর চেয়ে বেশি সময় লাগছে; (২) Security Group-এ ALB থেকে instance-এ যাওয়ার পথ বন্ধ; (৩) health check path ভুল। ASG activity history → instance-এর system log → ALB target health — এই ক্রমে পরীক্ষা করুন।

১১.৯ নিরাপত্তা, খরচ ও সাধারণ ভুল

নিরাপত্তা: Launch template-এ IMDSv2-only, EBS encryption, IAM role (access key নয়) — অধ্যায় ৯-এর সব নিয়ম ASG-তেও প্রযোজ্য; instance-এর tag-এ secret লিখবেন না। **খরচ:** ASG নিজে বিনামূল্যে, কিন্তু instance-গুলোর EC2, EBS ও data transfer চার্জ হয়; maximum capacity বেশি দিলে খরচের ঝুঁকি বাড়ে — AWS Budgets-এ ASG-র খরচের alert দিন; Spot instance ও Savings Plan দিয়ে খরচ কমান — সঠিক অঙ্ক AWS Pricing Calculator-এ যাচাই করুন। **সাধারণ ভুল:** (১) Minimum ১ দিয়ে production চালানো (একটি AZ ডাউন হলে অ্যাপ ডাউন); (২) grace period

খুব ছোট রাখা; (৩) ELB health check না চালিয়ে শুধু EC2 health check-এ ভরসা করা; (৪) maximum capacity অসীম রাখা; (৫) launch template-এর পুরনো version-এ আটকে থাকা; (৬) scheduled scaling-এর timezone ভুলে যাওয়া (UTC বনাম Asia/Dhaka — recurrence cron UTC-তে চলে)।

১১.১০ Production Scenario — TechNova-র Flash Sale

TechNova একটি flash sale ঘোষণা করল — ট্র্যাফিক স্বাভাবিকের পাঁচগুণ হবে। প্রস্তুতি: (১) scheduled scaling দিয়ে sale-এর এক ঘণ্টা আগে desired capacity ২ থেকে ৮ করা; (২) target tracking CPU ৬০%-এ রাখা — অপ্রত্যাশিত spike সামলাতে; (৩) maximum ১৫-তে বাড়ানো (সাময়িকভাবে); (৪) warm-up ৩০০ সেকেন্ড রাখা — নতুন instance প্রস্তুত হতে সময় পায়; (৫) CloudWatch dashboard-এ CPU, request count, ALB latency — সব একসঙ্গে দেখা। Sale শেষে scheduled scaling দিয়ে capacity আবার ২-এ নামানো। ফল: কোনো downtime নেই, বিল শুধু sale-এর সময়টুকুর জন্য বেড়েছে। এটাই elasticity-র আসল মূল্য।

১১.১১ Deployment কৌশল ও Chaos পরীক্ষা — গভীরে

ASG শুধু scaling-এর জন্য নয় — deployment-এর জন্যও। **Rolling update:** ASG ধীরে ধীরে পুরনো instance বদলে নতুন launch template version-এর instance আনে — একসঙ্গে একটি বা দুটি করে, প্রতিটি healthy হওয়ার পর পরেরটি। **Blue/green deployment:** নতুন ASG (green) পুরো তৈরি করে, ALB-র weighted target group দিয়ে ধীরে ধীরে ট্র্যাফিক সরানো হয়, সমস্যা হলে এক ক্লিকে পুরনোতে (blue) ফেরা যায়। **Canary deployment:** প্রথমে ৫% ট্র্যাফিক নতুন version-এ, metric ভালো থাকলে ২৫%, তারপর ১০০%। CodeDeploy-এর সঙ্গে ASG মিলিয়ে এই কৌশলগুলো স্বয়ংক্রিয় করা যায় — CodeDeploy deployment-এর সময় ASG-র instance-গুলোতে নতুন কোড বসায় ও health পরীক্ষা করে।

Instance refresh: ASG-র নিজস্ব বৈশিষ্ট্য — নতুন launch template version দিলে ASG নিজে থেকে instance-গুলো বদলে দেয়, আপনার নির্ধারণ করা গতিতে (যেমন একসঙ্গে ২৫%) ও health check-সহ। এটি rolling update-এর সবচেয়ে সহজ উপায়। মনে রাখবেন: instance refresh-এর সময় minimum healthy percentage নির্ধারণ করুন (যেমন ৯০%) — নইলে refresh-এর সময় capacity কমে অ্যাপ ধীর হয়ে যাবে।

Chaos পরীক্ষা — high availability প্রমাণ করুন: High availability দাবি করা সহজ, প্রমাণ করা কঠিন। প্রতি মাসে একটি game day করুন: (১) একটি AZ-র সব instance terminate করুন — ASG কত সময়ে capacity ফেরায়? (২) একটি instance-এর অ্যাপ বন্ধ করুন — ALB ও ASG কত দ্রুত সাড়া দেয়? (৩) NAT Gateway-এর route সরিয়ে দেখুন — অন্য AZ-র NAT কাজ করে কি? প্রতিটি পরীক্ষার ফল লিখে রাখুন — recovery time objective (RTO) ও recovery point objective (RPO) আসলে কত, তা অনুমান নয়, মাপা সংখ্যায় জানুন। AWS Fault Injection Simulator (FIS) দিয়ে এই পরীক্ষাগুলো নিয়ন্ত্রিতভাবে চালানো যায় — প্রথমে dev পরিবেশে, তারপর production-এ সতর্কভাবে।

কখন ASG ব্যবহার করবেন না: একটি মাত্র instance-এর workload (যেমন ছোট bastion বা একটিমাত্র ডেটাবেস) — সেখানে ASG-র বদলে সরাসরি EC2 + CloudWatch alarm যথেষ্ট। Stateful workload (যেমন একক-node ডেটাবেস) ASG-তে চালাবেন না — instance বদলে গেলে ডেটা হারাবে; ডেটাবেসের জন্য RDS Multi-AZ ব্যবহার করুন (Volume 11-এর পরের part-এ বিস্তারিত)। ASG stateless অ্যাপের জন্য — state বাইরে (RDS, ElastiCache, S3) রাখুন, instance-কে disposable ভাবুন।

১১.১২ সারাংশ

Auto Scaling Group আপনার infrastructure-কে self-healing ও elastic করে: desired capacity বজায় রাখে, অসুস্থ instance বদলায়, চাহিদা অনুযায়ী বাড়ে-কমে। Launch template হলো recipe, scaling policy হলো সিদ্ধান্তের নিয়ম, health check হলো সত্যের উৎস। Production-এ অন্তত দুটি AZ, minimum ২, target tracking + scheduled scaling —

এই সমন্বয়েই high availability আসে। আর মনে রাখবেন: scaling-এর সবচেয়ে ভালো পরীক্ষা হলো ইচ্ছা করে একটি instance মেরে দেখা — ASG কত দ্রুত তাকে ফিরিয়ে আনে।

১১.১৩ পর্যালোচনা প্রশ্ন

১. ASG-এর minimum, desired ও maximum capacity-র কাজ কী? ২. Launch template ও launch configuration-এর পার্থক্য কী? ৩. Target tracking scaling কীভাবে কাজ করে? ৪. Health check grace period কেন দরকার? ৫. EC2 health check বনাম ELB health check-এর পার্থক্য কী? ৬. Scheduled scaling কখন target tracking-এর চেয়ে ভালো? ৭. Flapping কী এবং কীভাবে থামাবেন? ৮. AZ rebalancing কী করে? ৯. একটি AZ ডাউন হলে ASG কী করে? ১০. Spot instance-এর interruption-এ ASG কীভাবে capacity ধরে রাখে?

১১.১৪ অনুশীলনী

অনুশীলন ১: TechNova ASG বানিয়ে একটি instance terminate করুন — ASG কত সময়ে নতুন instance আনে, তা মাপুন। **অনুশীলন ২:** Target tracking policy দিয়ে CPU load বাড়ান (stress tool দিয়ে) — instance বাড়তে দেখুন, load কমিয়ে instance কমতে দেখুন। **অনুশীলন ৩:** Scheduled scaling দিয়ে রাতের capacity 0 করুন, সকালে আবার ২ করুন — খরচের পার্থক্য AWS Cost Explorer-এ দেখুন।

পরিশিষ্ট • PART II — Networking & Compute

Appendix — Part II শব্দকোষ ও দ্রুত নির্দেশিকা

এই পরিশিষ্টে Part II-এর গুরুত্বপূর্ণ শব্দগুলো এক জায়গায় দেওয়া হলো — পরীক্ষার আগে ঝালিয়ে নেওয়ার জন্য।

শব্দকোষ

VPC: AWS-এর ভেতরে আপনার logically isolated private নেটওয়ার্ক। **Subnet:** VPC-র ভেতরে IP range-এর একটি ভাগ, একটি AZ-তে থাকে। **Route Table:** প্যাকেট কোথায় যাবে তার নিয়মের তালিকা। **Internet Gateway:** VPC ও ইন্টারনেটের মধ্যে managed দরজা। **NAT Gateway:** প্রাইভেট সাবনেট থেকে একমুখী outbound ইন্টারনেট অ্যাক্সেস। **VPC Endpoint:** ইন্টারনেট না ছুঁয়ে AWS সার্ভিসে যাওয়ার private রাস্তা। **Security Group:** stateful, instance-level virtual firewall — শুধু allow নিয়ম। **Network ACL:** stateless, subnet-level firewall — allow ও deny নিয়ম। **Route 53:** AWS-এর DNS সার্ভিস — hosted zone, Alias record, health check ও routing policy। **Transit Gateway:** অনেক VPC ও on-premises নেটওয়ার্কের central hub। **Direct Connect:** AWS-এ dedicated private লাইন। **EC2:** API-চালিত virtual server। **AMI:** instance চালুর template। **EBS:** persistent block storage। **IMDSv2:** token-ভিত্তিক instance metadata সার্ভিস — SSRF থেকে রক্ষা করে। **Session Manager:** port না খুলে instance পরিচালনার উপায়। **ALB:** Layer 7 load balancer — HTTP/HTTPS, path/host routing। **NLB:** Layer 4 load balancer — ultra-low latency, static IP। **Target Group:** load balancer-এর target-দের সমষ্টি ও health check-এর সেটিংস। **Auto Scaling Group:** desired capacity বজায় রাখা ও চাহিদা অনুযায়ী instance বাড়ানো-কমানোর সার্ভিস। **Launch Template:** ASG-র instance তৈরির blueprint।

দ্রুত নির্দেশিকা — TechNova Production Checklist

১. VPC `10.0.0.0/16`, অন্তত দুটি AZ, পাবলিক ও প্রাইভেট সাবনেট আলাদা। ২. প্রতি AZ-তে NAT Gateway; S3/DynamoDB-র জন্য gateway endpoint। ৩. ডেটাবেস সাবনেটে কোনো `0.0.0.0/0` route নেই। ৪. Security Group-এ SG reference, কখনো IP নয়; 22 port কোথাও খোলা নেই। ৫. IMDSv2-only, EBS encryption, Session Manager। ৬. ALB-তে HTTPS listener, ACM certificate, HTTP থেকে HTTPS redirect। ৭. ASG: minimum 2, দুটি AZ, ELB health check, target tracking 60% CPU। ৮. Flow Logs সব VPC-তে চালু; CloudWatch alarm-এ StatusCheckFailed ও 5XX। ৯. খরচ: NAT Gateway, ALB LCU ও data transfer — প্রতি মাসে AWS Pricing Calculator-এ যাচাই করুন ও Cost Explorer দেখুন। ১০. প্রতি মাসে একটি game day: একটি instance terminate করে ASG-র self-healing প্রমাণ করুন।

Part II এখানেই শেষ। Part III-এ আমরা ডেটা স্তরে যাব — RDS, DynamoDB, S3 ও caching — যেখানে এই নেটওয়ার্ক ও compute ভিত্তির ওপর আসল অ্যাপ্লিকেশন ডেটা থাকবে। এই Part II-এর প্রতিটি অধ্যায়ের CLI কমান্ড হাতে চালিয়ে দেখুন, কারণ পড়া আর করা — দুটি আলাদা দক্ষতা। একটি VPC নিজে বানানো, তাতে ALB ও ASG চালানো, তারপর ইচ্ছা করে একটি instance মেরে self-healing দেখা — এই অভিজ্ঞতাটুকুই আপনাকে পরীক্ষার প্রশ্নের চেয়ে production incident-এর জন্য বেশি প্রস্তুত করবে।

PART III • DEVOPS ENGINEERING MASTERY

Storage, Database & Application Services

Amazon S3, EBS/EFS/FSx ও ব্যাকআপ, RDS ও Aurora, DynamoDB ও ElastiCache, Route 53, CloudFront, ACM ও Edge আর্কিটেকচার।

অধ্যায় ১২ • PART III — Storage, Database & Application Services

Chapter 12 — Amazon S3 Engineering

১২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) **Amazon S3** (Simple Storage Service) কী, কেন এটি ক্লাউডের সবচেয়ে মৌলিক বিল্ডিং ব্লক, তা প্রথম নীতি (first principles) থেকে ব্যাখ্যা করতে; (২) **bucket** (বালতি — ডেটা রাখার ধারক) ও **object** (বস্তু — ফাইল ও তার metadata) তৈরি, আপলোড, সংস্করণ ও মুছে ফেলার কাজ Console ও AWS CLI দিয়ে করতে; (৩) সব **storage class** (সংরক্ষণ শ্রেণি) — Standard, Intelligent-Tiering, Standard-IA, One Zone-IA, Glacier Instant Retrieval, Glacier Flexible Retrieval, Glacier Deep Archive — এর মধ্যে তুলনা করে সঠিক শ্রেণি বেছে নিতে; (৪) **versioning** (সংস্করণ ব্যবস্থা), **lifecycle policy** (জীবনচক্র নীতি), **replication** (প্রতিলিপিকরণ) ও **presigned URL** (স্বাক্ষরিত সাময়িক লিংক) বাস্তবায়ন করতে; (৫) **SSE-S3, SSE-KMS, SSE-C** এনক্রিপশন, **bucket policy** (বালতি নীতি), **Block Public Access** (জনসাধারণের প্রবেশ বন্ধ) দিয়ে S3 সুরক্ষিত করতে; (৬) **static website hosting** (স্থির ওয়েবসাইট হোস্টিং) চালু করতে ও এর সীমাবদ্ধতা বুঝতে; এবং (৭) S3-র খরচ অপ্টিমাইজ করতে ও সাধারণ ভুল এড়াতে।

১২.২ পূর্বশর্ত ঝালাই (Prerequisite Refresher)

এই অধ্যায়ের জন্য Volume 11-এর আগের অধ্যায়গুলো থেকে আপনার জানা দরকার: **AWS IAM** (Identity and Access Management — কে কী করতে পারবে, তার নীতি), **region** (অঞ্চল — যেমন ap-south-1) ও **Availability Zone** (AZ — একটি region-এর ভেতরে আলাদা ডেটা সেন্টার), **VPC** (Virtual Private Cloud) ও **security group**, এবং AWS CLI v2-র মৌলিক ব্যবহার (`aws configure`, `aws s3 ls`)। যদি IAM policy-র JSON গঠন ভুলে গিয়ে থাকেন, মনে রাখুন: প্রতিটি policy-তে থাকে `Effect` (Allow/Deny), `Principal` (কার জন্য), `Action` (কী কাজ) ও `Resource` (কোন জিনিসের ওপর)। S3-র নিরাপত্তা এই একই ভাষায় লেখা হয়, শুধু Resource হয় bucket বা object-এর ARN (Amazon Resource Name)। আরেকটি গুরুত্বপূর্ণ ধারণা — **REST API**: S3 আসলে একটি HTTP API; Console, CLI, SDK — সবাই শেষ পর্যন্ত HTTPS request পাঠায়। এই বোঝাপড়া থাকলে presigned URL বা bucket policy-র আচরণ আর রহস্য মনে হবে না।

১২.৩ কেন S3 শিখতেই হবে — এটি কেন গুরুত্বপূর্ণ

প্রায় প্রতিটি AWS আর্কিটেকচারের নিচে S3 আছে। TechNova-র কথাই ধরুন: তাদের ওয়েবসাইটের ছবি, ব্যবহারকারীর আপলোড করা PDF, অ্যাপ্লিকেশনের লগ ফাইল, ডেটাবেসের ব্যাকআপ, CloudFront-এর origin, এমনকি AWS Lambda-র deployment package — সবই S3-তে থাকে। S3 ছাড়া ক্লাউডে ডেটা রাখার কোনো সস্তা, টেকসই ও অসীম স্কেলের জায়গা নেই। একটি পরিসংখ্যান মনে রাখুন: S3-র ডিজাইন করা হয়েছে ১১টি ৯ (99.999999999%) **durability** (স্থায়িত্ব — ডেটা হারিয়ে না যাওয়ার নিশ্চয়তা) দেওয়ার জন্য; অর্থাৎ ১০,০০০টি object-এর মধ্যে বছরে গড়ে ০.০০০০০০০০০১টি হারাতে পারে — কার্যত কখনোই নয়। কিন্তু এই স্থায়িত্বের উল্টো দিকে আছে **availability** (উপলব্ধতা — চাইলে ডেটা পাওয়া যাবে কি না), যা Standard শ্রেণিতে 99.99%। দুটি আলাদা জিনিস — ডেটা হারানো আর ডেটা সাময়িকভাবে না পাওয়া এক নয়। একজন DevOps প্রকৌশলী হিসেবে আপনার কাজ হলো এই পার্থক্য বোঝা, সঠিক storage class বেছে নেওয়া, ভুল করে bucket public না করা, এবং বিল নিয়ন্ত্রণে রাখা। ২০১৭ সালের পর থেকে বহু বিখ্যাত ডেটা লিকের মূলে ছিল ভুলভাবে public করা S3 bucket — এই অধ্যায়ে সেই ভুল যাতে আপনি না করেন, তার পুরো কৌশল শেখানো হবে।

১২.৪ প্রথম নীতি থেকে S3: আগে কী সমস্যা ছিল

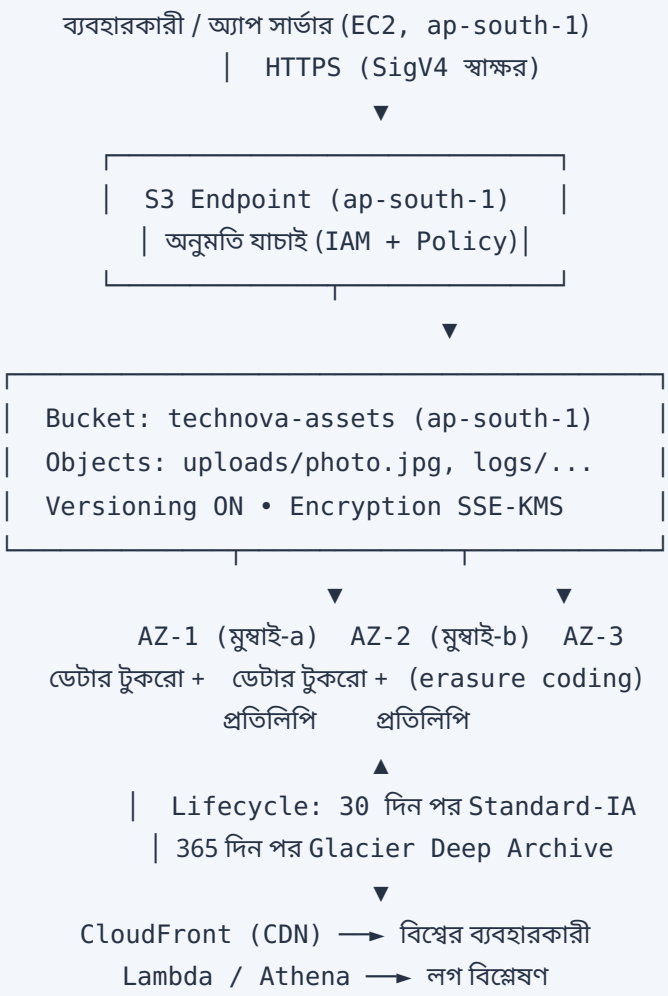
ক্লাউডের আগে কোম্পানিকে নিজের সার্ভার রুমে হার্ড ডিস্ক কিনতে হতো। ডেটা বাড়লে নতুন ডিস্ক, নতুন সার্ভার, নতুন র‍্যাক কিনতে হতো — সপ্তাহের পর সপ্তাহ অপেক্ষা। ডিস্ক নষ্ট হলে ডেটা হারাত, তাই RAID (একাধিক ডিস্কে ডেটা ছড়িয়ে রাখা) ও টেপ ব্যাকআপ লাগত। আর ফাইল সিস্টেমের (file system) সীমা ছিল: একটি ফোল্ডারে লাখ লাখ ফাইল রাখলে ধীর হয়ে যেত, ফাইলের নামের দৈর্ঘ্য ও পথের (path) সীমা ছিল, এবং একই ফাইল দুই সার্ভার থেকে ব্যবহার করা কঠিন ছিল। S3 এই সমস্যার উত্তরে এসেছে একটি ভিন্ন ধারণা নিয়ে — **object storage** (অবজেক্ট সংরক্ষণ)। এখানে কোনো ফোল্ডার গাছ (directory tree) নেই, কোনো ডিস্ক পার্টিশন নেই। আছে শুধু দুটি জিনিস: **bucket** — একটি নামযুক্ত ধারক, যার নাম সারা বিশ্বে অনন্য (globally unique); এবং **object** — ডেটা (যেকোনো ফাইল: ছবি, ভিডিও, ZIP, JSON) + একটি **key** (চাবি — object-এর নাম, যেমন `uploads/2026/photo.jpg`) + **metadata** (তথ্য সম্পর্কে তথ্য — যেমন Content-Type, আপলোডের সময়)। Key-তে স্ল্যাশ (/) থাকলেও আসলে কোনো ফোল্ডার তৈরি হয় না — Console শুধু স্ল্যাশ দেখে ফোল্ডারের মতো সাজিয়ে দেখায়, এটিকে বলে **prefix** (উপসর্গ)। এই সরলতাই S3-র শক্তি: কোনো সার্ভার পরিচালনা নেই, ধারণক্ষমতা কার্যত অসীম, আর আপনি শুধু যতটুকু রাখেন ততটুকুর জন্য দেন।

Object storage বনাম File storage বনাম Block storage: Block storage (যেমন EBS) হলো কাঁচা ডিস্ক — অপারেটিং সিস্টেম সেখানে ফাইল সিস্টেম বানায়। File storage (যেমন EFS) হলো ভাগ করা ফোল্ডার — অনেক সার্ভার একসঙ্গে ব্যবহার করে। Object storage (S3) হলো HTTP API দিয়ে ডেটা রাখা ও নেওয়া — কোনো সার্ভার মাউন্ট করতে হয় না, ইন্টারনেটের যেকোনো জায়গা থেকে REST API-তে পৌঁছানো যায়। প্রতিটির ব্যবহার আলাদা; ১৩ নম্বর অধ্যায়ে পূর্ণ তুলনা আছে।

১২.৫ S3 ভেতরে কীভাবে কাজ করে

আপনি যখন `aws s3 cp photo.jpg s3://technova-assets/uploads/photo.jpg` চালান, তখন যা ঘটে তা ধাপে ধাপে বুঝুন। (১) CLI ফাইলটি পড়ে HTTPS PUT request বানায়, তাতে আপনার IAM credential দিয়ে স্বাক্ষর (signature) যোগ করে। (২) Request যায় S3-র **endpoint**-এ (যেমন `s3.ap-south-1.amazonaws.com`)। (৩) S3 প্রথমে যাচাই করে: bucket আছে কি না, আপনার অনুমতি আছে কি না, bucket policy কী বলে। (৪) তারপর ডেটা লেখা হয় — কিন্তু একটি ডিস্কে নয়; S3 ডেটাকে ছোট ছোট টুকরো করে একাধিক Availability Zone-এর একাধিক ডিস্কে ছড়িয়ে দেয়, সঙ্গে **erasure coding** ও প্রতিলিপি রাখে যাতে দুটি AZ নষ্ট হলেও ডেটা বেঁচে থাকে। (৫) লেখা শেষ হলে S3 একটি **ETag** (entity tag — object-এর checksum-এর মতো পরিচয়) ফেরত দেয়। পড়ার সময় (GET) উল্টো প্রক্রিয়া: যেকোনো AZ থেকে টুকরোগুলো জোড়া লাগিয়ে ডেটা ফেরত দেওয়া হয়। এই কারণে S3-র **throughput** (প্রতি সেকেন্ডে কত ডেটা যায়) প্রায় অসীমভাবে বাড়ে — হাজার হাজার parallel request একসঙ্গে চলতে পারে। একটি গুরুত্বপূর্ণ বৈশিষ্ট্য: S3 এখন **strong read-after-write consistency** (শক্তিশালী সামঞ্জস্য) দেয় — অর্থাৎ PUT সফল হওয়ার পরপরই GET করলে নতুন ডেটাই পাবেন, পুরোনো নয়। ২০২০ সালের আগে eventual consistency ছিল; এখন আর সেই জটিলতা নেই।

চিত্র ১২.১ — S3 আর্কিটেকচার: TechNova-র ডেটা প্রবাহ



১২.৬ Bucket ও Object: নামকরণ, সীমা ও গঠন

Bucket-এর নাম সারা AWS-এ একটিই হতে পারে — `technova-assets` একবার কেউ নিলে আর কেউ নিতে পারবে না। নামে ছোট হাতের অক্ষর, সংখ্যা, হাইফেন ও ডট চলবে; বড় হাতের অক্ষর চলবে না; নাম ৩ থেকে ৬৩ অক্ষরের মধ্যে হতে হবে। একটি object সর্বোচ্চ ৫ টেরাবাইট (TB) হতে পারে, তবে একবারে PUT দিয়ে সর্বোচ্চ ৫ গিগাবাইট (GB); তার বেশি হলে **multipart upload** (বহু খণ্ডে আপলোড) করতে হয় — CLI স্বয়ংক্রিয়ভাবে বড় ফাইল ভেঙে আপলোড করে। প্রতিটি object-এর সঙ্গে থাকে **metadata**: system metadata (যেমন Content-Type, Content-Length, Last-Modified) ও user metadata (আপনার নিজের `x-amz-meta-` দিয়ে শুরু করা key-value)। আর থাকে **object tagging** — যেমন `project=website`, `env=prod` — যা দিয়ে lifecycle policy বা খরচ ভাগ (cost allocation) করা যায়। মনে রাখবেন: bucket তৈরি হয় একটি region-এ, কিন্তু bucket-এর নাম global; object-এর ডেটা সেই region-এর AZ-গুলোতেই থাকে, যদি না আপনি replication চালু করেন।

১২.৬.১ Console দিয়ে Bucket তৈরি

Console-এ S3 → Create bucket → নাম দিন `technova-assets-111122223333` (অ্যাকাউন্ট নম্বর যোগ করলে নাম অনন্য হওয়া সহজ হয়) → Region: Asia Pacific (Mumbai) ap-south-1 → **Block all public access** টিক দেওয়া আছে কি না দেখুন (ডিফল্টে চালু থাকে — এভাবেই রাখুন) → Bucket Versioning: Enable → Default encryption:

SSE-S3 বা SSE-KMS বেছে নিন → Create bucket। এই চারটি সিদ্ধান্ত — region, public access বন্ধ, versioning চালু, encryption চালু — প্রতিটি নতুন bucket-এর জন্য অভ্যাসে পরিণত করুন।

১২.৬.২ AWS CLI দিয়ে Bucket ও Object পরিচালনা

```
# Bucket তৈরি (region ap-south-1)
aws s3api create-bucket --bucket technova-assets-111122223333 \
  --region ap-south-1 \
  --create-bucket-configuration LocationConstraint=ap-south-1

# Bucket তালিকা দেখুন
aws s3 ls

# ফাইল আপলোড করুন (cp = copy)
aws s3 cp ./photo.jpg s3://technova-assets-111122223333/uploads/photo.jpg

# পুরো ফোল্ডার সিঙ্ক করুন (শুধু বদলানো ফাইল আপলোড হবে)
aws s3 sync ./website/ s3://technova-assets-111122223333/website/ --delete

# Object তালিকা ও বিস্তারিত দেখুন
aws s3 ls s3://technova-assets-111122223333/uploads/ --recursive --human-readable
aws s3api head-object --bucket technova-assets-111122223333 --key uploads/photo.jpg

# Object ডাউনলোড করুন
aws s3 cp s3://technova-assets-111122223333/uploads/photo.jpg ./downloaded.jpg
```

লক্ষ করুন: `aws s3` হলো উচ্চ-স্তরের (high-level) কমান্ড — cp, sync, mv সহজ কাজের জন্য। `aws s3api` হলো নিম্ন-স্তরের API — versioning, lifecycle, policy-র মতো সূক্ষ্ম নিয়ন্ত্রণের জন্য। দুটোই শিখুন; পরীক্ষায় ও বাস্তবে দুটোই লাগে।

> **PRO TIP:** `aws s3 sync --delete` খুব শক্তিশালী কিন্তু বিপজ্জনক — এটি destination-এ থাকা অতিরিক্ত ফাইল মুছে দেয়। প্রথমে `--dryrun` যোগ করে দেখুন কী হবে, তারপর আসল কমান্ড চালান। Production bucket-এ কখনো `--delete` ছাড়া যাচাই করে চালাবেন না।

১২.৭ Storage Classes — সঠিক শ্রেণি বেছে নেওয়া

S3-র সবচেয়ে বড় খরচ-সিদ্ধান্ত হলো storage class। একই ডেটা Standard-এ রাখলে দাম বেশি, Glacier Deep Archive-এ রাখলে অনেক কম — কিন্তু পেতে সময় লাগে ঘণ্টার পর ঘণ্টা। ভুল শ্রেণি বেছে নিলে হয় বিল বাড়ে, নয়তো প্রয়োজনের সময় ডেটা পাওয়া যায় না। নিচের টেবিলটি মুখস্থ না করে বোঝার চেষ্টা করুন: প্রতিটি শ্রেণি একটি trade-off (বিনিময়) — দাম বনাম পুনরুদ্ধারের গতি বনাম উপলব্ধতা।

Storage Class	কখন ব্যবহার করবেন	পুনরুদ্ধার সময়	AZ	মনে রাখার কথা
S3 Standard	প্রতিদিন ব্যবহৃত ডেটা — ওয়েবসাইট ছবি, অ্যাপ ফাইল	মিলিসেকেন্ড	৩+ AZ	ডিফল্ট; সবচেয়ে দামি সংরক্ষণ, সস্তা request
S3 Intelligent-Tiering	ব্যবহারের ধরন অজানা বা বদলায়	মিলিসেকেন্ড	৩+ AZ	স্বয়ংক্রিয়ভাবে স্তর বদলায়; ছোট monitoring ফি

S3 Standard-IA	মাসে এক-দুবার লাগে এমন ডেটা — ব্যাকআপ	মিলিসেকেন্ড	৩+ AZ	কম সংরক্ষণ দাম, বেশি retrieval ফি; ৩০ দিনের ন্যূনতম
S3 One Zone-IA	আবার তৈরি করা যায় এমন ডেটা	মিলিসেকেন্ড	১টি AZ	সস্তা, কিন্তু AZ নষ্ট হলে ডেটা হারাতে পারে
Glacier Instant Retrieval	ত্রৈমাসিকে একবার লাগে, তবু দ্রুত চাই	মিলিসেকেন্ড	৩+ AZ	আর্কাইভের দামে তাৎক্ষণিক পুনরুদ্ধার
Glacier Flexible Retrieval	বার্ষিক আর্কাইভ, কয়েক মিনিট-ঘণ্টা অপেক্ষা চলে	মিনিট–ঘণ্টা	৩+ AZ	Expedited/Standard/Bulk retrieval বিকল্প
Glacier Deep Archive	আইনি/নিয়ন্ত্রক সংরক্ষণ, বছরে একবারও লাগে না	ঘণ্টা (১২ পর্যন্ত)	৩+ AZ	সবচেয়ে সস্তা; ১৮০ দিনের ন্যূনতম

একটি বাস্তব উদাহরণ: TechNova-র ওয়েবসাইটের ছবি Standard-এ থাকে। ৩০ দিন না-খোলা ছবি lifecycle policy দিয়ে Standard-IA-তে যায়। এক বছরের পুরোনো লগ Deep Archive-এ যায়। আর ব্যবহারের ধরন না জানা থাকলে Intelligent-Tiering সবচেয়ে নিরাপদ — এটি ৩০ দিন না-ব্যবহৃত object নিজে থেকে সস্তা স্তরে নামিয়ে দেয়, আবার ব্যবহার হলে ফিরিয়ে আনে। তবে মনে রাখবেন: IA ও Glacier শ্রেণিতে **ন্যূনতম সংরক্ষণ সময়** আছে (৩০/৯০/১৮০ দিন) — তার আগে মুছলে বা স্তর বদলালে পুরো সময়ের দাম দিতে হয়। আর প্রতিটি শ্রেণিতে **request-এর দাম** আলাদা — Glacier থেকে বারবার ডেটা তোলা Standard-এর চেয়ে অনেক দামি হতে পারে। তাই দাম যাচাই না করে শুধু "সস্তা storage" দেখে সিদ্ধান্ত নেবেন না; AWS Pricing Calculator-এ যাচাই করুন।

```
# Object-এর storage class বদলান (CLI)
aws s3 cp s3://technova-assets-111122223333/logs/app.log \
  s3://technova-assets-111122223333/logs/app.log \
  --storage-class STANDARD_IA

# আপলোডের সময়ই শ্রেণি ঠিক করুন
aws s3 cp ./backup.zip s3://technova-assets-111122223333/backups/ \
  --storage-class DEEP_ARCHIVE
```

১২.৮ Versioning — ভুল করে মোছা থেকে বাঁচার উপায়

Versioning চালু থাকলে S3 প্রতিটি object-এর প্রতিটি সংস্করণ (version) আলাদা করে রাখে। আপনি `photo.jpg` ওভাররাইট করলে পুরোনোটি হারায় না — সেটি একটি পুরোনো version হয়ে থাকে, প্রতিটির আলাদা **version ID** থাকে। ভুল করে মুছে ফেললে কী হয়? S3 object-টি সত্যিই মোছে না — একটি **delete marker** (মোছার চিহ্ন) বসায়; version গুলো তখনো আছে, delete marker সরালেই ফাইল ফিরে আসে। এটাই versioning-এর জাদু। কিন্তু সাবধান: versioning চালু থাকলে প্রতিটি version-এর জন্য আলাদা storage বিল আসে — পুরোনো version জমতে থাকলে বিল বাড়ে। তাই versioning-এর সঙ্গে lifecycle policy জুড়ে দিন: যেমন, ৯০ দিনের পুরোনো version Deep Archive-এ পাঠান, ৩৬৫ দিন পর মুছে দিন। আরেকটি সুরক্ষা: **MFA Delete** — version মুছতে হলে MFA (multi-factor authentication) ডিভাইসের কোড লাগে; root অ্যাকাউন্টের জন্য এটি production bucket-এ চালু করা ভালো অভ্যাস। মনে রাখবেন: versioning একবার চালু করলে বন্ধ করা যায় না, শুধু suspend করা যায়; suspend করলেও আগের version গুলো থেকে যায়।

```
# Versioning চালু করুন
aws s3api put-bucket-versioning --bucket technova-assets-111122223333 \
  --versioning-configuration Status=Enabled
```

```
# Version তালিকা দেখুন
aws s3api list-object-versions --bucket technova-assets-111122223333 \
  --prefix uploads/photo.jpg

# নির্দিষ্ট version ডাউনলোড করুন
aws s3api get-object --bucket technova-assets-111122223333 \
  --key uploads/photo.jpg --version-id CHANGE-ME-VERSION-ID ./old-photo.jpg
```

১২.৯ Lifecycle Policy — ডেটার জীবনচক্র স্বয়ংক্রিয় করা

Lifecycle policy হলো S3-কে দেওয়া নিয়মের সেট: "এই বয়সের object এই শ্রেণিতে নাও, এই বয়সে মুছে দাও।" হাতে হাতে হাজার হাজার ফাইল সরানো অসম্ভব; policy একবার লিখলে S3 নিজে প্রতিদিন নিয়ম চালায়। TechNova-র লগ bucket-এর উদাহরণ দেখুন: ৩০ দিন পর Standard-IA, ৯০ দিন পর Glacier Flexible Retrieval, ৩৬৫ দিন পর মুছে ফেলা; আর অসম্পূর্ণ multipart upload ৭ দিন পর মুছে ফেলা (এটি ভুলে গেলে অসম্পূর্ণ খণ্ডের বিল চলতেই থাকে — খুব সাধারণ ভুল)। Lifecycle rule লেখা যায় Console-এ (Management → Create lifecycle rule) অথবা JSON ফাইলে।

```
{
  "Rules": [
    {
      "ID": "logs-to-archive",
      "Status": "Enabled",
      "Filter": { "Prefix": "logs/" },
      "Transitions": [
        { "Days": 30, "StorageClass": "STANDARD_IA" },
        { "Days": 90, "StorageClass": "GLACIER" }
      ],
      "Expiration": { "Days": 365 },
      "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
    }
  ]
}
# প্রয়োগ করুন:
# aws s3api put-bucket-lifecycle-configuration \
#   --bucket technova-assets-111122223333 \
#   --lifecycle-configuration file://lifecycle.json
```

এই JSON-এর প্রতিটি লাইনের অর্থ: `ID` নিয়মের নাম; `Filter.Prefix` বলে নিয়ম শুধু `logs/` দিয়ে শুরু হওয়া object-এ প্রযোজ্য; `Transitions` বলে ৩০ দিনে Standard-IA ও ৯০ দিনে Glacier-এ যাবে; `Expiration` বলে ৩৬৫ দিনে মুছে যাবে; `AbortIncompleteMultipartUpload` অসম্পূর্ণ আপলোড পরিষ্কার করে। সতর্কতা: Expiration দিয়ে production ডেটা মুছে ফেলার আগে নিয়মটি একটি পরীক্ষামূলক bucket-এ যাচাই করুন — lifecycle-এর ভুলে ডেটা মুছে গেলে versioning ছাড়া ফেরানো যায় না।

১২.১০ Encryption — ডেটা গোপন রাখা

S3-তে ডেটা দুইভাবে সুরক্ষিত থাকে: **encryption at rest** (সংরক্ষিত অবস্থায় এনক্রিপশন — ডিস্কে লেখা ডেটা গোপন) ও **encryption in transit** (পরিবহণে এনক্রিপশন — HTTPS/TLS দিয়ে যাতায়াতের সময় গোপন)। Transit-এর জন্য

সবসময় HTTPS ব্যবহার করুন; S3 bucket policy দিয়ে HTTP request বন্ধ করে দেওয়া যায়। At rest-এর জন্য তিনটি **server-side encryption** (SSE) বিকল্প আছে:

- **SSE-S3 (AES-256):** S3 নিজে key পরিচালনা করে। সবচেয়ে সহজ, কোনো অতিরিক্ত খরচ নেই। বেশিরভাগ সাধারণ ব্যবহারের জন্য যথেষ্ট।
- **SSE-KMS:** AWS Key Management Service-এর **customer managed key** (CMK) ব্যবহার করে। Key আলাদাভাবে পরিচালনা, rotation (নিয়মিত বদল) ও audit (CloudTrail-এ কে key ব্যবহার করল তার লগ) করা যায়। সংবেদনশীল ডেটা, যেমন ব্যবহারকারীর ব্যক্তিগত তথ্যের জন্য এটাই সঠিক। তবে KMS API call-এর আলাদা খরচ আছে — লাখ লাখ ছোট object-এ KMS ব্যবহার করলে বিল বাড়তে পারে।
- **SSE-C:** আপনি নিজের key দেন, S3 শুধু ব্যবহার করে, সংরক্ষণ করে না। নিজের key পরিচালনার পূর্ণ নিয়ন্ত্রণ চাইলে ব্যবহার করুন, তবে key হারালে ডেটা চিরতরে হারাবে — AWS-ও ফেরাতে পারবে না।

এছাড়া **client-side encryption** আছে — আপলোডের আগেই আপনার অ্যাপ ডেটা এনক্রিপ্ট করে; S3 তখন শুধু গোপন ডেটা দেখে। Default encryption চালু করলে bucket-এ আসা প্রতিটি object স্বয়ংক্রিয়ভাবে এনক্রিপ্ট হয়।

```
# Bucket-এর default encryption SSE-KMS করুন
aws s3api put-bucket-encryption --bucket technova-assets-111122223333 \
  --server-side-encryption-configuration '{
    "Rules": [{
      "ApplyServerSideEncryptionByDefault": {
        "SSEAlgorithm": "aws:kms",
        "KMSMasterKeyID": "CHANGE-ME-KMS-KEY-ID"
      },
      "BucketKeyEnabled": true
    }]
  }'
# BucketKeyEnabled চালু করলে KMS call কমে, খরচ কমে।
```

> **WARNING:** Encryption চালু থাকলেই bucket নিরাপদ নয়। Encryption ডিস্ক চুরি থেকে বাঁচায়, ভুল অনুমতি থেকে নয়। সবচেয়ে বড় ঝুঁকি হলো ভুল bucket policy বা public access — এনক্রিপ্ট করা ডেটাও public bucket থেকে যে কেউ ডাউনলোড করতে পারে। Encryption ও access control — দুটোই চাই।

১২.১১ Bucket Policy — লাইন ধরে ধরে ব্যাখ্যা

Bucket policy হলো bucket-এর সঙ্গে লাগানো একটি JSON নথি, যা বলে দেয় কে (Principal), কী (Action), কোন object-এ (Resource) কী করতে পারবে। IAM policy ব্যবহারকারীকে অনুমতি দেয়; bucket policy resource-কে সুরক্ষা দেয় — দুটি একসঙ্গে মূল্যায়ন হয়। নিচের নীতিটি TechNova-র একটি সাধারণ production প্যাটার্ন: শুধুমাত্র CloudFront (OAC — Origin Access Control) দিয়ে ছবি পড়া যাবে, আর সবাইকে HTTPS ব্যবহার করতে হবে।

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontRead",
      "Effect": "Allow",
      "Principal": { "Service": "cloudfront.amazonaws.com" },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::technova-assets-111122223333/*",
```

```
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::111122223333:distribution/CHANGE-ME-DIST-ID"
        }
      }
    },
    {
      "Sid": "DenyInsecureTransport",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::technova-assets-111122223333",
        "arn:aws:s3:::technova-assets-111122223333/*"
      ],
      "Condition": { "Bool": { "aws:SecureTransport": "false" } }
    }
  ]
}
```

লাইন ধরে ব্যাখ্যা: `Version: 2012-10-17` — policy ভাষার সংস্করণ, সবসময় এটাই লিখুন। প্রথম Statement (`Sid: AllowCloudFrontRead`): `Effect: Allow` মানে অনুমতি দেওয়া; `Principal: cloudfront.amazonaws.com` মানে শুধু CloudFront সার্ভিস; `Action: s3:GetObject` মানে শুধু পড়া, লেখা নয়; `Resource: .../*` মানে bucket-এর ভেতরের সব object; `Condition: AWS:SourceArn` মানে শুধুমাত্র TechNova-র নির্দিষ্ট CloudFront distribution — অন্য কারো distribution দিয়ে পড়া যাবে না। দ্বিতীয় Statement (`DenyInsecureTransport`): `Effect: Deny` সবসময় Allow-কে হারিয়ে দেয়; `Principal: *` মানে সবাই; `Action: s3:*` মানে সব কাজ; `Condition: aws:SecureTransport: false` মানে HTTP (অসুরক্ষিত) হলে সব বন্ধ — শুধু HTTPS চলবে। এই দুটি statement একসঙ্গে একটি নিরাপদ public-content bucket-এর ভিত্তি তৈরি করে।

```
# Policy প্রয়োগ করুন (আগে --dryrun নেই — পরীক্ষামূলক bucket-এ যাচাই করুন)
aws s3api put-bucket-policy --bucket technova-assets-111122223333 \
  --policy file://bucket-policy.json
# Policy দেখুন
aws s3api get-bucket-policy --bucket technova-assets-111122223333
```

১২.১২ Block Public Access ও একটি বাস্তব ভুলের কেস স্টাডি

Block Public Access (BPA) হলো S3-র চারটি সুইচের সেট — অ্যাকাউন্ট স্তরে ও bucket স্তরে: (১) নতুন public ACL বন্ধ করা, (২) যেকোনো public ACL উপেক্ষা করা, (৩) নতুন public bucket policy বন্ধ করা, (৪) যেকোনো public bucket policy উপেক্ষা করা। ডিফল্টে চারটিই চালু থাকে। যতক্ষণ না আপনার সত্যিই public bucket দরকার (যেমন static website), এগুলো বন্ধ করবেন না। আর public করতে হলে শুধুমাত্র নির্দিষ্ট bucket-এ, নির্দিষ্ট কারণে, নির্দিষ্ট সময়ের জন্য করুন — অ্যাকাউন্ট-স্তরের BPA কখনো বন্ধ করবেন না।

REAL WORLD — কেস স্টাডি: "পরীক্ষার জন্য খুলেছিলাম, বন্ধ করতে ভুলে গিয়েছিলাম": একটি কাল্পনিক কিন্তু অতি পরিচিত ঘটনা — TechNova-র এক ডেভেলপার static website পরীক্ষার জন্য bucket-টি public করলেন, bucket policy-তে `Principal: "*"` ও `Action: s3:GetObject` দিলেন। পরীক্ষা শেষ হলো, কিন্তু policy সরানো হলো না। দুই সপ্তাহ পর সেই bucket-এ ভুল করে ব্যবহারকারীদের ব্যক্তিগত PDF আপলোড করা হলো — একই bucket, একই policy। Search engine ও স্বয়ংক্রিয় স্ক্যানার public bucket খুঁজে পেল, ফাইলগুলো ছড়িয়ে পড়ল। ক্ষতি: শুধু ডেটা লিক নয় — গ্রাহকের আস্থা, আইনি ঝুঁকি, আর AWS বিল (কেউ বারবার বড় ফাইল ডাউনলোড করলে data transfer বিল)। শিক্ষা: (১) public ও private ডেটা কখনো এক bucket-এ রাখবেন না — আলাদা bucket করুন; (২) প্রতিটি public bucket-এর পাশে একটি মেয়াদ (expiry) ও পর্যালোচনার তারিখ লিখে রাখুন; (৩) AWS Config rule `s3-bucket-public-read-prohibited` ও GuardDuty চালু রাখুন যাতে public হওয়ার সঙ্গে সঙ্গে সতর্কবার্তা আসে; (৪) `aws s3api get-public-access-block` দিয়ে নিয়মিত যাচাই করুন।

১২.১৩ Replication, Presigned URL ও Static Website Hosting

Replication (প্রতিলিপিকরণ) হলো object স্বয়ংক্রিয়ভাবে অন্য bucket-এ কপি করা — একই region-এ (Same-Region Replication, SRR) বা অন্য region-এ (Cross-Region Replication, CRR)। কেন দরকার? দুর্যোগ থেকে বাঁচতে (এক region নষ্ট হলে অন্য region-এ ডেটা থাকে), আইনি কারণে ডেটা অন্য দেশে রাখতে, বা ব্যবহারকারীর কাছাকাছি ডেটা রাখতে। Replication-এর জন্য versioning দুই bucket-এই চালু থাকতে হয়, আর একটি IAM role দরকার যা S3-কে কপি করার অনুমতি দেয়। মনে রাখবেন: replication শুধু নতুন object-এর জন্য কাজ করে — আগের object কপি করতে **S3 Batch Replication** লাগে। আর replication-এর খরচ আছে — storage দুই জায়গায়, data transfer, request — Pricing Calculator-এ যাচাই করুন।

Presigned URL হলো একটি সাময়িক, স্বাক্ষরিত লিংক — যাতে অনুমতি ছাড়াই কেউ নির্দিষ্ট সময়ের জন্য একটি private object ডাউনলোড বা আপলোড করতে পারে। উদাহরণ: TechNova-র অ্যাপে ব্যবহারকারী প্রোফাইল ছবি আপলোড করতে চায় — সার্ভার নিজে ফাইল না নিয়ে একটি presigned PUT URL দেয় (মেয়াদ ১৫ মিনিট), ব্রাউজার সরাসরি S3-তে আপলোড করে। এতে সার্ভারের ব্যান্ডউইথ বাঁচে, আর URL-এর মেয়াদ শেষ হলে লিংক অকেজো হয়ে যায়। কিন্তু সাবধান: presigned URL যার হাতে যাবে সে-ই ব্যবহার করতে পারবে — তাই মেয়াদ ছোট রাখুন (৫–১৫ মিনিট), আর সংবেদনশীল ফাইলে কখনো দীর্ঘ মেয়াদ দেবেন না।

```
# Presigned GET URL তৈরি (মেয়াদ 900 সেকেন্ড = 15 মিনিট)
aws s3 presign s3://technova-assets-111122223333/private/report.pdf \
  --expires-in 900

# Presigned PUT URL (SDK দিয়ে তৈরি করতে হয়; CLI-তে সরাসরি নেই)
# Node.js (AWS SDK v3) উদাহরণ — সার্ভার কোডে ব্যবহার করুন:
# const url = await getSignedUrl(s3Client,
#   new PutObjectCommand({ Bucket, Key, ContentType }), { expiresIn: 900 });
```

Static website hosting: S3 bucket-কে ওয়েবসাইট হিসেবে চালানো যায় — HTML, CSS, JavaScript, ছবি রেখে। Console-এ Properties → Static website hosting → Enable → Index document: `index.html`, Error document: `error.html`। তখন bucket-এর website endpoint (যেমন `http://technova-assets-111122223333.s3-website.ap-south-1.amazonaws.com`) থেকে সাইট চলে। কিন্তু সীমাবদ্ধতা জানুন: (১) এটি শুধু **static** (স্থির) কনটেন্ট — কোনো সার্ভার-সাইড কোড (Node.js, PHP) চলে না; (২) HTTPS সরাসরি নেই — HTTPS চাইলে CloudFront সামনে বসাতে হয় (১৬ নম্বর অধ্যায় দেখুন); (৩) routing নিয়ম সীমিত; (৪) bucket-টি public করতে হয়, তাই শুধুমাত্র সত্যিকারের

public কনটেন্ট রাখুন। Production-এ সঠিক প্যাটার্ন: S3 (private) + CloudFront (OAC) + ACM সার্টিফিকেট + Route 53 — এতে HTTPS, caching ও নিরাপত্তা তিনটিই পাওয়া যায়।

১২.১৪ Production Scenario, নিরাপত্তা, খরচ ও ভুল

Production scenario — TechNova-র মিডিয়া পাইপলাইন: ব্যবহারকারী ছবি আপলোড করে → presigned URL দিয়ে S3-তে যায় (Standard, SSE-KMS, versioning ON) → S3 Event Notification Lambda-কে ডাকে → Lambda ছবির thumbnail বানিয়ে আরেকটি bucket-এ রাখে → CloudFront দিয়ে বিশ্বব্যাপী পরিবেশন করা হয় → ৯০ দিন পর মূল ছবি Intelligent-Tiering-এ যায়। এই একটি প্রবাহে S3-র প্রায় সব বৈশিষ্ট্য একসঙ্গে কাজ করে: presigned URL, event, Lambda, CloudFront, lifecycle। **নিরাপত্তা চেকলিস্ট:** BPA চালু, bucket policy-তে least privilege (সর্বনিম্ন অনুমতি), default encryption, versioning, MFA Delete, CloudTrail-এ S3 data event লগ, AWS Config rule, GuardDuty। **খরচ অপ্টিমাইজেশন:** lifecycle policy, Intelligent-Tiering, অসম্পূর্ণ multipart upload পরিষ্কার, S3 Storage Lens দিয়ে বড় bucket খুঁজে বের করা, CloudFront দিয়ে S3 থেকে বারবার ডাউনলোড কমানো (data transfer বিল কমে), request সংখ্যা কমাতে ছোট ফাইল একত্র করা। **সাধারণ ভুল:** (১) bucket public রেখে দেওয়া; (২) versioning ছাড়া production ডেটা রাখা; (৩) lifecycle ছাড়া versioning চালু করে বিল বাড়ানো; (৪) Glacier-এ রেখে বারবার ডেটা তোলা; (৫) `sync --delete` যাচাই ছাড়া চালানো; (৬) presigned URL-এর মেয়াদ দিনের পর দিন রাখা; (৭) এক bucket-এ public ও private ডেটা মেশানো। **Troubleshooting:** AccessDenied এলে — IAM policy, bucket policy, BPA, KMS key policy — চারটি জায়গা পরীক্ষা করুন; `aws s3api get-bucket-policy` ও CloudTrail-এর error দেখুন। Slow upload হলে multipart ও Transfer Acceleration বিবেচনা করুন। Object না পাওয়া গেলে (404) key-র বানান, version ID ও region যাচাই করুন।

> **TROUBLESHOOTING:** "Access Denied" মানেই IAM-এর দোষ নয়। S3-তে অনুমতি মূল্যায়ন হয় এই ক্রমে: BPA → bucket policy → IAM policy → ACL → KMS key policy। একটি Deny যেকোনো স্তরে থাকলে সব বন্ধ। `aws sts get-caller-identity` দিয়ে দেখুন আপনি কোন পরিচয়ে কাজ করছেন, তারপর IAM Policy Simulator-এ পরীক্ষা করুন।

১২.১৫ Event Notification, Object Lock, S3 Select ও উন্নত বৈশিষ্ট্য

S3 শুধু ডেটা রাখার জায়গা নয় — এটি একটি **event-driven** (ঘটনা-চালিত) প্ল্যাটফর্ম। **S3 Event Notification** হলো এমন একটি ব্যবস্থা যেখানে bucket-এ কোনো ঘটনা ঘটলেই (object তৈরি, মুছে ফেলা, প্রতিলিপি সম্পন্ন হওয়া) S3 স্বয়ংক্রিয়ভাবে একটি বার্তা পাঠায় — **Lambda function**, **SQS queue** বা **SNS topic**-এ। TechNova-র উদাহরণ: ব্যবহারকারী ছবি আপলোড করলেই একটি Lambda চলে, thumbnail বানায়, আরেকটি bucket-এ রাখে, আর DynamoDB-তে metadata লেখে — কোনো সার্ভার চালাতে হয় না, কোনো polling (বারবার জিজ্ঞেস করা) লাগে না। Event notification কনফিগার করার সময় **prefix ও suffix filter** ব্যবহার করুন — যেমন শুধু `uploads/` prefix ও `.jpg` suffix-এর জন্য Lambda ডাকুন; নইলে প্রতিটি লগ ফাইলের জন্য Lambda চলবে আর বিল বাড়বে। একটি সতর্কতা: একই bucket-এ event notification ও Lambda-র মধ্যে **loop** (চক্র) তৈরি হতে পারে — Lambda যদি একই bucket-এ ফাইল লেখে, আবার event ঘটে, আবার Lambda চলে — অসীম চক্র। তাই Lambda-র আউটপুট সবসময় আলাদা bucket বা আলাদা prefix-এ লিখুন।

S3 Object Lock হলো আইনি ও নিয়ন্ত্রক সংরক্ষণের বৈশিষ্ট্য: একটি object-কে নির্দিষ্ট সময়ের জন্য মুছে ফেলা বা বদলানো অসম্ভব করে দেয় — এমনকি root ব্যবহারকারীও মুছতে পারে না। দুটি mode আছে: **Governance mode** (বিশেষ অনুমতি থাকলে বদলানো যায় — সাধারণত compliance পরীক্ষার জন্য) ও **Compliance mode** (কেউ বদলাতে

পারে না — এমনকি root-ও নয় — মেয়াদ শেষ না হওয়া পর্যন্ত)। ব্যাংক, হাসপাতাল ও সরকারি কাজে যেখানে আইন বলে ডেটা ৭ বছর রাখতে হবে, সেখানে Object Lock ব্যবহার করা হয়। মনে রাখবেন: Object Lock চালু করতে bucket তৈরির সময়ই চালু করতে হয়, পরে নয়; আর versioning আবশ্যক। **S3 Select** হলো আরেকটি শক্তিশালী বৈশিষ্ট্য: পুরো ফাইল ডাউনলোড না করে SQL-এর মতো query দিয়ে ফাইলের ভেতর থেকে শুধু দরকারি অংশ নেওয়া যায় — যেমন ১ GB CSV ফাইল থেকে শুধু একটি কলাম। এতে data transfer ও সময় দুটোই বাঁচে। **S3 Transfer Acceleration** হলো CloudFront-এর edge location ব্যবহার করে আপলোড দ্রুত করা — দূরবর্তী ব্যবহারকারীর বড় ফাইল আপলোডে কার্যকর, তবে অতিরিক্ত ফি আছে — Pricing Calculator-এ যাচাই করুন। **S3 Storage Lens** হলো bucket-এর ব্যবহার ও খরচের ড্যাশবোর্ড — কোন bucket বড় হচ্ছে, কোথায় পুরোনো version জমছে, কোথায় encryption নেই — সব এক জায়গায় দেখা যায়। **S3 Inventory** দিয়ে bucket-এর সব object-এর তালিকা (CSV/Parquet) প্রতিদিন পাওয়া যায় — audit ও খরচ বিশ্লেষণের জন্য। আর **S3 Batch Operations** দিয়ে লাখ লাখ object-এ একসঙ্গে কাজ করা যায় — যেমন সব object-এর storage class বদলানো বা tag যোগ করা — একটি manifest ফাইল দিয়ে।

S3-র ব্যর্থতা ও পর্যবেক্ষণ: S3 ব্যর্থ হলে কী হয়? S3 নিজে প্রায় কখনোই বন্ধ হয় না, কিন্তু আপনার অ্যাপের দিক থেকে ব্যর্থতা দেখা দিতে পারে: ভুল region-এ bucket খোঁজা (bucket-এর নাম global হলেও ডেটা একটি region-এ থাকে — ভুল region-এর endpoint-এ request গেলে redirect হয়, ধীর হয়), IAM অনুমতির ভুল, KMS key নিষ্ক্রিয় (disabled) হয়ে গেলে এনক্রিপ্ট করা object পড়া যায় না, বা lifecycle-এর ভুলে ডেটা Glacier-এ চলে গেলে তাৎক্ষণিক পাওয়া যায় না। পর্যবেক্ষণের জন্য: **CloudWatch metrics** (`BucketSizeBytes`, `NumberOfObjects`, `AllRequests`, `4xxErrors`, `5xxErrors`), **CloudTrail** (কে কোন API call করল), **S3 server access logs** (প্রতিটি request-এর লগ — কে, কখন, কী চাইল), আর **Storage Lens**। একটি production নিয়ম: S3-র `4xxErrors`-এর ওপর alarm বসান — হঠাৎ 403 বাড়া মানে হয় অনুমতির ভুল, নয়তো কেউ আপনার bucket স্ক্যান করছে। **কখন S3 ব্যবহার করবেন না:** ডেটাবেসের মতো প্রতিটি লেখায় নিশ্চয়তা চাওয়া কাজে (RDS ব্যবহার করুন), ফাইল লক (lock) দরকার এমন কাজে (EFS ব্যবহার করুন), বা খুব ছোট, বারবার বদলানো ফাইলে যেখানে request-এর খরচ storage-এর চেয়ে বেশি হয়ে যায় — সেখানে EBS বা ElastiCache ভালো হতে পারে। **বিকল্প:** on-premises-এ MinIO বা Ceph (S3-সামঞ্জস্যপূর্ণ), অন্য ক্লাউডে Google Cloud Storage বা Azure Blob Storage — কিন্তু AWS-এর ভেতরে S3-ই মান।

> **CONCEPT — S3-র মূল্য নির্ধারণের চারটি স্তম্ভ:** (১) Storage — কত GB, কোন storage class-এ; (২) Requests — PUT, GET, LIST কতটি; (৩) Data transfer — ইন্টারনেটে কত GB গেল (AWS-এর ভেতরে একই region-এ transfer সাধারণত বিনামূল্যে, অন্য region বা ইন্টারনেটে ফি আছে); (৪) Management — replication, inventory, Storage Lens-এর মতো বৈশিষ্ট্য। বিল কমাতে চাইলে চারটিই দেখুন — শুধু storage class বদলালেই হবে না।

১২.১৬ সারাংশ, প্রশ্ন ও অনুশীলন

সারাংশ: S3 হলো object storage — bucket ও object, key ও metadata দিয়ে গঠিত, HTTP API দিয়ে পরিচালিত, ১১টি ৯ durability-সম্পন্ন। Storage class বেছে নেওয়া একটি খরচ বনাম গতির সিদ্ধান্ত; lifecycle policy সেই সিদ্ধান্ত স্বয়ংক্রিয় করে। Versioning ভুল থেকে বাঁচায়, encryption ডিস্ক-স্তরে সুরক্ষা দেয়, কিন্তু আসল নিরাপত্তা আসে bucket policy, BPA ও least privilege থেকে। Presigned URL সাময়িক প্রবেশ দেয়, replication দুর্যোগ থেকে বাঁচায়, আর static hosting শুধু স্থির সাইটের জন্য — production-এ CloudFront সামনে বসান।

Review Questions: (১) Object storage কীভাবে file storage থেকে আলাদা? (২) Bucket-এর নাম কেন globally unique হতে হয়? (৩) Standard বনাম Standard-IA বনাম Glacier Deep Archive — কখন কোনটি? (৪) Versioning চালু থাকলে মুছে ফেলা object কীভাবে ফেরাবেন? (৫) SSE-S3, SSE-KMS ও SSE-C-র মধ্যে পার্থক্য কী? (৬) Bucket

policy-র Deny কেন Allow-কে হারিয়ে দেয়? (৭) Block Public Access-এর চারটি সুইচ কী কী? (৮) Presigned URL কীভাবে কাজ করে ও এর ঝুঁকি কী? (৯) Lifecycle policy ছাড়া versioning চালু করলে বিলে কী প্রভাব পড়ে? (১০) Static website hosting-এ HTTPS কেন সরাসরি পাওয়া যায় না?

Practical Exercises: (১) একটি পরীক্ষামূলক bucket তৈরি করে versioning ও SSE-S3 চালু করুন, একটি ফাইল তিনবার বদলে তিনটি version তৈরি করুন, তারপর একটি version মুছে delete marker সরিয়ে ফাইল ফেরান। (২) উপরের bucket policy JSON-টি নিজের bucket-এ বসিয়ে HTTP request বন্ধ আছে কি না যাচাই করুন (curl দিয়ে http চেষ্টা করুন)। (৩) একটি lifecycle policy লিখুন যা ৩০ দিনে Standard-IA, ৯০ দিনে Glacier-এ নেয় ও অসম্পূর্ণ multipart upload ৭ দিনে মোছে; S3 Storage Lens দিয়ে bucket-এর খরচের চিত্র দেখুন।

অধ্যায় ১৩ • PART III — Storage, Database & Application Services

Chapter 13 — EBS, EFS, FSx and Backup

১৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) **block, file ও object storage** -এর মধ্যে প্রথম নীতি থেকে পার্থক্য করতে ও সঠিকটি বেছে নিতে; (২) **Amazon EBS** (Elastic Block Store)-এর volume type (gp3, gp2, io2, st1, sc1) তুলনা করে সঠিক volume বেছে নিতে ও IOPS/throughput বুঝতে; (৩) **EBS snapshot** (স্ন্যাপশট) দিয়ে ব্যাকআপ ও পুনরুদ্ধার করতে; (৪) **Amazon EFS** (Elastic File System)-এর আর্কিটেকচার, performance mode ও throughput mode ব্যাখ্যা করতে ও NFS দিয়ে মাউন্ট করতে; (৫) **Amazon FSx**-এর ব্যবহার — Windows File Server, Lustre, NetApp ONTAP, OpenZFS — কখন কোনটি, তা বলতে; (৬) **AWS Backup** দিয়ে কেন্দ্রীয় ব্যাকআপ নীতি বানাতে; এবং (৭) **RPO/RTO** (পুনরুদ্ধার লক্ষ্য) দিয়ে durability ও availability-র সিদ্ধান্ত নিতে।

১৩.২ পূর্বশর্ত ঝালাই (Prerequisite Refresher)

১২ নম্বর অধ্যায় থেকে S3 object storage-এর ধারণা, আর আগের volume-গুলো থেকে EC2 instance, Availability Zone ও IAM-এর ধারণা সঙ্গে নিন। নতুন শব্দগুলো আগেই চিনে নিন: **volume** (ভলিউম — একটি ভার্চুয়াল ডিস্ক), **IOPS** (Input/Output Operations Per Second — প্রতি সেকেন্ডে কতটি পড়া/লেখা হয়), **throughput** (প্রতি সেকেন্ডে কত মেগাবাইট যায়), **latency** (বিলম্ব — অনুরোধের উত্তর পেতে কত সময় লাগে), **mount** (মাউন্ট — একটি storage-কে ফাইল সিস্টেম হিসেবে যুক্ত করা, যেমন `/data`), **NFS** (Network File System — নেটওয়ার্ক দিয়ে ফাইল ভাগ করার প্রোটোকল) ও **SMB** (Windows-এর ফাইল ভাগ করার প্রোটোকল)। মনে রাখবেন: EC2 instance-এর **root volume** (যেখানে অপারেটিং সিস্টেম থাকে) সাধারণত EBS হয়; instance বন্ধ করলে root volume-এর ডেটা থেকে যায় (যদি DeleteOnTermination বন্ধ থাকে), কিন্তু instance-এর সঙ্গে আসা অস্থায়ী **instance store** ডিস্ক বন্ধ করলে মুছে যায় — এই পার্থক্যটি ব্যাকআপ পরিকল্পনার ভিত্তি।

১৩.৩ কেন এটি গুরুত্বপূর্ণ

S3 সব সমস্যার সমাধান নয়। ডেটাবেস চালাতে হয় block storage-এ — কারণ ডেটাবেস প্রতিটি লেখা সঙ্গে সঙ্গে ডিস্কে নিশ্চিত (durable) করতে চায়, আর সেটি শুধু block storage-ই দিতে পারে। আবার অনেক সার্ভার একই ফাইল ভাগ করে ব্যবহার করতে চাইলে (যেমন WordPress-এর uploads ফোল্ডার) file storage লাগে — S3 দিয়ে সেটি সরাসরি করা যায় না। আর Windows অ্যাপ্লিকেশন বা high-performance computing (HPC) কাজের জন্য বিশেষায়িত ফাইল সিস্টেম (FSx) লাগে। ভুল storage বেছে নিলে হয় অ্যাপ ধীর হয়, নয়তো ডেটা হারায়, নয়তো বিল আকাশছোঁয়া হয়। TechNova-র একটি বাস্তব উদাহরণ: তাদের PostgreSQL ডেটাবেস gp3 volume-এ চলে; তাদের মাইক্রোসার্ভিসগুলোর ভাগ করা কনফিগারেশন EFS-এ থাকে; আর ব্যবহারকারীর ছবি S3-তে থাকে। তিনটি storage, তিনটি ভিন্ন কাজ — এই অধ্যায়ে আপনি শিখবেন কোন কাজে কোনটি।

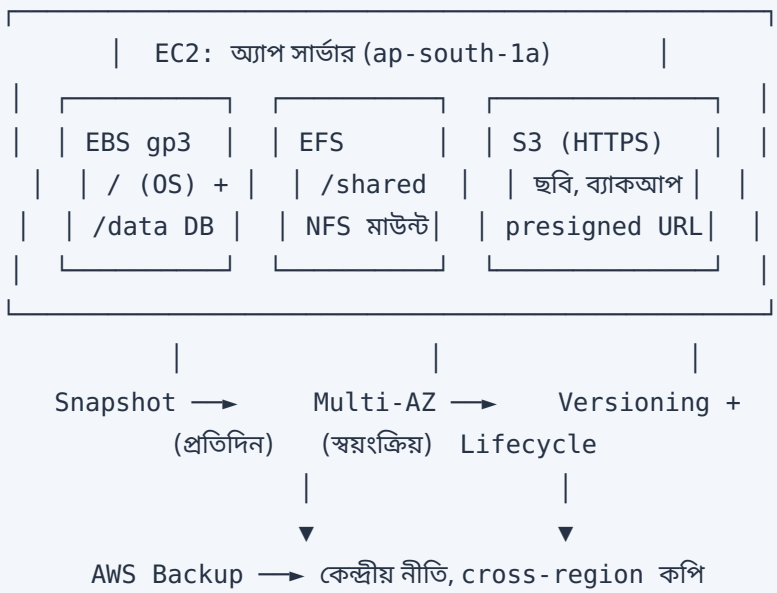
১৩.৪ প্রথম নীতি: Block বনাম File বনাম Object

তিনটি storage-কে তিনটি ভিন্ন জিনিস হিসেবে ভাবুন। **Block storage** হলো কাঁচা ডিস্কের টুকরো — অপারেটিং সিস্টেম এতে নিজের ফাইল সিস্টেম (ext4, XFS, NTFS) বানায়, আর প্রতিটি লেখা একটি নির্দিষ্ট block address-এ যায়। এটি সবচেয়ে দ্রুত ও নিম্ন-latency, কিন্তু একসঙ্গে শুধু একটি instance-এই যুক্ত (attach) করা যায়। **File storage** হলো ভাগ করা ফোল্ডার — NFS/SMB প্রোটোকলে অনেক সার্ভার একই ফাইল পড়তে ও লিখতে পারে, ফাইলের নাম, অনুমতি

(permission), লক (lock) সবই আছে। এটি block-এর চেয়ে ধীর, কিন্তু ভাগাভাগির জন্য অপরিহার্য। **Object storage** (S3) হলো HTTP API — কোনো মাউন্ট নেই, কোনো ফাইল লক নেই, শুধু PUT/GET। এটি সবচেয়ে সস্তা ও স্কেলযোগ্য, কিন্তু ডেটাবেসের মতো প্রতিটি লেখায় নিশ্চয়তা চাওয়া কাজে চলে না। এই তিনটির সীমা বোঝাই এই অধ্যায়ের মূল শিক্ষা।

বৈশিষ্ট্য	Block (EBS)	File (EFS / FSx)	Object (S3)
ব্যবহারের ধরন	একটি EC2-তে মাউন্ট করা ডিস্ক	অনেক EC2/Lambda থেকে ভাগ করা ফোল্ডার	HTTP API দিয়ে যেকোনো জায়গা থেকে
Latency	সবচেয়ে কম (মিলিসেকেন্ডের ভগ্নাংশ)	কম (কয়েক মিলিসেকেন্ড)	বেশি (দশ মিলিসেকেন্ড)
ভাগ করা যায়?	সাধারণত না (একটি instance)	হ্যাঁ — হাজারো ক্লায়েন্ট	হ্যাঁ — যেকোনো ক্লায়েন্ট
স্কেল	GB থেকে TB (একটি volume)	পেটাবাইট পর্যন্ত, স্বয়ংক্রিয়	কার্যত অসীম
সাধারণ ব্যবহার	ডেটাবেস, OS ডিস্ক, অ্যাপ ডেটা	ভাগ করা কনটেন্ট, হোম ডিরেক্টরি	ছবি, ব্যাকআপ, লগ, ডেটা লেক
ব্যর্থ হলে কী হয়	Volume নষ্ট হলে snapshot থেকে ফেরাতে হয়	AZ নষ্ট হলে অন্য AZ থেকে চলে	ডেটা হারানোর সম্ভাবনা কার্যত নেই

চিত্র ১৩.১ — TechNova-র তিন স্তরের Storage আর্কিটেকচার



১৩.৫ EBS গভীরে: Volume Type, IOPS ও Throughput

EBS volume হলো একটি AZ-এর ভেতরে থাকা নেটওয়ার্ক-সংযুক্ত ভার্চুয়াল ডিস্ক — EC2 instance-এর সঙ্গে যুক্ত হয়, কিন্তু instance-এর ভেতরের ডিস্ক নয়; নেটওয়ার্ক দিয়ে ডেটা যায়, তাই AZ-র বাইরে volume যুক্ত করা যায় না। Volume তৈরি হয় একটি AZ-তে, আর সেই AZ-র instance-এই শুধু attach করা যায়। Volume type বেছে নেওয়া মানে performance ও দামের মধ্যে সিদ্ধান্ত নেওয়া। **gp3** (General Purpose SSD) হলো আজকের ডিফল্ট: IOPS (৩,০০০

baseline, ১৬,০০০ পর্যন্ত) ও throughput (১২৫ MB/s baseline, ১,০০০ পর্যন্ত) আলাদাভাবে বাড়ানো যায়, দামও gp2-র চেয়ে ভালো। **gp2** পুরোনো — IOPS volume-এর আকারের সঙ্গে বাড়ে (burst credit সহ), ছোট volume-এ ধীর হতে পারে; নতুন কাজে gp3 ব্যবহার করুন। **io2 / io2 Block Express** হলো সবচেয়ে দ্রুত ও টেকসই (99.999% durability) — গুরুত্বপূর্ণ ডেটাবেসের জন্য, যেখানে প্রতিটি লেখার নিশ্চয়তা চাই; দাম সবচেয়ে বেশি। **st1** (Throughput Optimized HDD) বড়, ক্রমিক (sequential) পড়া/লেখার জন্য — যেমন লগ প্রসেসিং, বিগ ডেটা; ছোট, এলোমেলো (random) I/O-তে খারাপ। **sc1** (Cold HDD) সবচেয়ে সস্তা — খুব কম ব্যবহৃত ডেটার জন্য। একটি নিয়ম মনে রাখুন: ডেটাবেসে SSD (gp3/io2), লগ ও ব্যাচ কাজে HDD (st1/sc1), আর কখনোই ডেটাবেস sc1-এ চালাবেন না — latency-তে অ্যাপ ধীর হয়ে যাবে।

```
# gp3 volume তৈরি (100 GB, 3000 IOPS, 250 MB/s throughput)
aws ec2 create-volume --availability-zone ap-south-1a \
  --size 100 --volume-type gp3 --iops 3000 --throughput 250 \
  --tag-specifications 'ResourceType=volume,Tags=[{Key=Name,Value=technova-db}]'

# Volume EC2 instance-এ attach করুন
aws ec2 attach-volume --volume-id vol-CHANGE-ME \
  --instance-id i-CHANGE-ME --device /dev/sdf

# Volume-এর অবস্থা দেখুন
aws ec2 describe-volumes --volume-ids vol-CHANGE-ME \
  --query 'Volumes[0].[State,Size,VolumeType,Iops]'
```

Attach করার পর instance-এর ভেতরে `lsblk` দিয়ে ডিস্ক দেখুন, `mkfs -t xfs /dev/sdf` দিয়ে ফাইল সিস্টেম বানান (নতুন volume হলে), তারপর `mount /dev/sdf /data` দিয়ে মাউন্ট করুন। `/etc/fstab`-এ এন্ট্রি যোগ করুন যাতে reboot-এর পরও মাউন্ট থাকে — তবে device নামের বদলে UUID ব্যবহার করুন, কারণ reboot-এ device নাম বদলাতে পারে।

> **CONCEPT — IOPS বনাম Throughput:** IOPS হলো "প্রতি সেকেন্ডে কতটি কাজ", throughput হলো "প্রতি সেকেন্ডে কত ডেটা"। ডেটাবেসের ছোট ছোট লেখায় IOPS গুরুত্বপূর্ণ; বড় ফাইল কপিতে throughput গুরুত্বপূর্ণ। একটি volume-এর IOPS যথেষ্ট কিন্তু throughput কম হলে বড় ফাইল ধীরে যাবে; উল্টোটাও সত্যি। CloudWatch-এ `VolumeReadOps`, `VolumeWriteOps` ও `VolumeQueueLength` দেখে বুঝুন কোনটি আটকে আছে — queue length ধারাবাহিকভাবে ১-এর বেশি থাকলে volume-টি bottleneck (বাধা) হয়ে গেছে।

১৩.৬ EBS Snapshot — ব্যাকআপের ভিত্তি

Snapshot হলো EBS volume-এর একটি নির্দিষ্ট মুহূর্তের (point-in-time) কপি, যা S3-তে সংরক্ষিত হয় (আপনি সরাসরি S3-তে দেখতে পাবেন না — AWS পরিচালনা করে)। প্রথম snapshot পূর্ণ (full) কপি হয়; পরেরগুলো **incremental** (ক্রমবর্ধমান) — শুধু বদলানো block কপি হয়, তাই দ্রুত ও সস্তা। Snapshot থেকে নতুন volume বানানো যায় — যেকোনো AZ-তে, যেকোনো আকারে (মূলের সমান বা বড়), এমনকি অন্য region-এ কপি করেও। Snapshot-এর গুরুত্বপূর্ণ বৈশিষ্ট্য: এটি **crash-consistent** — অর্থাৎ snapshot নেওয়ার মুহূর্তে ডিস্কে যা ছিল, তা-ই ধরা হয়; কিন্তু ডেটাবেসের মতো অ্যাপ চলমান থাকলে মেমরিতে থাকা ডেটা ধরা পড়ে না। তাই ডেটাবেসের snapshot নেওয়ার আগে অ্যাপ quiesce (শান্ত) করুন, অথবা ডেটাবেসের নিজের ব্যাকআপ (যেমন pg_dump) সঙ্গে ব্যবহার করুন। Snapshot-এর খরচ: শুধু বদলানো block-এর জন্য, GB-প্রতি মাসিক হারে — Pricing Calculator-এ যাচাই করুন।

```
# Snapshot তৈরি করুন
aws ec2 create-snapshot --volume-id vol-CHANGE-ME \
  --description "technova-db daily backup 2026-11-09" \
```

```
  --tag-specifications 'ResourceType=snapshot,Tags=[{Key=Name,Value=technova-db-daily}]'

# Snapshot থেকে নতুন volume বানান (অন্য AZ-তে)
aws ec2 create-volume --availability-zone ap-south-1b \
  --snapshot-id snap-CHANGE-ME --volume-type gp3 --size 100

# Snapshot অন্য region-এ কপি করুন (দুর্যোগ পুনরুদ্ধারের জন্য)
aws ec2 copy-snapshot --source-region ap-south-1 \
  --source-snapshot-id snap-CHANGE-ME \
  --region ap-southeast-1 --description "DR copy"
```

১৩.৭ EFS — ভাগ করা ফাইল সিস্টেম

Amazon EFS হলো সম্পূর্ণ পরিচালিত NFS ফাইল সিস্টেম: আপনি শুধু ফাইল সিস্টেম তৈরি করেন, AWS পেছনে সার্ভার, ডিস্ক ও প্রতিলিপি সামলায়। EFS-এর সবচেয়ে বড় বৈশিষ্ট্য — এটি **elastic** (স্থিতিস্থাপক): ডেটা বাড়লে নিজে থেকে বাড়ে, কমলে কমে; আগে থেকে আকার ঠিক করতে হয় না। একই EFS হাজার হাজার EC2 instance, Lambda function ও container থেকে একসঙ্গে মাউন্ট করা যায় — সবাই একই ফাইল দেখে। EFS প্রতিটি AZ-তে একটি **mount target** (মাউন্ট লক্ষ্য — একটি IP ঠিকানা) তৈরি করে; instance সেই IP দিয়ে NFS-এ যুক্ত হয়। দুটি performance mode আছে: **General Purpose** (সাধারণ — বেশিরভাগ কাজ, যেমন ওয়েব কনটেন্ট) ও **Max I/O** (সর্বোচ্চ I/O — হাজার হাজার parallel কাজ, যেমন বিগ ডেটা)। Throughput mode: **Bursting** (ডেটার পরিমাণের সঙ্গে throughput বাড়ে — ছোট ফাইল সিস্টেমে throughput কম হতে পারে, এটি সাধারণ বিভ্রান্তি) ও **Provisioned** (আগে থেকে throughput কিনে নেওয়া — নির্দিষ্ট গতি নিশ্চিত)। Storage class: **Standard** ও **Infrequent Access (IA)** — lifecycle policy দিয়ে ৩০ দিন না-ছোঁয়া ফাইল IA-তে পাঠানো যায়, খরচ কমে। EFS-এর নিরাপত্তা: VPC-র ভেতরে থাকে, security group দিয়ে NFS পোর্ট (2049) নিয়ন্ত্রণ করুন, IAM authorization বা access point দিয়ে ফোল্ডার-স্তরে অনুমতি দিন, encryption at rest (KMS) ও in transit (TLS) চালু করুন।

```
# EFS ফাইল সিস্টেম তৈরি করুন
aws efs create-file-system --creation-token technova-shared \
  --performance-mode generalPurpose --throughput-mode bursting \
  --encrypted --tags Key=Name,Value=technova-shared

# Mount target তৈরি করুন (প্রতিটি AZ-তে, subnet ও security group দিয়ে)
aws efs create-mount-target --file-system-id fs-CHANGE-ME \
  --subnet-id subnet-CHANGE-ME --security-groups sg-CHANGE-ME

# EC2 instance-এর ভেতরে মাউন্ট করুন (Amazon Linux / Ubuntu)
# sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2 \
#   fs-CHANGE-ME.efs.ap-south-1.amazonaws.com:/ /shared
# EFS mount helper ব্যবহার করলে সহজ: sudo mount -t efs fs-CHANGE-ME:/ /shared
```

> **WARNING:** EFS-এর Bursting throughput mode-এ ছোট ফাইল সিস্টেমের throughput খুব কম হতে পারে — ১ GB ডেটায় মাত্র ~৫০ KB/s baseline। ছোট কিন্তু দ্রুত throughput চাইলে Provisioned mode বা Elastic Throughput ব্যবহার করুন, নইলে অ্যাপ হঠাৎ ধীর হয়ে যাবে এবং কারণ খুঁজে পাবেন না।

১৩.৮ FSx — বিশেষায়িত ফাইল সিস্টেম

Amazon FSx হলো বিশেষ কাজের জন্য সম্পূর্ণ পরিচালিত ফাইল সিস্টেমের পরিবার। চারটি প্রধান সদস্য: **FSx for Windows File Server** — Windows অ্যাপের জন্য SMB প্রোটোকল, Active Directory-র সঙ্গে যুক্ত হয়; যেসব কোম্পানির পুরোনো Windows অ্যাপ আছে, তারা EC2-তে ফাইল সার্ভার না চালিয়ে FSx ব্যবহার করে। **FSx for Lustre** — high-performance computing (HPC) ও মেশিন লার্নিংয়ের জন্য; হাজার হাজার parallel client থেকে একসঙ্গে GB/s গতিতে ডেটা পড়ে; S3-র সঙ্গে যুক্ত হয়ে ডেটা লেক থেকে সরাসরি কাজ করতে পারে। **FSx for NetApp ONTAP** — NetApp-এর পরিচিত ফাইল সিস্টেম, NFS/SMB/iSCSI একসঙ্গে, snapshot ও ক্লোনিংয়ের শক্তিশালী বৈশিষ্ট্য; যারা on-premises-এ NetApp ব্যবহার করে তারা সহজে ক্লাউডে আসতে পারে। **FSx for OpenZFS** — ZFS ফাইল সিস্টেম, snapshot, compression ও ক্লোনিং সহ; Linux-ভিত্তিক কাজের জন্য। নিয়ম: সাধারণ ভাগ করা ফোল্ডার চাইলে EFS; Windows/Active Directory চাইলে FSx for Windows; HPC/ML-তে বিপুল গতির parallel পড়া চাইলে FSx for Lustre; NetApp বা ZFS-এর নির্দিষ্ট বৈশিষ্ট্য চাইলে সেই FSx। FSx-এর খরচ EFS-এর চেয়ে বেশি হতে পারে — কারণ এটি নির্দিষ্ট performance-এর জন্য provisioned হয়; Pricing Calculator-এ যাচাই করুন।

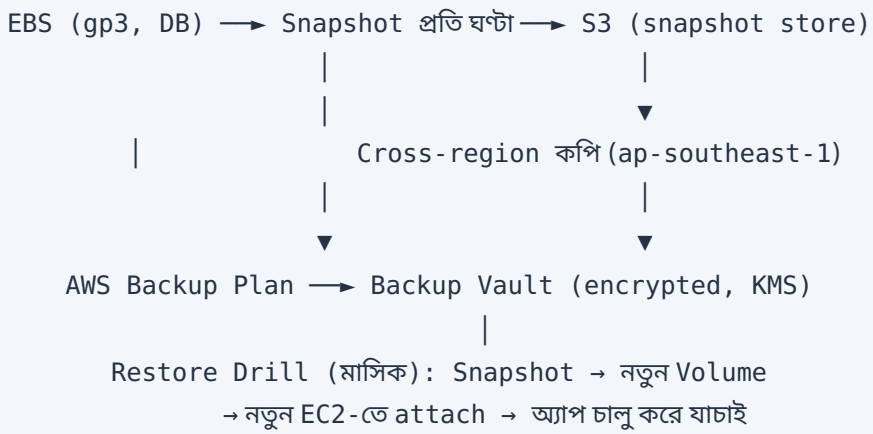
১৩.৯ AWS Backup, RPO/RTO ও পুনরুদ্ধার পরিকল্পনা

AWS Backup হলো কেন্দ্রীয় ব্যাকআপ সেবা — EBS, RDS, EFS, DynamoDB, S3-র ব্যাকআপ এক জায়গা থেকে নীতি দিয়ে পরিচালনা করা যায়। আপনি একটি **backup plan** (ব্যাকআপ পরিকল্পনা) বানান: কত ঘণ্টা পরপর ব্যাকআপ, কত দিন রাখা হবে, কোন vault-এ রাখা হবে, অন্য region-এ কপি হবে কি না। তারপর **backup selection** দিয়ে বলেন কোন resource (ট্যাগ দিয়ে, যেমন `backup=daily`) এই পরিকল্পনার আওতায় আসবে। দুটি শব্দ এখানে মুখ্য: **RPO** (Recovery Point Objective — কতটুকু ডেটা হারানো মেনে নেওয়া যায়; যেমন RPO ১ ঘণ্টা মানে সর্বোচ্চ ১ ঘণ্টার ডেটা হারাতে পারে, তাই প্রতি ঘণ্টায় ব্যাকআপ চাই) ও **RTO** (Recovery Time Objective — কত সময়ের মধ্যে সেবা ফিরিয়ে আনতে হবে; যেমন RTO ৪ ঘণ্টা মানে ৪ ঘণ্টার মধ্যে পুনরুদ্ধার করতে হবে)। RPO ছোট করতে ঘন ঘন ব্যাকআপ লাগে (খরচ বাড়ে); RTO ছোট করতে আগে থেকে প্রস্তুতি লাগে (standby resource, স্বয়ংক্রিয় স্ক্রিপ্ট)। একটি ব্যাকআপ কখনো পরীক্ষা না করলে সেটি ব্যাকআপ নয় — প্রতি মাসে একটি restore drill (পুনরুদ্ধার মহড়া) করুন: snapshot থেকে volume বানিয়ে, EFS থেকে ফাইল ফিরিয়ে, সত্যিই ডেটা ফেরে কি না দেখুন।

```
# AWS Backup দিয়ে EBS volume-এর on-demand ব্যাকআপ নিন
aws backup start-backup-job --backup-vault-name technova-vault \
  --resource-arn arn:aws:ec2:ap-south-1:111122223333:volume/vol-CHANGE-ME \
  --iam-role-arn arn:aws:iam::111122223333:role/service-role/AWSBackupDefaultServiceRole

# পুনরুদ্ধার (restore) — snapshot থেকে volume বানানোর মতোই,
# AWS Backup console-এ Recovery point → Restore দিয়ে যাচাই করুন।
```

চিত্র ১৩.২ — ব্যাকআপ ও পুনরুদ্ধার প্রবাহ (RPO ১ ঘণ্টা, RTO ৪ ঘণ্টা)



১৩.১০ Production Scenario, নিরাপত্তা, খরচ, ভুল ও Troubleshooting

Production scenario — TechNova-র ডেটাবেস সার্ভার: EC2-তে PostgreSQL চলে, ডেটা gp3 volume-এ (৩,০০০ IOPS provisioned), প্রতিদিন রাত ২টায় snapshot নেওয়া হয় (AWS Backup plan), snapshot অন্য region-এ কপি হয়, আর মাসে একবার restore drill হয়। ভাগ করা uploads ফোল্ডার EFS-এ (General Purpose, lifecycle দিয়ে ৩০ দিন পর IA), আর ব্যবহারকারীর ছবি S3-তে। এই নকশায় প্রতিটি storage তার শক্তির জায়গায় আছে। **নিরাপত্তা:** EBS ও EFS-এ encryption at rest (KMS) চালু করুন; snapshot শেয়ার করার সময় শুধুমাত্র নির্দিষ্ট অ্যাকাউন্টের সঙ্গে শেয়ার করুন — public snapshot কখনো করবেন না (public snapshot-এ ডেটা সবার জন্য খোলা); EFS-এর security group-এ শুধু NFS পোর্ট 2049, শুধু অ্যাপের security group থেকে; FSx-এ Active Directory-র অনুমতি least privilege রাখুন। **খরচ:** অব্যবহৃত (unattached) EBS volume মুছে ফেলুন — এগুলো চুপচাপ বিল তোলে; পুরোনো snapshot মুছে ফেলুন (incremental হলেও জমে বিল বাড়ে); gp2 থেকে gp3-তে যান (সাধারণত সস্তা ও দ্রুত); EFS-এ IA lifecycle চালু করুন; st1/sc1-তে ভুল করে ডেটাবেস চালাবেন না। **সাধারণ ভুল:** (১) instance store-কে স্থায়ী ডেটা ভেবে ব্যবহার করা; (২) snapshot না নিয়ে volume resize করা; (৩) DeleteOnTermination চালু রেখে instance terminate করে ডেটা হারানো; (৪) EFS-এর Bursting throughput না বুঝে ছোট ফাইল সিস্টেমে ধীরগতি; (৫) ব্যাকআপ নিয়ে কখনো restore পরীক্ষা না করা; (৬) snapshot public শেয়ার করা। **Troubleshooting:** Volume attach হচ্ছে না? — AZ মিলছে কি না দেখুন (volume ও instance একই AZ-তে হতে হবে)। মাউন্ট হচ্ছে না? — `dmesg` ও `mount` আউটপুট দেখুন, fstab-এ UUID ঠিক আছে কি না দেখুন। EFS ধীর? — throughput mode ও CloudWatch-এর `PercentIOLimit` দেখুন। Snapshot থেকে volume বানাতে দেরি হচ্ছে? — প্রথমবার lazy loading হয়; `fio` দিয়ে volume warm করুন অথবা EBS fast snapshot restore ব্যবহার করুন।

> **CHECK — এই অধ্যায়ের চেকলিস্ট:** প্রতিটি production volume-এ encryption চালু আছে কি? Snapshot/AWS Backup plan আছে কি? Restore drill গত মাসে হয়েছে কি? কোনো unattached volume বা পুরোনো snapshot পড়ে আছে কি? EFS-এর security group শুধু 2049 পোর্ট খোলা আছে কি?

১৩.১১ EBS Encryption, I/O বৈশিষ্ট্য, RAID ও FSx গভীরে

EBS encryption সম্পর্কে গভীরে যান: প্রতিটি EBS volume KMS key দিয়ে এনক্রিপ্ট করা যায় — ডেটা ডিস্কে লেখার আগেই এনক্রিপ্ট হয়, পড়ার সময় ডিক্রিপ্ট হয়, আর snapshot-ও এনক্রিপ্ট থাকে। **Default encryption** চালু করলে

অ্যাকাউন্টে তৈরি প্রতিটি নতুন volume স্বয়ংক্রিয়ভাবে এনক্রিপ্ট হয় — এটি একটি অ্যাকাউন্ট-স্তরের সেটিং, একবার চালু করলে ভুলে যাওয়া অসম্ভব। একটি গুরুত্বপূর্ণ নিয়ম: এনক্রিপ্ট করা volume-এর snapshot থেকে বানানো volume-ও এনক্রিপ্ট থাকে; আর এনক্রিপ্ট করা snapshot অন্য অ্যাকাউন্টের সঙ্গে শেয়ার করতে হলে KMS key-ও শেয়ার করতে হয় — নইলে অন্য অ্যাকাউন্ট snapshot দেখতে পেলেও ব্যবহার করতে পারবে না। KMS key নিষ্ক্রিয় বা মুছে ফেললে volume-এর ডেটা চিরতরে অকার্যকর হয়ে যায় — তাই production KMS key মুছে ফেলার আগে দুবার ভাবুন; AWS key মুছে ফেলার আগে ৭–৩০ দিনের অপেক্ষা দেয়, সেই সময়ে বাতিল করা যায়।

EBS I/O বৈশিষ্ট্য ও performance tuning: EBS volume-এর performance তিনটি সংখ্যায় মাপা হয় — IOPS, throughput ও latency। **Burst balance** (gp2-তে) একটি গুরুত্বপূর্ণ ধারণা: ছোট gp2 volume-এ baseline IOPS কম থাকে, কিন্তু burst credit জমা হয় — হঠাৎ বেশি I/O হলে credit খরচ হয়; credit শেষ হলে performance baseline-এ নেমে আসে — অ্যাপ হঠাৎ ধীর হয়ে যাওয়ার একটি সাধারণ কারণ। gp3-তে এই সমস্যা নেই — IOPS সরাসরি provisioned হয়। **Multi-attach** (io1/io2-তে) একটি বিশেষ বৈশিষ্ট্য: একটি volume একসঙ্গে একাধিক instance-এ যুক্ত করা যায় — ক্লাস্টার ফাইল সিস্টেমের (যেমন GFS2) জন্য; সাধারণ অ্যাপে ব্যবহার করবেন না, কারণ দুটি instance একই block-এ লিখলে ডেটা নষ্ট হতে পারে। **EBS RAID:** একাধিক EBS volume-কে RAID 0 (গতির জন্য — IOPS যোগ হয়, কিন্তু একটি volume নষ্ট হলে সব ডেটা যায়) বা RAID 1 (প্রতিলিপির জন্য) করে ব্যবহার করা যায় — তবে AWS-এর পরামর্শ হলো RAID-এর বদলে বড় volume বা io2 ব্যবহার করা, কারণ EBS নিজেই প্রতিলিপি রাখে। **Volume resize:** EBS volume-এর আকার বাড়ানো যায় চলমান instance-এই — `modify-volume` দিয়ে আকার বা type বদলান, তারপর OS-এর ভেতরে ফাইল সিস্টেম বাড়ান (`growpart` ও `xfs_growfs` বা `resize2fs`); type বদলাতে (যেমন gp2 থেকে gp3) কোনো downtime লাগে না, তবে পরিবর্তন সম্পূর্ণ হতে কিছু সময় লাগে — CloudWatch-এ দেখুন।

FSx গভীরে ও DataSync: FSx for Windows File Server-এ **Active Directory** যুক্ত করা যায় — ব্যবহারকারীর Windows অনুমতি সরাসরি কাজ করে; **FSx for Lustre**-এ S3-র সঙ্গে **data repository association** আছে — S3-র ডেটা Lustre-তে দেখা যায়, কাজ শেষে ফলাফল আবার S3-তে লেখা যায় — মেশিন লার্নিং প্রশিক্ষণের আদর্শ প্যাটার্ন; **FSx for NetApp ONTAP**-এ **FlexClone** দিয়ে মুহূর্তে ডেটার কপি বানানো যায় — পরীক্ষার জন্য ডেটাবেসের ক্লোন বানাতে অসাধারণ। আর **AWS DataSync** হলো on-premises থেকে AWS-এ (বা AWS-এর ভেতরে S3, EFS, FSx-এর মধ্যে) ডেটা স্থানান্তরের সেবা — শুধু বদলানো ফাইল কপি করে, ব্যান্ডউইথ বাঁচায়, আর transfer যাচাই করে। একটি কোম্পানি যখন নিজের ফাইল সার্ভার থেকে EFS-এ যায়, তখন DataSync-ই সঠিক উপায় — হাতে `rsync` চালানোর চেয়ে দ্রুত ও নির্ভরযোগ্য। **Durability ও availability আবার:** EBS volume-এর durability 99.999% (একটি AZ-র ভেতরে প্রতিলিপি), EFS-এর durability 99.999999999% (একাধিক AZ-তে প্রতিলিপি), S3-রও ১১টি ৯। কিন্তু availability আলাদা: EBS volume একটি AZ-তে থাকে — AZ নষ্ট হলে volume পাওয়া যায় না (snapshot থেকে অন্য AZ-তে বানাতে হয়); EFS ও S3 একাধিক AZ-তে থাকে — একটি AZ নষ্ট হলেও চলে। তাই গুরুত্বপূর্ণ অ্যাপে EFS বা Multi-AZ RDS ব্যবহার করুন, আর EBS-এ snapshot-এর ওপর নির্ভর করুন।

১৩.১২ Data Lifecycle Manager, EFS Access Point ও Instance Store গভীরে

Amazon Data Lifecycle Manager (DLM) হলো EBS snapshot-এর স্বয়ংক্রিয় সময়সূচি: আপনি একটি নীতি বানান — যেমন "প্রতিদিন রাত ২টায় snapshot নাও, ১৪ দিন রাখো, তারপর মুছে দাও" — আর DLM ট্যাগ দিয়ে চিহ্নিত volume-গুলোতে স্বয়ংক্রিয়ভাবে চালায়। AWS Backup-এর চেয়ে DLM সহজ ও EBS-কেন্দ্রিক; AWS Backup আরও ব্যাপক (RDS, EFS, DynamoDB-ও)। একটি গুরুত্বপূর্ণ বৈশিষ্ট্য: DLM-এ **fast snapshot restore** চালু করা যায় — snapshot থেকে volume বানানোর সময় প্রথমবারের lazy loading দূর হয়, volume সঙ্গে সঙ্গে পূর্ণ গতিতে চলে; তবে এর অতিরিক্ত খরচ আছে — শুধুমাত্র জরুরি পুনরুদ্ধারের volume-এ চালু করুন। **Snapshot Lifecycle:** পুরোনো snapshot মুছে ফেলার সময় মনে রাখুন — snapshot incremental হলেও প্রতিটি snapshot তার আগেরটির ওপর নির্ভর করে না এমনভাবে মুছে ফেলা যায়: AWS মুছে ফেলার সময় শুধুমাত্র সেই block মুছে যা আর কোনো snapshot-এ দরকার নেই —

তাই মাঝের snapshot মুছে ফেললেও পরের snapshot অক্ষত থাকে। তবু একটি নিয়ম: সবচেয়ে পুরোনো snapshot মুছে ফেলার আগে যাচাই করুন কোনো AMI (Amazon Machine Image) সেই snapshot ব্যবহার করছে কি না — AMI-র snapshot মুছে ফেললে AMI অকেজো হয়ে যায়।

EFS Access Point হলো EFS-এর একটি প্রবেশদ্বার: প্রতিটি অ্যাপের জন্য একটি access point বানিয়ে বলা যায় "এই অ্যাপ শুধু `/app1` ফোল্ডার দেখবে, আর সেই ফোল্ডারে লেখা ফাইলের মালিক হবে user 1001" — এতে একাধিক অ্যাপ একই EFS ভাগ করে ব্যবহার করতে পারে, কেউ অন্যের ফোল্ডারে ঢুকতে পারে না। Lambda-র সঙ্গে EFS ব্যবহার করলে access point প্রায় বাধ্যতামূলক — Lambda function-এর জন্য একটি access point বানিয়ে `/mnt/shared`-এ মাউন্ট করা যায় — এতে Lambda-র ৫১২ MB অস্থায়ী storage-এর সীমা পেরিয়ে বড় ফাইল প্রসেস করা যায়। **Instance store গভীরে:** কিছু EC2 instance type-এ (যেমন `i3`, `d3`) ভৌত ডিস্ক সরাসরি যুক্ত থাকে — এগুলো **instance store** — EBS-এর চেয়ে অনেক দ্রুত (NVMe SSD, লাখ লাখ IOPS), কিন্তু instance বন্ধ বা নষ্ট হলে ডেটা চিরতরে যায়। ব্যবহার: অস্থায়ী cache, buffer, বা এমন ডেটা যা হারালে আবার তৈরি করা যায় — যেমন Spark-এর shuffle ডেটা, বা ভিডিও transcoding-এর অস্থায়ী ফাইল। কখনোই instance store-এ একমাত্র কপি রাখবেন না — গুরুত্বপূর্ণ ডেটা সবসময় EBS বা S3-তে রাখুন। **Throughput ও latency-র বাস্তব সংখ্যা:** gp3 volume-এর latency সাধারণত ১ মিলিসেকেন্ডের কম, EFS-এর কয়েক মিলিসেকেন্ড, S3-র দশ মিলিসেকেন্ডের ওপরে — এই ক্রমটি মনে রাখুন: যত দ্রুত চাই, তত কাছে (block) রাখুন; যত বড় ও সস্তা চাই, তত দূরে (object) রাখুন। **খরচের শেষ কথা:** প্রতিমাসে AWS Cost Explorer-এ EBS, EFS ও snapshot-এর খরচ আলাদা করে দেখুন; `aws ec2 describe-volumes --filters Name=status,Values=available` দিয়ে unattached volume খুঁজুন — এগুলো চুপচাপ বিল তোলে; আর `aws ec2 describe-snapshots --owner-ids self` দিয়ে পুরোনো snapshot তালিকা করে অপ্রয়োজনীয়গুলো মুছে ফেলুন।

১৩.১৩ Snapshot শেয়ারিং, EFS Replication ও Backup Vault Lock

Snapshot শেয়ারিং ও AMI: একটি snapshot অন্য AWS অ্যাকাউন্টের সঙ্গে শেয়ার করা যায় — যেমন TechNova তাদের সফটওয়্যারের একটি image অন্য অ্যাকাউন্টে দিতে চাইলে snapshot শেয়ার করে; তবে এনক্রিপ্ট করা snapshot শেয়ার করতে KMS key-ও শেয়ার করতে হয়, আর শেয়ার করা snapshot থেকে অন্য অ্যাকাউন্ট volume বানাতে পারে কিন্তু snapshot-টি মুছতে পারে না। **AMI (Amazon Machine Image)** আসলে একটি EBS snapshot + metadata — একটি instance-এর সম্পূর্ণ কপি; AMI থেকে নতুন instance চালু করা যায় — auto scaling group-এ এটাই ব্যবহৃত হয়। একটি AMI তৈরি করার সময় instance-টি reboot হতে পারে (যদি `NoReboot` না দেওয়া হয়) — তাই production AMI বানানোর সময় রক্ষণাবেক্ষণ জানালা ব্যবহার করুন। **EFS Replication:** একটি EFS ফাইল সিস্টেম অন্য region-এর EFS-এ প্রতিলিপি করা যায় — দুর্যোগ পুনরুদ্ধারের জন্য; replication-এ শুধু বদলানো ফাইল কপি হয়, আর destination ফাইল সিস্টেম read-only থাকে যতক্ষণ না failover করা হয়। **AWS Backup Vault Lock:** একটি backup vault-কে **WORM** (Write Once Read Many) করা যায় — একবার lock করলে ব্যাকআপ কেউ মুছতে পারে না, এমনকি root-ও নয়, মেয়াদ শেষ না হওয়া পর্যন্ত — ransomware (মুক্তিপণ ভাইরাস) থেকে বাঁচার জন্য এটি অত্যন্ত গুরুত্বপূর্ণ: আক্রমণকারী আপনার সিস্টেমে ঢুকলেও ব্যাকআপ মুছতে পারবে না। Vault Lock চালু করার সময় **cooling-off period** থাকে — যেমন ৩ দিন — সেই সময়ে lock বাতিল করা যায়, তারপর আর নয় — তাই সাবধানে পরিকল্পনা করুন। **Backup-এর খরচের হিসাব:** EBS snapshot-এর খরচ GB-প্রতি মাসিক — কিন্তু মনে রাখুন, snapshot শুধু বদলানো block-এর জন্য নেওয়া হয় — প্রতিদিন snapshot নিলেও খরচ শুধু বদলানো অংশের জন্য; AWS Backup-এর **cold storage tier** আছে — ৯০ দিনের পুরোনো ব্যাকআপ সস্তা cold storage-এ যায় — দীর্ঘমেয়াদি সংরক্ষণের জন্য ভালো। **পুনরুদ্ধারের গতি:** snapshot থেকে volume বানানো সঙ্গে সঙ্গে হয়, কিন্তু প্রথমবারের পড়া ধীর হয় (lazy loading) — **fast snapshot restore** চালু করলে volume সঙ্গে সঙ্গে পূর্ণ গতিতে চলে; EFS থেকে ফাইল ফেরানো সঙ্গে সঙ্গে হয়; আর AWS Backup থেকে restore করার সময় **restore testing plan** দিয়ে স্বয়ংক্রিয়ভাবে পরীক্ষা করা যায় — AWS নিজে restore করে যাচাই করে যে ডেটা ঠিক আছে।

১৩.১৪ সারাংশ, প্রশ্ন ও অনুশীলন

সারাংশ: Block (EBS) একটি instance-এর ডিস্ক — দ্রুত, কিন্তু একটি AZ-তে; File (EFS/FSx) অনেকের ভাগ করা ফোল্ডার — EFS সাধারণ NFS, FSx বিশেষায়িত; Object (S3) HTTP API — সস্তা ও অসীম, কিন্তু ডেটাবেসের কাজে নয়। EBS-এ gp3 ডিফল্ট, io2 গুরুত্বপূর্ণ ডেটাবেসের জন্য, st1/sc1 বড় ক্রমিক কাজের জন্য। Snapshot incremental ব্যাকআপ দেয়, AWS Backup কেন্দ্রীয় নীতি দেয়, আর RPO/RTO দিয়ে ব্যাকআপের ঘনত্ব ও পুনরুদ্ধারের গতি ঠিক হয়। ব্যাকআপের আসল পরীক্ষা restore drill।

Review Questions: (১) Block, file ও object storage-এর মূল পার্থক্য কী? (২) gp3 কেন gp2-র চেয়ে ভালো? (৩) io2 কখন ব্যবহার করবেন? (৪) Snapshot incremental হওয়ার অর্থ কী? (৫) EFS-এর Bursting throughput mode-এ ছোট ফাইল সিস্টেম কেন ধীর হয়? (৬) FSx for Lustre কখন ব্যবহার করবেন? (৭) RPO ও RTO-র পার্থক্য কী? (৮) Instance store ও EBS-এর পার্থক্য কী? (৯) Snapshot public শেয়ার করলে কী ঝুঁকি? (১০) DeleteOnTermination কী এবং কেন সতর্ক থাকবেন?

Practical Exercises: (১) একটি gp3 volume তৈরি করে EC2-তে attach করুন, ফাইল সিস্টেম বানিয়ে মাউন্ট করুন, একটি ফাইল লিখে snapshot নিন, snapshot থেকে নতুন volume বানিয়ে অন্য instance-এ মাউন্ট করে ফাইলটি ফিরে পেয়েছেন কি না যাচাই করুন। (২) একটি EFS ফাইল সিস্টেম তৈরি করে দুটি EC2 instance থেকে একসঙ্গে মাউন্ট করুন — একটিতে লেখা ফাইল অন্যটিতে দেখা যায় কি না পরীক্ষা করুন। (৩) একটি AWS Backup plan বানান যা প্রতিদিন একটি ট্যাগ করা volume-এর ব্যাকআপ নেয় ও ৭ দিন রাখে; তারপর একটি restore drill করে সময় মাপুন (RTO যাচাই)।

অধ্যায় ১৪ • PART III — Storage, Database & Application Services

Chapter 14 — RDS and Aurora

১৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) **managed relational database** (পরিচালিত সম্পর্কযুক্ত ডেটাবেস) কী ও কেন নিজে EC2-তে ডেটাবেস চালানোর চেয়ে RDS ভালো, তা ব্যাখ্যা করতে; (২) RDS-এর engine (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server) বেছে নিতে; (৩) **Amazon Aurora**-র আর্কিটেকচার — storage layer, writer ও reader endpoint — ব্যাখ্যা করতে; (৪) **Multi-AZ** deployment ও **read replica** (পড়ার প্রতিলিপি)-র মধ্যে পার্থক্য করতে ও সঠিকভাবে ব্যবহার করতে; (৫) automated backup, snapshot ও **point-in-time recovery (PITR)** (নির্দিষ্ট মুহূর্তে ফেরানো) বাস্তবায়ন করতে; (৬) **parameter group** ও **option group** দিয়ে ডেটাবেস কনফিগার করতে; (৭) encryption, VPC ও security group দিয়ে ডেটাবেস সুরক্ষিত করতে; (৮) **DMS** (Database Migration Service) দিয়ে migration (স্থানান্তর) পরিকল্পনা করতে; এবং (৯) production ডেটাবেস পরিচালনা — monitoring, failover পরীক্ষা, connection পরিচালনা — করতে।

১৪.২ পূর্বশর্ত ঝালাই (Prerequisite Refresher)

এই অধ্যায়ে আপনার SQL-এর মৌলিক ধারণা থাকা দরকার: table, row, primary key, index, JOIN, transaction (ACID — Atomicity, Consistency, Isolation, Durability)। আর ১৩ নম্বর অধ্যায় থেকে EBS volume ও snapshot, VPC-র subnet (public বনাম private) ও security group, এবং KMS encryption-এর ধারণা সঙ্গে নিন। একটি গুরুত্বপূর্ণ শব্দ: **endpoint** — ডেটাবেসের DNS নাম ও পোর্ট (যেমন `technova-db.abcdef.ap-south-1.rds.amazonaws.com:5432`); অ্যাপ এই endpoint দিয়ে ডেটাবেসের সঙ্গে যুক্ত হয়, IP ঠিকানা দিয়ে নয় — কারণ failover হলে IP বদলাতে পারে, endpoint একই থাকে। আরেকটি শব্দ: **DB subnet group** — RDS কোন subnet-গুলোতে instance বানাতে পারবে, তার তালিকা; Multi-AZ-র জন্য কমপক্ষে দুটি AZ-তে subnet থাকতে হয়।

১৪.৩ কেন Managed Database — আগে কী সমস্যা ছিল

নিজে EC2-তে ডেটাবেস চালানো মানে: অপারেটিং সিস্টেম patch করা, ডেটাবেস সফটওয়্যার আপগ্রেড করা, ব্যাকআপ স্ক্রিপ্ট লেখা, disk পূর্ণ হলে volume বাড়ানো, primary নষ্ট হলে replica-কে primary বানানো (failover), আর রাত ৩টায় ডেটাবেস বন্ধ হয়ে গেলে ঘুম থেকে উঠে ঠিক করা — সবই আপনার দায়িত্ব। **Amazon RDS** (Relational Database Service) এই পরিচালনার ভার নেয়: AWS OS patch করে, ডেটাবেস engine আপগ্রেডের ব্যবস্থা করে, স্বয়ংক্রিয় ব্যাকআপ নেয়, storage নিজে থেকে বাড়ায় (autoscaling), Multi-AZ failover স্বয়ংক্রিয়ভাবে করে, আর monitoring (CloudWatch) দেয়। আপনি শুধু SQL লেখেন, schema বানান ও অ্যাপ যুক্ত করেন। এটি **managed service**-এর মূল ধারণা: undifferentiated heavy lifting (সাধারণ ভারী কাজ) AWS-কে দিয়ে, ব্যবসার যুক্তিতে মনোযোগ দেওয়া। তবে মনে রাখবেন: RDS আপনার SQL বা schema ঠিক করে না — খারাপ query, missing index বা ভুল schema-র দায়িত্ব আপনারই। Managed মানে "চিন্তামুক্ত" নয়, মানে "পরিচালনার ভার কম"।

১৪.৪ RDS Engine ও Instance নির্বাচন

RDS-এ ছয়টি engine চলে: **PostgreSQL** (TechNova-র পছন্দ — ওপেন সোর্স, শক্তিশালী, JSON ও extension সমৃদ্ধ), **MySQL** ও **MariaDB** (বহুল ব্যবহৃত, WordPress-এর জন্য পরিচিত), **Oracle** ও **Microsoft SQL Server** (লাইসেন্সসহ, পুরোনো এন্টারপ্রাইজ অ্যাপের জন্য), এবং **Aurora** (AWS-এর নিজস্ব, MySQL ও PostgreSQL-

সামঞ্জস্যপূর্ণ)। Engine বেছে নেওয়ার নিয়ম: নতুন অ্যাপ হলে PostgreSQL বা Aurora PostgreSQL; WordPress বা পরিচিত MySQL অ্যাপ হলে MySQL; পুরোনো Oracle/SQL Server অ্যাপ হলে সেই engine (লাইসেন্সের খরচ Pricing Calculator-এ যাচাই করুন)। **DB instance class** (যেমন `db.t3.medium`, `db.r6g.large`) হলো CPU ও RAM-এর আকার: `t3/t4g` burstable — ছোট, অনিয়মিত কাজের জন্য সস্তা; `r6g/r7g` memory-optimized — ডেটাবেসের জন্য সাধারণ পছন্দ, কারণ ডেটাবেস RAM-এ cache রাখে; `m6g/m7g` general purpose — ভারসাম্যপূর্ণ। Storage: **gp3** ডিফল্ট, IOPS ও throughput আলাদা করে বাড়ানো যায়; **io2** সবচেয়ে দ্রুত, গুরুত্বপূর্ণ ডেটাবেসের জন্য। **Storage autoscaling** চালু রাখুন — disk পূর্ণ হয়ে ডেটাবেস বন্ধ হয়ে যাওয়া একটি ক্লাসিক production দুর্ঘটনা, autoscaling তা ঠেকায়।

```
# PostgreSQL RDS instance তৈরি করুন (Multi-AZ, encrypted)
aws rds create-db-instance --db-instance-identifier technova-db \
  --db-instance-class db.t3.medium --engine postgres \
  --engine-version 16.3 --allocated-storage 100 \
  --storage-type gp3 --iops 3000 --storage-encrypted \
  --master-username technova_admin \
  --master-user-password 'CHANGE-ME-STRONG-PASSWORD' \
  --db-subnet-group-name technova-db-subnets \
  --vpc-security-group-ids sg-CHANGE-ME \
  --multi-az --backup-retention-period 7 \
  --deletion-protection --region ap-south-1

# Instance-এর অবস্থা ও endpoint দেখুন
aws rds describe-db-instances --db-instance-identifier technova-db \
  --query 'DBInstances[0].[DBInstanceStatus,Endpoint.Address,MultiAZ]'
```

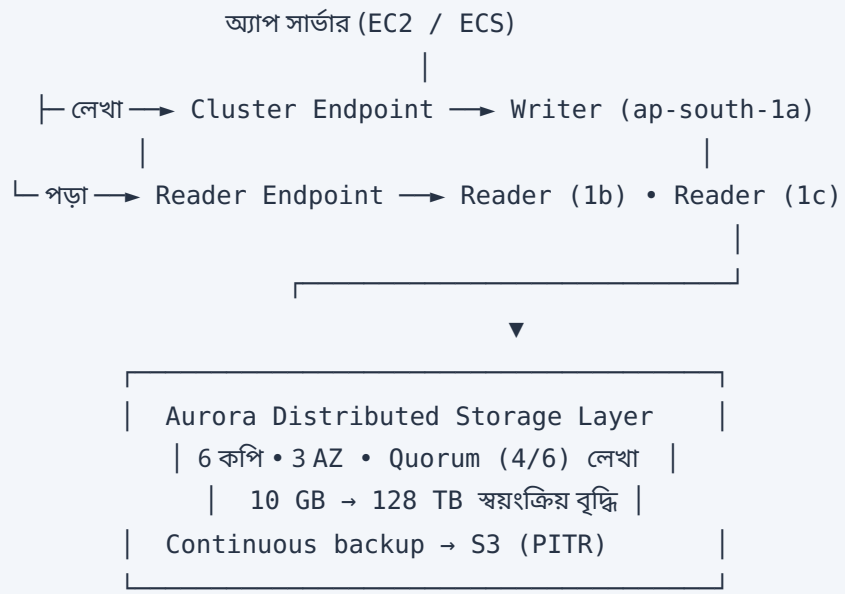
> **WARNING:** Master password কখনো কমান্ড লাইনে বা কোডে লিখবেন না — shell history-তে থেকে যায়। Production-এ **AWS Secrets Manager** ব্যবহার করুন ও RDS-এর "Manage master credentials in Secrets Manager" বিকল্প চালু করুন, যাতে পাসওয়ার্ড স্বয়ংক্রিয়ভাবে rotate (নিয়মিত বদল) হয়। উপরের কমান্ডটি শুধু শেখার জন্য; আসল পাসওয়ার্ড `CHANGE-ME-...` দিয়ে বদলান ও Secrets Manager-এ রাখুন।

১৪.৫ Aurora আর্কিটেকচার — কেন এটি আলাদা

Amazon Aurora হলো AWS-এর নিজস্ব ডেটাবেস engine, যা MySQL ও PostgreSQL-এর সঙ্গে সামঞ্জস্যপূর্ণ (compatible) — অর্থাৎ আপনার অ্যাপ প্রায় কোনো বদল ছাড়াই Aurora-তে চলতে পারে — কিন্তু ভেতরের আর্কিটেকচার সম্পূর্ণ ভিন্ন। সাধারণ RDS-এ ডেটাবেস instance-এর সঙ্গে EBS volume যুক্ত থাকে; Aurora-তে **compute ও storage আলাদা**। Storage layer হলো একটি বিতরণ করা (distributed), স্বয়ংক্রিয়ভাবে বাড়তে থাকা সিস্টেম: ডেটা ছয়টি কপিতে তিনটি AZ-তে ছড়িয়ে থাকে, আর প্রতিটি লেখা ছয়টি কপির মধ্যে কমপক্ষে চারটিতে নিশ্চিত (quorum) হলেই সফল ধরা হয়। এর ফলে: (১) storage ১০ GB থেকে ১২৮ TB পর্যন্ত নিজে থেকে বাড়ে — আগে থেকে আকার ঠিক করতে হয় না; (২) replica তৈরি হয় মিনিটে — কারণ replica-কে ডেটা কপি করতে হয় না, শুধু storage layer-এর সঙ্গে যুক্ত হতে হয়; (৩) crash recovery দ্রুত — ডেটা আগেই বিতরণ করা থাকে, তাই পুনরুদ্ধারে সময় লাগে না। Aurora-র **cluster** (গুচ্ছ)-এ একটি **writer instance** (লেখার instance — INSERT/UPDATE/DELETE এখানে যায়) ও শূন্য থেকে পনেরোটি **reader instance** (পড়ার instance — SELECT এখানে যায়) থাকে। দুটি endpoint আছে: **cluster endpoint** (writer-এর দিকে যায়) ও **reader endpoint** (reader-দের মধ্যে load balance করে)। Failover হলে Aurora স্বয়ংক্রিয়ভাবে একটি reader-কে writer বানায় — সাধারণত ৩০ সেকেন্ডের কম সময়ে। **Aurora Serverless v2** আরও এক ধাপ:

capacity (ACU — Aurora Capacity Unit) চাহিদা অনুযায়ী স্বয়ংক্রিয়ভাবে বাড়ে-কমে — অনিয়মিত workload-এর জন্য ভালো, তবে খরচের হিসাব সাবধানে করুন।

চিত্র ১৪.১ — Aurora Cluster আর্কিটেকচার (ap-south-1)



১৪.৬ Multi-AZ বনাম Read Replica — দুটি ভিন্ন জিনিস

এই দুটি প্রায়ই গুলিয়ে ফেলা হয়, কিন্তু এদের উদ্দেশ্য সম্পূর্ণ আলাদা। **Multi-AZ deployment** হলো **উচ্চ উপলব্ধতার** (high availability) জন্য: একটি primary instance একটি AZ-তে, আর একটি synchronous standby (সমন্বিত প্রস্তুত) instance অন্য AZ-তে — প্রতিটি লেখা standby-তেও লেখা হয়। Primary নষ্ট হলে AWS স্বয়ংক্রিয়ভাবে standby-কে primary বানায় (failover, সাধারণত ৬০–১২০ সেকেন্ড) — endpoint একই থাকে, অ্যাপকে কিছু করতে হয় না। Standby থেকে সরাসরি পড়া যায় না; এটি শুধু failover-এর জন্য। **Read replica** হলো **performance ও scale**-এর জন্য: primary থেকে asynchronous replication (অসমন্বিত প্রতিলিপি) দিয়ে ডেটা কপি হয়, replica থেকে SELECT query চালানো যায় — পড়ার চাপ ভাগ হয়। Replica-কে প্রয়োজনে আলাদা করে promote করে স্বতন্ত্র ডেটাবেস বানানো যায়। মনে রাখার সূত্র: Multi-AZ = "নষ্ট হলে বাঁচব", Read Replica = "পড়ার চাপ সামলাব"। দুটো একসঙ্গে ব্যবহার করা যায়: Multi-AZ primary + কয়েকটি read replica। আর **replication lag** (প্রতিলিপির বিলম্ব) মনে রাখুন: replica-তে ডেটা পৌঁছাতে কয়েক মিলিসেকেন্ড থেকে সেকেন্ড লাগতে পারে — লেখার পরপরই replica থেকে পড়লে পুরোনো ডেটা পেতে পারেন (eventual consistency)। তাই লেখার পরপরই পড়তে হলে writer endpoint ব্যবহার করুন।

```
# Read replica তৈরি করুন (একই region)
aws rds create-db-instance-read-replica \
  --db-instance-identifier technova-db-replica-1 \
  --source-db-instance-identifier technova-db --region ap-south-1

# Multi-AZ failover পরীক্ষা করুন (রক্ষণাবেক্ষণের জানালায় করুন)
aws rds reboot-db-instance --db-instance-identifier technova-db \
  --force-failover --region ap-south-1
```

১৪.৭ Backup, Snapshot ও Point-in-Time Recovery

RDS-এ তিন স্তরের পুনরুদ্ধার আছে। (১) **Automated backup:** প্রতিদিন একটি snapshot নেওয়া হয়, আর transaction log প্রতি ৫ মিনিটে সংরক্ষণ করা হয়। **Backup retention period** (যেমন ৭ দিন, সর্বোচ্চ ৩৫ দিন) ঠিক করে দেয় কতদিনের পেছনে ফেরা যাবে। (২) **Manual snapshot:** আপনি নিজে নেন — যেমন বড় পরিবর্তনের আগে; এগুলো retention period-এর আওতার বাইরে, যতদিন চান রাখা যায়, তবে storage বিল আসে। (৩) **Point-in-time recovery (PITR):** transaction log ব্যবহার করে যেকোনো নির্দিষ্ট সেকেন্ডে ফেরা যায় — যেমন ভুল করে `DELETE FROM users` চালানোর ৫ মিনিট আগের অবস্থায়। PITR আসলে একটি **নতুন** DB instance বানায় — পুরোনোটি বদলায় না; তারপর আপনি যাচাই করে অ্যাপকে নতুন instance-এর দিকে ঘুরিয়ে দেন। মনে রাখবেন: automated backup চালু না থাকলে (retention ০ দিন) PITR সম্ভব নয়। আর **deletion protection** চালু রাখুন — ভুল করে instance মুছে ফেলার সবচেয়ে সাধারণ দুর্ঘটনা এটি ঠেকায়; মুছতে হলে প্রথমে protection বন্ধ করতে হয়, যা একটি সচেতন পদক্ষেপ।

```
# Manual snapshot নিন (বড় deploy-এর আগে)
aws rds create-db-snapshot --db-instance-identifier technova-db \
  --db-snapshot-identifier technova-db-before-deploy-2026-11-09

# Point-in-time recovery: নির্দিষ্ট সময়ে নতুন instance বানান
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier technova-db \
  --target-db-instance-identifier technova-db-restored \
  --restore-time 2026-11-09T10:00:00+06:00 --region ap-south-1
```

১৪.৮ Parameter Group, Option Group ও নেটওয়ার্ক নিরাপত্তা

Parameter group হলো ডেটাবেস engine-এর কনফিগারেশন মানের সেট — যেমন PostgreSQL-এ `max_connections` (সর্বোচ্চ সংযোগ), `shared_buffers` (cache-এর আকার), `work_mem`। কিছু parameter বদলাতে reboot লাগে (static), কিছু সঙ্গে সঙ্গে কার্যকর হয় (dynamic)। ডিফল্ট parameter group বদলানো যায় না — নিজের parameter group বানিয়ে instance-এ যুক্ত করুন। একটি সাধারণ ভুল: `max_connections` বাড়িয়ে দেওয়া — প্রতিটি সংযোগ RAM খায়; বেশি সংযোগ মানে ধীর ডেটাবেস। এর বদলে **RDS Proxy** ব্যবহার করুন — এটি সংযোগ pooling (ভাগ করা) করে, Lambda-র মতো হাজার হাজার ছোট সংযোগ সামলায়, আর failover-এর সময় সংযোগ ধরে রাখে। **Option group** হলো engine-এর অতিরিক্ত বৈশিষ্ট্য — যেমন Oracle-এর জন্য, অথবা MariaDB/MySQL-এর audit plugin। **নেটওয়ার্ক নিরাপত্তা:** ডেটাবেস কখনো public subnet-এ রাখবেন না — private subnet-এ রাখুন, **Publicly accessible: No** করুন; security group-এ শুধুমাত্র অ্যাপ সার্ভারের security group থেকে ডেটাবেস পোর্ট (PostgreSQL 5432, MySQL 3306) খুলুন — কখনো `0.0.0.0/0` নয়; IAM database authentication ব্যবহার করলে পাসওয়ার্ড ছাড়াই IAM credential দিয়ে যুক্ত হওয়া যায় (PostgreSQL/MySQL-এ সমর্থিত); encryption at rest (KMS) ও in transit (SSL/TLS) চালু করুন।

```
# নিজের parameter group বানান ও মান বদলান
aws rds create-db-parameter-group --db-parameter-group-name technova-pg16 \
  --db-parameter-group-family postgres16 --description "TechNova PG16 tuning"
aws rds modify-db-parameter-group --db-parameter-group-name technova-pg16 \
  --parameters "ParameterName=max_connections,ParameterValue=200,ApplyMethod=pending-reboot"
# Instance-এ যুক্ত করুন (reboot লাগতে পারে)
aws rds modify-db-instance --db-instance-identifier technova-db \
  --db-parameter-group-name technova-pg16 --apply-immediately
```

১৪.৯ DMS দিয়ে Migration ও Production পরিচালনা

AWS Database Migration Service (DMS) হলো ডেটাবেস স্থানান্তরের সেবা: source ডেটাবেস (on-premises, অন্য ক্লাউড, বা অন্য RDS) থেকে target ডেটাবেসে ডেটা কপি করে — প্রথমে পূর্ণ load (full load), তারপর চলমান পরিবর্তন (CDC — Change Data Capture) ধরে রাখে, যাতে স্থানান্তরের সময় অ্যাপ বন্ধ রাখতে হয় না বা কম রাখতে হয়। DMS-এর তিনটি অংশ: **replication instance** (যে সার্ভার কাজ করে), **source ও target endpoint**, এবং **migration task**। Schema বদলাতে হলে (যেমন Oracle থেকে PostgreSQL) **AWS Schema Conversion Tool (SCT)** ব্যবহার করুন। Migration-এর আগে: source-এর ডেটা যাচাই করুন, target-এ index ও constraint পরীক্ষা করুন, একটি পরীক্ষামূলক migration চালান, আর cutover (চূড়ান্ত বদল)-এর জন্য একটি রক্ষণাবেক্ষণ জানালা ঠিক করুন। **Production পরিচালনা:** CloudWatch-এ `CPUUtilization`, `FreeableMemory`, `DatabaseConnections`, `ReadLatency`, `WriteLatency`, `ReplicaLag` দেখুন; **Performance Insights** চালু করুন — এটি দেখায় কোন query সবচেয়ে বেশি সময় নিচ্ছে; **Enhanced Monitoring** দিয়ে OS-স্তরের মেট্রিক দেখুন; slow query log চালু করে খারাপ query খুঁজুন; প্রতি ত্রৈমাসিকে failover পরীক্ষা করুন — failover কখনো পরীক্ষা না করলে, আসল failover-এর সময় কী হবে তা কেউ জানে না।

> **REAL WORLD — TechNova-র failover মহড়া:** TechNova প্রতি মাসের প্রথম শনিবার রাত ২টায় একটি পরিকল্পিত failover পরীক্ষা করে: `reboot-db-instance --force-failover` চালিয়ে দেখে অ্যাপ কতক্ষণে আবার যুক্ত হয়, কোনো error বাড়ে কি না, আর CloudWatch alarm ঠিকমতো বাজে কি না। একবার পরীক্ষায় দেখা গেল অ্যাপের connection pool failover-এর পর পুরোনো সংযোগ ধরে রেখেছে — সমাধান ছিল RDS Proxy চালু করা ও অ্যাপে retry logic যোগ করা। এই মহড়া না থাকলে আসল বিপদে এই সমস্যা production-এ ধরা পড়ত।

১৪.১০ Production Scenario, নিরাপত্তা, খরচ, ভুল ও Troubleshooting

Production scenario — TechNova-র অর্ডার ডেটাবেস: Aurora PostgreSQL cluster — writer + ২টি reader, Multi-AZ, cluster endpoint দিয়ে লেখা, reader endpoint দিয়ে রিপোর্ট query; backup retention ১৪ দিন, PITR চালু; RDS Proxy সামনে; Performance Insights চালু; CloudWatch alarm — CPU ৮০%-এর বেশি ৫ মিনিট, ReplicaLag ১০ সেকেন্ডের বেশি, FreeableMemory কম। **নিরাপত্তা চেকলিস্ট:** private subnet, security group-এ শুধু অ্যাপের SG, deletion protection, encryption at rest + in transit, Secrets Manager-এ credential, IAM authentication যেখানে সম্ভব, CloudTrail-এ RDS API লগ, parameter group-এ `rds.force_ssl=1` (PostgreSQL-এ SSL বাধ্যতামূলক)। **খরচ:** instance সারাদিন চলে — সঠিক আকার বেছে নিন (over-provisioning সবচেয়ে বড় অপচয়); development-এ instance বন্ধ করে রাখুন (RDS instance ৭ দিনের বেশি বন্ধ রাখা যায় না — তারপর AWS নিজে চালু করে দেয়, মনে রাখুন); Reserved Instance বা Aurora I/O-Optimized খরচের হিসাব Pricing Calculator-এ যাচাই করুন; Aurora-তে I/O-র খরচ আলাদা — I/O বেশি হলে I/O-Optimized ভালো হতে পারে। **সাধারণ ভুল:** (১) ডেটাবেস public করা; (২) Multi-AZ না চালিয়ে production চালানো; (৩) backup retention ০ রাখা; (৪) deletion protection বন্ধ রাখা; (৫) replica থেকে লেখার পরপরই পড়া (replication lag); (৬) `max_connections` বাড়িয়ে RAM শেষ করা; (৭) failover কখনো পরীক্ষা না করা; (৮) storage autoscaling বন্ধ রাখা। **Troubleshooting:** সংযোগ হচ্ছে না? — security group, subnet group, endpoint, DNS যাচাই করুন; `telnet endpoint 5432` বা `psql` দিয়ে পরীক্ষা করুন। ধীর query? — Performance Insights ও `EXPLAIN ANALYZE` দেখুন, index আছে কি না দেখুন। Disk পূর্ণ? — CloudWatch `FreeStorageSpace` দেখুন, autoscaling চালু করুন, পুরোনো log মুছুন। Failover-এর পর অ্যাপ যুক্ত হচ্ছে না? — DNS cache (TTL ৫ সেকেন্ড) ও connection pool-এর পুরোনো সংযোগ যাচাই করুন।

১৪.১১ Aurora Serverless, Global Database, Performance Insights ও ব্যর্থতার বিশ্লেষণ

Aurora Serverless v2 সম্পর্কে গভীরে যান: এটি Aurora-র এমন একটি mode যেখানে capacity **ACU** (Aurora Capacity Unit — প্রায় ২ GB RAM ও সংশ্লিষ্ট CPU) এককে মাপা হয়, আর চাহিদা অনুযায়ী ০.৫ ACU থেকে হাজার ACU পর্যন্ত স্বয়ংক্রিয়ভাবে বাড়ে-কমে। কখন ব্যবহার করবেন? অনিয়মিত workload — যেমন দিনে কয়েকবার চলা রিপোর্টিং, বা নতুন অ্যাপ যার চাহিদা জানা নেই। কখন ব্যবহার করবেন না? স্থির, বেশি চাপের workload — সেখানে provisioned instance সাধারণত সস্তা হয়; আর Serverless-এ কিছু বৈশিষ্ট্য (যেমন কিছু parameter) সীমিত। **Aurora Global Database** হলো এক ধাপ এগিয়ে: একটি primary region-এ লেখা হয়, আর পাঁচটি পর্যন্ত secondary region-এ ডেটা এক সেকেন্ডের কম replication lag-এ প্রতিলিপি হয় — বিশ্বব্যাপী অ্যাপের জন্য, আর দুর্যোগে অন্য region-কে primary বানানো যায় (managed planned failover)। **Aurora I/O-Optimized** হলো এমন একটি কনফিগারেশন যেখানে I/O-র আলাদা খরচ নেই — I/O-ভারী অ্যাপে (যেমন অনেক ছোট লেখা) এটি সস্তা হতে পারে; Pricing Calculator-এ আপনার I/O-র পরিমাণ দিয়ে হিসাব করুন।

Performance Insights ও query tuning গভীরে: Performance Insights-এর **Top SQL** দেখায় কোন query সবচেয়ে বেশি DB load তৈরি করছে — **DB load** হলো গড়ে কতটি সক্রিয় (active) সেশন চলছে; একটি ভালো নিয়ম: DB load CPU core-এর সংখ্যার কাছাকাছি থাকা স্বাভাবিক, তার চেয়ে অনেক বেশি মানে bottleneck। **Wait event** দেখায় query কীসের জন্য অপেক্ষা করছে — `IO` (ডিস্ক), `LWLock` (lock), `CPU` — প্রতিটির সমাধান আলাদা: IO wait বেশি হলে index বা IOPS বাড়ান, lock wait বেশি হলে transaction ছোট করুন, CPU বেশি হলে query optimize করুন। **EXPLAIN ANALYZE** দিয়ে query-র plan দেখুন — `Seq Scan` (পুরো table পড়া) দেখলে index যোগ করুন, `Nested Loop` বড় table-এ দেখলে JOIN-এর ধরন বদলান। **Index-এর খরচ** মনে রাখুন: প্রতিটি index লেখাকে ধীর করে (INSERT-এ সব index আপডেট হয়) ও storage খায় — অপ্রয়োজনীয় index মুছে ফেলুন, আর **pg_stat_user_indexes** দিয়ে দেখুন কোন index কখনো ব্যবহার হয় না। **Connection পরিচালনা:** প্রতিটি connection RAM ও CPU খায়; অ্যাপে connection pool (যেমন PgBouncer, বা RDS Proxy) ব্যবহার করুন; Lambda থেকে ডেটাবেসে যুক্ত হলে RDS Proxy প্রায় বাধ্যতামূলক — নইলে প্রতিটি Lambda invocation নতুন connection খুলে ডেটাবেসকে `too many connections` error-এ ফেলে দেয়।

ব্যর্থতা বিশ্লেষণ — ডেটাবেস নষ্ট হলে কী হয়: Primary instance নষ্ট হলে Multi-AZ-তে standby ৬০–১২০ সেকেন্ডে primary হয় — এই সময়ে অ্যাপের লেখা ব্যর্থ হবে, তাই অ্যাপে **retry logic** (পুনরায় চেষ্টা) থাকতে হবে। Storage পূর্ণ হলে ডেটাবেস লেখা বন্ধ করে দেয় — autoscaling ও CloudWatch alarm দিয়ে ঠেকান। **Failover-এর সময় DNS:** endpoint-এর DNS TTL ৫ সেকেন্ড — কিন্তু অ্যাপের DNS cache বা connection pool পুরোনো IP ধরে রাখতে পারে — তাই failover-এর পর অ্যাপ পুনরায় চালু বা connection pool refresh করা ভালো। **ডেটা নষ্ট হলে:** ভুল DELETE বা UPDATE-এর পর PITR দিয়ে নির্দিষ্ট সময়ে ফেরা যায় — তবে PITR নতুন instance বানায়, তাই অ্যাপকে নতুন endpoint-এ ঘোরাতে হয়; আর PITR-এর জন্য backup retention চালু থাকতে হয় — retention ০ থাকলে PITR অসম্ভব। **কখন RDS ব্যবহার করবেন না:** অত্যন্ত বড়, অনিয়মিত ডেটা (NoSQL ভালো), বা graph ডেটা (Neptune ভালো), বা পূর্ণ-পাঠ অনুসন্ধান (OpenSearch ভালো) — প্রতিটি কাজের জন্য সঠিক ডেটাবেস আছে। **পর্যবেক্ষণ চেকলিস্ট:** CloudWatch alarm — CPU ৮০%, FreeableMemory কম, FreeStorageSpace ২০%-এর কম, ReplicaLag ১০ সেকেন্ডের বেশি, DatabaseConnections সর্বোচ্চের ৮০%-এর বেশি; আর **Enhanced Monitoring** দিয়ে OS-স্তরের CPU, memory, disk I/O দেখুন।

১৪.১২ Blue/Green Deployment, IAM Authentication ও Cross-Region গভীরে

RDS Blue/Green Deployment হলো নিরাপদ আপগ্রেডের পদ্ধতি: আপনার চলমান ডেটাবেস (Blue) থেকে একটি সম্পূর্ণ কপি (Green) বানানো হয় — Green-এ নতুন engine version বা বড় schema পরিবর্তন পরীক্ষা করা যায়, আর সব ঠিক থাকলে এক ক্লিকে traffic Green-এ ঘুরিয়ে দেওয়া হয় (switchover) — পুরোনো Blue তখন standby হিসেবে থাকে, সমস্যা হলে ফিরে যাওয়া যায়। বড় production আপগ্রেডে এটি সোনালি মান: সরাসরি production-এ আপগ্রেড করার চেয়ে Blue/Green অনেক নিরাপদ, কারণ পরীক্ষা ও ফিরে যাওয়া (rollback) দুটোই সহজ। **Minor version upgrade:** RDS-এ minor version (যেমন PostgreSQL 16.2 থেকে 16.3) স্বয়ংক্রিয়ভাবে আপগ্রেড করা যায় — **maintenance window** (রক্ষণাবেক্ষণ জানালা — সপ্তাহে একটি নির্দিষ্ট সময়, যেমন রবিবার রাত ৩টা) ঠিক করে দিন, আর AWS সেই জানালায় patch করে। কিন্তু major version upgrade (যেমন 15 থেকে 16) সবসময় হাতে পরীক্ষা করে করুন — প্রথমে একটি snapshot থেকে পরীক্ষামূলক instance বানিয়ে আপগ্রেড পরীক্ষা করুন, অ্যাপের query ঠিকমতো চলে কি না দেখুন, তারপর production-এ করুন।

IAM Database Authentication গভীরে: PostgreSQL ও MySQL-এ IAM দিয়ে ডেটাবেসে যুক্ত হওয়া যায় — পাসওয়ার্ডের বদলে একটি অস্থায়ী token (১৫ মিনিটের মেয়াদ) তৈরি করা হয়, যা IAM credential দিয়ে স্বাক্ষরিত। সুবিধা: কোনো পাসওয়ার্ড সংরক্ষণ করতে হয় না, token স্বয়ংক্রিয়ভাবে মেয়াদ শেষ হয়, আর CloudTrail-এ কে যুক্ত হলো তার লগ থাকে। তবে সীমা আছে: প্রতি সেকেন্ডে সর্বোচ্চ ২০০টি নতুন সংযোগ — তাই IAM auth Lambda-র মতো অনেক ছোট সংযোগের কাজে RDS Proxy-র সঙ্গে ব্যবহার করুন। **Secrets Manager rotation:** RDS-এর master password Secrets Manager-এ রাখলে স্বয়ংক্রিয় rotation চালু করা যায় — যেমন প্রতি ৩০ দিনে পাসওয়ার্ড বদলায়, আর অ্যাপ স্বয়ংক্রিয়ভাবে নতুন পাসওয়ার্ড পায় (Secrets Manager-এর rotation Lambda-র মাধ্যমে)। একটি গুরুত্বপূর্ণ নিয়ম: rotation-এর সময় অ্যাপের পুরোনো সংযোগ ভেঙে যেতে পারে — তাই rotation রক্ষণাবেক্ষণ জানালায় করুন, আর অ্যাপে retry logic রাখুন। **Cross-region read replica:** একটি read replica অন্য region-এ বানানো যায় — বিশ্বব্যাপী পড়ার জন্য, বা দুর্যোগে অন্য region-কে primary বানানোর জন্য (promote)। মনে রাখবেন: cross-region replication-এ data transfer-এর খরচ আছে, আর replication lag একই region-এর চেয়ে বেশি হতে পারে — Pricing Calculator-এ যাচাই করুন। **Aurora-র Backtrack:** Aurora-তে একটি বিশেষ বৈশিষ্ট্য — **backtrack** — যার মাধ্যমে ডেটাবেসকে কয়েক ঘণ্টা পেছনে ফেরানো যায় নতুন instance না বানিয়েই (৭২ ঘণ্টা পর্যন্ত) — ভুল ডেটা পরিবর্তনের পর দ্রুত ফেরার জন্য অসাধারণ, তবে backtrack চালু থাকলে storage-এর অতিরিক্ত খরচ আছে। **ডেটাবেস migration-এর শেষ কথা:** DMS-এ migration task-এর **validation** চালু করুন — এটি source ও target-এর row গুনে মিলিয়ে দেখে; আর cutover-এর আগে একটি **rehearsal** (মহড়া) করুন — পুরো migration একবার পরীক্ষামূলকভাবে চালিয়ে সময় মাপুন, যাতে আসল cutover-এর সময় কতক্ষণ লাগবে তা জানা থাকে।

১৪.১৩ Aurora Cloning, RDS Events ও Storage Autoscaling গভীরে

Aurora cloning: Aurora-র সবচেয়ে চমৎকার বৈশিষ্ট্যগুলোর একটি — একটি cluster-এর **clone** বানানো যায় মিনিটের মধ্যে, ডেটা কপি না করেই — clone শুধু storage layer-এর সঙ্গে যুক্ত হয়, আর নতুন লেখা হলেই শুধু বদলানো page কপি হয় (copy-on-write)। ব্যবহার: production ডেটার একটি কপি নিয়ে পরীক্ষা করা — যেমন নতুন feature-র query পরীক্ষা, বা রিপোর্ট চালানো — production-কে প্রভাবিত না করে। Clone-এর খরচ শুধু বদলানো ডেটার জন্য — তাই বড় ডেটাবেসের clone বানানো সস্তা। **RDS event subscription:** RDS-এর ঘটনা (যেমন failover, backup সম্পন্ন, storage পূর্ণ হওয়ার কাছাকাছি) SNS topic-এ পাঠানো যায় — সেখান থেকে ইমেইল, Slack বা Lambda-তে সতর্কবার্তা যায়। একটি production নিয়ম: failover, backup ব্যর্থতা ও storage কম — এই তিনটি event-এ অবশ্যই alarm বসান। **Storage autoscaling গভীরে:** autoscaling-এ সর্বোচ্চ storage সীমা (maximum storage threshold) ঠিক করতে হয় — যেমন ১০০ GB থেকে সর্বোচ্চ ১,০০০ GB; যখন free space ১০%-এর কম হয়, AWS স্বয়ংক্রিয়ভাবে storage বাড়ায় — তবে মনে রাখুন, storage শুধু বাড়ানো যায়, কমানো যায় না — তাই autoscaling-এর সীমা সাবধানে ঠিক করুন, নইলে একবারের

স্পাইকে storage বেড়ে বিল স্থায়ীভাবে বেড়ে যাবে। **RDS Proxy গভীরে:** Proxy শুধু connection pooling করে না — এটি failover-এর সময় সংযোগ ধরে রাখে (connection pinning কমায়), IAM authentication সহজ করে, আর **read/write splitting** করে — লেখা writer-এ, পড়া reader-এ পাঠায় — অ্যাপকে দুটি endpoint মনে রাখতে হয় না। Lambda-র সঙ্গে RDS Proxy ব্যবহার করলে প্রতিটি Lambda invocation নতুন connection খোলে না — Proxy সংযোগ ভাগ করে নেয়। **ডেটাবেসের সাপ্তাহিক রুটিন:** প্রতি সপ্তাহে `VACUUM ANALYZE` (PostgreSQL-এ — মুছে ফেলা row-র জায়গা ফেরানো ও statistics আপডেট করা), slow query log পর্যালোচনা, index ব্যবহারের পরিসংখ্যান দেখা, আর backup সফল হয়েছে কি না যাচাই — এই চারটি কাজ একটি runbook-এ লিখে রাখুন। **খরচের শেষ কথা:** RDS-এর খরচের তিনটি অংশ — instance (সারাদিন চলে), storage (GB-প্রতি), আর I/O (io2 ও Aurora-তে) — Cost Explorer-এ তিনটি আলাদা করে দেখুন; development instance রাতে বন্ধ করে রাখলে instance-এর খরচ কমে — তবে মনে রাখুন, RDS instance ৭ দিনের বেশি বন্ধ রাখা যায় না।

১৪.১৪ সারাংশ, প্রশ্ন ও অনুশীলন

সারাংশ: RDS পরিচালনার ভার নেয় — patch, backup, failover, storage scaling; আপনি SQL ও schema-র দায়িত্বে থাকেন। Aurora compute ও storage আলাদা করে — ছয় কপি, তিন AZ, দ্রুত replica ও failover দেয়। Multi-AZ উপলব্ধতার জন্য, read replica পড়ার চাপের জন্য — দুটি আলাদা উদ্দেশ্য। Backup + transaction log দিয়ে point-in-time recovery সম্ভব; parameter group দিয়ে tuning, RDS Proxy দিয়ে সংযোগ পরিচালনা, আর DMS দিয়ে migration। Production-এর সোনালি নিয়ম: private subnet, deletion protection, backup retention, failover মহড়া — চারটিই অপরিহার্য।

Review Questions: (১) Managed database নিজে EC2-তে ডেটাবেস চালানোর চেয়ে কী কী কাজ কমায়? (২) Aurora-র storage layer সাধারণ RDS-এর EBS থেকে কীভাবে আলাদা? (৩) Cluster endpoint ও reader endpoint-এর পার্থক্য কী? (৪) Multi-AZ বনাম read replica — কখন কোনটি? (৫) PITR কীভাবে কাজ করে ও কেন নতুন instance বানায়? (৬) RDS Proxy কখন ব্যবহার করবেন? (৭) Parameter group-এ static বনাম dynamic parameter কী? (৮) DMS-এ full load বনাম CDC কী? (৯) ডেটাবেস কেন private subnet-এ রাখতে হয়? (১০) Replication lag কী এবং লেখার পরপরই পড়লে কী সমস্যা হয়?

Practical Exercises: (১) একটি PostgreSQL RDS instance (Multi-AZ, encrypted, backup retention ৭ দিন, deletion protection) তৈরি করুন; psql দিয়ে যুক্ত হয়ে একটি table বানিয়ে ডেটা ঢোকান। (২) একটি read replica তৈরি করে replica-তে SELECT চালান; primary-তে লেখার পর replica-তে কতক্ষণে ডেটা আসে তা মাপুন (replication lag)। (৩) ভুল করে একটি row মুছে ফেলার ভান করুন, তারপর PITR দিয়ে নির্দিষ্ট সময়ের নতুন instance বানিয়ে row-টি ফেরান; Performance Insights চালু করে সবচেয়ে ধীর query খুঁজুন।

অধ্যায় ১৫ • PART III — Storage, Database & Application Services

Chapter 15 — DynamoDB and ElastiCache

১৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) **NoSQL** (নন-রিলেশনাল) ডেটাবেসের ধারণা ও কখন relational বনাম NoSQL বেছে নিতে হয়, তা ব্যাখ্যা করতে; (২) **Amazon DynamoDB**-র **partition key** (বিভাজন চাবি) ও **sort key** (সাজানোর চাবি) দিয়ে table ডিজাইন করতে; (৩) access pattern (প্রবেশের ধরন) থেকে **GSI** (Global Secondary Index) ও **LSI** (Local Secondary Index) বেছে নিতে; (৪) **on-demand বনাম provisioned** capacity ও billing mode-র মধ্যে সিদ্ধান্ত নিতে; (৫) consistency (সামঞ্জস্য), transaction ও **TTL** (Time to Live) ব্যবহার করতে; (৬) DynamoDB-র নিরাপত্তা ও ব্যাকআপ বাস্তবায়ন করতে; (৭) **Redis** ও **Memcached**-এর পার্থক্য ও **ElastiCache** আর্কিটেকচার ব্যাখ্যা করতে; এবং (৮) **cache-aside, write-through, TTL ও stampede protection** (ধস সুরক্ষা) সহ caching কৌশল বাস্তবায়ন করতে।

১৫.২ পূর্বশর্ত ঝালাই (Prerequisite Refresher)

১৪ নম্বর অধ্যায় থেকে relational ডেটাবেস, primary key, index ও transaction-এর ধারণা সঙ্গে নিন — কারণ NoSQL বোঝার সবচেয়ে ভালো উপায় হলো relational-এর সঙ্গে তুলনা করা। নতুন শব্দ: **key-value** (চাবি-মান জোড়া — যেমন একটি dictionary), **item** (DynamoDB-র একটি row-এর সমতুল্য — attribute-এর সেট), **attribute** (একটি field, যেমন `username`), **partition** (বিভাজন — ডেটা ভাগ করে অনেক সার্ভারে রাখা), **cache** (ক্যাশ — দ্রুত স্মৃতি, যেখানে বারবার লাগা ডেটা রাখা হয়), **TTL** (একটি item কতক্ষণ বাঁচবে), **hot partition** (গরম বিভাজন — একটি partition-এ অতিরিক্ত চাপ)। মনে রাখবেন: DynamoDB-তে কোনো JOIN নেই, কোনো schema নেই (প্রতিটি item-এর attribute আলাদা হতে পারে), আর query শুধুমাত্র key দিয়ে হয় — এই সীমাবদ্ধতাই এর গতির রহস্য।

১৫.৩ কেন NoSQL — আগে কী সমস্যা ছিল

Relational ডেটাবেস অসাধারণ, কিন্তু তিনটি জায়গায় আটকে যায়। প্রথম: **scale** — একটি টেবিলে কোটি কোটি row হলে JOIN ও জটিল query ধীর হয়ে যায়; scale করতে হলে বড় সার্ভার কিনতে হয় (vertical scaling), যার সীমা আছে। দ্বিতীয়: **schema** — প্রতিটি row-তে একই column থাকতে হয়; ব্যবহারকারীর প্রোফাইলে নতুন field যোগ করতে হলে migration চালাতে হয়। তৃতীয়: **অনিয়মিত ডেটা** — যেমন সেশন (session), গেমের leaderboard, IoT-র sensor ডেটা — এগুলোর জন্য পূর্ণ relational শক্তি দরকার হয় না, দরকার গতি ও স্কেল। **DynamoDB** এই সমস্যার উত্তরে এসেছে: একটি সম্পূর্ণ পরিচালিত key-value ও document ডেটাবেস, যা যেকোনো আকারে scale করে, একক সংখ্যার মিলিসেকেন্ডে (single-digit millisecond) উত্তর দেয়, আর serverless — কোনো সার্ভার পরিচালনা নেই। কিন্তু এর মূল্য আছে: আপনাকে আগে থেকে ভাবতে হয় অ্যাপ কীভাবে ডেটা পড়বে (access pattern), কারণ পরে নতুন query যোগ করা relational-এর মতো সহজ নয়। এই অধ্যায়ের মূল শিক্ষা হলো সেই ডিজাইন-চিন্তা।

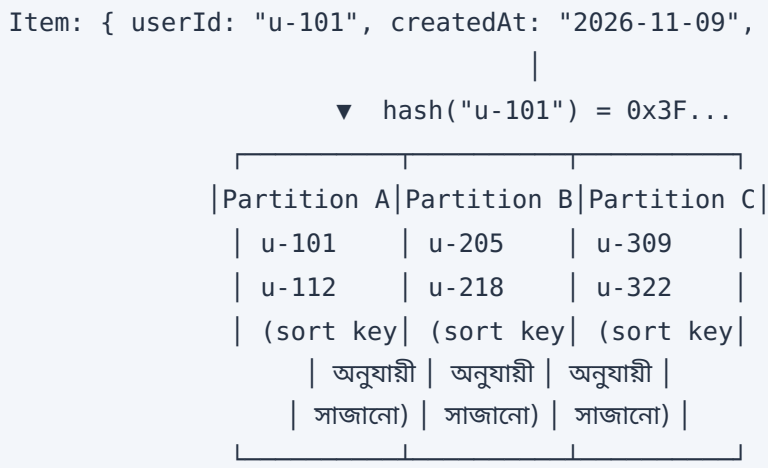
১৫.৪ DynamoDB প্রথম নীতি: Partition Key, Sort Key ও কীভাবে ডেটা ছড়ায়

DynamoDB-র একটি table-এ প্রতিটি item-এর একটি **primary key** থাকে — হয় শুধু **partition key** (যেমন `userId`), নয়তো partition key + **sort key** (যেমন `userId` + `createdAt`)। ভেতরে যা ঘটে: DynamoDB partition key-এর **hash** (একটি গাণিতিক ফাংশন) বের করে, সেই hash অনুযায়ী item-টি একটি নির্দিষ্ট **partition**-এ (একটি ভৌত সার্ভার-গুচ্ছ) রাখে। Sort key থাকলে একই partition key-এর item গুলো sort key অনুযায়ী সাজানো থাকে — তাই একটি

ব্যবহারকারীর সব অর্ডার তারিখ অনুযায়ী পড়া যায় একটি query-তে। এই নকশার সবচেয়ে গুরুত্বপূর্ণ নিয়ম: **partition key যত সমানভাবে ছড়াবে, performance তত ভালো হবে।** যদি সব item-এর partition key একই হয় (যেমন `status=active`), সব ডেটা একটি partition-এ জমবে — এটাই **hot partition** — throughput-এর সীমায় আটকে যাবে, আর অন্য partition গুলো ফাঁকা থাকবে। ভালো partition key-র উদাহরণ: `userId`, `orderId`, `deviceId` — যেগুলোর মান অনেক ও সমানভাবে ছড়ানো। খারাপ উদাহরণ: `country` (মাত্র কয়েকটি মান), `status`, তারিখ (সবাই আজকের তারিখে লেখে)। একটি বাস্তব কৌশল: partition key-তে এলোমেলো suffix যোগ করা (write sharding — যেমন `2026-11-09#1` থেকে `#10`) যাতে লেখা ছড়িয়ে যায়।

Access pattern আগে, table পরে: DynamoDB ডিজাইনের সোনালি নিয়ম — প্রথমে লিখুন অ্যাপ কী কী query করবে (যেমন: "userId দিয়ে সব অর্ডার দিন", "email দিয়ে ব্যবহারকারী খুঁজুন", "গত ৭ দিনের সবচেয়ে জনপ্রিয় পণ্য দিন")। তারপর সেই query গুলোর জন্য key ও index বেছে নিন। Relational-এ আপনি প্রথমে table বানান, পরে যেকোনো query চালান; DynamoDB-তে উল্টো — query-ই table-এর নকশা ঠিক করে।

চিত্র ১৫.১ — DynamoDB Partition: Hash দিয়ে ডেটা ছড়ানো



১৫.৫ Query, Scan, GSI ও LSI — কীভাবে ডেটা পড়বেন

DynamoDB-তে ডেটা পড়ার দুটি মূল উপায়: **Query** (দ্রুত ও সস্তা) — partition key দিয়ে (ও চাইলে sort key দিয়ে) item খোঁজা; এটি শুধু সংশ্লিষ্ট partition-এ যায়, তাই দ্রুত। **Scan** (ধীর ও দামি) — পুরো table পড়ে filter করা; বড় table-এ Scan চালানো মানে সব partition পড়া — খরচ ও সময় দুটোই বেশি। নিয়ম: Scan শুধুমাত্র ছোট table-এ বা একবারের কাজে (যেমন migration) ব্যবহার করুন; অ্যাপের নিয়মিত query-তে কখনো Scan নয়। কিন্তু অ্যাপের তো partition key ছাড়া অন্য field দিয়ে খুঁজতে হয় — যেমন email দিয়ে ব্যবহারকারী খোঁজা, অথচ partition key হলো `userId`। এখানে আসে **secondary index** (দ্বিতীয় সূচক): **GSI (Global Secondary Index)** — যেকোনো attribute-কে partition key (ও sort key) বানিয়ে একটি আলাদা index তৈরি করা; GSI-র নিজের throughput ও খরচ আলাদা, আর এটি table তৈরি হওয়ার পরেও যোগ করা যায়। **LSI (Local Secondary Index)** — একই partition key, কিন্তু ভিন্ন sort key; শুধুমাত্র table তৈরির সময় বানানো যায়, সর্বোচ্চ ৫টি, আর ১০ GB-র সীমা আছে। বাস্তবে GSI-ই প্রায় সবসময় ব্যবহার করা হয়; LSI শুধুমাত্র বিশেষ ক্ষেত্রে। একটি গুরুত্বপূর্ণ বিষয়: GSI-তে **projection** (কোন attribute index-এ কপি হবে) ঠিক করতে হয়

— `KEYS_ONLY` (শুধু key, সস্তা), `INCLUDE` (কিছু attribute), `ALL` (সব attribute, দামি কিন্তু query-তে table-এ ফিরতে হয় না)।

```
# DynamoDB table তৈরি করুন (partition key + sort key, on-demand)
aws dynamodb create-table --table-name technova-orders \
  --attribute-definitions AttributeName=userId,AttributeType=S \
    AttributeName=createdAt,AttributeType=S \
    AttributeName=email,AttributeType=S \
  --key-schema AttributeName=userId,KeyType=HASH \
    AttributeName=createdAt,KeyType=RANGE \
  --billing-mode PAY_PER_REQUEST \
  --global-secondary-indexes 'IndexName=email-
index,KeySchema=[{AttributeName=email,KeyType=HASH}],Projection={ProjectionType=ALL}' \
  --region ap-south-1

# Item লিখুন
aws dynamodb put-item --table-name technova-orders --item '{
  "userId": {"S": "u-101"}, "createdAt": {"S": "2026-11-09T10:00:00Z"},
  "email": {"S": "nahid@technova.example"}, "total": {"N": "1500"} }' --region ap-south-1

# Query করুন (partition key দিয়ে — দ্রুত)
aws dynamodb query --table-name technova-orders \
  --key-condition-expression "userId = :u" \
  --expression-attribute-values '{":u": {"S": "u-101"}}' --region ap-south-1
```

১৫.৬ Capacity, Billing Mode, Consistency ও Transaction

DynamoDB-র দুটি billing mode আছে। **On-demand (PAY_PER_REQUEST):** আপনি শুধু যত request করেন তত দেন; কোনো capacity আগে থেকে ঠিক করতে হয় না; অনিয়মিত বা নতুন workload-এর জন্য ভালো; তবে প্রতি request-এর দাম provisioned-এর চেয়ে বেশি হতে পারে। **Provisioned:** আপনি **RCU** (Read Capacity Unit — প্রতি সেকেন্ডে একটি strongly consistent ৪ KB পড়া) ও **WCU** (Write Capacity Unit — প্রতি সেকেন্ডে একটি ১ KB লেখা) কিনে নেন; স্থির workload-এ সস্তা; **auto scaling** চালু করলে চাহিদা অনুযায়ী RCU/WCU বাড়ে-কমে; আর **reserved capacity** কিনলে আরও সস্তা হয়। ভুল capacity-র পরিণতি: provisioned-এ capacity শেষ হলে request **throttle** (সীমিত) হয় — `ProvisionedThroughputExceededException` আসে; তাই CloudWatch-এ `ConsumedReadCapacityUnits` ও `ThrottledRequests` দেখুন। **Consistency: eventually consistent read** (ডিফল্ট, সস্তা — লেখার পরপরই পড়লে পুরোনো ডেটা আসতে পারে) বনাম **strongly consistent read** (সবসময় নতুন ডেটা, বেশি RCU খায়)। **Transaction:** DynamoDB **TransactWriteItems** দিয়ে একসঙ্গে একাধিক item-এর লেখা নিশ্চিত করে (সব হবে, নয়তো কিছুই হবে না) — যেমন অর্ডার তৈরি ও stock কমানো একসঙ্গে; তবে transaction-এর খরচ বেশি, তাই শুধু যেখানে সত্যিই দরকার সেখানেই ব্যবহার করুন। **TTL:** একটি attribute-এ Unix timestamp রাখলে সেই সময়ে item স্বয়ংক্রিয়ভাবে মুছে যায় — সেশন, অস্থায়ী token, লগের জন্য আদর্শ; TTL-তে মুছে যাওয়া item-এর খরচ নেই।

বিষয়	On-Demand	Provisioned (+ Auto Scaling)
কখন ভালো	অনিয়মিত, নতুন, স্পাইকযুক্ত workload	স্থির, অনুমানযোগ্য workload
খরচ	প্রতি request-এ দাম বেশি, কিন্তু শূন্য ব্যবহারে শূন্য বিল	ব্যবহার না করলেও capacity-র বিল
পরিচালনা	কিছু করতে হয় না	RCU/WCU ও auto scaling ঠিক করতে হয়
ঝুঁকি	হঠাৎ স্পাইকে বিল বাড়তে পারে	ভুল capacity-তে throttle হয়

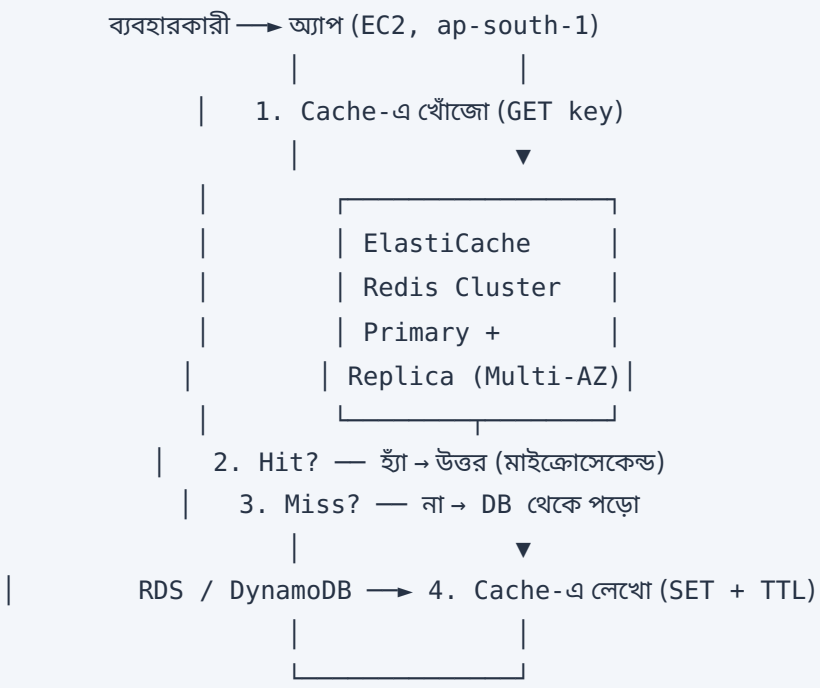
১৫.৭ DynamoDB নিরাপত্তা, ব্যাকআপ ও অন্যান্য বৈশিষ্ট্য

নিরাপত্তা: DynamoDB-তে encryption at rest ডিফল্টে চালু (AWS owned key, বা নিজের KMS key ব্যবহার করতে পারেন); in transit সবসময় HTTPS; access নিয়ন্ত্রণ IAM policy দিয়ে — table-স্তরে, এমনকি **fine-grained access control** দিয়ে item-স্তরে (যেমন একটি ব্যবহারকারী শুধু নিজের item পড়তে পারে — `dynamodb:LeadingKeys` condition দিয়ে); VPC থেকে **VPC endpoint** (gateway endpoint) দিয়ে ইন্টারনেট ছাড়াই DynamoDB-তে যাওয়া যায়। **ব্যাকআপ: on-demand backup** (যেকোনো সময় পূর্ণ ব্যাকআপ) ও **point-in-time recovery (PITR)** — ৩৫ দিনের ভেতরে যেকোনো সেকেন্ডে ফেরা যায়; PITR চালু রাখলে continuous backup-এর খরচ আছে — Pricing Calculator-এ যাচাই করুন। **DynamoDB Streams:** প্রতিটি পরিবর্তনের (insert/update/delete) একটি ক্রমিক লগ — Lambda দিয়ে ধরে অন্য কাজ করা যায় (যেমন অর্ডার তৈরি হলে ইমেইল পাঠানো); **Global Tables:** একই table একাধিক region-এ প্রতিলিপি — বিশ্বব্যাপী অ্যাপের জন্য, multi-region লেখা সম্ভব; **DAX (DynamoDB Accelerator):** in-memory cache — মাইক্রোসেকেন্ড latency চাইলে, তবে খরচ বেশি। একটি সতর্কতা: **empty string ও null**-এর আচরণ, **400 KB item সীমা** (বড় ফাইল DynamoDB-তে রাখবেন না — S3-তে রেখে DynamoDB-তে শুধু key রাখুন), আর **1 MB query সীমা** (বড় ফলাফলে pagination — `LastEvaluatedKey` ব্যবহার করুন) মনে রাখুন।

১৫.৮ ElastiCache: Redis বনাম Memcached

Cache হলো ডেটার দ্রুত কপি রাখার জায়গা — মূল ডেটাবেস ধীর, কিন্তু cache RAM-এ থাকে, তাই উত্তর আসে মাইক্রোসেকেন্ডে। **Amazon ElastiCache** হলো সম্পূর্ণ পরিচালিত in-memory cache সেবা — দুটি engine: **Redis** ও **Memcached**। **Redis** হলো আধুনিক, সমৃদ্ধ: শুধু key-value নয় — list, set, sorted set, hash, stream, pub/sub, Lua scripting, persistence (ডিস্কে সংরক্ষণ), replication ও clustering সমর্থন করে; ডেটা টাইপ সমৃদ্ধ বলে leaderboard, session, rate limiting, queue — সবই Redis-এ করা যায়। **Memcached** হলো সহজ ও দ্রুত: শুধু key-value, কোনো persistence নেই, multi-threaded (অনেক core ব্যবহার করে); শুধুমাত্র সহজ caching চাইলে ভালো। নিয়ম: নতুন কাজে প্রায় সবসময় Redis বেছে নিন; Memcached শুধুমাত্র তখনই, যখন আপনার অ্যাপ আগে থেকে Memcached-এর জন্য লেখা। ElastiCache-এর **architecture:** একটি **cluster** (গুচ্ছ)-এ এক বা একাধিক **node** (Redis-এ primary + replica, Memcached-এ অনেক node); **shard** (ভাগ) দিয়ে ডেটা ভাগ করা যায় (Redis Cluster mode); **Multi-AZ** চালু করলে primary নষ্ট হলে replica স্বয়ংক্রিয়ভাবে primary হয়; **node type** (যেমন `cache.t3.micro`, `cache.r6g.large`) দিয়ে RAM ও CPU ঠিক হয়। ElastiCache VPC-র ভেতরে থাকে — কখনো public করবেন না; security group-এ শুধু অ্যাপের SG থেকে Redis পোর্ট 6379 খুলুন; encryption in transit (TLS) ও at rest চালু করুন; Redis AUTH token ব্যবহার করুন।

চিত্র ১৫.২ — ElastiCache (Redis) আর্কিটেকচার: Cache-Aside প্যাটার্ন



Cache hit হলে DB-তে চাপ পড়ে না — এটাই caching-এর লাভ

১৫.৯ Caching কৌশল: Cache-Aside, Write-Through, TTL ও Stampede

Cache-aside (lazy loading): সবচেয়ে সাধারণ প্যাটার্ন — অ্যাপ প্রথমে cache-এ খোঁজে; **hit** (পাওয়া গেলে) সেখান থেকে উত্তর দেয়; **miss** (না পেলে) ডেটাবেস থেকে পড়ে, তারপর cache-এ লিখে রাখে। সুবিধা: শুধু যা লাগে তা-ই cache-এ থাকে; অসুবিধা: প্রথমবার miss হলে ধীর, আর ডেটা বদলালে cache পুরোনো হতে পারে। **Write-through:** লেখার সময় ডেটাবেস ও cache দুটোতেই লেখা হয় — cache সবসময় নতুন থাকে, কিন্তু প্রতিটি লেখা ধীর হয় (দুটো জায়গায় লেখা)। **Write-behind (write-back):** প্রথমে cache-এ লেখা হয়, পরে ডেটাবেসে — দ্রুত, কিন্তু cache নষ্ট হলে ডেটা হারাতে পারে। **TTL:** প্রতিটি cache entry-তে মেয়াদ (যেমন ৩০০ সেকেন্ড) দিন — মেয়াদ শেষ হলে entry মুছে যায়, পরেরবার নতুন ডেটা আসে; TTL ছাড়া cache মানে পুরোনো ডেটা চিরকাল থেকে যাওয়ার ঝুঁকি। **Cache invalidation** (অকার্যকর করা): ডেটা বদলালে সংশ্লিষ্ট cache key মুছে দিন (delete) — যেমন ব্যবহারকারীর প্রোফাইল বদলালে `user:u-101` key মুছে দিন; ভুল key মুছলে পুরোনো ডেটা দেখাবে — এটি caching-এর সবচেয়ে কঠিন সমস্যা, প্রবাদ আছে: "কম্পিউটার বিজ্ঞানে দুটি কঠিন সমস্যা — cache invalidation ও নামকরণ।" **Cache stampede (thundering herd):** একটি জনপ্রিয় key-র TTL একসঙ্গে শেষ হলে হাজার হাজার request একসঙ্গে ডেটাবেসে ছুটে যায় — ডেটাবেস বসে যেতে পারে। প্রতিকার: (১) TTL-তে এলোমেলো jitter (যেমন ৩০০ ± ৩০ সেকেন্ড) যোগ করুন; (২) **lock** ব্যবহার করুন — একজনই ডেটাবেস থেকে নতুন ডেটা আনবে, বাকিরা অপেক্ষা করবে; (৩) **stale-while-revalidate** — পুরোনো ডেটা দেখিয়ে পেছনে নতুন ডেটা আনুন। আর **cache eviction** (RAM পূর্ণ হলে কোন entry মুছবে): Redis-এ `maxmemory-policy` ঠিক করুন — সাধারণত `allkeys-lru` (সবচেয়ে কম ব্যবহৃতটি মুছবে)।

```
# Node.js-এ cache-aside উদাহরণ (Redis, ioredis)
# const cached = await redis.get(`user:${userId}`);
# if (cached) return JSON.parse(cached);          // cache hit
# const user = await db.getUser(userId);          // cache miss → DB
# await redis.set(`user:${userId}`, JSON.stringify(user), 'EX', 300); // TTL 300s
# return user;
```

```
# // প্রোফাইল বদলালে: await redis.del(`user:${userId}`); // invalidation

# Redis CLI দিয়ে যাচাই করুন (EC2-র ভেতর থেকে)
# redis-cli -h CHANGE-ME-CLUSTER-ENDPOINT -p 6379 --tls -a 'CHANGE-ME-AUTH-TOKEN' ping
# redis-cli ... SET user:u-101 '{"name":"Nahid"}' EX 300
# redis-cli ... GET user:u-101
```

> **PRO TIP:** Cache-এ কখনোই সংবেদনশীল ডেটা (পাসওয়ার্ড, টোকেন, ব্যক্তিগত তথ্য) TTL ছাড়া রাখবেন না, আর Redis-এর AUTH token Secrets Manager-এ রাখুন। আর একটি নিয়ম: cache হলো performance-এর জন্য, সত্যের উৎস (source of truth) নয় — cache মুছে গেলে অ্যাপ যেন ডেটাবেস থেকে আবার তৈরি করতে পারে, সেভাবে লিখুন।

১৫.১০ Production Scenario, নিরাপত্তা, খরচ, ভুল ও Troubleshooting

Production scenario — TechNova-র সেশন ও leaderboard: ব্যবহারকারীর সেশন DynamoDB-তে (TTL ২৪ ঘণ্টা, on-demand), পণ্যের তালিকা ElastiCache Redis-এ (cache-aside, TTL ৩০০ সেকেন্ড + jitter), গেমের leaderboard Redis sorted set-এ, আর অর্ডার Aurora-তে। এই নকশায় প্রতিটি ডেটা তার সঠিক জায়গায়: সেশন NoSQL-এ, গরম ডেটা cache-এ, লেনদেন relational-এ। **নিরাপত্তা:** DynamoDB-তে IAM least privilege (table-স্তরে), KMS encryption, VPC endpoint; Redis-এ private subnet, security group শুধু অ্যাপ থেকে, TLS ও AUTH চালু, CloudTrail লগ। **খরচ:** DynamoDB on-demand-এ হঠাৎ স্পাইকে বিল বাড়ে — CloudWatch-এ `ConsumedReadCapacityUnits` ও বিল alarm বসান; GSI-র খরচ আলাদা — অপ্রয়োজনীয় GSI মুছে ফেলুন; Redis-এ সঠিক node type বেছে নিন — `cache.t3.micro` দিয়ে শুরু করে CloudWatch-এর `Evictions` ও `MemoryUsage` দেখে বাড়ান; TTL ছাড়া cache মানে RAM পূর্ণ হয়ে eviction — খরচ ও performance দুটোই নষ্ট। **সাধারণ ভুল:** (১) hot partition key বেছে নেওয়া; (২) অ্যাপে Scan ব্যবহার করা; (৩) GSI-তে ভুল projection; (৪) provisioned capacity না বুঝে throttle; (৫) cache-এ TTL না দেওয়া; (৬) cache invalidation ভুলে পুরোনো ডেটা দেখানো; (৭) Redis public করা; (৮) DynamoDB-তে বড় ফাইল (৪০০ KB-র বেশি) রাখা। **Troubleshooting:** DynamoDB throttle হলে — CloudWatch `ThrottledRequests` দেখুন, partition key ছড়ানো আছে কি না দেখুন, on-demand-এ যান বা auto scaling বাড়ান। Redis-এ hit ratio কম হলে — `INFO stats`-এ `keyspace_hits` বনাম `keyspace_misses` দেখুন, TTL ও key নকশা যাচাই করুন। Cache stampede হলে — jitter ও lock যোগ করুন। Redis সংযোগ হচ্ছে না? — security group, VPC, TLS ও AUTH যাচাই করুন।

১৫.১১ DynamoDB Streams, Global Tables, DAX ও Redis ডেটা কাঠামো গভীরে

DynamoDB Streams + Lambda প্যাটার্ন গভীরে বুঝুন: প্রতিটি পরিবর্তন stream-এ একটি record হিসেবে আসে — `INSERT`, `MODIFY`, `REMOVE` — সঙ্গে পুরোনো ও নতুন ছবি (old/new image)। Lambda সেই record ধরে অন্য কাজ করতে পারে: অর্ডার তৈরি হলে ইমেইল পাঠানো, ব্যবহারকারীর প্রোফাইল বদলালে search index (OpenSearch) আপডেট করা, বা audit log S3-তে লেখা। এই প্যাটার্নকে বলে **change data capture (CDC)** — ডেটাবেসের পরিবর্তনই অ্যাপের অন্য অংশকে চালায়। একটি সতর্কতা: stream-এর record ২৪ ঘণ্টা থাকে; Lambda ব্যর্থ হলে retry হয় — তাই Lambda-র কাজ **idempotent** (একই কাজ দুবার করলেও ফল একই) হতে হবে — যেমন "ইমেইল পাঠাও" idempotent নয়, "ইমেইল পাঠানো হয়েছে বলে চিহ্নিত করো" idempotent। **Global Tables:** একটি table একাধিক region-এ প্রতিলিপি হয় — প্রতিটি region-এ লেখা যায় (multi-region write), আর প্রতিটি লেখা অন্য region-এ এক সেকেন্ডের কম সময়ে পৌঁছায়।

কিন্তু **conflict** (দ্বন্দ্ব) হতে পারে: দুটি region-এ একই item একসঙ্গে বদলালে শেষ লেখাটি জেতে (last-writer-wins) — তাই Global Tables শুধুমাত্র তখনই ব্যবহার করুন যখন region-গুলো ভিন্ন ভিন্ন item লেখে, বা দ্বন্দ্ব মেনে নেওয়া যায়। **DAX (DynamoDB Accelerator):** DynamoDB-র সামনে একটি in-memory cache — পড়া মাইক্রোসেকেন্ডে আসে, লেখা write-through হয়; কিন্তু DAX-এর খরচ বেশি ও পরিচালনা জটিল — প্রথমে ElastiCache বিবেচনা করুন, DAX শুধুমাত্র তখনই যখন DynamoDB-র পড়া মাইক্রোসেকেন্ডে চাই। **PartiQL:** DynamoDB-তে SQL-এর মতো query লেখার ভাষা — শেখার জন্য ভালো, কিন্তু production-এ key-based API ব্যবহার করুন — PartiQL-এর পেছনেও একই Query/Scan চলে, শুধু সিনট্যাক্স সহজ।

Redis ডেটা কাঠামো — কেন Redis শুধু cache নয়: Redis-এর শক্তি তার ডেটা টাইপে। **String:** সাধারণ key-value, counter (`INCR` দিয়ে page view গোনা)। **Hash:** একটি key-র ভেতরে অনেক field — ব্যবহারকারীর প্রোফাইল রাখার জন্য। **List:** ক্রমিক তালিকা — queue বানাতে (`LPUSH` / `BRPOP`)। **Set:** অনন্য মানের সেট — "এই পণ্যটি কারা পছন্দ করেছে" — আর `SINTER` দিয়ে দুটি সেটের মিল বের করা যায়। **Sorted Set:** score অনুযায়ী সাজানো — leaderboard-এর জন্য আদর্শ (`ZADD`, `ZRANGE`)। **Pub/Sub:** একটি channel-এ বার্তা পাঠানো, অনেক subscriber শোনা — real-time notification-এর জন্য। **Stream:** append-only লগ — event sourcing-এর জন্য। **Rate limiting:** Redis দিয়ে সহজে করা যায় — `INCR` দিয়ে প্রতি মিনিটে কত request এলো গুনুন, সীমা ছাড়ালে 429 উত্তর দিন। **Distributed lock:** `SET key value NX EX 30` দিয়ে একটি lock নেওয়া যায় — শুধুমাত্র একজনই একটি কাজ করবে (যেমন cache stampede-র সময় একজনই ডেটাবেস থেকে আনবে)। একটি সতর্কতা: Redis-এর lock সঠিকভাবে বাস্তবায়ন করা কঠিন — **Redlock** অ্যালগরিদম বা প্রস্তুত লাইব্রেরি ব্যবহার করুন, নিজে লিখবেন না। **Redis persistence:** ElastiCache Redis-এ **RDB snapshot** ও **AOF (Append Only File)** দিয়ে ডিস্কে সংরক্ষণ করা যায় — cache হিসেবে ব্যবহার করলে persistence বন্ধ রাখা যায়, কিন্তু session বা queue হিসেবে ব্যবহার করলে চালু রাখুন — নইলে node restart হলে ডেটা হারাবে। **ElastiCache Serverless:** নতুন বিকল্প — node পরিচালনা না করে শুধু ব্যবহার অনুযায়ী দাম দেওয়া — অনিয়মিত workload-এর জন্য ভালো, তবে খরচের হিসাব সাবধানে করুন।

ব্যর্থতা ও পর্যবেক্ষণ: DynamoDB-তে **throttling** সবচেয়ে সাধারণ ব্যর্থতা — CloudWatch-এ `ThrottledRequests` ও `ConsumedReadCapacityUnits` দেখুন; hot partition হলে **adaptive capacity** (স্বয়ংক্রিয়ভাবে capacity ধার দেওয়া) সাহায্য করে, কিন্তু সঠিক key ডিজাইনই আসল সমাধান। Redis node নষ্ট হলে Multi-AZ-তে replica স্বয়ংক্রিয়ভাবে primary হয় — কিন্তু failover-এর সময় কয়েক সেকেন্ডের জন্য লেখা বন্ধ থাকতে পারে — অ্যাপে retry রাখুন। Redis-এর RAM পূর্ণ হলে **eviction** শুরু হয় — `Evictions` মেট্রিক দেখুন; eviction বাড়লে node বড় করুন বা TTL কমান। **কখন DynamoDB ব্যবহার করবেন না:** জটিল JOIN, ad-hoc query, বা রিপোর্টিং — সেখানে RDS বা Athena ভালো; আর **কখন Redis ব্যবহার করবেন না:** স্থায়ী, একমাত্র ডেটা হিসেবে — Redis RAM-এ থাকে, ডিস্কের ডেটাবেসের মতো টেকসই নয়; persistence থাকলেও ডেটাবেসের বিকল্প নয়। **খরচের সূক্ষ্মতা:** DynamoDB-তে GSI-র লেখার খরচ table-এর লেখার সমান — প্রতিটি GSI মানে প্রতিটি লেখায় অতিরিক্ত খরচ; অপ্রয়োজনীয় GSI মুছে ফেলুন। ElastiCache-এ data transfer: একই AZ-তে বিনামূল্যে, অন্য AZ-তে ফি — তাই ElastiCache cluster অ্যাপের সঙ্গে একই AZ-তে রাখার চেষ্টা করুন (তবে Multi-AZ-র জন্য replica অন্য AZ-তে থাকবে — এটি availability-র খরচ, মেনে নিন)।

১৫.১২ Single-Table Design, Sparse Index ও Cache Warming গভীরে

Single-table design হলো DynamoDB-র উন্নত নকশা-পদ্ধতি: একটি table-এ অনেক ধরনের entity (ব্যবহারকারী, অর্ডার, পণ্য) একসঙ্গে রাখা হয় — প্রতিটি item-এর partition key-তে entity-র ধরন ও ID একসঙ্গে লেখা হয়, যেমন `USER#u-101` ও `ORDER#o-5001`। কেন? কারণ DynamoDB-তে JOIN নেই — সম্পর্কিত ডেটা একসঙ্গে রাখলে একটি Query-তেই সব পাওয়া যায় (যেমন একটি ব্যবহারকারী ও তার সব অর্ডার এক Query-তে)। **Adjacency list pattern:** একই partition key-র ভেতরে ভিন্ন sort key দিয়ে সম্পর্ক রাখা — যেমন partition key `USER#u-101`, sort key `ORDER#2026-11-09#o-5001` — এতে "একজন ব্যবহারকারীর সব অর্ডার" ও "একটি অর্ডারের বিস্তারিত" দুটোই একটি

Query-তে পাওয়া যায়। এই নকশা প্রথমে কঠিন মনে হয়, কিন্তু বড় অ্যাপে এটি DynamoDB-র সবচেয়ে শক্তিশালী প্যাটার্ন। একটি সতর্কতা: single-table design-এ ভুল করলে পরে বদলানো কঠিন — তাই প্রথমে কাগজে সব access pattern লিখুন, তারপর key ডিজাইন করুন। **Sparse index:** GSI-তে শুধুমাত্র সেই item গুলোই আসে যাদের index-এর key attribute আছে — যেমন শুধু `status=premium` ব্যবহারকারীদের একটি attribute থাকলে GSI-তে শুধু তারাই থাকবে — এতে GSI ছোট ও সস্তা হয়, আর "সব premium ব্যবহারকারী খুঁজুন" query দ্রুত হয়। **Composite sort key:** sort key-তে একাধিক মান `#` দিয়ে জোড়া লাগানো — যেমন `2026-11-09#PENDING` — এতে `begins_with` দিয়ে "আজকের সব pending অর্ডার" খোঁজা যায় — একটি sort key দিয়ে অনেক query করা যায়।

Cache warming ও cold start: নতুন Redis cluster বা DynamoDB table চালু করলে cache খালি থাকে — প্রথম ব্যবহারকারীরা সবাই cache miss পায়, ডেটাবেসে হঠাৎ চাপ পড়ে — এটাকে বলে **cold start**। প্রতিকার: **cache warming** — production-এ যাওয়ার আগে একটি স্ক্রিপ্ট দিয়ে জনপ্রিয় ডেটা cache-এ ভরে দিন (যেমন সবচেয়ে বেশি দেখা ১,০০০টি পণ্য)। **ElastiCache cluster mode গভীরে:** Redis Cluster mode-এ ডেটা ১৬,৩৮৪টি slot-এ ভাগ হয়, প্রতিটি shard কিছু slot ধারণ করে — shard বাড়ালে capacity ও throughput বাড়ে; কিন্তু cluster mode-এ multi-key operation (যেমন একসঙ্গে দুটি key পড়া) শুধুমাত্র তখনই চলে যখন key গুলো একই slot-এ থাকে — **hash tag** (`{user:u-101}:profile` ও `{user:u-101}:orders`) দিয়ে একই slot-এ রাখা যায়। **Memcached-এর cluster:** Memcached-এ কোনো replication নেই — একটি node নষ্ট হলে তার ডেটা হারায় (cache-এ ডেটা হারানো মেনে নেওয়া যায়, কারণ ডেটাবেস থেকে আবার তৈরি করা যায়) — তাই Memcached শুধুমাত্র pure cache হিসেবে ব্যবহার করুন। **খরচের শেষ কথা:** DynamoDB-তে **standard-IA table class** আছে — কম ব্যবহৃত ডেটার জন্য সস্তা storage, তবে পড়ার খরচ বেশি; ElastiCache-এ **Reserved Node** কিনলে ১/৩ বছরের জন্য ছাড় পাওয়া যায় — স্থির workload-এ ব্যবহার করুন। আর সবচেয়ে গুরুত্বপূর্ণ: cache ও ডেটাবেসের খরচ আলাদা করে CloudWatch ও Cost Explorer-এ দেখুন — cache-এর খরচ ডেটাবেসের খরচ কমাচ্ছে কি না, তা সংখ্যায় যাচাই করুন।

১৫.১৩ Auto Scaling, On-Demand ঝুঁকি ও ElastiCache পর্যবেক্ষণ গভীরে

DynamoDB auto scaling গভীরে: Provisioned mode-এ auto scaling একটি **target utilization** (যেমন ৭০%) ধরে রাখে — ব্যবহার ৭০%-এর বেশি হলে capacity বাড়ায়, কম হলে কমায়। কিন্তু auto scaling তাৎক্ষণিক নয় — বাড়তে কয়েক মিনিট লাগে; হঠাৎ স্পাইক (যেমন একটি ভাইরাল পোস্ট) হলে সেই কয়েক মিনিটে throttle হতে পারে। প্রতিকার: **burst capacity** (জমে থাকা অব্যবহৃত capacity) ব্যবহার করা হয় প্রথমে, আর **on-demand mode**-এ গেলে স্পাইকের ঝুঁকি থাকে না — তবে বিলের ঝুঁকি থাকে। একটি বাস্তব কৌশল: স্থির workload-এ provisioned + auto scaling, অনিয়মিত workload-এ on-demand — আর দুটোর মধ্যে বদলানো যায় যেকোনো সময় (তবে ২৪ ঘণ্টায় একবারের বেশি বদলানো যায় না)। **On-demand-এর লুকানো ঝুঁকি:** on-demand-এ কোনো সীমা নেই — একটি bug যদি অসীম loop-এ লেখে, বিলও অসীম হতে পারে — তাই on-demand table-এ **AWS Budgets alarm** বসান (যেমন দিনে ১০ ডলারের বেশি হলে সতর্কবার্তা), আর CloudWatch-এ `ConsumedWriteCapacityUnits` দেখুন। **DynamoDB-র item সীমা আবার:** ৪০০ KB item সীমা, ১ MB query ফলাফল, প্রতি second-এ একটি partition-এ ১,০০০ WCU ও ৩,০০০ RCU — এই সংখ্যাগুলো মনে রাখুন; বড় ফাইল S3-তে রাখুন, DynamoDB-তে শুধু metadata ও S3 key রাখুন। **ElastiCache পর্যবেক্ষণ:** CloudWatch-এ `CacheHitRatio` (hit ভাগ total — ৮০%-এর বেশি ভালো), `Evictions` (RAM পূর্ণ হয়ে মুছে ফেলা — বাড়লে node বড় করুন), `CPUUtilization`, `NetworkBytesIn/Out`, `CurrConnections` দেখুন। **Redis slow log:** `SLOWLOG GET 10` দিয়ে দেখুন কোন কমান্ড ধীর — সাধারণত `KEYS *` (সব key খোঁজা — production-এ কখনো চালাবেন না, `SCAN` ব্যবহার করুন) বা বড় `SMEMBERS` ধীর হয়। **Cache-এর সাপ্তাহিক রুটিন:** hit ratio দেখুন, eviction দেখুন, সবচেয়ে বড় key খুঁজুন (`--bigkeys`), TTL ছাড়া key আছে কি না দেখুন (`TTL key` দিয়ে -১ মানে কোনো মেয়াদ নেই) — TTL ছাড়া key মানে RAM-এ চিরকাল থেকে যাওয়া — নিয়মিত পরিষ্কার করুন।

১৫.১৪ সারাংশ, প্রশ্ন ও অনুশীলন

সারাংশ: DynamoDB হলো পরিচালিত NoSQL — partition key দিয়ে ডেটা ছড়ায়, sort key দিয়ে সাজায়, Query দ্রুত, Scan দামি, GSI অন্য field দিয়ে খোঁজার উপায়। On-demand সহজ, provisioned সস্তা — workload অনুযায়ী বেছে নিন। ElastiCache (Redis) হলো in-memory cache — cache-aside সবচেয়ে সাধারণ প্যাটার্ন, TTL ও invalidation ছাড়া cache বিপজ্জনক, আর stampede protection ছাড়া জনপ্রিয় key ডেটাবেসকে বসিয়ে দিতে পারে। Cache কখনোই সত্যের উৎস নয় — ডেটাবেসই সত্যের উৎস।

Review Questions: (১) Relational বনাম NoSQL — কখন কোনটি? (২) Hot partition কী এবং কীভাবে এড়াবেন? (৩) Query বনাম Scan-এর পার্থক্য কী? (৪) GSI বনাম LSI — কখন কোনটি? (৫) On-demand বনাম provisioned — কখন কোনটি? (৬) Eventually consistent বনাম strongly consistent read কী? (৭) Redis বনাম Memcached — কখন কোনটি? (৮) Cache-aside বনাম write-through-এর পার্থক্য কী? (৯) Cache stampede কী এবং প্রতিকার কী? (১০) DynamoDB-তে বড় ফাইল রাখা কেন ভুল?

Practical Exercises: (১) একটি DynamoDB table বানান (userId + createdAt), ১০টি item লিখুন, Query দিয়ে একটি ব্যবহারকারীর সব item পড়ুন, তারপর একটি GSI যোগ করে email দিয়ে খুঁজুন; Scan-এর খরচ (ConsumedReadCapacityUnits) Query-র সঙ্গে তুলনা করুন। (২) একটি Redis cluster (ElastiCache) তৈরি করে Node.js অ্যাপে cache-aside বাস্তবায়ন করুন — প্রথম request-এ miss, দ্বিতীয়টিতে hit হচ্ছে কি না মাপুন; TTL ৬০ সেকেন্ড দিয়ে মেয়াদ শেষে কী হয় দেখুন। (৩) একটি জনপ্রিয় key-তে stampede অনুকরণ করুন (একসঙ্গে ১০০টি request) — তারপর TTL jitter ও lock যোগ করে ডেটাবেসে চাপ কত কমল তা মাপুন।

অধ্যায় ১৬ • PART III — Storage, Database & Application Services

Chapter 16 — Route 53, CloudFront, ACM and Edge Architecture

১৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) **DNS** (Domain Name System) কীভাবে নামকে IP ঠিকানায় বদলায়, তা প্রথম নীতি থেকে ব্যাখ্যা করতে; (২) **Amazon Route 53**-এ **hosted zone** (হোস্টেড জোন) তৈরি করে record (A, AAAA, CNAME, MX, TXT, NS, SOA) পরিচালনা করতে; (৩) routing policy — **simple, weighted, latency-based, failover, geolocation** (ও geoproximity) — বেছে নিতে ও বাস্তবায়ন করতে; (৪) **health check** (স্বাস্থ্য পরীক্ষা) দিয়ে failover স্বয়ংক্রিয় করতে; (৫) **Amazon CloudFront** distribution তৈরি করে origin, cache behavior ও invalidation কনফিগার করতে; (৬) **AWS Certificate Manager (ACM)** দিয়ে HTTPS সার্টিফিকেট পরিচালনা করতে; (৭) **WAF** (Web Application Firewall) দিয়ে CDN-স্তরে সুরক্ষা দিতে; এবং (৮) সম্পূর্ণ edge আর্কিটেকচার — Route 53 → CloudFront → S3/ALB → EC2 — নকশা করতে।

১৬.২ পূর্বশর্ত ঝালাই (Prerequisite Refresher)

আগের অধ্যায়গুলো থেকে S3 static website hosting (অধ্যায় ১২), ALB (Application Load Balancer) ও EC2, VPC ও security group, এবং ACM-এর আগে TLS/SSL-এর মৌলিক ধারণা সঙ্গে নিন। নতুন শব্দ: **DNS record** (একটি নামের জন্য একটি নিয়ম — যেমন `www.technova.example → 203.0.113.10`), **TTL** (DNS record কতক্ষণ cache-এ থাকবে), **CDN** (Content Delivery Network — বিশ্বজুড়ে ছড়ানো সার্ভার যা কনটেন্ট ব্যবহারকারীর কাছাকাছি থেকে দেয়), **edge location** (CDN-এর কাছাকাছি সার্ভার), **origin** (মূল উৎস — যেখান থেকে CDN কনটেন্ট আনে, যেমন S3 bucket বা ALB), **TLS certificate** (HTTPS-এর পরিচয়পত্র — ব্রাউজার যাচাই করে সাইটটি আসল কি না), **CNAME** (একটি নামকে আরেকটি নামের দিকে নির্দেশ করা)। মনে রাখবেন: DNS-এর ভুল মানে ব্যবহারকারী আপনার সাইটেই পৌঁছাতে পারবে না — DNS হলো ইন্টারনেটের ফোনবুক, আর ফোনবুক ভুল হলে কেউ ফোন করতে পারে না।

১৬.৩ কেন Edge Architecture — আগে কী সমস্যা ছিল

আপনার সার্ভার মুম্বাইয়ে (ap-south-1), ব্যবহারকারী ঢাকায়, লন্ডনে, সিডনিতে। ঢাকার ব্যবহারকারীর request মুম্বাই পর্যন্ত যেতে সময় লাগে, লন্ডনের আরও বেশি, সিডনির আরও বেশি — দূরত্ব মানেই latency। আর প্রতিটি request মূল সার্ভারে গেলে সার্ভারের চাপ বাড়ে, বিল বাড়ে, আর সার্ভার নষ্ট হলে সাইট বন্ধ। **Edge architecture** এই সমস্যার উত্তরে এসেছে তিনটি স্তরে: (১) **Route 53 (DNS)** ব্যবহারকারীকে সবচেয়ে কাছের ও সুস্থ সার্ভারের দিকে পাঠায়; (২) **CloudFront (CDN)** কনটেন্ট বিশ্বজুড়ে edge location-এ cache করে — ব্যবহারকারী কাছের edge থেকে পায়, মূল সার্ভারে যেতে হয় না; (৩) **ACM + WAF** HTTPS ও সুরক্ষা দেয় edge-তেই, মূল সার্ভারে আক্রমণ পৌঁছানোর আগেই। এই তিনটি একসঙ্গে একটি ওয়েবসাইটকে দ্রুত, সস্তা ও নিরাপদ করে — আর এটাই আধুনিক web architecture-এর মান।

১৬.৪ DNS প্রথম নীতি: নাম কীভাবে ঠিকানায় বদলায়

আপনি ব্রাউজারে `www.technova.example` লিখলে যা ঘটে: (১) ব্রাউজার প্রথমে নিজের cache, তারপর OS-এর cache, তারপর **recursive resolver** (সাধারণত আপনার ISP বা 8.8.8.8) জিজ্ঞেস করে। (২) Resolver **root server**-কে জিজ্ঞেস করে `.example`-র নাম সার্ভার কোথায়, তারপর TLD সার্ভারকে জিজ্ঞেস করে `technova.example`-র নাম

সার্ভার কোথায়, তারপর **authoritative name server** (এখানে Route 53)-কে জিজ্ঞেস করে `www`-র IP কী। (৩) Route 53 উত্তর দেয় — যেমন একটি **A record** (IPv4 ঠিকানা, যেমন `203.0.113.10`) বা **AAAA record** (IPv6)। (৪) Resolver সেই উত্তর TTL অনুযায়ী cache করে, ব্রাউজার IP-তে যুক্ত হয়। এই পুরো প্রক্রিয়াকে বলে **DNS resolution**। Route 53 হলো AWS-এর authoritative DNS সেবা — অত্যন্ত উপলব্ধ (100% SLA), বিশ্বজুড়ে ছড়ানো, আর AWS-এর অন্য সেবার সঙ্গে গভীরভাবে যুক্ত (যেমন ALB-র দিকে **alias record** দিয়ে নির্দেশ করা যায় — সাধারণ CNAME-এর চেয়ে ভালো, কারণ alias record root domain-এও (`technova.example`, `www` ছাড়া) ব্যবহার করা যায়, আর এর জন্য অতিরিক্ত খরচ নেই)।

Record Type	কী করে	উদাহরণ
A	নাম → IPv4 ঠিকানা	`www → 203.0.113.10`
AAAA	নাম → IPv6 ঠিকানা	`www → 2001:db8::1`
CNAME	নাম → আরেকটি নাম	`blog → technova.example`
Alias (Route 53)	নাম → AWS resource (ALB, CloudFront, S3)	`www → d111111abcdef8.cloudfront.net`
MX	ইমেইল সার্ভার নির্দেশ করে	`@ → mail.technova.example`
TXT	যাচাইকরণ তথ্য (SPF, DKIM, ACM যাচাই)	`@ → "v=spf1 ..."`
NS / SOA	নাম সার্ভার ও জোনের তথ্য	Hosted zone তৈরিতে স্বয়ংক্রিয়

```
# Route 53 hosted zone তৈরি করুন
aws route53 create-hosted-zone --name technova.example \
  --caller-reference technova-2026-11-09 \
  --hosted-zone-config Comment="TechNova public zone"

# A record (alias নয় — সাধারণ A record) তৈরি করুন
# Route 53-এ record বানাতে change-batch JSON ব্যবহার করুন:
# aws route53 change-resource-record-sets --hosted-zone-id CHANGE-ME-ZONE-ID \
#   --change-batch file://record.json
```

১৬.৫ Routing Policy — ব্যবহারকারীকে কোথায় পাঠাবেন

একটি নামের জন্য একাধিক উত্তর থাকলে Route 53 কীভাবে বেছে নেয়, সেটাই **routing policy**। পাঁচটি প্রধান নীতি: **Simple:** একটি মান ফেরত দেয় — একটিই সার্ভার থাকলে। **Weighted:** প্রতিটি উত্তরে একটি **weight** (ভার) থাকে — যেমন নতুন সংস্করণে ১০% ট্রাফিক, পুরোনোটিতে ৯০% (canary deployment — ধীরে ধীরে নতুন সংস্করণ পরীক্ষা করা)। **Latency-based:** ব্যবহারকারীর সবচেয়ে কম latency-র region-এর দিকে পাঠায় — ঢাকার ব্যবহারকারীকে মুম্বাই, লন্ডনের ব্যবহারকারীকে ইউরোপের region-এ। **Failover:** primary সুস্থ থাকলে সেখানে, নষ্ট হলে secondary-তে — health check-এর সঙ্গে ব্যবহার করা হয়; দুর্যোগ পুনরুদ্ধারের ভিত্তি। **Geolocation:** ব্যবহারকারীর ভৌগোলিক অবস্থান অনুযায়ী — যেমন বাংলাদেশের ব্যবহারকারীকে বাংলা সাইটে, অন্য দেশের ব্যবহারকারীকে ইংরেজি সাইটে; আইনি কারণে (কিছু কনটেন্ট কিছু দেশে দেখানো যাবে না) এটি অপরিহার্য। আরেকটি আছে — **Geoproximity:** ভৌগোলিক দূরত্ব ও bias দিয়ে ট্রাফিক ভাগ করা — যেমন একটি region-কে বেশি ট্রাফিক দেওয়া। নীতি বেছে নেওয়ার নিয়ম: একটিই সার্ভার হলে Simple; ধীরে ধীরে বদলাতে Weighted; বিশ্বব্যাপী দ্রুততা চাইলে Latency-based; দুর্যোগ থেকে বাঁচতে Failover + health check; দেশ অনুযায়ী কনটেন্ট বদলাতে Geolocation।

১৬.৬ Health Check — অসুস্থ সার্ভার থেকে ট্রাফিক সরানো

Health check হলো Route 53-র নিয়মিত পরীক্ষা: প্রতি ১০ বা ৩০ সেকেন্ডে একটি endpoint-এ (যেমন `https://203.0.113.10/health`) request পাঠিয়ে দেখা হয় সেটি 2xx উত্তর দেয় কি না। তিনবার ব্যর্থ হলে endpoint-কে **unhealthy** (অসুস্থ) ধরা হয়, আর Route 53 সেই endpoint-এর দিকে আর ট্রাফিক পাঠায় না — failover policy-তে secondary-তে পাঠায়। Health check-এর গুরুত্বপূর্ণ বিষয়: (১) `/health` endpoint-টি হালকা হতে হবে — ডেটাবেসের ভারী query নয়, শুধু "আমি বেঁচে আছি" উত্তর; (২) health check বিশ্বজুড়ে একাধিক জায়গা থেকে আসে — আপনার firewall-এ Route 53 health checker-এর IP range খুলে রাখতে হয়; (৩) **CloudWatch alarm-ভিত্তিক health check** দিয়ে যেকোনো মেট্রিকের (যেমন CPU, error rate) ভিত্তিতে failover করা যায়; (৪) health check-এর খরচ আছে — প্রতিটি health check-এর জন্য মাসিক ফি, Pricing Calculator-এ যাচাই করুন। একটি সাধারণ ভুল: health check-এ মূল পেজ (`/`) ব্যবহার করা — মূল পেজ ভারী হলে health check নিজেই সার্ভারকে ধীর করে দেয়।

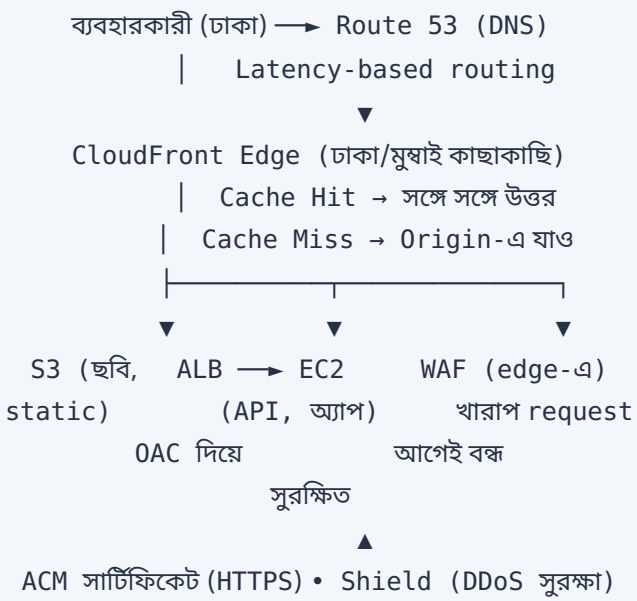
```
# Health check তৈরি করুন
aws route53 create-health-check --caller-reference technova-health-1 \
  --health-check-config '{
    "Type": "HTTPS", "ResourcePath": "/health",
    "FullyQualifiedDomainName": "www.technova.example",
    "Port": 443, "RequestInterval": 30, "FailureThreshold": 3 }'

# Failover record: primary + secondary, health check যুক্ত করে
# Console-এ: Create record → Routing policy: Failover → Primary/Secondary
# → Health check ID যুক্ত করুন → প্রতিটি record-এ আলাদা মান দিন।
```

১৬.৭ CloudFront — CDN গভীরে

Amazon CloudFront হলো AWS-এর CDN: বিশ্বজুড়ে ৬০০+ edge location-এ কনটেন্ট cache করে। কাজের প্রবাহ: (১) ব্যবহারকারী `www.technova.example`-এ request পাঠায় — Route 53 তাকে সবচেয়ে কাছের CloudFront edge-এর দিকে পাঠায়। (২) Edge দেখে কনটেন্ট cache-এ আছে কি না: **hit** হলে সঙ্গে সঙ্গে দেয় (মিলিসেকেন্ড); **miss** হলে **origin** (মূল উৎস — S3 bucket বা ALB) থেকে আনে, cache-এ রাখে, তারপর দেয়। (৩) পরের ব্যবহারকারী একই কনটেন্ট edge থেকে পায় — origin-এ চাপ পড়ে না। CloudFront-এর মূল উপাদান: **distribution** (বিতরণ — CloudFront-এর কনফিগারেশন), **origin** (S3, ALB, EC2, বা অন্য ওয়েবসাইট), **cache behavior** (কোন পথ — যেমন `/images/*` — কতক্ষণ cache হবে), **TTL** (Min/Default/Max — cache-এ কতক্ষণ থাকবে), **cache key** (কোন query string, header, cookie cache-কে আলাদা করে), **origin access control (OAC)** (S3 origin-কে শুধু CloudFront-ই পড়তে পারবে — সরাসরি S3 URL কাজ করবে না; পুরোনো OAI-এর বদলে OAC ব্যবহার করুন)। **Invalidation** (অকার্যকর করা): কনটেন্ট বদলালে cache-এর পুরোনো কপি মুছে ফেলতে হয় — `/*` দিয়ে সব মুছলে খরচ বেশি; নির্দিষ্ট পথ (`/images/logo.png`) দিয়ে মুছুন; আর সবচেয়ে ভালো হলো **versioned filename** (`app-v2.js`) ব্যবহার করা — নতুন ফাইলের নাম মানেই নতুন cache key, invalidation লাগে না।

চিত্র ১৬.১ — সম্পূর্ণ Edge আর্কিটেকচার: TechNova




```
# CloudFront invalidation তৈরি করুন (নির্দিষ্ট পথ — সস্তা)
aws cloudfront create-invalidation --distribution-id CHANGE-ME-DIST-ID \
  --paths "/images/logo.png" "/css/main.css"

# Distribution-এর অবস্থা দেখুন
aws cloudfront get-distribution --id CHANGE-ME-DIST-ID \
  --query 'Distribution.[Status,DomainName,Origins.Items[0].DomainName]'
```

CloudFront Console-এ distribution তৈরি করার ধাপ: Create distribution → Origin domain: S3 bucket বা ALB-র DNS নাম → Origin access: OAC (S3 হলে, bucket policy স্বয়ংক্রিয়ভাবে আপডেট করতে দিন) → Default cache behavior: Viewer protocol policy: **Redirect HTTP to HTTPS** → Allowed HTTP methods: GET, HEAD (static) বা GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE (API) → Cache policy: CachingOptimized (static) বা CachingDisabled (API) → Price class: Use all edge locations (সবচেয়ে দ্রুত) বা Use only North America and Europe (সস্তা — তবে বাংলাদেশের ব্যবহারকারীর জন্য ধীর হতে পারে, তাই TechNova-র জন্য সব edge ব্যবহার করুন) → Alternate domain name (CNAME): `www.technova.example` → Custom SSL certificate: ACM থেকে বেছে নিন → Create distribution। Distribution তৈরি হতে ৫–১৫ মিনিট লাগে — `Status: Deployed` হওয়া পর্যন্ত অপেক্ষা করুন।

> **PRO TIP:** CloudFront-এর **cache policy** ও **origin request policy** আলাদা জিনিস — cache policy ঠিক করে কী cache key-তে যাবে, origin request policy ঠিক করে origin-এ কী পাঠানো হবে। API-র জন্য CachingDisabled ব্যবহার করুন, নইলে ব্যক্তিগত ডেটা cache হয়ে অন্য ব্যবহারকারীকে দেখাতে পারে — একটি গুরুতর নিরাপত্তা ভুল।

১৬.৮ ACM — HTTPS সার্টিফিকেট পরিচালনা

HTTPS ছাড়া আধুনিক ওয়েবসাইট চলে না — ব্রাউজার HTTP সাইটকে "Not Secure" বলে, আর ব্যবহারকারীর ডেটা (পাসওয়ার্ড, টোকেন) খোলা অবস্থায় যায়। HTTPS-এর জন্য দরকার একটি **TLS certificate** — যা প্রমাণ করে সাইটটি

আসলেই `technova.example`। **AWS Certificate Manager (ACM)** এই সার্টিফিকেট বিনামূল্যে দেয় ও স্বয়ংক্রিয়ভাবে নবায়ন (renew) করে — আপনাকে আর সার্টিফিকেট কিনতে, ইনস্টল করতে বা মেয়াদ শেষের ভয়ে থাকতে হয় না। ACM-এর দুটি গুরুত্বপূর্ণ নিয়ম: (১) CloudFront-এর জন্য সার্টিফিকেট **us-east-1** region-এ থাকতে হয় (CloudFront global সেবা, সার্টিফিকেট us-east-1 থেকে নেয়); ALB-র জন্য সার্টিফিকেট সেই region-এ থাকতে হয় যেখানে ALB আছে (যেমন ap-south-1)। (২) সার্টিফিকেট যাচাই (validation) করতে হয় — **DNS validation** (Route 53-এ একটি CNAME record যোগ করা — সবচেয়ে সহজ, Route 53 ব্যবহার করলে ACM নিজে record যোগ করে দেয়) বা **email validation** (ডোমেইনের মালিকের ইমেইলে লিংক পাঠানো)। DNS validation-ই ব্যবহার করুন — এটি স্বয়ংক্রিয়ভাবে নবায়ন হয়। ACM সার্টিফিকেট শুধু AWS-এর সেবায় (CloudFront, ALB, API Gateway) ব্যবহার করা যায় — EC2-তে সরাসরি ইনস্টল করা যায় না; EC2-তে নিজের সার্টিফিকেট চাইলে Let's Encrypt বা অন্য CA থেকে নিতে হয়।

```
# ACM সার্টিফিকেট অনুরোধ করুন (CloudFront-এর জন্য us-east-1-এ)
aws acm request-certificate --domain-name technova.example \
  --subject-alternative-names www.technova.example \
  --validation-method DNS --region us-east-1

# সার্টিফিকেটের অবস্থা দেখুন (ISSUED হলে ব্যবহার করা যায়)
aws acm describe-certificate --certificate-arn arn:aws:acm:us-east-1:111122223333:certificate/
CHANGE-ME \
  --region us-east-1 --query 'Certificate.[Status,DomainName]'
```

১৬.৯ WAF, Shield ও CDN নিরাপত্তা

AWS WAF (Web Application Firewall) হলো HTTP request পরীক্ষা করার সেবা — খারাপ request edge-তেই বন্ধ করে দেয়, মূল সার্ভারে পৌঁছানোর আগেই। WAF-এ **rule** (নিয়ম) লেখা যায়: যেমন SQL injection (ডেটাবেসে খারাপ query ঢোকানো) চেষ্টা বন্ধ করা, cross-site scripting (XSS) বন্ধ করা, একটি IP থেকে অতিরিক্ত request (rate limiting — যেমন ৫ মিনিটে ২০০০-এর বেশি request হলে বন্ধ) করা, বা নির্দিষ্ট দেশ থেকে আসা request বন্ধ করা (geo blocking)। AWS পরিচালিত **managed rule** (যেমন AWSManagedRulesCommonRuleSet, AWSManagedRulesSQLiRuleSet) ব্যবহার করুন — নিজে নিয়ম লেখার চেয়ে নিরাপদ ও আপডেটেড থাকে। WAF CloudFront distribution বা ALB-র সঙ্গে যুক্ত করা যায়; CloudFront-এর সঙ্গে যুক্ত করলে সুরক্ষা edge-তেই হয় — আক্রমণকারী মূল সার্ভারের কাছাকাছিও আসতে পারে না। **AWS Shield Standard** সব AWS গ্রাহকের জন্য বিনামূল্যে DDoS (Distributed Denial of Service — হাজার হাজার ভুয়া request দিয়ে সাইট বসিয়ে দেওয়া) সুরক্ষা দেয়; **Shield Advanced** (দামি) আরও গভীর সুরক্ষা ও ২৪/৭ সহায়তা দেয় — বড়, আক্রমণের ঝুঁকিযুক্ত সাইটের জন্য। আরেকটি গুরুত্বপূর্ণ বৈশিষ্ট্য: **CloudFront signed URL / signed cookie** — শুধুমাত্র অনুমোদিত ব্যবহারকারীই নির্দিষ্ট কনটেন্ট দেখতে পারবে (যেমন পেইড ভিডিও) — S3 presigned URL-এর CDN সংস্করণ।

১৬.১০ Production Scenario, নিরাপত্তা, খরচ, ভুল ও Troubleshooting

Production scenario — TechNova-র সম্পূর্ণ সেটআপ: Route 53-এ `technova.example` hosted zone; `www` alias record → CloudFront distribution; CloudFront-এর origin: S3 (static ছবি, OAC) ও ALB (API, ap-south-1); ACM সার্টিফিকেট (us-east-1) দিয়ে HTTPS; WAF managed rule + rate limiting; health check দিয়ে ALB-র failover — primary ap-south-1, secondary ap-southeast-1; CloudFront access log S3-তে, CloudWatch alarm — 5xx error ১%-এর বেশি হলে, origin latency বেশি হলে। **নিরাপত্তা চেকলিস্ট:** HTTP → HTTPS redirect চালু; S3 origin OAC দিয়ে সুরক্ষিত (সরাসরি S3 URL বন্ধ); WAF managed rule চালু; Shield Standard চালু; Route 53-এ DNSSEC বিবেচনা করুন (DNS উত্তরের সত্যতা যাচাই); ACM সার্টিফিকেটের মেয়াদ ও validation যাচাই করুন। **খরচ:**

Route 53-এ hosted zone-এর মাসিক ফি + প্রতি মিলিয়ন query-র ফি; CloudFront-এ data transfer (edge থেকে ব্যবহারকারী) ও request-এর ফি — price class দিয়ে নিয়ন্ত্রণ করুন; invalidation-এর প্রথম ১,০০০টি পথ প্রতি মাসে বিনামূল্যে, তারপর ফি — তাই versioned filename ব্যবহার করুন; ACM সার্টিফিকেট বিনামূল্যে; WAF-এ প্রতিটি rule ও request-এর ফি — Pricing Calculator-এ যাচাই করুন। **সাধারণ ভুল:** (১) CloudFront-এর সার্টিফিকেট ভুল region-এ (us-east-1 নয়) বানানো; (২) API response cache করে ব্যক্তিগত ডেটা ফাঁস করা; (৩) invalidation-এ `/*` বারবার চালিয়ে বিল বাড়ানো; (৪) health check-এ ভারী পেজ ব্যবহার করা; (৫) DNS TTL খুব বেশি রাখা — failover-এর সময় পুরোনো IP cache-এ থেকে যায়; (৬) S3 bucket সরাসরি public রেখে CloudFront-এর OAC না ব্যবহার করা; (৭) WAF ছাড়া public API চালানো। **Troubleshooting:** সাইট খুলছে না? — `nslookup www.technova.example` ও `dig` দিয়ে DNS যাচাই করুন, Route 53 record ঠিক আছে কি না দেখুন। পুরোনো কনটেন্ট দেখাচ্ছে? — CloudFront cache: invalidation চালান বা versioned filename ব্যবহার করুন; `curl -I` দিয়ে `X-Cache: Hit from cloudfront` বনাম `Miss` দেখুন। HTTPS error? — ACM সার্টিফিকেট `ISSUED` কি না, সঠিক region-এ কি না, CNAME ঠিক আছে কি না দেখুন। 403 error? — S3 bucket policy-তে OAC-র ARN ঠিক আছে কি না, WAF rule কোনো বৈধ request বন্ধ করছে কি না দেখুন (WAF log CloudWatch-এ দেখুন)।

> **WARNING:** DNS পরিবর্তনের প্রভাব সঙ্গে সঙ্গে দেখা যায় না — TTL অনুযায়ী পুরোনো উত্তর cache-এ থাকতে পারে (সাধারণত ৩০০ সেকেন্ড, তবে কিছু resolver বেশি সময় ধরে রাখে)। গুরুত্বপূর্ণ পরিবর্তনের (যেমন failover) আগে TTL কমিয়ে (৬০ সেকেন্ড) রাখুন, পরিবর্তনের পর আবার বাড়ান। আর কখনোই DNS পরিবর্তন শুক্রবার রাতে করবেন না — ভুল হলে সারা সপ্তাহান্তে সাইট বন্ধ থাকতে পারে।

১৬.১১ Lambda@Edge, Origin Shield, CloudFront Functions ও ব্যর্থতার বিশ্লেষণ

Lambda@Edge হলো CloudFront-এর সঙ্গে যুক্ত Lambda — যা edge location-এ চলে, ব্যবহারকারীর কাছাকাছি। চারটি trigger point আছে: **viewer request** (ব্যবহারকারীর request আসার সময় — যেমন URL বদলানো, header যোগ করা, বা দেশ অনুযায়ী redirect), **viewer response** (উত্তর যাওয়ার সময় — যেমন security header যোগ করা), **origin request** (origin-এ যাওয়ার আগে — যেমন URL বদলে S3-র key ঠিক করা), **origin response** (origin থেকে উত্তর আসার সময় — যেমন error পেজ বদলানো)। ব্যবহার: A/B testing (ব্যবহারকারীকে দুটি সংস্করণের একটিতে পাঠানো), geo-based redirect (বাংলাদেশের ব্যবহারকারীকে `/bn/`-তে পাঠানো), authentication যাচাই (JWT token পরীক্ষা করে origin-এ যেতে দেওয়া)। **CloudFront Functions** হলো Lambda@Edge-এর হালকা সংস্করণ — শুধু JavaScript, শুধু viewer request/response-এ চলে, কিন্তু অনেক সস্তা ও দ্রুত (sub-millisecond) — header বদলানো বা redirect-এর মতো সহজ কাজে এটাই ব্যবহার করুন; ভারী কাজে (যেমন ডেটাবেস যাচাই) Lambda@Edge। একটি সতর্কতা: Lambda@Edge-এর code us-east-1-এ deploy করতে হয়, আর প্রতিটি edge-এ replicate হতে সময় লাগে — পরিবর্তনের পর পরীক্ষা করতে কয়েক মিনিট অপেক্ষা করুন।

Origin Shield ও performance tuning গভীরে: **Origin Shield** হলো CloudFront-এর একটি অতিরিক্ত cache স্তর — সব edge-এর miss প্রথমে একটি কেন্দ্রীয় Shield region-এ যায়, সেখানে hit হলে origin-এ যেতে হয় না। ফলে origin-এর চাপ অনেক কমে — বিশেষত যখন একই কনটেন্ট বিশ্বের অনেক edge থেকে চাওয়া হয়। Origin Shield-এর অতিরিক্ত ফি আছে, কিন্তু origin-এর data transfer ও load কমে — Pricing Calculator-এ হিসাব করুন। **Cache key tuning:** query string, header ও cookie — যা cache key-তে রাখবেন, তা-ই আলাদা cache entry বানায়। ভুল করে সব query string cache key-তে রাখলে প্রতিটি tracking parameter (`?utm_source=...`) আলাদা cache entry বানায় — hit ratio কমে যায়। নিয়ম: শুধুমাত্র সেই query string/header রাখুন যা কনটেন্ট বদলায়। **Compression:** CloudFront

স্বয়ংক্রিয়ভাবে gzip/br দিয়ে compress করতে পারে — text, CSS, JS-এর আকার ৭০% পর্যন্ত কমে — সবসময় চালু রাখুন। **HTTP/2 ও HTTP/3:** CloudFront ডিফল্টে HTTP/2 সমর্থন করে, HTTP/3 চালু করা যায় — একাধিক ফাইল একসঙ্গে দ্রুত যায়। **Real-time log:** CloudFront-এর access log সাধারণত কয়েক মিনিট দেরিতে আসে; তাৎক্ষণিক পর্যবেক্ষণ চাইলে **real-time logs** (Kinesis Data Firehose-এ) চালু করুন।

ব্যর্থতা বিশ্লেষণ — Edge ব্যর্থ হলে কী হয়: CloudFront edge নষ্ট হলে Route 53 ব্যবহারকারীকে অন্য edge-এ পাঠায় — ব্যবহারকারী কিছু টের পায় না। Origin (S3 বা ALB) নষ্ট হলে CloudFront **custom error page** দেখাতে পারে (যেমন 503-এর বদলে সুন্দর "শীঘ্রই ফিরছি" পেজ) — Console-এ Error Pages-এ কনফিগার করুন। **Origin failover:** CloudFront-এ একটি origin group বানানো যায় — primary origin ব্যর্থ হলে secondary origin-এ যায় — Route 53 failover-এর CDN সংস্করণ। **DNS ব্যর্থতা:** Route 53 নিজে প্রায় কখনো বন্ধ হয় না (100% SLA), কিন্তু আপনার hosted zone-এর ভুল (যেমন ভুল NS record, বা ডোমেইন registrar-এ ভুল name server) মানে সাইট অদৃশ্য — ডোমেইন কেনার সময় registrar-এ Route 53-র চারটি NS record ঠিকভাবে বসান, আর `dig NS technova.example` দিয়ে যাচাই করুন। **সার্টিফিকেট মেয়াদ:** ACM স্বয়ংক্রিয়ভাবে নবায়ন করে, কিন্তু DNS validation-এর CNAME record মুছে ফেললে নবায়ন ব্যর্থ হয় — CNAME record কখনো মুছবেন না। **কখন CloudFront ব্যবহার করবেন না:** শুধুমাত্র একটি region-এর ব্যবহারকারী ও অল্প ট্রাফিক থাকলে সরাসরি ALB যথেষ্ট হতে পারে; আর অত্যন্ত গতিশীল (প্রতিটি request-এ আলাদা) কনটেন্টে cache-এর লাভ কম — তবে WAF ও DDoS সুরক্ষার জন্য CloudFront তখনো মূল্যবান। **পর্যবেক্ষণ চেকলিস্ট:** CloudWatch-এ `Requests`, `BytesDownloaded`, `4xxErrorRate`, `5xxErrorRate`, `OriginLatency`, `CacheHitRate` দেখুন; CacheHitRate ৮০%-এর কম হলে cache policy যাচাই করুন; 5xx বাড়লে origin-এর স্বাস্থ্য দেখুন।

১৬.১২ Private Hosted Zone, Signed URL/Cookie ও WAF গভীরে

Route 53 Resolver ও private hosted zone: শুধু public DNS নয় — VPC-র ভেতরে **private hosted zone** বানানো যায়, যার record শুধু VPC-র ভেতর থেকে দেখা যায়। ব্যবহার: microservice-গুলো একে অপরকে নাম দিয়ে ডাকে — যেমন `orders.internal.technova.example` — ইন্টারনেটে এই নামের অস্তিত্ব নেই, শুধু VPC-র ভেতরে কাজ করে। **Route 53 Resolver endpoint** দিয়ে on-premises DNS ও AWS DNS একসঙ্গে যুক্ত করা যায় — hybrid আর্কিটেকচারে (নিজের ডেটা সেন্টার + AWS) এটি অপরিহার্য: inbound endpoint দিয়ে on-premises থেকে AWS-এর নাম জিজ্ঞেস করা যায়, outbound endpoint দিয়ে AWS থেকে on-premises-এর নাম। **DNS query logging:** Route 53-এ query log চালু করলে প্রতিটি DNS জিজ্ঞাসা CloudWatch Logs-এ যায় — কে কোন নাম জিজ্ঞেস করল, কখন — নিরাপত্তা বিশ্লেষণের জন্য (যেমন ম্যালওয়্যার কোন domain-এ যুক্ত হওয়ার চেষ্টা করছে) এটি মূল্যবান। একটি সতর্কতা: DNS log-এ অনেক ডেটা হয় — খরচের হিসাব করে শুধু প্রয়োজনীয় hosted zone-এ চালু করুন।

CloudFront signed URL ও signed cookie গভীরে: শুধুমাত্র অনুমোদিত ব্যবহারকারীকে কনটেন্ট দেখাতে দুটি উপায় আছে। **Signed URL:** একটি নির্দিষ্ট ফাইলের জন্য একটি স্বাক্ষরিত লিংক — যেমন পেইড ভিডিওর একটি ফাইল — মেয়াদ শেষ হলে লিংক অকেজো। **Signed cookie:** একাধিক ফাইলের জন্য — যেমন একটি পুরো কোর্সের সব ভিডিও — ব্যবহারকারী একবার cookie পেলে সব ফাইল দেখতে পারে, প্রতিটি ফাইলের জন্য আলাদা URL লাগে না। দুটোতেই একটি **key pair** (CloudFront key group) দিয়ে স্বাক্ষর করা হয় — private key আপনার কাছে থাকে, public key CloudFront-এ থাকে। একটি গুরুত্বপূর্ণ নিয়ম: signed URL/cookie ব্যবহার করলে S3 origin OAC দিয়ে সুরক্ষিত থাকতে হবে — নইলে কেউ সরাসরি S3 URL দিয়ে কনটেন্ট পেয়ে যাবে। **WAF গভীরে:** WAF-এ **rate-based rule** দিয়ে প্রতি IP-তে সীমা বসান — যেমন ৫ মিনিটে ২,০০০ request-এর বেশি হলে IP-টি ৫ মিনিটের জন্য বন্ধ; **geo-match rule** দিয়ে নির্দিষ্ট দেশ থেকে আসা request বন্ধ বা অনুমতি দিন; **managed rule group**-এ AWS-এর নিয়মিত আপডেট করা নিয়ম থাকে — যেমন `AWSManagedRulesAmazonIpReputationList` (খারাপ খ্যাতির IP বন্ধ করে) ও `AWSManagedRulesKnownBadInputsRuleSet`। WAF-এর **count mode** দিয়ে প্রথমে নিয়ম পরীক্ষা করুন — নিয়মটি

কত request বন্ধ করত তা দেখুন, তারপর block mode-এ যান — নইলে বৈধ ব্যবহারকারী বন্ধ হয়ে যেতে পারে। **Shield Advanced:** DDoS আক্রমণের সময় Shield Advanced-এর **DDoS Response Team (DRT)** ২৪/৭ সাহায্য করে, আর আক্রমণের সময় বাড়তি CloudFront/ELB খরচের সুরক্ষা (cost protection) দেয় — বড় ইভেন্টের (যেমন নির্বাচন, খেলার ফাইনাল) আগে চালু করার কথা ভাবুন। **Edge-এর শেষ কথা:** একটি সম্পূর্ণ edge আর্কিটেকচারে প্রতিটি স্তর একটি কাজ করে — Route 53 সঠিক জায়গায় পাঠায়, CloudFront কাছ থেকে দেয়, WAF খারাপ request থামায়, ACM HTTPS দেয়, আর origin শুধু আসল কাজ করে। কোনো একটি স্তর বাদ দিলে পুরো কাঠামো দুর্বল হয় — তাই production-এ চারটি একসঙ্গে ব্যবহার করুন।

১৬.১৩ Traffic Flow, Cache Policy গভীরে ও Multi-Origin আর্কিটেকচার

Route 53 Traffic Flow: জটিল routing নীতি (যেমন weighted + health check + geolocation একসঙ্গে) দৃশ্যমানভাবে বানানোর জন্য Traffic Flow ব্যবহার করা যায় — একটি visual editor-এ policy বানিয়ে version করা যায়, আর প্রয়োজনে আগের version-এ ফেরা যায়। বড়, বিশ্বব্যাপী অ্যাপে Traffic Flow-ই সঠিক উপায় — হাতে record বানানোর চেয়ে ভুল কম হয়। **CloudFront cache policy গভীরে:** AWS-এর প্রস্তুত policy আছে — `CachingOptimized` (query string ও cookie ছাড়া — static কনটেন্টের জন্য), `CachingDisabled` (API-র জন্য), `UseOriginCacheControlHeaders` (origin-এর Cache-Control header মেনে চলা)। নিজের policy বানাতে চাইলে ভাবুন: কোন query string কনটেন্ট বদলায়? (যেমন `?size=large` বদলায়, `?utm_source=facebook` বদলায় না) — শুধু প্রথমটি cache key-তে রাখুন। **Origin request policy:** origin-এ কী পাঠাবেন তা ঠিক করে — যেমন ব্যবহারকারীর আসল IP (`X-Forwarded-For`) বা `Accept-Language` header — cache key-তে না রেখে origin-এ পাঠানো যায় — এতে cache hit ratio বাড়ে, আবার origin সঠিক তথ্য পায়। **Multi-origin আর্কিটেকচার:** একটি CloudFront distribution-এ একাধিক origin থাকতে পারে — যেমন `/images/*` S3 থেকে, `/api/*` ALB থেকে, `/videos/*` অন্য S3 bucket থেকে — প্রতিটি পথের জন্য আলাদা cache behavior। আর **origin group** দিয়ে failover: primary S3 bucket ব্যর্থ হলে secondary bucket থেকে কনটেন্ট আসে। **CloudFront-এর খরচের হিসাব:** data transfer (edge থেকে ব্যবহারকারী) সবচেয়ে বড় অংশ — price class দিয়ে নিয়ন্ত্রণ করুন; request-এর খরচ (GET/POST) ছোট কিন্তু অনেক request হলে বাড়ে; invalidation-এর প্রথম ১,০০০ পথ বিনামূল্যে; আর **field-level encryption** (যেমন ক্রেডিট কার্ড নম্বর edge-তেই এনক্রিপ্ট করা) ব্যবহার করলে অতিরিক্ত ফি আছে। **DNS-এর শেষ কথা:** ডোমেইন কেনার সময় **auto-renew** চালু রাখুন — মেয়াদ শেষে ডোমেইন হারানো একটি ভয়াবহ ও সাধারণ দুর্ঘটনা; আর registrar-এর contact ইমেইল সবসময় আপডেট রাখুন — সার্টিফিকেট ও ডোমেইনের সতর্কবার্তা সেখানেই যায়।

১৬.১৪ সারাংশ, প্রশ্ন ও অনুশীলন

সারাংশ: Route 53 হলো ইন্টারনেটের ফোনবুক — নামকে IP-তে বদলায়, routing policy দিয়ে ট্রাফিক ভাগ করে, health check দিয়ে অসুস্থ সার্ভার থেকে ট্রাফিক সরায়। CloudFront হলো CDN — edge-এ cache করে কনটেন্ট দ্রুত দেয় ও origin-এর চাপ কমায়; OAC দিয়ে S3 সুরক্ষিত রাখুন, versioned filename দিয়ে invalidation এড়ান। ACM বিনামূল্যে HTTPS সার্টিফিকেট দেয় ও নবায়ন করে — CloudFront-এর জন্য us-east-1 মনে রাখুন। WAF ও Shield edge-তেই আক্রমণ থামায়। এই চারটি একসঙ্গে একটি দ্রুত, নিরাপদ ও বিশ্বব্যাপী ওয়েবসাইটের ভিত্তি।

Review Questions: (১) DNS resolution ধাপে ধাপে কীভাবে হয়? (২) Alias record CNAME থেকে কেন ভালো? (৩) Weighted বনাম latency-based routing — কখন কোনটি? (৪) Health check কীভাবে failover চালায়? (৫) CloudFront-এ cache hit বনাম miss কী? (৬) OAC কী এবং OAI থেকে কেন ভালো? (৭) Invalidation-এর বদলে versioned filename কেন ভালো? (৮) CloudFront-এর সার্টিফিকেট কেন us-east-1-এ থাকতে হয়? (৯) WAF-এর managed rule কী? (১০) DNS TTL failover-এর সময় কেন গুরুত্বপূর্ণ?

Practical Exercises: (১) একটি Route 53 hosted zone তৈরি করে একটি A record ও একটি CNAME record বানান; `dig` দিয়ে যাচাই করুন। (২) একটি S3 bucket-এ static সাইট রেখে CloudFront distribution (OAC + HTTPS redirect + ACM সার্টিফিকেট) তৈরি করুন; `curl -I` দিয়ে cache hit/miss দেখুন, তারপর একটি ফাইল বদলে invalidation চালিয়ে পরিবর্তন যাচাই করুন। (৩) একটি health check তৈরি করে একটি failover routing policy বানান — primary endpoint বন্ধ করে (security group-এ পোর্ট বন্ধ করে) দেখুন Route 53 secondary-তে ট্রাফিক পাঠায় কি না।

PART IV • DEVOPS ENGINEERING MASTERY

Containers, Serverless & Delivery

AWS কনটেইনার ইঞ্জিনিয়ারিং — ECS ও EKS — Serverless ও Event-Driven আর্কিটেকচার, AWS CI/CD, AWS-এ Infrastructure as Code।

অধ্যায় ১৭ • PART IV

Chapter 17 — AWS Container Engineering

১৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) ECR-এ কনটেইনার ইমেজ সংরক্ষণ, স্ক্যান ও সংস্করণ নিয়ন্ত্রণ করতে; (২) ECS-এর আর্কিটেকচার — ক্লাস্টার, টাস্ক ডেফিনিশন, সার্ভিস, টাস্ক — ব্যাখ্যা করতে; (৩) EC2 লঞ্চ টাইপ ও Fargate-এর মধ্যে সিদ্ধান্ত নিতে; (৪) সম্পূর্ণ টাস্ক ডেফিনিশন JSON লিখে প্রতিটি ফিল্ড ব্যাখ্যা করতে; (৫) Application Load Balancer-এর সঙ্গে ECS সার্ভিস যুক্ত করতে; (৬) EKS ও Kubernetes-এর মূল ধারণা — পড, ডিপ্লয়মেন্ট, সার্ভিস, ইনগ্রেস, স্টোরেজ — AWS প্রেক্ষাপটে প্রয়োগ করতে; (৭) কনটেইনার নেটওয়ার্কিং (awsvpc, Service Discovery) ও স্টোরেজ (EBS, EFS) বেছে নিতে; (৮) ব্লু-গ্রিন ও ক্যানারি ডিপ্লয়মেন্ট চালাতে; (৯) কনটেইনার নিরাপত্তা ও খরচ নিয়ন্ত্রণ করতে।

১৭.২ পূর্বজ্ঞান ঝালাই

আগের অধ্যায়গুলো থেকে আপনার জানা আছে: EC2, VPC, সাবনেট, সিকিউরিটি গ্রুপ, IAM রোল, ALB ও CloudWatch। Docker সম্পর্কে মনে রাখুন: **ইমেজ** হলো অ্যাপ্লিকেশন ও তার নির্ভরতার অপরিবর্তনীয় (immutable) প্যাকেজ; **কনটেইনার** হলো সেই ইমেজের চলমান ইনস্ট্যান্স; **রেজিস্ট্রি** হলো ইমেজ সংরক্ষণের জায়গা। Kubernetes-এর ক্ষেত্রে মনে রাখুন: **পড** হলো সবচেয়ে ছোট ডিপ্লয়যোগ্য একক, **নোড** হলো কর্মী মেশিন, **কন্ট্রোল প্লেন** হলো মস্তিষ্ক। এই অধ্যায়ে আমরা দেখব AWS কীভাবে এই ধারণাগুলোকে পরিচালিত (managed) সেবায় রূপ দিয়েছে — যাতে আপনি সার্ভার না চালিয়েও কনটেইনার চালাতে পারেন, আবার প্রয়োজনে পুরো Kubernetes নিয়ন্ত্রণও পেতে পারেন।

১৭.৩ কেন কনটেইনার, কেন AWS-এ?

TechNova-র MERN অ্যাপ্লিকেশনটি প্রথমে একটি EC2 ইনস্ট্যান্সে চলত: Node.js সার্ভার, MongoDB, Nginx — সব এক মেশিনে। সমস্যা ছিল তিনটি। প্রথমত, **পরিবেশের অমিল** — ডেভেলপারের ল্যাপটপে চলত, প্রোডাকশনে ভাঙত। দ্বিতীয়ত, **স্কেলিং** — ট্রাফিক বাড়লে পুরো মেশিন বড় করতে হতো, অ্যাপের শুধু ব্যস্ত অংশটুকু নয়। তৃতীয়ত, **ডিপ্লয়মেন্টের ভয়** — নতুন সংস্করণ মানেই ম্যানুয়াল ধাপ, ভুল হলে ঘণ্টার ডাউনটাইম। কনটেইনার এই তিনটি সমস্যার উত্তর: একই ইমেজ সর্বত্র চলে, প্রতিটি সেবা আলাদা স্কেল হয়, এবং ডিপ্লয়মেন্ট হয় ঘোষণামূলক (declarative) — আপনি বলেন "তিনটি কপি চাই", প্ল্যাটফর্ম তা বজায় রাখে।

AWS-এ কনটেইনারের দুটি প্রধান পথ: **ECS** (Elastic Container Service) — AWS-এর নিজস্ব, সরল ও গভীরভাবে সমন্বিত অর্কেস্ট্রেটর; এবং **EKS** (Elastic Kubernetes Service) — পরিচালিত Kubernetes, যা স্ট্যান্ডার্ড Kubernetes API দেয়। কোনটি নেবেন, তা নির্ভর করে দলের দক্ষতা, বহনযোগ্যতা (portability) ও নিয়ন্ত্রণের প্রয়োজনে — এ অধ্যায়ের শেষে সিদ্ধান্ত টেবিলে তা দেখবেন।

> **মূল ধারণা:** অর্কেস্ট্রেটর (orchestrator) হলো সেই সফটওয়্যার যা সিদ্ধান্ত নেয় — কোন কনটেইনার কোন মেশিনে চলবে, কত কপি চলবে, একটি মারা গেলে কী হবে, ট্রাফিক কীভাবে বণ্টন হবে। আপনি ইচ্ছা ঘোষণা করেন; অর্কেস্ট্রেটর বাস্তবতাকে সেই ইচ্ছার সঙ্গে মিলিয়ে রাখে।

১৭.৪ ECR — কনটেইনার ইমেজের ভান্ডার

ECR (Elastic Container Registry) হলো AWS-এর পরিচালিত প্রাইভেট Docker রেজিস্ট্রি। আগে দলগুলো নিজেরা রেজিস্ট্রি চালাত বা পাবলিক Docker Hub-এর ওপর নির্ভর করত — যেখানে প্রাইভেট ইমেজের সীমা, নিরাপত্তা স্ক্যান ও IAM নিয়ন্ত্রণ ছিল দুর্বল। ECR সেই সমস্যার সমাধান: প্রতিটি রিপজিটরি IAM পলিসি দিয়ে সুরক্ষিত, ইমেজ স্ক্যানিং দুর্বলতা (vulnerability) ধরে, এবং লাইফসাইকেল পলিসি পুরোনো ইমেজ মুছে খরচ কমায়।

ECR কীভাবে কাজ করে: আপনি `docker build` করে ইমেজ বানান, `docker push` দিয়ে ECR রিপজিটরিতে তোলেন। ECS বা EKS টাস্ক চালুর সময় AWS সেই ইমেজ টেনে আনে — IAM রোলের মাধ্যমে, কোনো দীর্ঘস্থায়ী পাসওয়ার্ড ছাড়াই। ইমেজ ট্যাগ করুন অর্থবহভাবে: `latest` কখনো প্রোডাকশনে ব্যবহার করবেন না; Git কমিট SHA বা সংস্করণ নম্বর (`v1.4.2`) ব্যবহার করুন, যাতে রোলব্যাক মানেই আগের ট্যাগে ফেরা।

```
aws ecr create-repository \
  --repository-name technova/api \
  --image-scanning-configuration scanOnPush=true \
  --region ap-south-1
aws ecr get-login-password --region ap-south-1 | \
  docker login --username AWS --password-stdin \
  111122223333.dkr.ecr.ap-south-1.amazonaws.com
docker build -t technova/api:v1.4.2 .
docker tag technova/api:v1.4.2 \
  111122223333.dkr.ecr.ap-south-1.amazonaws.com/technova/api:v1.4.2
docker push 111122223333.dkr.ecr.ap-south-1.amazonaws.com/technova/api:v1.4.2
```

উপরের ধাপগুলো: রিপজিটরি তৈরি (push-এ স্ক্যান চালু), ECR-এ লগইন (অস্থায়ী টোকেন, ১২ ঘণ্টা মেয়াদ), ইমেজ বিল্ড, ট্যাগ ও পুশ। **ব্যর্থ হলে কী হয়:** পুশ ব্যর্থ হলে সাধারণত IAM অনুমতি বা ভুল রিজিয়ন দায়ী — `ecr:PutImage` অনুমতি যাচাই করুন। **বিলিং:** সংরক্ষিত ডেটার পরিমাণ ও ডেটা ট্রান্সফারের জন্য চার্জ হয় — AWS Pricing Calculator-এ যাচাই করুন। **নিরাপত্তা:** ইমেজ স্ক্যানিং চালু রাখুন, `latest` ট্যাগের ওপর নির্ভর করবেন না, রিপজিটরি পলিসিতে শুধু প্রয়োজনীয় অ্যাকাউন্টকে `pull` দিন।

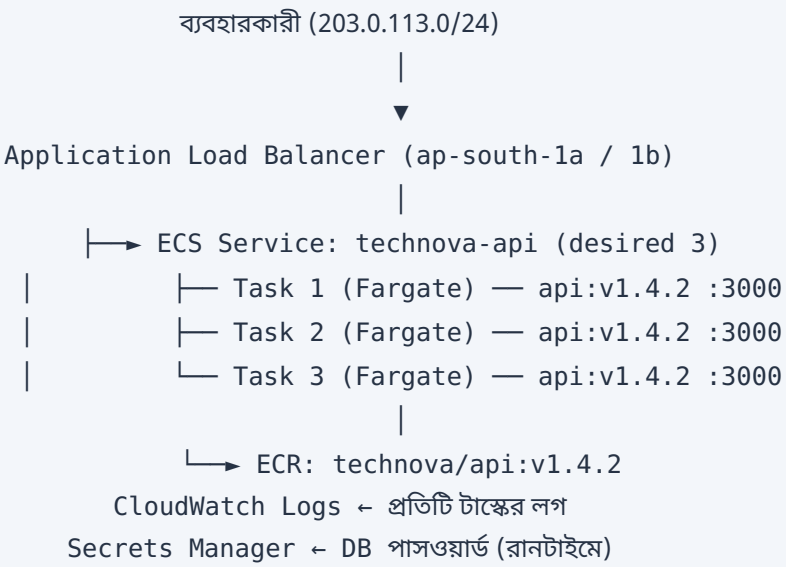
> **WARNING:** ECR-এ কখনো সিক্রেট (API কী, ডেটাবেস পাসওয়ার্ড) ইমেজের ভেতরে বেক করবেন না। ইমেজ যে কেউ pull করতে পারলে সিক্রেট ফাঁস হয়। সিক্রেট দিন Secrets Manager বা SSM Parameter Store থেকে, রানটাইমে — ২২ অধ্যায়ে বিস্তারিত।

১৭.৫ ECS আর্কিটেকচার — ভেতরের কাজ

ECS হলো AWS-এর কনটেইনার অর্কেস্ট্রেটর। এর মূল উপাদান চারটি: **ক্লাস্টার** (cluster) — ক্যাপাসিটির যৌক্তিক গোষ্ঠী; **টাস্ক ডেফিনিশন** (task definition) — "কী চালাব" তার নীলনকশা (ইমেজ, CPU, মেমোরি, পোর্ট, পরিবেশ ভেরিয়েবল); **টাস্ক** (task) — সেই নীলনকশার একটি চলমান ইনস্ট্যান্স; **সার্ভিস** (service) — নির্দিষ্ট সংখ্যক টাস্ক চালু রাখার দায়িত্বপ্রাপ্ত নিয়ন্ত্রক, যা লোড ব্যালান্সারের সঙ্গে যুক্ত হয়।

কাজের প্রবাহ: আপনি সার্ভিসে বলেন "api সার্ভিসের ৩টি টাস্ক চাই"। ECS স্কেডিউলার উপযুক্ত জায়গা খুঁজে টাস্ক চালু করে। কোনো টাস্ক ক্র্যাশ করলে সার্ভিস নতুন টাস্ক চালু করে — এটাই **desired count বনাম actual count** -এর স্ব-নিরাময় (self-healing) চক্র। ALB-এর সঙ্গে যুক্ত থাকলে নতুন টাস্ক স্বয়ংক্রিয়ভাবে টার্গেট গ্রুপে নিবন্ধিত হয়, পুরোনোটি সরিয়ে নেওয়া হয়।

ECS আর্কিটেকচার — TechNova API



অন্যান্য সেবার সঙ্গে যোগাযোগ: ECS টাস্ক IAM **টাস্ক রোল** ব্যবহার করে S3, DynamoDB বা Secrets Manager-এ পৌঁছায়; **এক্সিকিউশন রোল** ব্যবহার করে ECR থেকে ইমেজ টানা ও CloudWatch-এ লগ লেখার জন্য। এই দুই রোল আলাদা রাখা নিরাপত্তার মূল নিয়ম — এক্সিকিউশন রোল শুধু অবকাঠামোর কাজ করে, টাস্ক রোল শুধু অ্যাপের কাজ করে। **ব্যর্থ হলে:** টাস্ক বারবার থামলে ECS ইভেন্টে কারণ দেখুন (`CannotPullContainerError` মানে ইমেজ/অনুমতি সমস্যা; `OutOfMemory` মানে মেমোরি সীমা কম)। **মনিটরিং:** CloudWatch-এ CPU/Memory ব্যবহার, সার্ভিস ইভেন্ট ও লগ। **বিলিং:** Fargate-এ ভাড়া নেওয়া vCPU ও মেমোরি অনুযায়ী; EC2 লঞ্চ টাইপে অন্তর্নিহিত EC2-এর জন্য — AWS Pricing Calculator-এ যাচাই করুন।

১৭.৬ EC2 লঞ্চ টাইপ বনাম Fargate

EC2 লঞ্চ টাইপে আপনি নিজে EC2 ইনস্ট্যান্সের ক্লাস্টার চালান, তাতে ECS এজেন্ট চলে; আপনার নিয়ন্ত্রণ সর্বোচ্চ — ইনস্ট্যান্সের ধরন, স্টোরেজ, নেটওয়ার্কিং — কিন্তু প্যাচিং, স্কেলিং ও ক্যাপাসিটি পরিকল্পনার দায়িত্বও আপনার। **Fargate** হলো সার্ভারলেস কম্পিউট: আপনি শুধু CPU ও মেমোরি চান, AWS মেশিন বরাদ্দ করে, প্যাচ করে, স্কেল করে। দাম সাধারণত প্রতি ইউনিটে বেশি, কিন্তু অপ্রয়োজনীয় ক্যাপাসিটি ও পরিচালনার খরচ বাঁচে।

বিষয়	EC2 লঞ্চ টাইপ	Fargate
নিয়ন্ত্রণ	ইনস্ট্যান্স, AMI, স্টোরেজ সম্পূর্ণ	শুধু CPU/মেমোরি/নেটওয়ার্ক সেটিংস
পরিচালনা	প্যাচিং, স্কেলিং আপনার	AWS-এর
খরচের ধরন	ইনস্ট্যান্স ভাড়া (অলস সময়েও)	চলমান টাস্কের vCPU/মেমোরি
উপযুক্ত	দীর্ঘস্থায়ী, স্থির লোড; বিশেষ হার্ডওয়্যার	পরিবর্তনশীল লোড, ছোট দল, দ্রুত শুরু
শুরু করতে	ক্লাস্টার ক্যাপাসিটি প্রভাইডার লাগে	শুধু টাস্ক ডেফিনিশন

TechNova-র জন্য সুপারিশ: API ও ওয়েব সেবা Fargate-এ শুরু করুন; ব্যাচ বা GPU-নির্ভর কাজে EC2 লঞ্চ টাইপ বিবেচনা করুন। **Capacity Provider** হলো EC2 অটো স্কেলিং গ্রুপকে ECS-এর সঙ্গে যুক্ত করার সেতু — চাহিদা বাড়লে ইনস্ট্যান্স বাড়ে।

১৭.৭ সম্পূর্ণ ECS টাস্ক ডেফিনিশন — ফিল্ড ধরে ধরে

নিচের JSON-টি TechNova API-এর প্রোডাকশন-ঘেঁষা টাস্ক ডেফিনিশন। প্রতিটি ফিল্ডের ব্যাখ্যা পরে দেওয়া হলো — এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ অংশ।

```
{
  "family": "technova-api",
  "networkMode": "awsvpc",
  "executionRoleArn": "arn:aws:iam::111122223333:role/ecsTaskExecutionRole",
  "taskRoleArn": "arn:aws:iam::111122223333:role/technova-api-task-role",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512",
  "memory": "1024",
  "containerDefinitions": [
    {
      "name": "api",
      "image": "111122223333.dkr.ecr.ap-south-1.amazonaws.com/technova/api:v1.4.2",
      "essential": true,
      "portMappings": [
        { "containerPort": 3000, "protocol": "tcp" }
      ],
      "environment": [
        { "name": "NODE_ENV", "value": "production" },
        { "name": "REGION", "value": "ap-south-1" }
      ],
      "secrets": [
        {
          "name": "DATABASE_URL",
          "valueFrom": "arn:aws:secretsmanager:ap-south-1:111122223333:secret:technova/db-
CHANGE-ME"
        }
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/technova-api",
          "awslogs-region": "ap-south-1",
          "awslogs-stream-prefix": "api"
        }
      },
      "healthCheck": {
        "command": ["CMD-SHELL", "curl -f http://localhost:3000/health || exit 1"],
        "interval": 30,
        "timeout": 5,
        "retries": 3,
        "startPeriod": 60
      }
    }
  ]
}
```

ফিল্ড	অর্থ ও সিদ্ধান্ত
`family`	টাস্ক ডেফিনিশনের পরিবার/নাম; নতুন সংস্করণে রিভিশন নম্বর বাড়ে, রোলব্যাক মানে আগের রিভিশনে ফেরা।
`networkMode: awsvpc`	প্রতিটি টাস্ক নিজের ENI ও প্রাইভেট IP পায়; সিকিউরিটি গ্রুপ টাস্ক-স্তরে প্রয়োগ হয়। Fargate-এ বাধ্যতামূলক।
`executionRoleArn`	ECR pull ও CloudWatch লগের অনুমতি; শুধু অবকাঠামোর কাজ।
`taskRoleArn`	অ্যাপের অনুমতি (S3, DynamoDB); ন্যূনতম সুযোগ (least privilege) দিন।
`cpu / memory`	Fargate-এ বৈধ জোড়া মেনে চলুন (512 CPU → 1024 MB); কম দিলে OOM, বেশি দিলে খরচ।
`essential: true`	এই কনটেইনার থামলে পুরো টাস্ক থামে; সাইডকার (যেমন লগ শিপার) হলে false।
`secrets`	Secrets Manager থেকে রানটাইমে মান আসে; JSON-এ আসল মান লেখা হয় না।
`logConfiguration`	awslogs ড্রাইভার লগ CloudWatch Logs-এ পাঠায়; গ্রুপ আগে তৈরি করুন।
`healthCheck`	`/health` এন্ডপয়েন্ট ৩০ সেকেন্ড অন্তর পরীক্ষা; ব্যর্থ হলে ECS টাস্ক প্রতিস্থাপন করে।

নিবন্ধন ও সার্ভিস তৈরি:

```
aws ecs register-task-definition \
  --cli-input-json file://task-def.json --region ap-south-1
aws ecs create-cluster --cluster-name technova-prod \
  --region ap-south-1
aws ecs create-service \
  --cluster technova-prod \
  --service-name technova-api \
  --task-definition technova-api:1 \
  --desired-count 3 --launch-type FARGATE \
  --platform-version LATEST \
  --network-configuration \
  "awsvpcConfiguration={subnets=[subnet-CHANGE-ME-1,subnet-CHANGE-ME-2],securityGroups=[sg-CHANGE-ME],assignPublicIp=DISABLED}" \
  --region ap-south-1
```

লক্ষ করুন: `assignPublicIp=DISABLED` — টাস্ক প্রাইভেট সাবনেটে থাকে, বাইরে যায় NAT Gateway দিয়ে; ভেতরে আসে শুধু ALB দিয়ে। এটাই নিরাপদ নকশা।

> **PRO TIP:** টাস্ক ডেফিনিশন JSON হাতে না লিখে CloudFormation, CDK বা Terraform-এ লিখুন (২০ অধ্যায়) — তাহলে সংস্করণ নিয়ন্ত্রণ, পর্যালোচনা ও রোলব্যাক সব Git-এ থাকে।

১৭.৮ ECS-এ লোড ব্যালান্সিং ও সার্ভিস ডিসকভারি

ALB-এর সঙ্গে ECS সার্ভিস যুক্ত করলে সার্ভিস তৈরি বা আপডেটের সময় আপনি টার্গেট গ্রুপ, কনটেইনার নাম ও পোর্ট বলে দেন। নতুন টাস্ক চালু হলে ALB স্বয়ংক্রিয়ভাবে হেলথ চেক করে ট্রাফিক দিতে শুরু করে; পুরোনো টাস্ক সরানোর আগে

ড্রেইনিং (deregistration delay) সময় দেয়, যাতে চলমান অনুরোধ শেষ হতে পারে। পাথ-ভিত্তিক রাউটিং দিয়ে এক ALB দিয়ে `/api/*` → API সার্ভিস, `/*` → ওয়েব সার্ভিস — এভাবে একাধিক সার্ভিস চালানো যায়।

সার্ভিস-থেকে-সার্ভিস যোগাযোগের দুটি পথ: **Service Discovery** (Cloud Map ভিত্তিক DNS নাম, যেমন `api.technova.local`) — অভ্যন্তরীণ কলের জন্য; এবং **Service Connect** — ECS-এর নতুন, সহজ সেবা-জাল (service mesh) ধাঁচের ব্যবস্থা, যেখানে সার্ভিস নামেই কল করা যায়, লোড ব্যালান্সিং ও পর্যবেক্ষণ অন্তর্ভুক্ত। শুরুতে Service Discovery-ই যথেষ্ট; জটিল মাইক্রোসার্ভিসে Service Connect বা App Mesh বিবেচনা করুন।

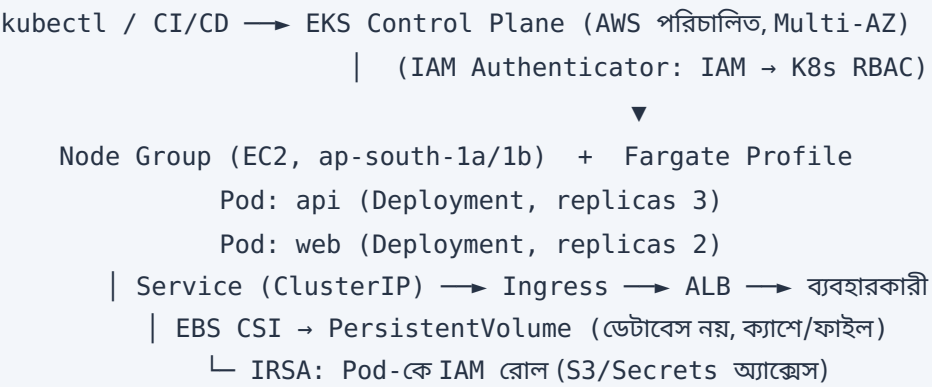
অটো স্কেলিং: ECS Service Auto Scaling — CloudWatch অ্যালার্ম (যেমন CPU ৭০%-এর বেশি) বা ALB অনুরোধ-সংখ্যার ভিত্তিতে desired count বাড়ায়-কমায়। টার্গেট ট্র্যাকিং নীতি সবচেয়ে সহজ: "CPU ৬০%-এ রাখো" বললেই হলো।

১৭.৯ EKS ও Kubernetes — AWS-এ

EKS হলো পরিচালিত Kubernetes কন্ট্রোল প্লেন। আগে নিজে Kubernetes চালাতে হলে etcd, API সার্ভার, স্কেডিউলার — সব নিজে প্যাচ ও ব্যাকআপ করতে হতো। EKS কন্ট্রোল প্লেন AWS চালায় (মাল্টি-AZ, স্বয়ংক্রিয় প্যাচ), আপনি শুধু **ওয়ার্কার নোড** (EC2 বা Fargate) ও ওয়ার্কলোড সামলান। এর ফলে আপনি স্ট্যান্ডার্ড `kubectl`, Helm চার্ট ও Kubernetes API ব্যবহার করতে পারেন — দল Kubernetes জানলে শেখার খরচ কমে, আর প্রয়োজনে অন্য ক্লাউডেও একই YAML চলে।

Kubernetes-এর মূল বস্তুগুলো, AWS প্রেক্ষাপটে: **পড** — এক বা একাধিক কনটেইনারের গোষ্ঠী, একই নেটওয়ার্ক নেমস্পেস ভাগ করে; **ডিপ্লয়মেন্ট** — পডের কাঙ্ক্ষিত সংখ্যা ও রোলিং আপডেটের নিয়ম; **সার্ভিস** — পডগুলোর সামনে স্থিতিশীল ঠিকানা ও লোড ব্যালান্সিং; **ইনগ্রেস** — HTTP রাউটিং (AWS-এ AWS Load Balancer Controller ALB তৈরি করে); **ConfigMap/Secret** — কনফিগ ও সিক্রেট; **নেমস্পেস** — পরিবেশ বা দল অনুযায়ী বিভাজন।

EKS আর্কিটেকচার — TechNova



IRSA (IAM Roles for Service Accounts) হলো EKS নিরাপত্তার হৃদয়: প্রতিটি Kubernetes সার্ভিস অ্যাকাউন্ট একটি IAM রোলের সঙ্গে যুক্ত হয়; পড সেই রোলের অস্থায়ী ক্রেডেনশিয়াল পায় — নোডের শেয়ার্ড ক্রেডেনশিয়াল ব্যবহারের পুরোনো ঝুঁকিপূর্ণ পথ বন্ধ হয়। **নেটওয়ার্কিং:** Amazon VPC CNI পডকে VPC-র আসল IP দেয় — ফলে সিকিউরিটি গ্রুপ ও নেটওয়ার্ক পলিসি সরাসরি প্রযোজ্য হয়; তবে IP ঠিকানা ফুরিয়ে যেতে পারে, তাই সাবনেট পরিকল্পনা জরুরি। **স্টোরেজ:** EBS CSI ড্রাইভার স্থায়ী ভলিউম দেয় (এক AZ-এ সীমাবদ্ধ), EFS CSI শেয়ার্ড ফাইল সিস্টেম দেয় (মাল্টি-AZ)। ডেটাবেস পডে না চালিয়ে RDS ব্যবহার করাই প্রোডাকশন নিয়ম — ২৪ অধ্যায়ে দেখবেন।

```
eksctl create cluster --name technova-prod \
  --region ap-south-1 --version 1.32 \
```

```
  --nodegroup-name standard \
  --node-type t3.medium --nodes 2 --nodes-min 2 --nodes-max 4 \
  --managed
kubectl get nodes
kubectl create deployment technova-api \
  --image=111122223333.dkr.ecr.ap-south-1.amazonaws.com/technova/api:v1.4.2
kubectl scale deployment technova-api --replicas=3
```

কখন EKS, কখন ECS: দল Kubernetes জানে, মাল্টি-ক্লাউড বহনযোগ্যতা চায়, বা জটিল মাইক্রোসার্ভিস জাল আছে — EKS। সরলতা, দ্রুত শুরু ও AWS সমন্বয় চায় — ECS। **বিলিং সতর্কতা:** EKS কন্ট্রোল প্লেন ঘণ্টায় চার্জ হয়, নোডের EC2 আলাদা — AWS Pricing Calculator-এ যাচাই করুন।

১৭.১০ কনটেইনার নেটওয়ার্কিং ও স্টোরেজ — সিদ্ধান্ত

নেটওয়ার্কিং মোড: ECS-এ `awsvpc` মানেই প্রতি টাস্কে ENI — নেটওয়ার্ক বিচ্ছিন্নতা (isolation) সেরা, কিন্তু ENI সীমা ও IP পরিকল্পনা খেয়াল রাখুন। `bridge` মোড পুরোনো EC2-ধাঁচের; নতুন কাজে এড়িয়ে চলুন। স্টোরেজ: কনটেইনার **এফিমেরাল** (ephemeral) — টাস্ক মরলে তার লোকাল ডিস্ক যায়; স্থায়ী ডেটা কখনো কনটেইনারে রাখবেন না। ফাইল শেয়ার করতে EFS মাউন্ট করুন, ডেটাবেসের জন্য RDS/DynamoDB ব্যবহার করুন, লগ CloudWatch-এ পাঠান।

১৭.১১ ডিপ্লয়মেন্ট প্যাটার্ন

রোলিং আপডেট: ECS সার্ভিস ধাপে ধাপে পুরোনো টাস্ক সরিয়ে নতুন আনে — `minimumHealthyPercent` ও `maximumPercent` দিয়ে নিয়ন্ত্রণ করুন (যেমন ১০০/২০০ মানে ডিপ্লয় চলাকালে দ্বিগুণ টাস্ক চলতে পারে)। **ব্লু-গ্রিন:** CodeDeploy নতুন পরিবেশ (গ্রিন) তৈরি করে, পরীক্ষা করে, তারপর ট্রাফিক সরায়; ব্যর্থ হলে এক ক্লিকে ফেরত। **ক্যানারি:** প্রথমে ১০% ট্রাফিক নতুন সংস্করণে, মেট্রিক ভালো থাকলে ধীরে ধীরে ১০০%। TechNova-র API-তে ব্লু-গ্রিন, অভ্যন্তরীণ টুলে রোলিং — ঝুঁকি অনুযায়ী প্যাটার্ন বাছুন।

১৭.১২ প্রোডাকশন পরিস্থিতি — TechNova শুক্রবার রাতের ট্রাফিক

TechNova-র একটি প্রচারণায় শুক্রবার রাতে ট্রাফিক পাঁচ গুণ বাড়ল। Fargate সার্ভিসে টার্গেট ট্র্যাকিং (CPU ৬০%) আগে থেকে চালু ছিল বলে desired count ৩ থেকে ১২-তে উঠল; ALB হেলথ চেক ব্যর্থ টাস্ক সরিয়ে দিল; কোনো ডাউনটাইম হয়নি। শিক্ষা: অটো স্কেলিং ডিপ্লয়ের দিনে নয়, শান্ত দিনে পরীক্ষা করতে হয় — লোড টেস্টে স্কেল-আপ সময় মাপুন (ইমেজ pull সময়সহ)। আরেকটি ঘটনা: একটি ডিপ্লয়ে ভুল পরিবেশ ভেরিয়েবলের কারণে সব নতুন টাস্ক `/health`-এ ব্যর্থ হলো; ECS পুরোনো টাস্ক রেখে দিল কারণ `minimumHealthyPercent 100` ছিল — রোলিং ডিপ্লয় নিজেই রোলব্যাক হিসেবে কাজ করল।

১৭.১৩ নিরাপত্তা, খরচ, ভুল ও সমস্যা সমাধান

নিরাপত্তা: ইমেজ স্ক্যানিং ও স্বাক্ষর (signing) চালু করুন; রুট হিসেবে কনটেইনার চালাবেন না (`user` নির্দিষ্ট করুন); টাস্ক রোলে ন্যূনতম সুযোগ; প্রাইভেট সাবনেট + ALB; সিক্রেট রানটাইমে। **খরচ:** CPU/মেমোরি সঠিক মাপুন (CloudWatch মেট্রিক দেখে ডান-আকার করুন), অপ্রয়োজনীয় `latest` ইমেজের লাইফসাইকেল পলিসি দিন, Fargate Spot ব্যাচ কাজে বিবেচনা করুন; EKS কন্ট্রোল প্লেন ও NAT Gateway বিলযোগ্য — AWS Pricing Calculator-এ যাচাই করুন।

> **সাধারণ ভুল:** (১) `latest` ট্যাগে ডিপ্লয় — কোন সংস্করণ চলছে কেউ জানে না; (২) হেলথ চেক না রাখা — ভাঙা টাস্ক ট্রাফিক পেতে থাকে; (৩) একটিমাত্র AZ-এ সার্ভিস — AZ বিভ্রাটে পুরো সেবা যায়; (৪) লগ ড্রাইভার না দেওয়া — ক্র্যাশের কারণ অজানা থাকে; (৫) টাস্ক রোলে `*` অনুমতি।

সমস্যা সমাধান: টাস্ক চালু হচ্ছে না → ECS কনসোলে সার্ভিস ইভেন্ট দেখুন, `Stopped reason` পড়ুন; ইমেজ pull ব্যর্থ → ECR অনুমতি, NAT/এন্ডপয়েন্ট, ট্যাগ যাচাই; ALB ৫০২ → টার্গেট হেলথ চেক, সিকিউরিটি গ্রুপ (ALB → টাস্ক পোর্ট ৩০০০), হেলথ পাথ; EKS পড Pending → নোড ক্যাপাসিটি, `kubectl describe pod`; OOMKilled → মেমোরি বাড়ান।

১৭.১৪ সারাংশ

ECR ইমেজ সংরক্ষণ করে, ECS সরল অর্কেস্ট্রেশন দেয়, Fargate সার্ভারহীন কনটেইনার দেয়, EKS পরিচালিত Kubernetes দেয়। টাস্ক ডেফিনিশন হলো চুক্তি — CPU, মেমোরি, ইমেজ, সিক্রেট, লগ ও হেলথ চেক এক জায়গায়। ALB, অটো স্কেলিং ও ব্লু-গ্রিন ডিপ্লয়মেন্ট মিলে প্রোডাকশন-উপযোগী সেবা তৈরি হয়; নিরাপত্তা মানে ন্যূনতম সুযোগ, স্ক্যান করা ইমেজ ও প্রাইভেট নেটওয়ার্ক।

১৭.১৫ পর্যালোচনা প্রশ্ন

১. ECR, ECS ও EKS-এর ভূমিকা আলাদা করে ব্যাখ্যা করুন। ২. টাস্ক ডেফিনিশন, টাস্ক ও সার্ভিসের সম্পর্ক কী? ৩. execution role ও task role আলাদা কেন জরুরি? ৪. Fargate কখন EC2 লঞ্চ টাইপের চেয়ে ভালো? ৫. awsvpc নেটওয়ার্ক মোডের সুবিধা ও সীমা কী? ৬. হেলথ চেক না থাকলে ডিপ্লয়ে কী বিপদ? ৭. IRSA কী সমস্যার সমাধান করে? ৮. ব্লু-গ্রিন বনাম ক্যানারি — কখন কোনটি? ৯. কনটেইনারে স্থায়ী ডেটা রাখা কেন ভুল? ১০. EKS কন্ট্রোল প্লেনের খরচ কীভাবে হিসাব করবেন?

১৭.১৬ অনুশীলন

১. ECR রিপজিটরি তৈরি করে একটি Node.js অ্যাপের ইমেজ push করুন; স্ক্যান ফলাফল দেখুন। ২. উপরের টাস্ক ডেফিনিশন দিয়ে Fargate সার্ভিস (desired ২) চালু করুন, ALB যুক্ত করুন, `/health` পরীক্ষা করুন। ৩. টার্গেট ট্র্যাকিং অটো স্কেলিং সেট করে লোড টেস্টে স্কেল-আপ পর্যবেক্ষণ করুন। ৪. eksctl দিয়ে পরীক্ষামূলক EKS ক্লাস্টার তৈরি করে একটি Deployment স্কেল করুন, তারপর মুছে ফেলুন (খরচ এড়াতে)।

অধ্যায় ১৮ • PART IV

Chapter 18 — Serverless and Event-Driven Architecture

১৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Lambda-র এক্সিকিউশন মডেল, কোল্ড স্টার্ট ও কনকারেন্সি ব্যাখ্যা করতে; (২) API Gateway দিয়ে HTTP API বানাতে; (৩) SQS, SNS ও EventBridge-এর মধ্যে সঠিক সেবা বেছে নিতে; (৪) ইভেন্ট-চালিত নকশা আঁকতে; (৫) রিট্রাই, ডেড-লেটার কিউ ও আইডেমপোটেন্সি (idempotency) বাস্তবায়ন করতে; (৬) সার্ভারলেস পর্যবেক্ষণ (CloudWatch, X-Ray) সাজাতে; (৭) সার্ভারলেস নিরাপত্তা ও খরচ বিশ্লেষণ করতে।

১৮.২ পূর্বজ্ঞান ঝালাই

মনে রাখুন: ১৭ অধ্যায়ে কনটেইনার সবসময় চলমান থাকে — টাকা ওঠে চলমান সময়ের জন্য। সার্ভারলেস উল্টো দর্শন: কোড চলে শুধু ঘটনা (event) ঘটলে, বাকি সময় কিছুই চলে না, কিছুই খরচ হয় না। IAM, CloudWatch Logs ও API ধারণা আগের অধ্যায় থেকে জানা ধরে নেওয়া হচ্ছে। এখানে নতুন শব্দ: **ইভেন্ট** (কোনো কিছু ঘটেছে — যেমন ফাইল আপলোড, অর্ডার তৈরি), **ট্রিগার** (যা ফাংশন চালু করে), **কোল্ড স্টার্ট** (প্রথম চালুর বিলম্ব)।

১৮.৩ কেন সার্ভারলেস? আগের সমস্যাটি কী ছিল

TechNova-র ছবি রিসাইজ করার কাজটি আগে একটি EC2 সার্ভারে ক্রন জব হিসেবে চলত। রাতে কোনো আপলোড না থাকলেও সার্ভার চলত ও বিল উঠত; সকালে হঠাৎ শত আপলোড এলে কিউ জমে যেত, সার্ভার বড় করতে হতো হাতে। সার্ভারলেস এই সমস্যার উত্তর: **Lambda** ফাংশন চলে শুধু S3-তে ফাইল এলে, একসঙ্গে হাজারটি আপলোড এলে হাজারটি সমান্তরাল চালনা হয়, আর শান্ত সময়ে খরচ শূন্যের কাছাকাছি। তবে সব কাজ সার্ভারলেসের উপযুক্ত নয় — দীর্ঘস্থায়ী, ধারাবাহিক সংযোগ বা ভারী কম্পিউটের কাজে কনটেইনার বা EC2-ই ভালো।

> **মূল ধারণা:** সার্ভারলেস মানে সার্ভার নেই — তা নয়; মানে সার্ভার দেখা, প্যাচ করা ও স্কেল করার দায়িত্ব আপনার নয়। আপনি শুধু কোড ও ঘটনার নিয়ম লেখেন; AWS বাকিটা সামলায়। এই বিনিময়ে আপনি নিয়ন্ত্রণ ছাড়েন: রানটাইম, সময়সীমা ও সমান্তরালতার সীমা AWS ঠিক করে।

১৮.৪ AWS Lambda — ভেতরের কাজ

Lambda হলো ইভেন্ট-চালিত কম্পিউট সেবা। আপনি একটি ফাংশন (Node.js 20 হ্যান্ডলার, যেমন `exports.handler`) ও তার **রানটাইম** বেছে দেন; AWS সেটি চালায়। **এক্সিকিউশন মডেল:** ঘটনা এলে Lambda একটি **এক্সিকিউশন এনভায়রনমেন্ট** তৈরি করে — রানটাইম, কোড ও `/tmp` ডিস্কসহ একটি বিচ্ছিন্ন পরিবেশ। একই পরিবেশ কিছুক্ষণ গরম (warm) থাকে; পরের ঘটনা এলে পুনরায় ব্যবহার হয় — এটাই **ওয়ার্ম স্টার্ট**। অনেকক্ষণ ঘটনা না এলে পরিবেশ মুছে যায়; পরের ঘটনায় নতুন পরিবেশ বানাতে সময় লাগে — এটাই **কোল্ড স্টার্ট** (সাধারণত শত মিলিসেকেন্ড, VPC-তে বা বড় প্যাকেজে বেশি)।

গুরুত্বপূর্ণ সীমা: একবার চালনায় সর্বোচ্চ ১৫ মিনিট; মেমোরি ১২৮ MB থেকে ১০,২৪০ MB; মেমোরি বাড়ালে CPU-ও বাড়ে (আনুপাতিক)। **কনকারেন্সি** হলো একসঙ্গে কত চালনা চলতে পারে — অ্যাকাউন্ট-স্তরে সীমা থাকে (ডিফল্ট ১,০০০, বাড়ানো

যায়); **রিজার্ভড কনকারেন্সি** দিয়ে একটি ফাংশনের জন্য কোটা সংরক্ষণ করা যায়, আর **প্রভিশনড কনকারেন্সি** দিয়ে কিছু পরিবেশ আগেই গরম রাখা যায় (খরচ বাড়ে)।

```
// handler.js — Node.js 20
exports.handler = async (event) => {
  console.log(JSON.stringify(event));
  const orderId = event.orderId;
  // আইডেমপোটেন্সি: একই orderId দুবার এলে দ্বিতীয়বার কিছু করবেন না
  return { statusCode: 200, body: JSON.stringify({ orderId }) };
};
aws lambda create-function --function-name technova-orders \
  --runtime nodejs20.x --role arn:aws:iam::111122223333:role/lambda-exec \
  --handler handler.handler --zip-file fileb://function.zip \
  --timeout 30 --memory-size 512 --region ap-south-1
aws lambda invoke --function-name technova-orders \
  --payload '{"orderId":"ORD-1001"}' out.json --region ap-south-1
```

অন্যান্য সেবার সঙ্গে যোগাযোগ: Lambda IAM রোল দিয়ে DynamoDB, S3, SQS-এ পৌঁছায়; VPC-তে চললে ENI দিয়ে প্রাইভেট রিসোর্সে (RDS) যায় — তবে VPC কোল্ড স্টার্ট বাড়াতে পারে। পরিবেশ ভেরিয়েবলে সিক্রেট লিখবেন না — Secrets Manager থেকে নিন। **ব্যর্থ হলে:** সমকালীন (synchronous) কলে ত্রুটি সরাসরি কলারকে ফেরে; অসমন্বিত (asynchronous) কলে Lambda দুবার পুনরায় চেষ্টা করে, তারপর DLQ-তে পাঠায়। **মনিটরিং:** CloudWatch মেট্রিক (Invocations, Errors, Duration, Throttles) ও লগ; X-Ray দিয়ে ট্রেস। **বিলিং:** অনুরোধ সংখ্যা ও চলার সময় (GB-সেকেন্ড) অনুযায়ী — AWS Pricing Calculator-এ যাচাই করুন; অলস সময়ে কোনো চার্জ নেই।

> **WARNING:** Lambda ফাংশনে ডেটাবেস সংযোগ প্রতিবার খুললে সংযোগ ফুরিয়ে যায়। RDS Proxy ব্যবহার করুন, অথবা সংযোগ পুলিং করুন এবং হ্যান্ডলারের বাইরে ক্লায়েন্ট পুনঃব্যবহার করুন (ওয়ার্ম পরিবেশে তা বেঁচে থাকে)।

১৮.৫ API Gateway — HTTP-এর দরজা

API Gateway হলো পরিচালিত API সামনের দরজা: রাউটিং, প্রমাণীকরণ (authentication), হার-সীমা (throttling), API কী, CORS ও ব্যবহার পরিকল্পনা (usage plan) দেয়। দুটি ধরন: **REST API** (সম্পূর্ণ বৈশিষ্ট্য, অনুরোধ/উত্তর রূপান্তর) ও **HTTP API** (সহজ, সস্তা, দ্রুত — অধিকাংশ নতুন কাজে এটাই যথেষ্ট)। Lambda-র সঙ্গে যুক্ত করলে প্রতিটি HTTP অনুরোধ একটি Lambda ঘটনা হয়; উত্তর Lambda ফেরত দেয়।

কাজের প্রবাহ: ক্লায়েন্ট → API Gateway (প্রমাণীকরণ যাচাই, থ্রটলিং) → Lambda → DynamoDB। **ব্যর্থ হলে:** ৫০২ মানে Lambda ত্রুটি বা ভুল উত্তর-ফরম্যাট; ৫০৪ মানে সময়সীমা (API Gateway-এর ২৯ সেকেন্ড সীমা মনে রাখুন — দীর্ঘ কাজ Lambda-তে অসমন্বিতভাবে করুন, ফলাফল SQS/পোলিং দিয়ে দিন)। **নিরাপত্তা:** IAM, Cognito বা Lambda অথরাইজার দিয়ে প্রমাণীকরণ; WAF যুক্ত করে আক্রমণ ঠেকান। **বিলিং:** অনুরোধ সংখ্যা অনুযায়ী — AWS Pricing Calculator-এ যাচাই করুন।

১৮.৬ SQS, SNS ও EventBridge — তিনটি বার্তা-সেবা, তিনটি কাজ

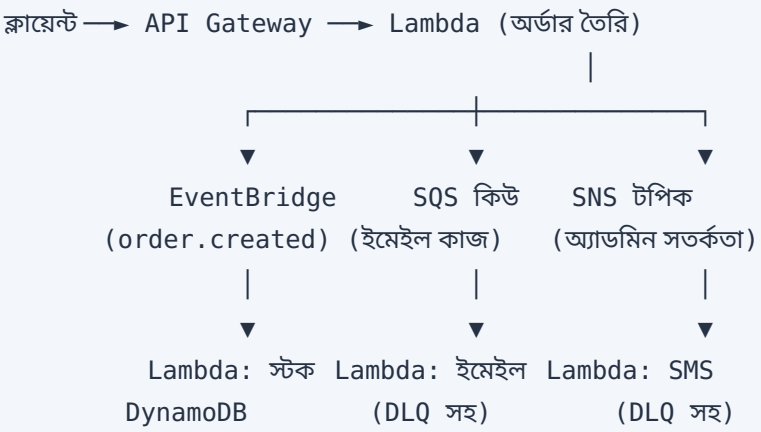
নতুনরা এই তিনটিকে গুলিয়ে ফেলেন; পার্থক্য পরিষ্কার: **SQS** (Simple Queue Service) হলো **কিউ** — বার্তা জমা হয়, একজন গ্রাহক (consumer) একবার টেনে প্রক্রিয়া করে; কাজের ভারসাম্য ও পুনঃচেষ্টার জন্য। **SNS** (Simple Notification Service) হলো **পাব/সাব** — এক প্রকাশক, অনেক গ্রাহক (ইমেইল, SMS, Lambda, SQS); এক ঘটনা অনেককে জানাতে।

EventBridge হলো **ইভেন্ট বাস** — নিয়ম (rule) দিয়ে ঘটনা ছেঁকে নির্দিষ্ট লক্ষ্যে পাঠানো; AWS সেবা, SaaS অ্যাপ ও নিজের অ্যাপের ঘটনা এক জায়গায়।

সেবা	ধরন	কখন ব্যবহার	কখন নয়
SQS	কিউ (pull)	কাজের সারি, ভারসাম্য, পুনঃচেষ্টা	এক ঘটনা অনেককে জানাতে
SNS	পাব/সাব (push)	বিজ্ঞপ্তি, ফ্যান-আউট	ক্রম বা একবার-মাত্র নিশ্চয়তা চাইলে
EventBridge	ইভেন্ট বাস + নিয়ম	সেবার মধ্যে ঘটনা রাউটিং, সময়সূচি	সরল কিউ যথেষ্ট হলে

SQS-এর গভীরে: স্ট্যান্ডার্ড কিউ প্রায়-অসীম স্কেল দেয় কিন্তু ক্রমের নিশ্চয়তা দেয় না; **FIFO কিউ** ক্রম ও ঠিক-একবার প্রক্রিয়াকরণ দেয়, কিন্তু থ্রুপুট সীমিত। **ভিজিবিলিটি টাইমআউট** — বার্তা টানার পর যতক্ষণ অন্য কেউ দেখবে না; প্রক্রিয়া এর চেয়ে বেশি সময় নিলে বার্তা আবার দেখা দেয় — তাই টাইমআউট প্রক্রিয়ার সময়ের চেয়ে বড় রাখুন। **ডেড-লেটার কিউ (DLQ):** নির্দিষ্টবার ব্যর্থ হলে বার্তা DLQ-তে যায় — সেখান থেকে পরীক্ষা করে আবার চালু (redrive) করা যায়।

TechNova ইভেন্ট-চালিত অর্ডার প্রবাহ



১৮.৭ ইভেন্ট-চালিত নকশা — নিয়ম ও বিপদ

ইভেন্ট-চালিত নকশায় সেবাগুলো সরাসরি একে অপরকে ডাকে না; তারা ঘটনা প্রকাশ করে, আগ্রহীরা শোনে। এতে **ঢিলেঢালা সংযোগ** (loose coupling) হয় — নতুন গ্রাহক যোগ করা মানে নতুন নিয়ম, পুরোনো কোড ছোঁয়া নয়। তবে তিনটি বিপদ: **দ্বৈত ডেলিভারি** — SQS/SNS অন্তত-একবার (at-least-once) দেয়, তাই গ্রাহককে **আইডেমপোটেন্ট** হতে হয় (একই ঘটনা দুবার এলে ফল একই); **ক্রম** — ঘটনার ক্রম নিশ্চিত নয়, তাই নকশায় ক্রম-নির্ভরতা এড়ান বা FIFO ব্যবহার করুন; **দৃশ্যমানতা** — ঘটনা কোথায় আটকাল তা খুঁজতে ট্রেসিং (X-Ray, correlation ID) অপরিহার্য।

আইডেমপোটেন্সি বাস্তবায়ন: প্রতিটি ঘটনায় অনন্য ID রাখুন; DynamoDB-তে সেই ID শর্তসাপেক্ষে (conditional write) লিখুন — আগে থাকলে প্রক্রিয়া এড়িয়ে যান। অর্থপ্রদানের মতো কাজে এটি বাধ্যতামূলক — দুবার চার্জ মানেই গ্রাহক হারানো।

১৮.৮ রিট্রাই, DLQ ও কনকারেন্সি — প্রোডাকশন নিয়ম

রিট্রাই নীতি: ক্ষণস্থায়ী ব্যর্থতায় (থ্রটলিং, নেটওয়ার্ক) **এক্সপোনেনশিয়াল ব্যাকঅফ** দিয়ে পুনঃচেষ্টা করুন; স্থায়ী ব্যর্থতায় (ভুল ডেটা) দ্রুত DLQ-তে পাঠান — অযথা রিট্রাই খরচ ও বিলম্ব বাড়ায়। SQS-এ `maxReceiveCount` (যেমন ৩) ঠিক

করুন; Lambda অসমন্বিত কলে ২ বার রিট্রাই করে। **কনকারেন্সি নিয়ন্ত্রণ:** ডাউনস্ট্রিম ডেটাবেসের সীমা অনুযায়ী ফাংশনের রিজার্ভড কনকারেন্সি সীমিত করুন — নইলে হাজার Lambda একসঙ্গে ডেটাবেস ডুবিয়ে দেবে। **থ্রটলিং হলে:** CloudWatch-এ Throttles মেট্রিক দেখুন, কনকারেন্সি বাড়ান বা SQS দিয়ে ভারসাম্য করুন।

১৮.৯ প্রোডাকশন পরিস্থিতি — TechNova-র দ্বৈত ইমেইল

TechNova-র অর্ডার নিশ্চিতকরণ ইমেইল কিছু গ্রাহক দুবার পেলেন। তদন্তে দেখা গেল: SQS ভিজিবিলিটি টাইমআউট ৩০ সেকেন্ড, কিন্তু ইমেইল পাঠাতে ৪৫ সেকেন্ড লাগছিল — বার্তা আবার দেখা দিয়ে দ্বিতীয় Lambda চালু হচ্ছিল। সমাধান দুটি: টাইমআউট ৯০ সেকেন্ড করা, এবং আইডেমপোটেন্সি কী (orderId) দিয়ে DynamoDB-তে পাঠানো-চিহ্ন রাখা। শিক্ষা: কিউয়ের সময়সীমা সবসময় প্রক্রিয়ার সময়ের চেয়ে বড় রাখুন, আর at-least-once মানেই আইডেমপোটেন্সি বাধ্যতামূলক।

১৮.৯ক Step Functions — দীর্ঘ ও জটিল কর্মপ্রবাহ

Lambda একবারে সর্বোচ্চ ১৫ মিনিট চলে — কিন্তু অনেক ব্যবসায়িক প্রক্রিয়া তার চেয়ে দীর্ঘ ও ধাপে ধাপে চলে: অর্ডার যাচাই, অর্থপ্রদান, স্টক কমানো, চালান তৈরি, ইমেইল পাঠানো — প্রতিটি ধাপে ভিন্ন সেবা, ভিন্ন ব্যর্থতার ধরন, মানুষের অনুমোদনও লাগতে পারে। এই ধরনের কাজে Lambda-কে নিজে সমন্বয় করতে গেলে কোড জটিল হয়: কোন ধাপে আটকাল, কোথা থেকে আবার শুরু করব, কতবার পুনঃচেষ্টা করব — সব হাতে লিখতে হয়। **Step Functions** হলো AWS-এর পরিচালিত কর্মপ্রবাহ (workflow) সেবা: আপনি একটি **স্টেট মেশিন** (state machine) আঁকেন — ধাপগুলো, তাদের ক্রম, শর্ত, পুনঃচেষ্টা ও ত্রুটি-পথ — JSON (Amazon States Language) বা Workflow Studio দিয়ে; AWS সেই মেশিন চালায়, প্রতিটি ধাপের অবস্থা সংরক্ষণ করে, ব্যর্থ ধাপে পুনঃচেষ্টা করে।

Step Functions-এর মূল অবস্থা (state) ধরন: **Task** — একটি Lambda চালানো বা AWS API ডাকা; **Choice** — শর্ত অনুযায়ী পথ বদলানো (যেমন অর্থপ্রদান সফল হলে এগিয়ে যাওয়া, ব্যর্থ হলে ক্ষতিপূরণ ধাপ); **Parallel** — একসঙ্গে একাধিক ধাপ চালানো (যেমন স্টক কমানো ও চালান তৈরি একসঙ্গে); **Wait** — নির্দিষ্ট সময় অপেক্ষা করা; **Succeed/Fail** — সমাপ্তি। **সাগা (Saga) প্যাটার্ন** হলো বিতরণ-করা লেনদেনের উত্তর: প্রতিটি ধাপের সঙ্গে একটি ক্ষতিপূরণ (compensating) ধাপ থাকে — অর্থপ্রদান সফল হলেও স্টক না মিললে অর্থ ফেরত দেওয়ার ধাপ চলে। এভাবে দুই-ধাপে কমিট (two-phase commit) ছাড়াই ধারাবাহিকতা রক্ষা করা যায়।

TechNova-র অর্ডার প্রক্রিয়া Step Functions-এ কেমন দেখায়: প্রথমে ValidateOrder (Lambda) — অর্ডার বৈধ কি না; তারপর Choice — অবৈধ হলে Fail; বৈধ হলে Parallel — ChargePayment (Lambda) ও ReserveStock (Lambda) একসঙ্গে; দুটোই সফল হলে CreateInvoice (Lambda) ও SendEmail (SQS-এ বার্তা); কোনো ধাপ ব্যর্থ হলে Catch ধরে ক্ষতিপূরণ — RefundPayment বা ReleaseStock। প্রতিটি চালনার (execution) ইতিহাস Step Functions কনসোলে দেখা যায় — কোন ধাপে কত সময় লাগল, ইনপুট-আউটপুট কী ছিল — তদন্ত তখন অনেক সহজ। **দুটি ধরন:** Standard Workflow — দীর্ঘস্থায়ী, ঠিক-একবার চালনা, বিস্তারিত ইতিহাস (অর্ডার প্রক্রিয়ার জন্য এটাই); Express Workflow — উচ্চ-হার, স্বল্পস্থায়ী, কম খরচে (ইভেন্ট প্রক্রিয়াকরণের জন্য)। **মনিটরিং:** CloudWatch মেট্রিক (ExecutionsStarted, ExecutionsFailed) ও লগ; **বিলিং:** স্টেট পরিবর্তন (state transition) সংখ্যা অনুযায়ী — AWS Pricing Calculator-এ যাচাই করুন। **কখন ব্যবহার করবেন না:** একটিমাত্র Lambda-তে শেষ হয় এমন সরল কাজে Step Functions অযথা জটিলতা; আর খুব উচ্চ-হারের (প্রতি সেকেন্ডে হাজার) সরল রাউটিংয়ে EventBridge-ই যথেষ্ট।

আরেকটি গুরুত্বপূর্ণ সেবা: **EventBridge Scheduler** — নির্দিষ্ট সময়ে বা নিয়মিত বিরতিতে Lambda বা অন্য লক্ষ্য চালানো (পুরোনো CloudWatch Events-এর সময়সূচি-অংশের আধুনিক রূপ)। TechNova-র দৈনিক রিপোর্ট, সাপ্তাহিক ব্যাকআপ যাচাই বা প্রতি ঘণ্টার ক্যাশে পরিষ্কার — এগুলো Scheduler দিয়ে সাজান, Lambda-র ভেতরে ক্রন লজিক লিখবেন না। Scheduler-এর সুবিধা: সময় অঞ্চল (timezone) সমর্থন, পুনঃচেষ্টা নীতি, ও এককালীন (one-time) সময়সূচি — যা CloudWatch Events-এ ছিল না।

PRO TIP: কর্মপ্রবাহের প্রতিটি ধাপের ইনপুট-আউটপুট ছোট রাখুন — বড় ডেটা S3-তে রেখে শুধু তার রেফারেন্স (S3 কী) পাস করুন। Step Functions-এর ইতিহাসে বড় পেলোড থাকলে খরচ ও দৃশ্যমানতা দুটোই খারাপ হয়।

১৮.১০ নিরাপত্তা, খরচ, ভুল ও সমস্যা সমাধান

নিরাপত্তা: প্রতিটি ফাংশনে ন্যূনতম IAM রোল; API Gateway-এ প্রমাণীকরণ ছাড়া পাবলিক এন্ডপয়েন্ট নয়; পরিবেশ ভেরিয়েবলে সিক্রেট নয়; SQS/SNS-এ এনক্রিপশন (SSE) চালু; Lambda-র কোডে ইনপুট যাচাই। **খরচ:** Lambda-তে মেমোরি বড় করলে সময় কমে — মোট খরচ কমতেও পারে, পরীক্ষা করে ঠিক করুন; প্রভিশনড কনকারেন্সি সবসময় চালু রাখা ব্যয়বহুল; API Gateway ও ডেটা ট্রান্সফার বিলযোগ্য — AWS Pricing Calculator-এ যাচাই করুন।

সাধারণ ভুল: (১) কোল্ড স্টার্ট না মেপে অভিযোগ করা; (২) DLQ না রাখা — ব্যর্থ বার্তা হারিয়ে যায়; (৩) ভিজিবিলিটি টাইমআউট ভুল; (৪) Lambda-তে ১৫ মিনিটের বেশি কাজ জোর করা — তার বদলে Step Functions বা ECS; (৫) API Gateway ছাড়া Lambda URL পাবলিক রাখা।

সমস্যা সমাধান: Lambda সময়সীমা ছাড়ালে → Duration মেট্রিক ও লগ দেখুন, মেমোরি/টাইমআউট বাড়ান; SQS বার্তা আটকে থাকলে → DLQ, ভিজিবিলিটি টাইমআউট, Lambda ত্রুটির লগ; API Gateway ৫০০ → Lambda লগ ও ইন্টিগ্রেশন উত্তর-ফরম্যাট; ইভেন্ট পৌঁছাচ্ছে না → EventBridge নিয়মের প্যাটার্ন ও লক্ষ্যের IAM অনুমতি যাচাই।

১৮.১১ সারাংশ

Lambda ঘটনা ঘটলে চলে, API Gateway তার দরজা, SQS কাজের সারি, SNS বিজ্ঞপ্তি, EventBridge ঘটনার রাউটার। সার্ভারলেসের সাফল্য নির্ভর করে আইডেমপোটেন্সি, DLQ, সঠিক টাইমআউট ও কনকারেন্সি নিয়ন্ত্রণের ওপর — এগুলো ছাড়া সার্ভারলেস দ্রুতই অব্যবস্থাপনায় পরিণত হয়।

১৮.১২ পর্যালোচনা প্রশ্ন

১. কোল্ড স্টার্ট কী এবং কীভাবে কমাবেন? ২. SQS বনাম SNS বনাম EventBridge — উদাহরণসহ পার্থক্য। ৩. আইডেমপোটেন্সি কেন বাধ্যতামূলক? ৪. ভিজিবিলিটি টাইমআউট ভুল হলে কী হয়? ৫. রিজার্ভড বনাম প্রভিশনড কনকারেন্সি। ৬. API Gateway-এর ২৯ সেকেন্ড সীমা কীভাবে সামলাবেন? ৭. DLQ ছাড়া কী হারাবেন? ৮. Lambda কখন ব্যবহার করবেন না? ৯. FIFO কিউ কখন দরকার? ১০. X-Ray ছাড়া ইভেন্ট খুঁজতে অসুবিধা কী?

১৮.১৩ অনুশীলন

১. Lambda + API Gateway দিয়ে একটি অর্ডার API বানান; DynamoDB-তে লিখুন। ২. SQS কিউ + DLQ যুক্ত করে ইমেইল-পাঠানো Lambda বানান; ইচ্ছা করে ব্যর্থ করে DLQ-তে বার্তা দেখুন। ৩. EventBridge নিয়ম দিয়ে S3 আপলোডে একটি Lambda চালু করুন। ৪. একই ঘটনা দুবার পাঠিয়ে আপনার হ্যান্ডলার আইডেমপোটেন্ট কি না পরীক্ষা করুন।

অধ্যায় ১৯ • PART IV

Chapter 19 — AWS CI/CD Engineering

১৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) CodePipeline, CodeBuild ও CodeDeploy-এর ভূমিকা ব্যাখ্যা করতে; (২) সোর্স থেকে ডিপ্লয় পর্যন্ত সম্পূর্ণ পাইপলাইন নকশা করতে; (৩) GitHub Actions-এর সঙ্গে AWS যুক্ত করতে; (৪) OIDC দিয়ে দীর্ঘস্থায়ী কী ছাড়া প্রমাণীকরণ করতে; (৫) আর্টিফ্যাক্ট ব্যবস্থাপনা, পরিবেশ-উন্নয়ন (promotion) ও অনুমোদন গেট সাজাতে; (৬) রোলব্যাক ও ডিপ্লয়মেন্ট কৌশল বেছে নিতে; (৭) CI/CD নিরাপত্তা নিশ্চিত করতে।

১৯.২ পূর্বজ্ঞান ঝালাই

মনে রাখুন: ১৭ অধ্যায়ে ECS ডিপ্লয়মেন্ট, ১৮ অধ্যায়ে Lambda ডিপ্লয়মেন্ট দেখেছেন — কিন্তু সেগুলো ছিল হাতে চালানো কমান্ড। **CI** (Continuous Integration) মানে প্রতিটি কমিটে স্বয়ংক্রিয়ভাবে বিল্ড ও পরীক্ষা; **CD** (Continuous Delivery/Deployment) মানে পরীক্ষা পেরোলে স্বয়ংক্রিয়ভাবে (বা অনুমোদন সাপেক্ষে) ডিপ্লয়। Git, Docker বিল্ড ও IAM রোলের ধারণা এখানে কাজে লাগবে।

১৯.৩ কেন CI/CD? আগের সমস্যাটি কী ছিল

TechNova-তে একসময় ডিপ্লয় মানেই ছিল শুক্রবার রাতে একজন সিনিয়র ডেভেলপারের ল্যাপটপ থেকে ম্যানুয়াল ধাপ — SSH, git pull, npm install, pm2 restart। একবার ভুল ব্রাঞ্চ ডিপ্লয় হয়ে প্রোডাকশন ভেঙেছিল, আর কেউ জানত না ঠিক কোন কমিট চলছে। CI/CD এই সমস্যার উত্তর: প্রতিটি পরিবর্তন একই স্বয়ংক্রিয় পথে যায় — বিল্ড, পরীক্ষা, নিরাপত্তা স্ক্যান, স্টেজিং, অনুমোদন, প্রোডাকশন — এবং প্রতিটি ডিপ্লয়ের রেকর্ড থাকে: কোন কমিট, কে অনুমোদন দিল, কখন।

> **মূল ধারণা:** পাইপলাইন (pipeline) হলো পরিবর্তনের পরিবহন-পথ। ভালো পাইপলাইনের বৈশিষ্ট্য: দ্রুত প্রতিক্রিয়া (ব্যর্থতা মিনিটে জানা যায়), পুনরুৎপাদনযোগ্যতা (একই কমিট সবসময় একই আর্টিফ্যাক্ট দেয়), ও নিরাপত্তা (প্রোডাকশনে যাওয়ার আগে গেট)।

১৯.৪ CodePipeline, CodeBuild, CodeDeploy — তিনটি সেবা, একটি প্রবাহ

CodePipeline হলো অর্কেস্ট্রেটর — ধাপগুলো (stage) সাজায়: Source → Build → Test → Deploy। **CodeBuild** হলো বিল্ড কর্মী — পরিচালিত বিল্ড পরিবেশে `buildspec.yml` অনুযায়ী কোড বিল্ড, পরীক্ষা ও প্যাকেজ করে। **CodeDeploy** হলো ডিপ্লয়কারী — EC2, Lambda বা ECS-এ নিরাপদে নতুন সংস্করণ বসায়, ব্লু-গ্রিন ও ক্যানারি কৌশলসহ, স্বয়ংক্রিয় রোলব্যাকসহ।

কাজের প্রবাহ: GitHub-এ push → CodePipeline সোর্স ধাপে কোড নেয় → CodeBuild টেস্ট চালিয়ে Docker ইমেজ বানিয়ে ECR-এ পাঠায় → CodeDeploy ECS সার্ভিসে নতুন টাস্ক ডেফিনিশন বসায় → CloudWatch অ্যালার্ম খারাপ হলে স্বয়ংক্রিয় রোলব্যাক। **ব্যর্থ হলে:** কোন ধাপে ব্যর্থ তা পাইপলাইন ইতিহাসে দেখা যায়; CodeBuild লগ CloudWatch-এ থাকে। **বিলিং:** CodePipeline প্রতি সক্রিয় পাইপলাইনে, CodeBuild বিল্ড মিনিটে — AWS Pricing Calculator-এ যাচাই করুন।

```
# buildspec.yml — CodeBuild-এর নির্দেশনা
version: 0.2
phases:
  pre_build:
    commands:
      - echo Logging in to ECR...
      - aws ecr get-login-password --region ap-south-1 | docker login
        --username AWS --password-stdin
        111122223333.dkr.ecr.ap-south-1.amazonaws.com
  build:
    commands:
      - echo Building image...
      - docker build -t technova/api:$CODEBUILD_RESOLVED_SOURCE_VERSION .
      - docker push 111122223333.dkr.ecr.ap-south-1.amazonaws.com/technova/api:
$CODEBUILD_RESOLVED_SOURCE_VERSION
  post_build:
    commands:
      - echo Build completed
artifacts:
  files:
    - imagedefinitions.json
```

লক্ষ করুন: ইমেজ ট্যাগ হিসেবে Git কমিট SHA (`CODEBUILD_RESOLVED_SOURCE_VERSION`) ব্যবহার করা হয়েছে — `latest` নয়। `imagedefinitions.json` CodeDeploy-কে বলে কোন ইমেজ ডিপ্লয় করতে হবে।

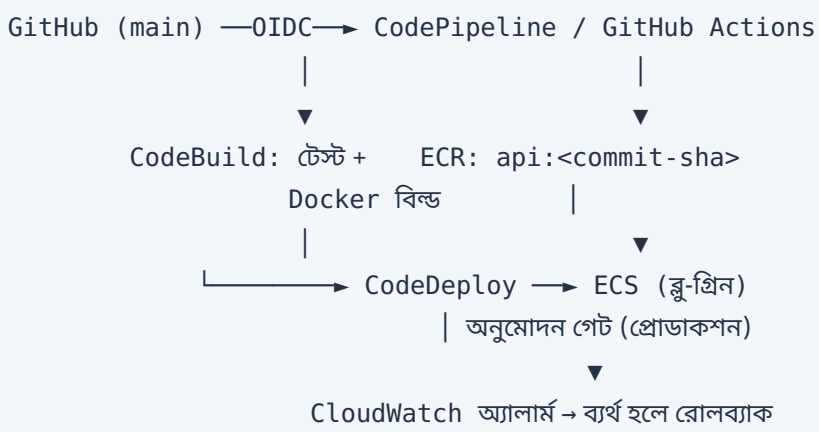
১৯.৫ GitHub Actions + AWS — জনপ্রিয় সংমিশ্রণ

অনেক দল CI হিসেবে GitHub Actions ব্যবহার করে, CD হিসেবে AWS সেবা। মূল প্রশ্ন: GitHub কীভাবে AWS-এ প্রমাণীকরণ করবে? পুরোনো পথ ছিল দীর্ঘস্থায়ী IAM অ্যাক্সেস কী GitHub Secrets-এ রাখা — ফাঁস হলে বিপদ, ঘোরানো (rotation) ঝামেলা। আধুনিক পথ **OIDC** (OpenID Connect): GitHub Actions AWS-এর কাছে একটি স্বল্পস্থায়ী টোকেন দেখায়, AWS IAM পরিচয় প্রদানকারী (identity provider) তা যাচাই করে অস্থায়ী ক্রেডেনশিয়াল দেয় — কোনো স্থায়ী কী সংরক্ষণ করতে হয় না।

```
# .github/workflows/deploy.yml (অংশবিশেষ)
permissions:
  id-token: write
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::111122223333:role/github-actions-deploy
          aws-region: ap-south-1
      - run: aws sts get-caller-identity
```

IAM রোলে **শর্ত** (condition) দিন: শুধু `technova/api` রিপজিটরির `main` ব্রাঞ্চ থেকেই এই রোল নেওয়া যাবে। এতে অন্য রিপজিটরি বা ব্রাঞ্চ থেকে আক্রমণ হলেও রোল মিলবে না। **নিরাপত্তা নিয়ম:** রোলের অনুমতি ন্যূনতম রাখুন (শুধু ECR push ও ECS update), কখনো `AdministratorAccess` দেবেন না।

TechNova CI/CD পাইপলাইন



১৯.৬ আর্টিফ্যাক্ট, পরিবেশ-উন্নয়ন ও অনুমোদন গেট

আর্টিফ্যাক্ট হলো বিল্ডের ফলাফল — Docker ইমেজ, zip ফাইল, CloudFormation টেমপ্লেট। নিয়ম: একবার বিল্ড, সর্বত্র একই আর্টিফ্যাক্ট। স্টেজিংয়ে পরীক্ষিত ইমেজটিই প্রোডাকশনে যাবে; প্রতিটি পরিবেশে আলাদা বিল্ড করবেন না — তাতে "স্টেজিংয়ে চলেছিল, প্রোডাকশনে ভাঙল" হয়। আর্টিফ্যাক্ট S3 বা ECR-এ সংরক্ষণ করুন, সংস্করণসহ।

পরিবেশ-উন্নয়ন (promotion): dev → staging → production। প্রতিটি পরিবেশে একই পাইপলাইন ধাপ, শুধু কনফিগ আলাদা (পরিবেশ ভেরিয়েবল, সিক্রেট)। **অনুমোদন গেট:** প্রোডাকশন ধাপের আগে মানুষের অনুমোদন — CodePipeline-এ ম্যানুয়াল অনুমোদন অ্যাকশন, অথবা GitHub-এ environment protection rule। জরুরি হটফিক্সের জন্য গেট এড়ানোর পথ রাখবেন না — বরং গেট দ্রুত করার ব্যবস্থা করুন (স্ল্যাক বিজ্ঞপ্তি, অন-কল)।

১৯.৭ ডিপ্লয়মেন্ট কৌশল ও রোলব্যাক

রোলিং: ধাপে ধাপে প্রতিস্থাপন — সহজ, কিন্তু কিছুক্ষণ দুই সংস্করণ একসঙ্গে চলে। **ব্লু-গ্রিন:** নতুন পরিবেশ পুরোপুরি তৈরি, পরীক্ষা করে ট্রাফিক সরানো — রোলব্যাক এক ক্লিকে। **ক্যানারি:** প্রথমে ১০% ট্রাফিক, মেট্রিক দেখে বাড়ানো — ঝুঁকি সবচেয়ে কম। **রোলব্যাক পরিকল্পনা:** আগের আর্টিফ্যাক্ট ট্যাগ সংরক্ষণ করুন, ডেটাবেস মাইগ্রেশন পশ্চাদমুখী-সামঞ্জস্যপূর্ণ (backward-compatible) রাখুন — কোড রোলব্যাক হলেও পুরোনো কোড নতুন স্কিমায় চলতে হবে। ডেটাবেস পরিবর্তন আগে, কোড পরে — এই ক্রমই নিরাপদ ডিপ্লয়ের মূলমন্ত্র।

১৯.৮ প্রোডাকশন পরিস্থিতি — TechNova-র ভাঙা ডিপ্লয়

এক সোমবার সকালে প্রোডাকশন ডিপ্লয় ব্যর্থ হলো — CodeBuild সফল, কিন্তু ECS টাস্ক `/health`-এ ব্যর্থ। CodeDeploy স্বয়ংক্রিয়ভাবে আগের সংস্করণে ফিরিয়ে দিল, গ্রাহক কিছু টের পেলেন না। পরে দেখা গেল নতুন কোডে পরিবেশ ভেরিয়েবলের নাম ভুল ছিল। শিক্ষা: (১) স্বয়ংক্রিয় রোলব্যাক জীবন বাঁচায় — সব ডিপ্লয়ে চালু রাখুন; (২) স্টেজিংয়ে হেলথ চেকসহ সম্পূর্ণ ধাপ পরীক্ষা করুন; (৩) ডিপ্লয়ের পর ১৫ মিনিট মেট্রিক পর্যবেক্ষণ করুন — "ডিপ্লয় শেষ" মানে "সফল" নয়।

১৯.৯ নিরাপত্তা, খরচ, ভুল ও সমস্যা সমাধান

নিরাপত্তা: OIDC ব্যবহার করুন, দীর্ঘস্থায়ী কী নয়; বিল্ড লগে সিক্রেট ছাপাবেন না; CodeBuild-এ `privileged mode` শুধু Docker বিল্ডের প্রয়োজনে; পাইপলাইনের IAM রোলে ন্যূনতম সুযোগ; আর্টিফ্যাক্ট S3 বাকেটে এনক্রিপশন ও সংস্করণ।

খরচ: CodeBuild মিনিট, CodePipeline সক্রিয় পাইপলাইন, S3 আর্টিফ্যাক্ট সংরক্ষণ — AWS Pricing Calculator-এ যাচাই করুন; অপ্রয়োজনীয় পাইপলাইন চালু রাখবেন না।

> **সাধারণ ভুল:** (১) GitHub Secrets-এ স্থায়ী AWS কী; (২) প্রতিটি পরিবেশে আলাদা বিল্ড; (৩) অনুমোদন গেট ছাড়া সরাসরি প্রোডাকশন; (৪) রোলব্যাক পরিকল্পনাহীন ডেটাবেস মাইগ্রেশন; (৫) বিল্ড লগে সিক্রেট।

সমস্যা সমাধান: পাইপলাইন আটকে গেলে → ধাপের লগ ও IAM অনুমতি দেখুন; OIDC ব্যর্থ হলে → ট্রাস্ট পলিসির শর্ত (রিপজিটরি/ব্রাঞ্চ) মিলিয়ে দেখুন; CodeDeploy ব্যর্থ হলে → অ্যালার্ম, হেলথ চেক ও AppSpec ফাইল যাচাই; ইমেজ pull ব্যর্থ হলে → ECR অনুমতি ও ট্যাগ।

১৯.১০ সারাংশ

CI/CD হলো পরিবর্তনের নিরাপদ পথ: CodePipeline সাজায়, CodeBuild বানায়, CodeDeploy বসায়, GitHub Actions + OIDC আধুনিক সংমিশ্রণ দেয়। একবার বিল্ড, অনুমোদন গেট, স্বয়ংক্রিয় রোলব্যাক — এই তিন নিয়ম মানলে ডিপ্লয় ভয়ের বিষয় থাকে না।

১৯.১১ পর্যালোচনা প্রশ্ন

১. CodePipeline, CodeBuild ও CodeDeploy-এর ভূমিকা কী? ২. OIDC কেন দীর্ঘস্থায়ী কী-র চেয়ে নিরাপদ? ৩. একবার বিল্ড নীতি কেন জরুরি? ৪. অনুমোদন গেট কখন রাখবেন? ৫. ব্লু-গ্রিন বনাম ক্যানারি। ৬. ডেটাবেস মাইগ্রেশনে রোলব্যাক কীভাবে নিরাপদ রাখবেন? ৭. buildspec.yml-এর ধাপগুলো কী? ৮. স্বয়ংক্রিয় রোলব্যাক কী ট্রিগার করে? ৯. GitHub Actions-এ IAM রোলের শর্ত কেন দরকার? ১০. পাইপলাইন ব্যর্থ হলে প্রথমে কোথায় দেখবেন?

১৯.১২ অনুশীলন

১. GitHub Actions + OIDC দিয়ে ECR-এ ইমেজ push করার ওয়ার্কফ্লো বানান। ২. CodePipeline দিয়ে সোর্স → বিল্ড → ECS ডিপ্লয় পাইপলাইন তৈরি করুন। ৩. প্রোডাকশন ধাপে ম্যানুয়াল অনুমোদন গেট যুক্ত করুন। ৪. ইচ্ছা করে ভাঙা ডিপ্লয় করে স্বয়ংক্রিয় রোলব্যাক পর্যবেক্ষণ করুন।

অধ্যায় ২০ • PART IV

Chapter 20 — Infrastructure as Code on AWS

২০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Infrastructure as Code (IaC)-এর নীতি ব্যাখ্যা করতে; (২) CloudFormation, CDK ও Terraform-এর মধ্যে পার্থক্য করতে; (৩) Terraform দিয়ে AWS রিসোর্স তৈরি, পরিবর্তন ও ধ্বংস করতে; (৪) স্টেট ব্যবস্থাপনা, লকিং ও ব্যাকএন্ড সাজাতে; (৫) মডিউল দিয়ে পুনঃব্যবহারযোগ্য স্থাপত্য বানাতে; (৬) ড্রিফট শনাক্ত করতে; (৭) IaC-কে CI/CD-তে যুক্ত করতে; (৮) সিদ্ধান্ত টেবিল দিয়ে সঠিক টুল বেছে নিতে।

২০.২ পূর্বজ্ঞান ঝালাই

আগের অধ্যায়গুলোতে আপনি কনসোল ও CLI দিয়ে রিসোর্স তৈরি করেছেন। সমস্যা: কনসোলে ক্লিক করে বানানো অবকাঠামোর কোনো ইতিহাস থাকে না — কে কী বদলাল, কেন, কখন — কিছুই জানা যায় না, আর একই পরিবেশ আবার বানানো যায় না। **IaC** মানে অবকাঠামো কোড হিসেবে লেখা — Git-এ সংস্করণ, পর্যালোচনা (code review) ও স্বয়ংক্রিয় প্রয়োগ। মনে রাখুন: VPC, সাবনেট, IAM রোল, S3 বাকেট — সবই এখন কোডের লাইন।

২০.৩ কেন IaC? প্রথম নীতি

TechNova-র স্টেজিং পরিবেশ একবার হাতে বানানো হয়েছিল; প্রোডাকশন বানাতে গিয়ে দেখা গেল স্টেজিংয়ের সিকিউরিটি গ্রুপের নিয়ম কেউ লিখে রাখেনি — দুই পরিবেশের অমিলে বাগ ধরা পড়ল প্রোডাকশনে। IaC-এর উত্তর: **ঘোষণামূলক** (declarative) কোড — আপনি বলেন "এই রকম চাই", টুল বাস্তবতাকে সেভাবে মেলায়। সুবিধা: পুনরুৎপাদনযোগ্যতা, পর্যালোচনা, রোলব্যাক (Git revert মানেই আগের অবকাঠামো), ও নিরীক্ষা (audit)।

> **মূল ধারণা:** IaC-তে দুটি ধারা: **ঘোষণামূলক** — "কী চাই" বলেন (CloudFormation, Terraform), টুল "কীভাবে" ঠিক করে; **আদেশমূলক** (imperative) — ধাপে ধাপে নির্দেশ দেন (স্ক্রিপ্ট)। ঘোষণামূলক ধারাই আধুনিক মান — কারণ টুল নিজেই পার্থক্য (diff) বের করে শুধু প্রয়োজনীয় পরিবর্তন প্রয়োগ করে।

২০.৪ CloudFormation — AWS-এর নিজস্ব IaC

CloudFormation হলো AWS-এর নেটিভ IaC: YAML/JSON টেমপ্লেটে রিসোর্স ঘোষণা করেন, **স্ট্যাক** হিসেবে ডিপ্লয় করেন। সুবিধা: AWS-এর সঙ্গে গভীর সমন্বয়, নতুন সেবার সমর্থন সবার আগে, কোনো আলাদা স্টেট ফাইল সামলাতে হয় না (AWS-ই স্টেট রাখে), ড্রিফট শনাক্তকরণ অন্তর্নিহিত। সীমা: শুধু AWS-এ চলে, টেমপ্লেট বড় হলে জটিল হয়, প্রোগ্রামিং যুক্তি (লুপ, শর্ত) সীমিত।

```
AWSTemplateFormatVersion: '2010-09-09'
Description: TechNova S3 bucket
Resources:
  AppBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: technova-app-111122223333
      VersioningConfiguration:
        Status: Enabled
```

```
      PublicAccessBlockConfiguration:
        BlockPublicAcls: true
        BlockPublicPolicy: true
        IgnorePublicAcls: true
        RestrictPublicBuckets: true
Outputs:
  BucketName:
    Value: !Ref AppBucket
```

ডিপ্লয়: `aws cloudformation deploy --template-file bucket.yaml --stack-name technova-app --region ap-south-1`। **স্ট্যাক লাইফসাইকেল:** তৈরি → আপডেট (চেঞ্জ সেট পর্যালোচনা করে প্রয়োগ) → মুছে ফেলা। **চেঞ্জ সেট** হলো প্রয়োগের আগে পার্থক্য দেখার নিরাপদ উপায় — প্রোডাকশনে সবসময় চেঞ্জ সেট দেখে প্রয়োগ করুন।

২০.৫ AWS CDK — কোড দিয়ে অবকাঠামো

CDK (Cloud Development Kit) হলো CloudFormation-এর ওপরে প্রোগ্রামিং ভাষার স্তর: TypeScript, Python বা Go-তে অবকাঠামো লেখেন, CDK তা CloudFormation টেমপ্লেটে রূপান্তরিত করে। সুবিধা: লুপ, ফাংশন, ক্লাস — পুনঃব্যবহার সহজ; **কনস্ট্রাক্ট** (construct) দিয়ে উচ্চ-স্তরের ধারণা (যেমন "একটি ECS সার্ভিস ALB-সহ") এক লাইনে প্রকাশ করা যায়। যারা কোড লিখতে স্বাচ্ছন্দ্য, তাদের জন্য CDK দ্রুততর; তবে শেষ পর্যন্ত এটি CloudFormation-ই — তাই AWS-সীমা একই থাকে।

২০.৬ Terraform — বহু-ক্লাউড মান

Terraform হলো HashiCorp-এর ওপেন-সোর্স IaC, **প্রভাইডার** ধারণায় চলে: AWS প্রভাইডার AWS রিসোর্স বোঝে, অন্য প্রভাইডার অন্য ক্লাউড বোঝে — একই ভাষায় বহু প্ল্যাটফর্ম। **স্টেট** হলো Terraform-এর স্মৃতি: কোন রিসোর্স বাস্তবে আছে, তার মানচিত্র। স্টেট স্থানীয়ভাবে রাখা যায়, কিন্তু দলগত কাজে **রিমোট ব্যাকএন্ড** (S3 + DynamoDB লক) বাধ্যতামূলক — নইলে দুজন একসঙ্গে প্রয়োগ করে স্টেট নষ্ট করবে।

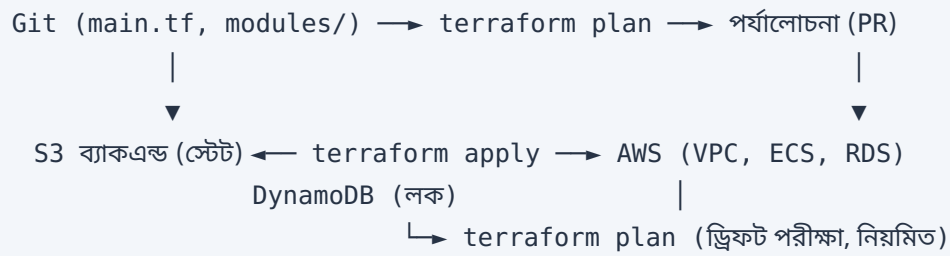
```
# main.tf
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    bucket         = "technova-tfstate-111122223333"
    key            = "prod/terraform.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "technova-tf-locks"
    encrypt        = true
  }
}
provider "aws" { region = "ap-south-1" }

resource "aws_s3_bucket" "app" {
  bucket = "technova-app-111122223333"
}
resource "aws_s3_bucket_versioning" "app" {
  bucket = aws_s3_bucket.app.id
  versioning_configuration { status = "Enabled" }
}
```

```
terraform init
terraform plan    # কী বদলাবে, দেখুন — প্রয়োগের আগে
terraform apply   # নিশ্চিত হয়ে প্রয়োগ করুন
terraform destroy # সাবধানে — সব মুছে যাবে
```

মডিউল: পুনঃব্যবহারযোগ্য প্যাকেজ — যেমন `modules/vpc` একবার লিখে dev, staging, prod-এ ব্যবহার। মডিউলে ভেরিয়েবল (input) ও আউটপুট স্পষ্ট রাখুন। **ড্রিফট:** কেউ কনসোলে হাতে বদলালে স্টেট ও বাস্তবতার অমিল হয় — `terraform plan` তা দেখায়; নিয়মিত `plan` চালিয়ে ড্রিফট ধরুন, আর হাতে বদল নিষিদ্ধ করুন।

Terraform কর্মপ্রবাহ — TechNova



২০.৭ সিদ্ধান্ত টেবিল — কোন টুল কখন

বিষয়	CloudFormation	CDK	Terraform
ভাষা	YAML/JSON	TypeScript/Python/Go	HCL
পরিসর	শুধু AWS	শুধু AWS	বহু ক্লাউড + SaaS
স্টেট	AWS সামলায়	AWS সামলায়	আপনি সামলান (S3)
শেখার খরচ	কম	মাঝারি (কোড জানতে হয়)	মাঝারি
পুনঃব্যবহার	নেস্টেড স্ট্যাক	কনস্ট্রাক্ট (সেরা)	মডিউল (ভালো)
নতুন AWS সেবা	সবার আগে	সবার আগে	কিছুটা দেরিতে
উপযুক্ত	শুধু AWS, সরল কাজ	AWS + কোড-প্রেমী দল	মাল্টি-ক্লাউড, বড় দল

TechNova-র সুপারিশ: শুধু AWS-এ থাকলে CDK বা CloudFormation; ভবিষ্যতে অন্য ক্লাউডের সম্ভাবনা থাকলে Terraform। যেটাই নিন, একটি টুলে স্থির থাকুন — তিনটি মেশানো মানে তিনটি স্টেট সামলানো।

২০.৮ IaC + CI/CD, নিরাপত্তা ও খরচ

IaC-কে পাইপলাইনে যুক্ত করুন: PR-এ `terraform plan` চালিয়ে পার্থক্য মন্তব্য হিসেবে দেখান; `main`-এ মার্জ হলে `apply` — প্রোডাকশনে অনুমোদন গেটসহ। **নিরাপত্তা:** স্টেট ফাইলে সিক্রেট থাকতে পারে — S3 ব্যাকএন্ড এনক্রিপ্ট করুন, সংস্করণ চালু করুন, পাবলিক অ্যাক্সেস বন্ধ করুন; `terraform plan` আউটপুটে সিক্রেট ছাপাবেন না; Checkov/tfsec দিয়ে নীতি পরীক্ষা করুন। **খরচ:** IaC নিজে প্রায় বিনামূল্যে, কিন্তু ভুল কোডে বড় রিসোর্স তৈরি হতে পারে — `plan` পর্যালোচনা ও বাজেট অ্যালার্ম বাধ্যতামূলক করুন।

২০.৮ক পলিসি অ্যাজ কোড ও ওয়ার্কস্পেস — IaC-এর পরের ধাপ

IaC লেখা শিখলেন; এবার প্রশ্ন: কেউ ভুল করে (বা ইচ্ছা করে) পাবলিক S3 বাকেট বা 0.0.0.0/0 SSH নিয়ম কোডে লিখলে কে ঠেকাবে? উত্তর: **পলিসি অ্যাজ কোড** (Policy as Code)। **OPA / Checkov / tfsec**-এর মতো টুল CI-তে `plan` বা টেমপ্লেট পরীক্ষা করে: "কোনো S3 বাকেট পাবলিক হতে পারবে না", "সব EBS ভলিউম এনক্রিপ্ট করা থাকতে হবে", "প্রতিটি রিসোর্সে Environment ট্যাগ থাকতে হবে" — নিয়ম ভাঙলে পাইপলাইন ব্যর্থ হয়, ডিপ্লয় হয় না। এভাবে নিরাপত্তা নীতি মানুষের মনে রাখার ওপর নির্ভর করে না; এটি পাইপলাইনের অংশ হয়। AWS-এর নিজস্ব পথও আছে: CloudFormation Guard ও AWS Config নিয়ম — Config নিয়ম ভাঙলে স্বয়ংক্রিয় সংশোধন চালানো যায়।

Terraform ওয়ার্কস্পেস হলো একই কোডে একাধিক পরিবেশ চালানোর উপায়: `terraform workspace new staging` দিয়ে আলাদা স্টেট, একই কোড — তবে সতর্কতা: পরিবেশের পার্থক্য শুধু ভেরিয়েবলে রাখুন; ওয়ার্কস্পেসে কোডের শর্ত (if environment == "prod") বাড়তে থাকলে মডিউল ভাগ করা ভালো। বড় দলে আরও ভালো প্যাটার্ন: প্রতিটি পরিবেশের জন্য আলাদা ডিরেক্টরি, একই মডিউল ব্যবহার করে — এতে `plan` পর্যালোচনা স্পষ্ট থাকে। **স্টেটের সুরক্ষা:** স্টেট ফাইলে কখনো কখনো সংবেদনশীল মান থাকে — তাই S3 ব্যাকএন্ডে এনক্রিপশন, সংস্করণ ও কঠোর IAM নীতি বাধ্যতামূলক; স্টেট পড়ার অনুমতি শুধু CI রোল ও সীমিত মানুষের। **ইমপোর্ট:** হাতে বানানো পুরোনো রিসোর্সকে IaC-তে আনতে `terraform import` ব্যবহার করুন — প্রথমে কোড লিখুন, তারপর import করুন, তারপর `plan`-এ কোনো পরিবর্তন দেখা যাচ্ছে না কি না যাচাই করুন; এটাই প্রমাণ করে কোড বাস্তবতার সঙ্গে মিলেছে।

২০.৯ প্রোডাকশন পরিস্থিতি — TechNova-র ড্রিফট

একবার প্রোডাকশন ALB-এর হেলথ চেক পাথ কেউ কনসোলে বদলে দিলেন — Terraform পরের `plan`-এ তা ধরে ফেলল। দল সিদ্ধান্ত নিল: কনসোলে বদল নয়, কোডে বদল; জরুরি বদল হলেও পরে কোডে তুলে আনতে হবে। শিক্ষা: ড্রিফট শনাক্তকরণ শুধু টুল নয়, সংস্কৃতি — "কোডই সত্যের উৎস" (source of truth) মেনে চলতে হয়।

> **সাধারণ ভুল:** (১) স্টেট ফাইল Git-এ কমিট করা; (২) লক ছাড়া রিমোট স্টেট; (৩) `plan` না দেখে `apply`; (৪) হার্ডকোড করা সিক্রেট; (৫) `terraform destroy` প্রোডাকশনে সুরক্ষা ছাড়া চালানো — `prevent_destroy` ব্যবহার করুন।

সমস্যা সমাধান: স্টেট লক আটকে গেলে → লক ধারক যাচাই করে `force-unlock` (সাবধানে); plan-এ অপ্রত্যাশিত ধ্বংস দেখালে → কোডের পরিবর্তন পর্যালোচনা করুন, `terraform state` দেখুন; প্রভাইডার সংস্করণ সমস্যা → সংস্করণ পিন করুন।

২০.১০ সারাংশ

IaC মানে অবকাঠামোকে কোডের মতো আচরণ করানো — সংস্করণ, পর্যালোচনা, পুনরুৎপাদন। CloudFormation AWS-নেটিভ, CDK কোড-বান্ধব, Terraform বহু-ক্লাউড। স্টেট সুরক্ষা, মডিউল ও ড্রিফট পরীক্ষা — এই তিনটি ছাড়া IaC অসম্পূর্ণ।

২০.১১ পর্যালোচনা প্রশ্ন

১. IaC কেন কনসোলের চেয়ে ভালো? ২. স্টেট কী এবং কেন S3-তে রাখবেন? ৩. ড্রিফট কীভাবে ধরবেন? ৪. CloudFormation বনাম Terraform — দুটি পার্থক্য। ৫. মডিউলের সুবিধা কী? ৬. চেঞ্জ সেট কেন দেখবেন? ৭. CDK কীভাবে

CloudFormation-এ রূপান্তরিত হয়? ৮. স্টেটে সিক্রেট থাকলে কী করবেন? ৯. CI/CD-তে plan কীভাবে ব্যবহার করবেন? ১০. `prevent_destroy` কখন দরকার?

২০.১২ অনুশীলন

১. Terraform দিয়ে S3 বাকেট + সংস্করণ তৈরি করুন; plan ও apply করুন। ২. S3 ব্যাকএন্ড + DynamoDB লক সাজান। ৩. একটি VPC মডিউল লিখে দুটি পরিবেশে ব্যবহার করুন। ৪. কনসোলে হাতে একটি পরিবর্তন করে plan-এ ড্রিফট দেখুন, তারপর কোডে ঠিক করুন।

PART V • DEVOPS ENGINEERING MASTERY

Production AWS Engineering

Monitoring, Logging ও Auditing, AWS সিকিউরিটি ইঞ্জিনিয়ারিং, নির্ভরযোগ্যতা ও Disaster Recovery, প্রোডাকশন আর্কিটেকচার ও FinOps।

অধ্যায় ২১ • PART V

Chapter 21 — Monitoring, Logging and Auditing

২১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) CloudWatch মেট্রিক, লগ, ড্যাশবোর্ড ও অ্যালার্ম সাজাতে; (২) CloudTrail দিয়ে API কার্যকলাপ নিরীক্ষা করতে; (৩) AWS Config দিয়ে কনফিগারেশন সম্মতি (compliance) পরীক্ষা করতে; (৪) X-Ray ও OpenTelemetry দিয়ে বিতরণ-করা ট্রেসিং করতে; (৫) কেন্দ্রীভূত পর্যবেক্ষণ নকশা করতে; (৬) ঘটনা তদন্ত করতে; (৭) ভালো সতর্কতা (alert) নকশা করতে — সতর্কতা-ক্লান্তি (alert fatigue) এড়িয়ে।

২১.২ পূর্বজ্ঞান ঝালাই

আগের অধ্যায়গুলোতে CloudWatch-এর নাম এসেছে — লগ, মেট্রিক, অ্যালার্ম হিসেবে। এখানে আমরা পূর্ণ ছবি দেখব। তিনটি স্তম্ভ মনে রাখুন: **মেট্রিক** (সংখ্যা — CPU কত, অনুরোধ কত), **লগ** (ঘটনার বিবরণ — কী হলো, কখন), **ট্রেস** (অনুরোধের যাত্রা — কোন সেবায় কত সময় লাগল)। IAM ও S3-এর ধারণা কাজে লাগবে — লগ সংরক্ষণ ও নিরীক্ষায়।

২১.৩ কেন পর্যবেক্ষণ? অন্ধকারে উড়ান নয়

TechNova-র API এক রাতে ধীর হয়ে গেল — গ্রাহক অভিযোগ করলেন, কিন্তু কেউ জানত না কারণ: ডেটাবেস? নেটওয়ার্ক? নাকি নতুন ডিপ্লয়? লগ ছড়িয়ে ছিল পাঁচ জায়গায়, মেট্রিক দেখার ড্যাশবোর্ড ছিল না। পর্যবেক্ষণ (observability) এই সমস্যার উত্তর: সিস্টেমের ভেতরের অবস্থা বাইরে থেকে জিজ্ঞাসা করে জানা যায় — মেট্রিক, লগ ও ট্রেস মিলিয়ে। মনে রাখবেন: **মনিটরিং** হলো জানা প্রশ্নের উত্তর দেখা (ড্যাশবোর্ড); **পর্যবেক্ষণযোগ্যতা** হলো অজানা প্রশ্নের উত্তর খুঁজে বের করার ক্ষমতা।

> **মূল ধারণা:** ভালো পর্যবেক্ষণের চারটি সোনালি সংকেত (golden signals): লেটেন্সি (কত দেরি), ট্রাফিক (কত অনুরোধ), ত্রুটি (কত ব্যর্থ), সম্পৃক্ততা (saturation — কতটা ভর্তি)। প্রতিটি সেবার জন্য এই চারটি মাপুন; বাকি সব বোনাস।

২১.৪ CloudWatch মেট্রিক, লগ, ড্যাশবোর্ড ও অ্যালার্ম

CloudWatch Metrics হলো সময়-ধারাবাহিক সংখ্যা — AWS সেবাগুলো স্বয়ংক্রিয়ভাবে মেট্রিক পাঠায় (EC2 CPU, ALB অনুরোধ, Lambda সময়কাল), আর আপনি **কাস্টম মেট্রিক** পাঠাতে পারেন (যেমন ব্যবসায়িক মেট্রিক: অর্ডার সংখ্যা)। **CloudWatch Logs** হলো লগ সংগ্রহ — Lambda, ECS টাস্ক, API Gateway-এর লগ **লগ গ্রুপ** ও **লগ স্ট্রিমে** জমা হয়; **Logs Insights** দিয়ে SQL-ধাঁচের প্রশ্ন করে খোঁজা যায়। **ড্যাশবোর্ড** হলো মেট্রিক ও লগের একত্রিত চিত্র — প্রোডাকশন ড্যাশবোর্ডে সোনালি সংকেত, ডিপ্লয় চিহ্ন ও ব্যবসায়িক মেট্রিক রাখুন।

অ্যালার্ম: মেট্রিক সীমা ছাড়ালে পদক্ষেপ — SNS বিজ্ঞপ্তি, অটো স্কেলিং, বা Lambda চালানো। ভালো অ্যালার্মের নিয়ম: উপসর্গে নয়, **ব্যবহারকারীর অভিজ্ঞতায়** অ্যালার্ম দিন (যেমন "৫ মিনিটে ত্রুটি হার ৫%-এর বেশি")। CPU ৯০% নিজে সমস্যা নয় — যদি ব্যবহারকারী ঠিক থাকেন।

```
aws cloudwatch put-metric-alarm \
  --alarm-name technova-api-high-errors \
  --metric-name HTTPCode_Target_5XX_Count \
```

```
  --namespace AWS/ApplicationELB \
  --statistic Sum --period 300 --threshold 50 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 1 \
  --alarm-actions arn:aws:sns:ap-south-1:111122223333:technova-alerts \
  --region ap-south-1
aws logs filter-log-events \
  --log-group-name /ecs/technova-api \
  --filter-pattern "ERROR" --region ap-south-1
```

ব্যর্থ হলে: অ্যালার্ম না বাজলে থ্রেশহোল্ড, সময়কাল ও মেট্রিকের নাম যাচাই করুন; লগ না এলে IAM অনুমতি (logs:PutLogEvents) ও লগ গ্রুপের অস্তিত্ব দেখুন। **বিলিং সতর্কতা:** লগ সংরক্ষণ (retention) ও মেট্রিক অ্যালার্ম বিলযোগ্য — লগ retention ৩০ দিনে সীমিত করুন, পুরোনো লগ S3-তে আর্কাইভ করুন — AWS Pricing Calculator-এ যাচাই করুন।

২১.৫ CloudTrail — কে কী করল

CloudTrail হলো AWS API কলের নিরীক্ষা-লগ: কে (কোন IAM ব্যবহারকারী/রোল), কখন, কোন API ডাকল, কোন রিসোর্সে, সফল না ব্যর্থ — সব লেখা থাকে। আগে কোনো রিসোর্স মুছে গেলে বা নীতি বদলে গেলে দায়ী খুঁজে পাওয়া যেত না; CloudTrail সেই প্রশ্নের উত্তর দেয়। **ট্রেইল** (trail) তৈরি করে সব রিজিয়নের ঘটনা S3 বাকেটে পাঠান, লগ ফাইল যাচাইকরণ (validation) চালু করুন — যাতে কেউ লগ বদলাতে না পারে। **CloudTrail Insights** অস্বাভাবিক API কার্যকলাপ ধরে।

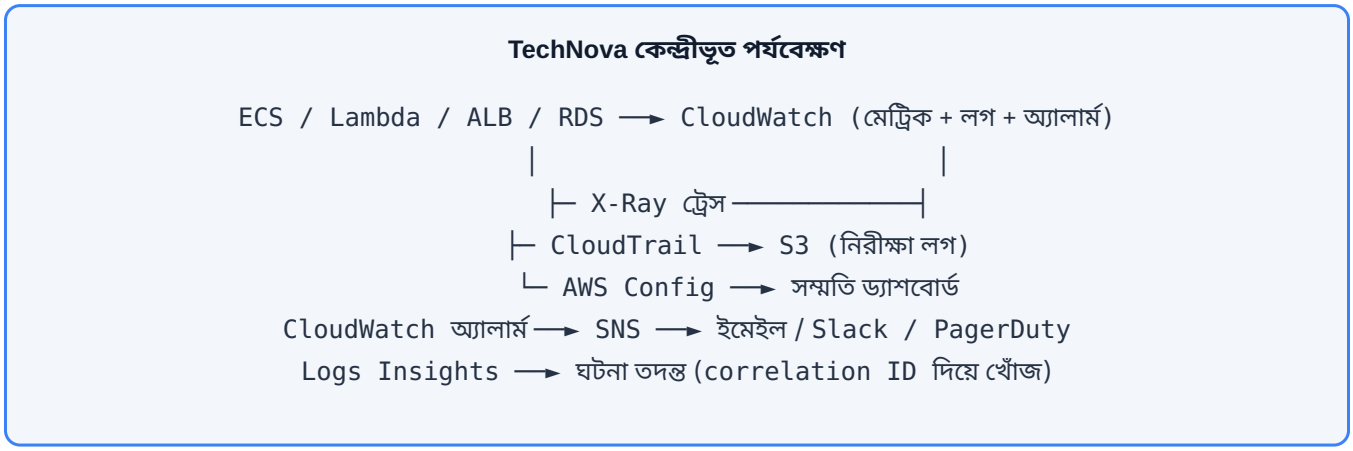
ব্যবহার: নিরাপত্তা ঘটনায় প্রথমে CloudTrail দেখুন — "গত রাতে কে এই বাকেটের নীতি বদলাল?" প্রশ্নের উত্তর এখানে। **নিরাপত্তা:** ট্রেইলের S3 বাকেটে পাবলিক অ্যাক্সেস বন্ধ, সংস্করণ ও MFA Delete বিবেচনা করুন; CloudTrail নিজে বন্ধ করার অনুমতি সীমিত রাখুন।

২১.৬ AWS Config — কনফিগারেশন সম্মতি

AWS Config রিসোর্সের কনফিগারেশনের ইতিহাস রাখে ও **নিয়ম** (rules) দিয়ে পরীক্ষা করে: "S3 বাকেট পাবলিক নয় তো?", "EBS ভলিউম এনক্রিপ্ট করা তো?" নিয়ম ভাঙলে Config তা চিহ্নিত করে, প্রয়োজনে স্বয়ংক্রিয় সংশোধন (remediation) চালায়। CloudTrail বলে "কে বদলাল", Config বলে "এখন অবস্থা কেমন" — দুটি একসঙ্গে সম্পূর্ণ ছবি দেয়। **Conformance Pack** দিয়ে একগুচ্ছ নিয়ম একসঙ্গে প্রয়োগ করুন।

২১.৭ X-Ray ও OpenTelemetry — বিতরণ-করা ট্রেসিং

মাইক্রোসার্ভিসে একটি অনুরোধ পাঁচটি সেবা ঘোরে — কোথায় সময় গেল, তা লগ দিয়ে বোঝা যায় না। **X-Ray** হলো AWS-এর ট্রেসিং সেবা: প্রতিটি অনুরোধে একটি **ট্রেস ID** যায়, প্রতিটি সেবা **সেগমেন্ট** যোগ করে — ফলে সেবা-মানচিত্র (service map) ও সময়-রেখা দেখা যায়। **OpenTelemetry** হলো ভেন্ডর-নিরপেক্ষ মান — একই ইন্সট্রুমেন্টেশন দিয়ে X-Ray বা অন্য প্ল্যাটফর্মে ট্রেস পাঠানো যায়। নতুন কাজে OpenTelemetry দিয়ে শুরু করুন — ভবিষ্যতে প্ল্যাটফর্ম বদলালেও কোড বদলাতে হয় না।



কেন্দ্রীভূত পর্যবেক্ষণ: সব অ্যাকাউন্টের লগ ও মেট্রিক একটি পর্যবেক্ষণ অ্যাকাউন্টে পাঠান (CloudWatch cross-account observability) — মাল্টি-অ্যাকাউন্টে এটাই মান। প্রতিটি লগ লাইনে **correlation ID** (অনুরোধের অনন্য ID) রাখুন — তাহলে এক অনুরোধের সব লগ একসঙ্গে খুঁজে পাবেন।

২১.৮ ঘটনা তদন্ত ও সতর্কতা নকশা

ঘটনা তদন্তের ধাপ: (১) ড্যাশবোর্ডে সোনালি সংকেত দেখুন — কখন শুরু হলো? (২) ডিপ্লয়/পরিবর্তন ইতিহাস মেলান — CloudTrail, CodePipeline; (৩) ট্রেস দিয়ে ধীর সেবা খুঁজুন; (৪) Logs Insights দিয়ে ত্রুটির ধরন গুনুন। **সতর্কতা নকশা:** প্রতিটি অ্যালার্মের সঙ্গে রানবুক (runbook) যুক্ত করুন — "এই অ্যালার্ম বাজলে কী করবেন"। যে অ্যালার্মে কোনো পদক্ষেপ নেই, সেটি অ্যালার্ম নয় — শব্দদূষণ; মুছে ফেলুন বা থ্রেশহোল্ড বদলান। **সতর্কতা-ক্লান্তি** এড়াতে: গুরুত্ব অনুযায়ী স্তর (critical → পেজার, warning → স্ল্যাক), নীরব সময় (suppression) ও সারাংশ বিজ্ঞপ্তি ব্যবহার করুন।

২১.৯ প্রোডাকশন পরিস্থিতি — TechNova-র ধীর সকাল

এক সকালে API লেটেন্সি তিন গুণ বাড়ল, কিন্তু CPU স্বাভাবিক। X-Ray ট্রেস দেখাল সময় যাচ্ছে DynamoDB কলে — একটি নতুন কোডে N+1 কোয়েরি (প্রতিটি অর্ডারে আলাদা কোয়েরি) চালু হয়েছিল। CloudWatch অ্যালার্ম লেটেন্সিতে বেজেছিল, ড্যাশবোর্ডে ডিপ্লয় চিহ্ন দেখে দল দ্রুত আগের সংস্করণে ফিরল। শিক্ষা: লেটেন্সি অ্যালার্ম CPU অ্যালার্মের চেয়ে মূল্যবান; ট্রেস ছাড়া ধীরতার কারণ খুঁজতে ঘণ্টা লাগত।

২১.৯ক Logs Insights, SLO ও কাস্টম মেট্রিক — গভীর পর্যবেক্ষণ

CloudWatch Logs শুধু লগ জমা রাখার জায়গা নয় — **Logs Insights** দিয়ে আপনি লগের ওপর প্রশ্ন চালাতে পারেন, যেন লগ একটি ডেটাবেস। উদাহরণ: গত এক ঘণ্টায় কোন API পাথে সবচেয়ে বেশি ৫০০ ত্রুটি হয়েছে? কোন ব্যবহারকারীর অনুরোধ সবচেয়ে ধীর? প্রশ্নের ভাষা SQL-ধাঁচের — `fields`, `filter`, `stats`, `sort`। একটি উপযোগী প্রশ্ন সংরক্ষণ করে রাখুন — ঘটনার সময় নতুন করে প্রশ্ন লেখার সময় থাকে না। লগে কাঠামো (structured logging, JSON) রাখলে Insights প্রশ্ন অনেক শক্তিশালী হয়: প্রতিটি লগ লাইনে `level`, `service`, `requestId`, `durationMs` ফিল্ড থাকলে "ধীর অনুরোধের শীর্ষ ১০" এক লাইনে বের করা যায়।

```
fields @timestamp, @message
| filter @message like /ERROR/
| stats count() as errorCount by service
| sort errorCount desc
| limit 10
```

SLO (Service Level Objective) হলো পর্যবেক্ষণকে ব্যবসায়িক প্রতিশ্রুতিতে বাঁধা: যেমন "৯৯.৯% অনুরোধ ৫০০ মিলিসেকেন্ডের ভেতরে সফল হবে"। CloudWatch-এ **Service Level Objectives** বৈশিষ্ট্য দিয়ে SLO সংজ্ঞায়িত করুন, **ত্রুটি বাজেট** (error budget) দেখুন — বাজেট শেষ হলে নতুন বৈশিষ্ট্যের বদলে নির্ভরযোগ্যতায় কাজ করুন। এটি SRE (Site Reliability Engineering) অনুশীলনের কেন্দ্র: অ্যালার্ম শুধু মেট্রিকে নয়, বাজেট পোড়ার হারে (burn rate) দিন — "বাজেট ১৪ দিনে শেষ হয়ে যাবে" এমন অ্যালার্ম অনেক বেশি কার্যকর।

কাস্টম মেট্রিক ও এমবেডেড মেট্রিক: ব্যবসায়িক ঘটনা — অর্ডার তৈরি, অর্থপ্রদান সফল, সাইনআপ — CloudWatch-এ কাস্টম মেট্রিক হিসেবে পাঠান। সরাসরি `PutMetricData` ডাকলে খরচ ও জটিলতা বাড়ে; ভালো পথ **Embedded Metric Format (EMF)** — লগ লাইনে JSON হিসেবে মেট্রিক লিখুন, CloudWatch স্বয়ংক্রিয়ভাবে তা মেট্রিকে রূপান্তর করে। এতে একই লগ থেকে তদন্ত ও ড্যাশবোর্ড দুটোই হয়। **সতর্কতা:** উচ্চ-কার্ডিনালিটি মাত্রা (dimension) — যেমন প্রতিটি ব্যবহারকারীর ID মাত্রা হিসেবে — মেট্রিকের সংখ্যা বিস্ফোরিত করে খরচ বাড়ায়; মাত্রা হিসেবে শুধু সীমিত মানের জিনিস (সেবা, পরিবেশ, অঞ্চল) ব্যবহার করুন।

২১.১০ নিরাপত্তা, খরচ, ভুল ও সমস্যা সমাধান

নিরাপত্তা: লগে সিক্রেট বা PII ছাপাবেন না; লগ গ্রুপে এনক্রিপশন; CloudTrail বাকেট সুরক্ষিত; অ্যালার্মের SNS টপিকে শুধু প্রয়োজনীয় গ্রাহক। **খরচ:** বিস্তারিত (detailed) মনিটরিং, উচ্চ-রেজোলিউশন মেট্রিক ও দীর্ঘ লগ retention ব্যয়বহুল — retention নীতি ও স্যাম্পলিং ঠিক করুন — AWS Pricing Calculator-এ যাচাই করুন।

> **সাধারণ ভুল:** (১) শুধু CPU-তে অ্যালার্ম — ব্যবহারকারীর অভিজ্ঞতায় নয়; (২) লগ retention অসীম; (৩) correlation ID না রাখা; (৪) অ্যালার্মে রানবুক না থাকা; (৫) CloudTrail বন্ধ বা এক রিজিয়নে সীমিত।

সমস্যা সমাধান: মেট্রিক দেখা যাচ্ছে না → নেমস্পেস/নাম ও সময়সীমা যাচাই; লগ খুঁজে পাচ্ছেন না → লগ গ্রুপ, retention ও IAM অনুমতি; ট্রেস ভাঙা → স্যাম্পলিং হার ও SDK কনফিগারেশন; অ্যালার্ম বাজছে না → evaluation period ও "insufficient data" আচরণ।

২১.১১ সারাংশ

CloudWatch মেট্রিক ও লগ দেয়, CloudTrail "কে কী করল" বলে, Config "এখন কেমন আছে" বলে, X-Ray অনুরোধের যাত্রা দেখায়। সোনালি সংকেতে অ্যালার্ম, correlation ID-তে তদন্ত, রানবুকে প্রতিক্রিয়া — এই তিনটি মিলে প্রোডাকশন পর্যবেক্ষণ সম্পূর্ণ হয়।

২১.১২ পর্যালোচনা প্রশ্ন

১. মেট্রিক, লগ ও ট্রেসের পার্থক্য কী? ২. সোনালি সংকেত চারটি কী? ৩. CloudTrail বনাম Config — কে কী বলে? ৪. Logs Insights কী কাজে লাগে? ৫. correlation ID কেন জরুরি? ৬. ভালো অ্যালার্মের বৈশিষ্ট্য কী? ৭. সতর্কতা-ক্লান্তি কীভাবে এড়াবেন? ৮. X-Ray বনাম OpenTelemetry। ৯. লগ retention কীভাবে খরচ নিয়ন্ত্রণ করে? ১০. ঘটনা তদন্তের প্রথম তিন ধাপ কী?

২১.১৩ অনুশীলন

১. একটি ড্যাশবোর্ড বানান: ALB লেটেন্সি, 5XX, Lambda ত্রুটি। ২. ত্রুটি-হারে অ্যালার্ম + SNS বিজ্ঞপ্তি সাজান। ৩. Logs Insights দিয়ে গত ঘণ্টার ERROR গুনুন। ৪. CloudTrail-এ গত ২৪ ঘণ্টায় IAM নীতি পরিবর্তন খুঁজুন।

অধ্যায় ২২ • PART V

Chapter 22 — AWS Security Engineering

২২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) IAM নিরাপত্তা নীতি (ন্যূনতম সুযোগ, MFA, রোল) প্রয়োগ করতে; (২) KMS দিয়ে এনক্রিপশন সাজাতে; (৩) Secrets Manager ও Systems Manager ব্যবহার করতে; (৪) GuardDuty, Security Hub ও Inspector দিয়ে হুমকি শনাক্ত করতে; (৫) নিরাপত্তা ফলাফল (finding) তদন্ত ও প্রতিক্রিয়া জানাতে; (৬) ডেটা, নেটওয়ার্ক ও সরবরাহ-শৃঙ্খল (supply chain) সুরক্ষা নকশা করতে; (৭) নিরাপত্তা নিরীক্ষা চালাতে।

২২.২ পূর্বজ্ঞান ঝালাই

আগের অধ্যায়গুলোতে IAM রোল, সিকিউরিটি গ্রুপ, KMS-এর নাম, Secrets Manager ও CloudTrail এসেছে। এখানে আমরা সেগুলোকে একটি পূর্ণ নিরাপত্তা কাঠামোয় বাঁধব। মনে রাখুন AWS-এর **ভাগ করা দায়িত্ব মডেল** (shared responsibility): AWS সুরক্ষিত রাখে ক্লাউড নিজেকে (হার্ডওয়্যার, হাইপারভাইজার); আপনি সুরক্ষিত রাখেন ক্লাউডে আপনার যা আছে (ডেটা, IAM, নেটওয়ার্ক কনফিগ, অ্যাপ কোড)।

২২.৩ কেন নিরাপত্তা প্রকৌশল? হুমকির চিত্র

TechNova-র এক ডেভেলপারের অ্যাক্সেস কী GitHub-এ ফাঁস হয়েছিল — কয়েক ঘণ্টায় কেউ তা দিয়ে EC2 চালিয়ে ক্রিপ্টো মাইনিং শুরু করল। বিল হাজার ডলার ছাড়াল, তারপর অ্যালার্ম বাজল। এই ঘটনা শেখায়: নিরাপত্তা কোনো এককালীন সেটিং নয় — এটি স্তরে স্তরে প্রতিরক্ষা (defense in depth): পরিচয়, নেটওয়ার্ক, ডেটা, অ্যাপ ও পর্যবেক্ষণ — প্রতিটি স্তরে নিয়ন্ত্রণ। একটি স্তর ভাঙলেও পরের স্তর ঠেকায়।

> **মূল ধারণা: ন্যূনতম সুযোগ** (least privilege) — প্রতিটি পরিচয় (ব্যবহারকারী, রোল, সেবা) শুধু তার কাজের জন্য প্রয়োজনীয় অনুমতি পাবে, আর কিছু নয়। **শূন্য-বিশ্বাস** (zero trust) — ভেতরের নেটওয়ার্ককেও বিশ্বাস করবেন না; প্রতিটি অনুরোধ প্রমাণীকরণ ও অনুমোদন যাচাই করে।

২২.৪ IAM নিরাপত্তা — গভীরে

IAM নিরাপত্তার নিয়ম: (১) রুট অ্যাকাউন্টে MFA চালু করুন, রুটের অ্যাক্সেস কী কখনো বানাবেন না, রোজকার কাজে রুট ব্যবহার করবেন না; (২) মানুষের জন্য IAM ব্যবহারকারী + MFA, অস্থায়ী কাজে রোল; (৩) দীর্ঘস্থায়ী অ্যাক্সেস কী এড়িয়ে চলুন — যেখানে সম্ভব OIDC বা Instance Role; (৪) **পারমিশন বাউন্ডারি** (permissions boundary) দিয়ে সর্বোচ্চ সীমা বেঁধে দিন; (৫) **Access Analyzer** দিয়ে অব্যবহৃত অনুমতি ও বাইরে ভাগ করা রিসোর্স খুঁজুন; (৬) নিয়মিত **ক্রেডেনশিয়াল রিপোর্ট** দেখুন — পুরোনো কী, MFA ছাড়া ব্যবহারকারী।

```
aws iam get-credential-report --region ap-south-1
aws iam generate-service-last-accessed-details \
  --arn arn:aws:iam::111122223333:user/CHANGE-ME-user
aws accessanalyzer list-findings \
  --analyzer-arn arn:aws:access-analyzer:ap-south-1:111122223333:analyzer/CHANGE-ME \
  --region ap-south-1
```

নীতি লেখার সময় **শর্ত** (condition) ব্যবহার করুন: শুধু নির্দিষ্ট IP থেকে, শুধু MFA-সহ, শুধু নির্দিষ্ট ট্যাগযুক্ত রিসোর্সে। **SCP** (Service Control Policy) — Organizations-এ অ্যাকাউন্ট-স্তরে সর্বোচ্চ সীমা (যেমন "কেউ CloudTrail বন্ধ করতে পারবে না")।

২২.৫ KMS, Secrets Manager ও Systems Manager

KMS (Key Management Service) হলো এনক্রিপশন চাবির পরিচালিত ভান্ডার: **CMK** (customer managed key) দিয়ে S3, EBS, RDS, Secrets Manager এনক্রিপ্ট করুন; চাবির ব্যবহার CloudTrail-এ লেখা হয় — কে কখন ডিক্রিপ্ট করল, তা নিরীক্ষা করা যায়। চাবি ঘোরানো (rotation) চালু করুন; চাবি মুছতে সাবধানে — মুছলে ডেটা চিরতরে অগম্য হয়।

Secrets Manager হলো সিক্রেটের জীবনচক্র: সংরক্ষণ, স্বয়ংক্রিয় ঘোরানো (RDS পাসওয়ার্ডের জন্য অন্তর্নিহিত), অ্যাক্সেস নিরীক্ষা। **Systems Manager Parameter Store** হলো হালকা বিকল্প — কনফিগ ও ছোট সিক্রেটের জন্য (SecureString)। নিয়ম: কোডে বা Git-এ সিক্রেট নয়; পরিবেশ ভেরিয়েবলে সিক্রেট নয় — রানটাইমে Secrets Manager থেকে নিন। **Systems Manager Session Manager** দিয়ে SSH পোর্ট না খুলে EC2-তে নিরাপদে প্রবেশ করুন — Bastion হোস্টের পুরোনো পথ এড়িয়ে চলুন।

২২.৬ GuardDuty, Security Hub, Inspector — শনাক্তকরণ

GuardDuty হলো হুমকি শনাক্তকরণ: CloudTrail, VPC Flow Logs ও DNS লগ বিশ্লেষণ করে অস্বাভাবিক আচরণ ধরে — যেমন ফাঁস হওয়া কী দিয়ে অস্বাভাবিক রিজিয়নে API কল, বা EC2 থেকে পরিচিত খারাপ IP-তে সংযোগ। **Security Hub** হলো কেন্দ্রীয় ড্যাশবোর্ড — GuardDuty, Inspector, Config ও অন্যান্য সেবার ফলাফল (finding) এক জায়গায়, CIS বেঞ্চমার্কের বিরুদ্ধে স্কোরসহ। **Inspector** হলো দুর্বলতা স্ক্যানার — EC2 ও কনটেইনার ইমেজে পরিচিত দুর্বলতা (CVE) ও নেটওয়ার্ক উন্মুক্ততা খুঁজে বের করে।

TechNova নিরাপত্তা স্তর

বাইরে: `WAF + Shield (DDoS)` ⟶ `ALB` ⟶ প্রাইভেট সাবনেট (`ECS/RDS`)
পরিচয়: `IAM + MFA + OIDC + SCP`
ডেটা: `KMS` এনক্রিপশন + `Secrets Manager`
শনাক্তকরণ: `GuardDuty + Security Hub + Inspector + Config`
নিরীক্ষা: `CloudTrail` (সব রিজিয়ন) ⟶ `S3` (লক করা বাকেট)

ফলাফল প্রতিক্রিয়া: প্রতিটি finding-এর তীব্রতা (severity) দেখুন; critical-এ তাৎক্ষণিক পদক্ষেপ — আক্রান্ত কী নিষ্ক্রিয় করুন, ইনস্ট্যান্স বিচ্ছিন্ন করুন, CloudTrail দিয়ে পরিসর যাচাই করুন। Security Hub-এ finding-এর জীবনচক্র সামলান: নতুন → তদন্তাধীন → সমাধান; ভুয়া সতর্কতা (false positive) নথিভুক্ত করে দমন (suppress) করুন।

২২.৭ ডেটা, নেটওয়ার্ক ও সরবরাহ-শৃঙ্খল সুরক্ষা

ডেটা সুরক্ষা: বিশ্রামে (at rest) এনক্রিপশন — S3, EBS, RDS-এ KMS; চলাচলে (in transit) TLS — ALB ও API Gateway-এ HTTPS বাধ্যতামূলক; S3 বাকেটে পাবলিক অ্যাক্সেস বন্ধ, সংস্করণ ও Object Lock বিবেচনা করুন। **নেটওয়ার্ক সুরক্ষা:** প্রাইভেট সাবনেট, সিকিউরিটি গ্রুপে ন্যূনতম পোর্ট, NACL দ্বিতীয় স্তর, VPC Flow Logs চালু, WAF দিয়ে সাধারণ আক্রমণ (SQLi, XSS) ঠেকান, Shield দিয়ে DDoS সুরক্ষা। **সরবরাহ-শৃঙ্খল:** নির্ভরতা (dependency)

স্ক্যান করুন (npm audit, Dependabot), ইমেজ স্ক্যানিং চালু করুন, CI-তে স্বাক্ষরিত আর্টিফ্যাক্ট যাচাই করুন, তৃতীয় পক্ষের প্যাকেজ পিন করুন।

২২.৮ প্রোডাকশন পরিস্থিতি — TechNova-র ফাঁস হওয়া কী

GuardDuty সতর্ক করল: একটি অ্যাক্সেস কী দিয়ে অস্বাভাবিক রিজিয়ন থেকে EC2 চালু হচ্ছে। প্রতিক্রিয়া ছিল: (১) কীটি তাৎক্ষণিক নিষ্ক্রিয়; (২) CloudTrail দিয়ে কী দিয়ে কী করা হয়েছে যাচাই; (৩) অননুমোদিত EC2 বন্ধ, বিল পরীক্ষা; (৪) Git ইতিহাস থেকে কী সরানো ও ঘোরানো; (৫) OIDC-তে স্থানান্তর করে স্থায়ী কী বাদ দেওয়া। শিক্ষা: শনাক্তকরণ দ্রুত হলে ক্ষতি কমে — GuardDuty চালু রাখা ও সতর্কতায় প্রতিক্রিয়া জানা থাকা জরুরি।

২২.৮ক WAF, Shield ও ঘটনা প্রতিক্রিয়া — বাইরের আক্রমণ ঠেকানো

আগের অংশে পরিচয় ও ডেটা সুরক্ষা দেখেছেন; এবার বাইরের আক্রমণের সামনের সারি। **AWS WAF** (Web Application Firewall) হলো HTTP স্তরের ফিল্টার: ALB, CloudFront বা API Gateway-এর সামনে বসে অনুরোধ পরীক্ষা করে — SQL ইনজেকশন, ক্রস-সাইট স্ক্রিপ্টিং (XSS), পরিচিত খারাপ বট, ভৌগোলিক ব্লক, হার-সীমা (rate limit)। নিয়ম লেখা হয় **ওয়েব ACL**-এ: প্রথমে Count মোডে চালিয়ে দেখুন কোন নিয়ম কত অনুরোধ ধরছে (ভুল নিয়মে বৈধ ব্যবহারকারী আটকে যেতে পারে), তারপর Block মোডে যান। AWS পরিচালিত নিয়ম-সেট (managed rule groups) দিয়ে শুরু করুন — সাধারণ আক্রমণের জন্য নিজে নিয়ম লেখার দরকার নেই।

AWS Shield হলো DDoS সুরক্ষা: Shield Standard স্বয়ংক্রিয়ভাবে সব AWS গ্রাহককে দেওয়া হয় (নেটওয়ার্ক ও পরিবহন স্তরের আক্রমণের বিরুদ্ধে); **Shield Advanced** হলো অর্থপ্রদানকারী স্তর — উন্নত শনাক্তকরণ, আক্রমণের সময় ২৪/৭ সহায়তা দল (DDoS Response Team), ও আক্রমণের কারণে বাড়তি খরচের সুরক্ষা। TechNova-র মতো ছোট দলের জন্য Shield Standard + WAF হার-সীমা সাধারণত যথেষ্ট; Shield Advanced তখনই, যখন DDoS মানে সরাসরি আয় বন্ধ হওয়া।

ঘটনা প্রতিক্রিয়া পরিকল্পনা (Incident Response Plan): একটি নিরাপত্তা ঘটনায় প্রথম এক ঘণ্টা সবচেয়ে মূল্যবান — তাই আগে থেকে ঠিক করে রাখুন: কে ঘটনা পরিচালক, কাকে কখন জানাবেন, কীভাবে প্রমাণ সংরক্ষণ করবেন (লগ, স্ন্যাপশট — মুছে ফেলবেন না, বিচ্ছিন্ন করবেন), কখন AWS Support-কে জানাবেন। ছয়টি ধাপ মনে রাখুন: প্রস্তুতি → শনাক্তকরণ → নিয়ন্ত্রণ (containment) → নির্মূল (eradication) → পুনরুদ্ধার → শিক্ষা (lessons learned)। নিয়ন্ত্রণের প্রথম পদক্ষেপ প্রায়ই সহজ: আক্রান্ত কী নিষ্ক্রিয় করা, আক্রান্ত ইনস্ট্যান্সের সিকিউরিটি গ্রুপ বদলে বিচ্ছিন্ন করা, সন্দেহজনক IP WAF-এ ব্লক করা। **প্রমাণ সংরক্ষণ:** আক্রান্ত রিসোর্স মুছে ফেলবেন না — তার EBS স্ন্যাপশট নিন, CloudTrail লগ সংরক্ষণ করুন; তদন্তের জন্য এগুলো অপরিহার্য। **যোগাযোগ:** গ্রাহককে জানাতে হলে সত্য ও সুনির্দিষ্ট বলুন — কী ঘটেছে, কী প্রভাব, কী করা হচ্ছে; অনুমান প্রচার করবেন না।

Amazon Macie একটি অতিরিক্ত সেবা যা এই অধ্যায়ের ছবি সম্পূর্ণ করে: এটি S3-তে সংবেদনশীল ডেটা (ব্যক্তিগত তথ্য, ক্রেডিট কার্ড নম্বর) খুঁজে বের করে ও শ্রেণিবদ্ধ করে — TechNova-র গ্রাহক ডেটা কোন বাকেটে আছে, তা না জানলে সুরক্ষাও দেওয়া যায় না। Macie চালু করে নিয়মিত রিপোর্ট দেখুন; সংবেদনশীল ডেটা পাওয়া গেলে সেই বাকেটে এনক্রিপশন, অ্যাক্সেস লগ ও পাবলিক ব্লক যাচাই করুন।

> **WARNING:** নিরাপত্তা ঘটনায় সবচেয়ে বড় ভুল হলো তাড়াহুড়ো করে প্রমাণ মুছে ফেলা — আক্রান্ত ইনস্ট্যান্স "পরিষ্কার" করতে গিয়ে লগ মুছে ফেললে তদন্ত অসম্ভব হয়ে যায়। প্রথমে স্ন্যাপশট ও লগ সংরক্ষণ করুন, তারপর পরিষ্কার করুন।

২২.৯ নিরাপত্তা নিরীক্ষা, ভুল ও সমস্যা সমাধান

নিরীক্ষা: নিয়মিত Security Hub স্কোর, Access Analyzer ফলাফল, ক্রেডেনশিয়াল রিপোর্ট ও Config সম্মতি পর্যালোচনা করুন; ত্রৈমাসিক টেবিল-টপ অনুশীলন (tabletop exercise) করুন — "কী ফাঁস হলে কী করব?" **সাধারণ ভুল:** (১) রুট অ্যাকাউন্ট দৈনন্দিন ব্যবহার; (২) S3 বাকেট পাবলিক; (৩) সিকিউরিটি গ্রুপে 0.0.0.0/0 দিয়ে SSH; (৪) Git-এ সিক্রেট; (৫) GuardDuty বন্ধ রাখা; (৬) KMS চাবি ছাড়া এনক্রিপশন। **সমস্যা সমাধান:** অনুমতি ত্রুটি → CloudTrail-এ AccessDenied ঘটনা, IAM Policy Simulator; এনক্রিপশন ত্রুটি → KMS কী নীতি ও গ্রান্ট; GuardDuty finding → তীব্রতা, প্রভাবিত রিসোর্স, CloudTrail মিলিয়ে তদন্ত।

২২.১০ সারাংশ

নিরাপত্তা মানে স্তরে স্তরে প্রতিরক্ষা: IAM-এ ন্যূনতম সুযোগ, KMS-এ এনক্রিপশন, Secrets Manager-এ সিক্রেট, GuardDuty/Security Hub/Inspector-এ শনাক্তকরণ, CloudTrail-এ নিরীক্ষা। একটি স্তর ভাঙলেও পরের স্তর ঠেকায় — এই নকশাই প্রোডাকশন নিরাপত্তা।

২২.১১ পর্যালোচনা প্রশ্ন

১. ভাগ করা দায়িত্ব মডেল কী? ২. ন্যূনতম সুযোগ কীভাবে প্রয়োগ করবেন? ৩. GuardDuty বনাম Security Hub বনাম Inspector। ৪. KMS চাবি মুছলে কী হয়? ৫. Secrets Manager বনাম Parameter Store। ৬. Session Manager কেন Bastion-এর চেয়ে ভালো? ৭. SCP কী কাজে লাগে? ৮. ফাঁস হওয়া কী পেলে প্রথম পাঁচ পদক্ষেপ কী? ৯. WAF কী ঠেকায়? ১০. Access Analyzer কী খুঁজে দেয়?

২২.১২ অনুশীলন

১. ক্রেডেনশিয়াল রিপোর্ট দেখে MFA ছাড়া ব্যবহারকারী ও পুরোনো কী খুঁজুন। ২. GuardDuty ও Security Hub চালু করে স্কোর দেখুন। ৩. একটি S3 বাকেটে KMS এনক্রিপশন ও পাবলিক অ্যাক্সেস ব্লক সাজান। ৪. একটি টেবিল-টপ অনুশীলন লিখুন: "ফাঁস হওয়া কী" পরিস্থিতিতে প্রতিক্রিয়া পরিকল্পনা।

অধ্যায় ২৩ • PART V

Chapter 23 — Reliability, Backup and Disaster Recovery

২৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) উচ্চ-উপলভ্যতা (high availability) ও দোষ-সহনশীলতা (fault tolerance) আলাদা করতে; (২) ব্যাকআপ কৌশল নকশা করতে; (৩) RPO ও RTO সংজ্ঞায়িত ও পরিমাপ করতে; (৪) চারটি পুনরুদ্ধার কৌশল (backup & restore, pilot light, warm standby, multi-site active/active) তুলনা করতে; (৫) ক্রস-AZ ও ক্রস-Region পুনরুদ্ধার সাজাতে; (৬) ডেটাবেস পুনরুদ্ধার করতে; (৭) ঘটনা প্রতিক্রিয়া, রানবুক ও পোস্টমর্টেম লিখতে; (৮) নির্ভরযোগ্যতা পরীক্ষা (game day, chaos) চালাতে।

২৩.২ পূর্বজ্ঞান ঝালাই

মনে রাখুন: AZ (Availability Zone) হলো একটি রিজিয়নের ভেতরে বিচ্ছিন্ন ডেটা সেন্টার; রিজিয়ন হলো ভৌগোলিক এলাকা। RDS Multi-AZ, ALB মাল্টি-AZ, S3-এর ১১-নাইন স্থায়িত্ব — এগুলো আগের অধ্যায়ে এসেছে। এখানে আমরা শিখব: কোনো কিছু ভাঙলে কত দ্রুত ও কতটুকু ডেটা নিয়ে ফিরবেন — এবং তার জন্য আগে থেকে কী প্রস্তুতি নিতে হয়।

২৩.৩ কেন নির্ভরযোগ্যতা? ব্যর্থতা অবশ্যম্ভাবী

TechNova-র ডেটাবেস একবার একটি AZ-এর বিভ্রাটে বন্ধ হয়ে গেল — কারণ ডেটাবেস ছিল Single-AZ। গ্রাহক দুই ঘণ্টা সেবা পাননি। শিক্ষা: প্রশ্ন "ভাঙবে কি না" নয়, প্রশ্ন "ভাঙলে কী হবে"। **উচ্চ-উপলভ্যতা** মানে ব্যর্থতা সত্ত্বেও সেবা চলা (যেমন Multi-AZ); **দোষ-সহনশীলতা** মানে কোনো উপাদান ভাঙলেও ব্যবহারকারী কিছু টের পায় না (যেমন ALB-এর পেছনে একাধিক টাস্ক)। নির্ভরযোগ্যতা কোনো বৈশিষ্ট্য নয় — এটি নকশার সিদ্ধান্তের সমষ্টি।

> **মূল ধারণা: RPO** (Recovery Point Objective) — কতটুকু ডেটা হারানো মেনে নেওয়া যায় (যেমন ৫ মিনিটের ডেটা); **RTO** (Recovery Time Objective) — কতক্ষণে সেবা ফিরবে (যেমন ১ ঘণ্টা)। RPO/RTO যত ছোট, খরচ তত বেশি — ব্যবসার সঙ্গে আলোচনা করে ঠিক করুন, প্রযুক্তিবিদ একা ঠিক করবেন না।

২৩.৪ ব্যাকআপ কৌশল — ৩-২-১ নিয়ম

৩-২-১ নিয়ম: ডেটার ৩টি কপি, ২টি ভিন্ন মাধ্যমে, ১টি ভিন্ন স্থানে। AWS-এ: RDS স্বয়ংক্রিয় ব্যাকআপ (পয়েন্ট-ইন-টাইম পুনরুদ্ধার), স্ন্যাপশট, S3 সংস্করণ ও **AWS Backup** দিয়ে কেন্দ্রীয় নীতি (কোন রিসোর্স, কত ঘন ঘন, কতদিন রাখা হবে)। **ব্যাকআপ পরীক্ষা** বাধ্যতামূলক — ব্যাকআপ আছে মানেই পুনরুদ্ধার হবে, তা নয়; নিয়মিত পুনরুদ্ধার অনুশীলন করুন। **অপরিবর্তনীয় ব্যাকআপ:** S3 Object Lock বা ভল্ট লক দিয়ে ব্যাকআপ মুছে ফেলা বা বদলানো ঠেকান — র‍্যানসমওয়্যার থেকে রক্ষা পেতে।

```
aws backup create-backup-plan --backup-plan file://plan.json --region ap-south-1
aws rds create-db-snapshot \
  --db-instance-identifier technova-db \
  --db-snapshot-identifier technova-db-manual-20261109 \
  --region ap-south-1
aws rds restore-db-instance-from-db-snapshot \
```

```
  --db-instance-identifier technova-db-restored \
  --db-snapshot-identifier technova-db-manual-20261109 \
  --region ap-south-1
```

২৩.৫ চারটি পুনরুদ্ধার কৌশল

কৌশল	RTO / RPO	খরচ	কীভাবে
Backup & Restore	ঘণ্টা / ঘণ্টা	সবচেয়ে কম	ব্যাকআপ থেকে নতুন করে তৈরি
Pilot Light	দশ মিনিট-ঘণ্টা / মিনিট	কম	মূল ডেটা অন্য রিজিয়নে সিঙ্ক, সেবা বন্ধ; প্রয়োজনে চালু
Warm Standby	মিনিট / সেকেন্ড-মিনিট	মাঝারি	ছোট আকারে সেবা চলমান, প্রয়োজনে স্কেল
Multi-site Active/Active	সেকেন্ড / প্রায় শূন্য	সবচেয়ে বেশি	দুই রিজিয়নে পূর্ণ সেবা, ট্রাফিক ভাগ

TechNova-র জন্য: মূল সেবায় Pilot Light (RDS ক্রস-Region রিড রেপ্লিকা + S3 ক্রস-Region প্রতিলিপি), গুরুত্বপূর্ণ পেমেন্ট পথে Warm Standby বিবেচনা করুন। Active/Active শুধু তখনই, যখন কয়েক মিনিটের বিভ্রাটও ব্যবসায়িকভাবে অসহনীয়।

২৩.৬ ক্রস-AZ ও ক্রস-Region পুনরুদ্ধার

ক্রস-AZ: ALB দুই AZ-তে, ECS টাস্ক দুই AZ-তে, RDS Multi-AZ — একটি AZ গেলে অন্যটি চালিয়ে যায়। **ক্রস-Region:** S3 Cross-Region Replication, RDS ক্রস-Region রিড রেপ্লিকা (প্রয়োজনে প্রোমোট করে প্রাইমারি বানানো যায়), Route 53 হেলথ চেক + ফেইলওভার রাউটিং, DynamoDB গ্লোবাল টেবিল। **DNS ফেইলওভার:** Route 53 প্রাইমারি রিজিয়নের স্বাস্থ্য পরীক্ষা করে, ব্যর্থ হলে সেকেন্ডারিতে ট্রাফিক পাঠায় — TTL ছোট রাখুন (যেমন ৬০ সেকেন্ড)।

TechNova DR — Pilot Light

```
        প্রাইমারি (ap-south-1)          DR (ap-southeast-1)
     ALB + ECS (3 টাস্ক)                    ECS (বন্ধ, IaC প্রস্তুত)
RDS Multi-AZ ──রেপ্লিকেশন──►          RDS রিড রেপ্লিকা (প্রোমোটযোগ্য)
      S3 ──CRR──►                            S3 (প্রতিলিপি)
          Route 53 হেলথ চেক: ব্যর্থ হলে ──► DR রিজিয়নে ট্রাফিক
RPO: ~মিনিট (রেপ্লিকেশন বিলম্ব) • RTO: ~৩০ মিনিট (IaC + প্রোমোশন)
```

২৩.৭ ডেটাবেস পুনরুদ্ধার, ঘটনা প্রতিক্রিয়া ও পোস্টমর্টেম

ডেটাবেস পুনরুদ্ধার: RDS পয়েন্ট-ইন-টাইম পুনরুদ্ধার (PITR) — নির্দিষ্ট সময়ে ফেরা যায়; স্ন্যাপশট থেকে পুনরুদ্ধার; রেপ্লিকা প্রোমোশন। পুনরুদ্ধারের পর অ্যাপের সংযোগ-স্ট্রিং বদলাতে হবে — তাই ডেটাবেস ঠিকানা Secrets Manager-এ রাখুন, কোডে হার্ডকোড করবেন না। **ঘটনা প্রতিক্রিয়া:** ভূমিকা ঠিক করুন (ঘটনা পরিচালক, যোগাযোগ, প্রযুক্তি), যোগাযোগের চ্যানেল (একটি স্ল্যাক চ্যানেল), ও অবস্থা পাতা (status page)। **রানবুক:** প্রতিটি পরিচিত ব্যর্থতার জন্য ধাপে ধাপে নির্দেশনা — "RDS বিভ্রাট হলে..."। **পোস্টমর্টেম:** দোষারোপহীন (blameless) পর্যালোচনা — কী হলো, কেন হলো, কী শিখলাম, কী বদলাব; পদক্ষেপের মালিক ও সময়সীমা ঠিক করুন।

২৩.৮ নির্ভরযোগ্যতা পরীক্ষা — গেম ডে

পরিকল্পনা না পরীক্ষা করলে তা কাগজেই থাকে। **গেম ডে** (game day) হলো নিয়ন্ত্রিত অনুশীলন: স্টেজিংয়ে একটি AZ বন্ধ করার ভান করুন, একটি টাস্ক মেরে ফেলুন, ডেটাবেস ফেইলওভার করুন — দেখুন সিস্টেম কী করে। **AWS Fault Injection Service (FIS)** দিয়ে নিয়ন্ত্রিতভাবে ব্যর্থতা ঢোকানো যায়। ছোট থেকে শুরু করুন: প্রথমে একটি ECS টাস্ক বন্ধ করা, তারপর AZ ফেইলওভার — প্রোডাকশনে যাওয়ার আগে স্টেজিংয়ে আত্মবিশ্বাস তৈরি করুন।

২৩.৯ প্রোডাকশন পরিস্থিতি — TechNova-র AZ বিভ্রাট

এক সকালে ap-south-1a-তে বিভ্রাট হলো। TechNova-র ALB ও ECS দুই AZ-তে ছিল বলে সেবা চলল; RDS Multi-AZ স্বয়ংক্রিয়ভাবে 1b-তে ফেইলওভার করল (প্রায় ৬০ সেকেন্ডের বিলম্ব); গ্রাহক শুধু একটি সংক্ষিপ্ত ধীরতা টের পেলেন। পরে পোস্টমর্টেমে দেখা গেল একটি ব্যাচ কাজ Single-AZ-এ ছিল — তা Multi-AZ করা হলো। শিক্ষা: নির্ভরযোগ্যতা মানে সব উপাদান Multi-AZ — একটি Single-AZ উপাদানই পুরো শৃঙ্খলের দুর্বলতম বিন্দু।

২৩.১০ নিরাপত্তা, খরচ, ভুল ও সমস্যা সমাধান

নিরাপত্তা: ব্যাকআপ এনক্রিপ্ট করুন, ব্যাকআপ মুছে ফেলার অনুমতি সীমিত করুন, DR রিজিয়নেও একই নিরাপত্তা নীতি প্রয়োগ করুন। **খরচ:** Multi-AZ ও ক্রস-Region প্রতিলিপি বিলযোগ্য — ডেটা ট্রান্সফার ও রেপ্লিকা খরচ হিসাব করুন — AWS Pricing Calculator-এ যাচাই করুন; ব্যাকআপ retention অতিরিক্ত দীর্ঘ রাখবেন না।

> **সাধারণ ভুল:** (১) ব্যাকআপ নেওয়া কিন্তু কখনো পুনরুদ্ধার পরীক্ষা না করা; (২) RPO/RTO ব্যবসার সঙ্গে ঠিক না করা; (৩) Single-AZ ডেটাবেস; (৪) DR রানবুক না থাকা; (৫) পুনরুদ্ধারের পর সংযোগ-স্ট্রিং বদলাতে ভুলে যাওয়া।

সমস্যা সমাধান: ফেইলওভার হচ্ছে না → হেলথ চেক, Route 53 TTL, রেপ্লিকা বিলম্ব দেখুন; পুনরুদ্ধার ধীর → স্ন্যাপশট আকার, IOPS, IaC সময় পরীক্ষা; ব্যাকআপ ব্যর্থ → AWS Backup কাজের ইতিহাস ও IAM অনুমতি।

২৩.১১ সারাংশ

RPO বলে কতটুকু ডেটা হারানো চলবে, RTO বলে কতক্ষণে ফিরবেন। Backup & Restore থেকে Active/Active — খরচ ও গতির বিনিময়। Multi-AZ দৈনন্দিন নির্ভরযোগ্যতা, ক্রস-Region বিপর্যয় পুনরুদ্ধার; আর পরীক্ষা ছাড়া কোনো পরিকল্পনাই নির্ভরযোগ্য নয়।

২৩.১২ পর্যালোচনা প্রশ্ন

১. RPO বনাম RTO। ২. উচ্চ-উপলভ্যতা বনাম দোষ-সহনশীলতা। ৩. চারটি পুনরুদ্ধার কৌশলের তুলনা। ৪. ৩-২-১ নিয়ম কী? ৫. RDS PITR কীভাবে কাজ করে? ৬. Route 53 ফেইলওভার কীভাবে হয়? ৭. গেম ডে কেন দরকার? ৮. দোষারোপহীন পোস্টমর্টেম কেন? ৯. Pilot Light কখন যথেষ্ট? ১০. ব্যাকআপ পরীক্ষা কেন বাধ্যতামূলক?

২৩.১৩ অনুশীলন

১. RDS স্ন্যাপশট নিয়ে নতুন ইনস্ট্যান্সে পুনরুদ্ধার করুন। ২. AWS Backup পরিকল্পনা তৈরি করে একটি পরীক্ষামূলক পুনরুদ্ধার করুন। ৩. আপনার অ্যাপের RPO/RTO লিখে Pilot Light নকশা আঁকুন। ৪. স্টেজিংয়ে একটি গেম ডে চালান: একটি ECS টাস্ক বন্ধ করে পর্যবেক্ষণ করুন।

অধ্যায় ২৪ • PART V

Chapter 24 — Production AWS Architecture and FinOps

২৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) প্রোডাকশন স্থাপত্যের নীতি ব্যাখ্যা করতে; (২) মাল্টি-অ্যাকাউন্ট শাসন (governance) নকশা করতে; (৩) মাল্টি-Region স্থাপত্যের সিদ্ধান্ত নিতে; (৪) খরচ অপ্টিমাইজেশন (FinOps) চালাতে; (৫) ক্ষমতা পরিকল্পনা (capacity planning) করতে; (৬) পরিচালন-প্রস্তুতি (operational readiness) যাচাই করতে; (৭) স্থাপত্য সিদ্ধান্ত রেকর্ড (ADR) লিখতে; (৮) TechNova-র সম্পূর্ণ প্রোডাকশন MERN স্থাপত্য নকশা করতে ও ক্যাপস্টোন ওয়াকথ্রু করতে।

২৪.২ পূর্বজ্ঞান ঝালাই

এই বইয়ে আপনি শিখেছেন: VPC ও নেটওয়ার্কিং, EC2 ও কনটেইনার, সার্ভারলেস, CI/CD, IaC, পর্যবেক্ষণ, নিরাপত্তা ও পুনরুদ্ধার। এই অধ্যায়ে সেগুলো একসঙ্গে বেঁধে একটি সম্পূর্ণ প্রোডাকশন স্থাপত্য তৈরি করব — TechNova-র MERN অ্যাপ্লিকেশনকে কেন্দ্র করে। মনে রাখুন: প্রোডাকশন মানে শুধু "চলছে" নয় — মানে নিরাপদ, পর্যবেক্ষণযোগ্য, পুনরুদ্ধারযোগ্য ও খরচ-সচেতনভাবে চলছে।

২৪.৩ প্রোডাকশন স্থাপত্যের নীতি

পাঁচটি নীতি: (১) **সরলতা** — সবচেয়ে সরল নকশা যা প্রয়োজন মেটায়; জটিলতা নিজেই ঝুঁকি। (২) **ঢিলেঢালা সংযোগ** — সেবাগুলো ঘটনা ও API দিয়ে কথা বলে, সরাসরি নির্ভরতা নয়। (৩) **ব্যর্থতা ধরে নকশা** — প্রতিটি উপাদান ভাঙবে ধরে নিন; Multi-AZ, রিট্রাই, সার্কিট ব্রেকার। (৪) **সবকিছু কোড হিসেবে** — IaC, পাইপলাইন, নীতি। (৫) **পর্যবেক্ষণযোগ্যতা প্রথমে** — লগ, মেট্রিক, ট্রেস ছাড়া ডিপ্লয় নয়। এই নীতিগুলো AWS Well-Architected Framework-এর ছয়টি স্তম্ভের (পরিচালন উৎকর্ষ, নিরাপত্তা, নির্ভরযোগ্যতা, কর্মক্ষমতা দক্ষতা, খরচ অপ্টিমাইজেশন, স্থায়িত্ব) সঙ্গে মিলে যায় — প্রোডাকশন পর্যালোচনায় এই ছয়টি দিয়ে যাচাই করুন।

২৪.৪ মাল্টি-অ্যাকাউন্ট শাসন

একটি অ্যাকাউন্টে সবকিছু রাখা মানে একটি ভুলে সব যাওয়া। **AWS Organizations** দিয়ে অ্যাকাউন্ট আলাদা করুন: dev, staging, prod, security (লগ/নিরীক্ষা), shared-services (CI/CD, ECR)। **SCP** দিয়ে সীমা বেঁধে দিন (যেমন prod-এ CloudTrail বন্ধ করা যাবে না); **Control Tower** দিয়ে নতুন অ্যাকাউন্টের মানসম্মত ভিত্তি (landing zone) তৈরি করুন। লগ সব অ্যাকাউন্ট থেকে security অ্যাকাউন্টে পাঠান; বিলিং একত্রে (consolidated billing) দেখুন। **ট্যাগিং নীতি:** প্রতিটি রিসোর্সে `Environment`, `Team`, `CostCenter` ট্যাগ বাধ্যতামূলক — নইলে খরচের হিসাব মেলানো যায় না।

২৪.৫ মাল্টি-Region স্থাপত্য — কখন, কীভাবে

মাল্টি-Region মানে দ্বিগুণ জটিলতা ও খরচ — তাই আগে প্রশ্ন করুন: নিয়ন্ত্রক প্রয়োজন (ডেটা আবাস), লেটেন্সি (ব্যবহারকারী কোথায়), নাকি DR? বেশিরভাগ দলের জন্য Multi-AZ + Pilot Light DR যথেষ্ট; Active/Active শুধু বৈশ্বিক সেবায়। মাল্টি-Region নকশায়: ডেটা প্রতিলিপি (S3 CRR, DynamoDB গ্লোবাল টেবিল), Route 53 লেটেন্সি-ভিত্তিক রাউটিং, প্রতিটি রিজিয়নে একই IaC — রিজিয়ন শুধু একটি প্যারামিটার।

২৪.৬ FinOps — খরচ প্রকৌশল

FinOps হলো খরচকে প্রকৌশলগত সিদ্ধান্ত হিসেবে দেখা: (১) **দৃশ্যমানতা** — Cost Explorer, বাজেট অ্যালার্ম, ট্যাগ দিয়ে দল-অনুযায়ী খরচ; (২) **অপ্টিমাইজেশন** — সঠিক আকার (right-sizing), অপ্রয়োজনীয় রিসোর্স বন্ধ, S3 লাইফসাইকেল (পুরোনো ডেটা কম খরচের শ্রেণিতে), Savings Plans/Reserved Instances স্থির লোডে; (৩) **শাসন** — বাজেট, অনুমোদন, নিয়মিত পর্যালোচনা। **সতর্কতা:** NAT Gateway, ALB, EKS কন্ট্রোল প্লেন, ডেটা ট্রান্সফার ও CloudWatch লগ — এই "নীরব" খরচগুলো প্রায়ই বাজেট ছাড়ায় — AWS Pricing Calculator-এ যাচাই করুন। মাসে একবার খরচ পর্যালোচনা সভা করুন — খরচ কারও একার দায়িত্ব নয়, দলের।

খরচের উৎস	নিয়ন্ত্রণ
অলস EC2 / বড় টাস্ক	Right-sizing, অটো স্কেলিং, Fargate Spot
NAT Gateway	VPC এন্ডপয়েন্ট (S3, DynamoDB), NAT ভাগ করা
ডেটা ট্রান্সফার	একই AZ-তে রাখা, CloudFront ক্যাশিং
CloudWatch লগ	Retention সীমিত, অপ্রয়োজনীয় লগ বন্ধ
S3 সংরক্ষণ	লাইফসাইকেল নীতি, অসম্পূর্ণ আপলোড মোছা

২৪.৭ ক্ষমতা পরিকল্পনা, নিরাপত্তা শাসন ও পরিচালন-প্রস্তুতি

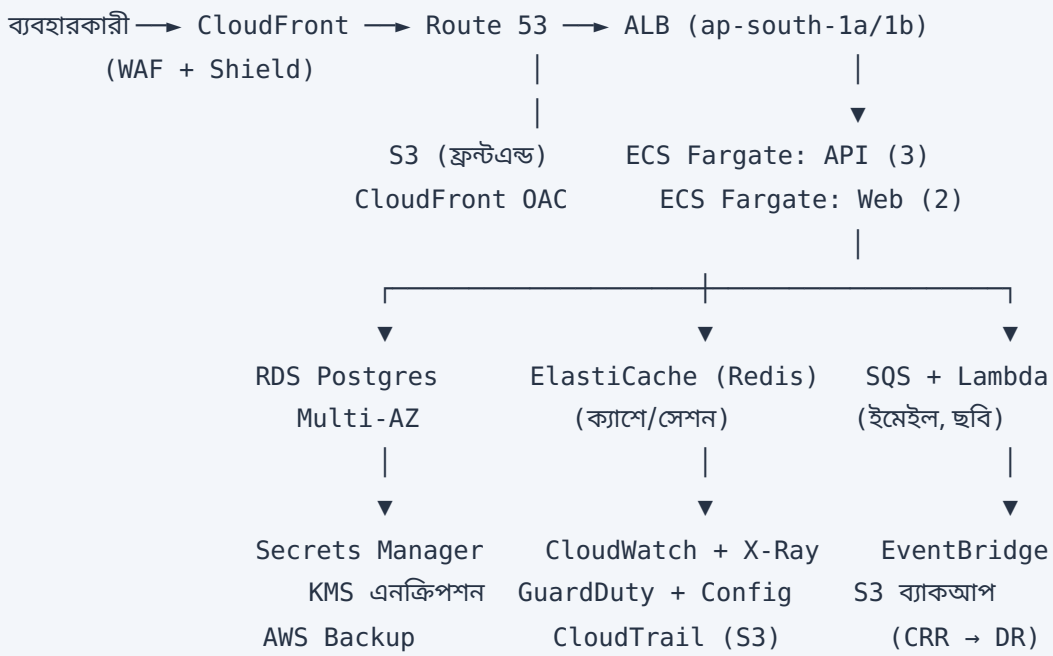
ক্ষমতা পরিকল্পনা: সর্বোচ্চ লোড অনুমান করুন (প্রচারণা, উৎসব), লোড টেস্ট করুন, কোটা (quota) আগেই বাড়ানোর অনুরোধ করুন — বিভ্রাটের দিনে কোটা বাড়াতে সময় লাগে। **নিরাপত্তা শাসন:** Security Hub স্কোর, Config সম্মতি, নিয়মিত অনুমতি পর্যালোচনা, ঘটনা প্রতিক্রিয়া পরিকল্পনা। **পরিচালন-প্রস্তুতি চেকলিস্ট:** ড্যাশবোর্ড আছে কি? অ্যালার্মে রানবুক আছে কি? ব্যাকআপ পরীক্ষা হয়েছে কি? রোলব্যাক পরীক্ষা হয়েছে কি? অন-কল তালিকা হালনাগাদ কি? ডিপ্লয়ের আগে এই চেকলিস্টে টিক দিন — প্রোডাকশন লঞ্চের শর্ত হিসেবে।

২৪.৮ স্থাপত্য সিদ্ধান্ত রেকর্ড (ADR)

ADR হলো ছোট নথি: প্রেক্ষাপট, সিদ্ধান্ত, বিকল্প, পরিণতি। উদাহরণ: "ECS বনাম EKS — ECS বেছে নেওয়া হলো, কারণ দল ছোট ও AWS সমন্বয় চায়; পরিণতি: Kubernetes বহনযোগ্যতা ছাড়া হলো; পুনর্বিবেচনা: দল ৫ জনের বেশি হলে।" ADR লিখলে ছয় মাস পরে কেউ জিজ্ঞেস করলে উত্তর থাকে — "কেন এভাবে বানানো হলো?" সিদ্ধান্তের ইতিহাসই স্থাপত্যের স্মৃতি।

২৪.৯ TechNova সম্পূর্ণ প্রোডাকশন MERN স্থাপত্য

TechNova Production — সম্পূর্ণ স্থাপত্য (ap-south-1)



প্রবাহ: ব্যবহারকারী CloudFront দিয়ে ফ্রন্টএন্ড (S3) পান; API কল ALB দিয়ে ECS Fargate-এ যায়; API RDS-এ ডেটা লেখে, Redis-এ ক্যাশ করে; ভারী কাজ (ইমেইল, ছবি প্রক্রিয়াকরণ) SQS + Lambda-তে যায়; ঘটনা EventBridge দিয়ে সেবাগুলোকে জানায়। নিরাপত্তা: প্রাইভেট সাবনেট, KMS, Secrets Manager, WAF, GuardDuty। পর্যবেক্ষণ: CloudWatch ড্যাশবোর্ড, X-Ray, অ্যালার্ম + SNS। ডেলিভারি: GitHub Actions (OIDC) → CodePipeline → ECR → ECS ব্লু-গ্রিন। DR: RDS রেপ্লিকা + S3 CRR, Route 53 ফেইলওভার। শাসন: Organizations (prod/staging/dev/security), SCP, ট্যাগিং, FinOps পর্যালোচনা।

কেন এই নকশা: প্রতিটি সিদ্ধান্তের পেছনে কারণ আছে — Fargate কারণ দল ছোট; RDS কারণ পরিচালিত ব্যাকআপ; SQS+Lambda কারণ ভারী কাজ API-কে ধীর করে না; CloudFront কারণ বৈশ্বিক লেটেন্সি কমে। কোনো উপাদানই "সেরা প্রযুক্তি" বলে নেওয়া হয়নি — নেওয়া হয়েছে "TechNova-র দল, বাজেট ও ঝুঁকির জন্য সঠিক" বলে।

২৪.১০ ক্যাপস্টোন ওয়াকথ্রু — শূন্য থেকে প্রোডাকশন

ধাপ ১ — ভিত্তি: Organizations-এ অ্যাকাউন্ট, VPC (২ AZ, পাবলিক/প্রাইভেট সাবনেট), IAM রোল, KMS চাবি — Terraform দিয়ে। **ধাপ ২ — ডেটা:** RDS Multi-AZ, ElastiCache, S3 বাকেট (সংস্করণ + এনক্রিপশন), Secrets Manager — ব্যাকআপ নীতি চালু। **ধাপ ৩ — অ্যাপ:** ECR, ECS Fargate সার্ভিস (API + Web), ALB, Route 53, CloudFront — হেলথ চেক ও অটো স্কেলিংসহ। **ধাপ ৪ — ইভেন্ট:** SQS + Lambda (ইমেইল/ছবি), EventBridge নিয়ম, DLQ। **ধাপ ৫ — ডেলিভারি:** GitHub Actions + OIDC, CodePipeline, ব্লু-গ্রিন ডিপ্লয়, অনুমোদন গেট। **ধাপ ৬ — পর্যবেক্ষণ ও নিরাপত্তা:** CloudWatch ড্যাশবোর্ড ও অ্যালার্ম, X-Ray, CloudTrail, GuardDuty, Security Hub, Config। **ধাপ ৭ — DR ও প্রস্তুতি:** Pilot Light, গেম ডে, রানবুক, পোস্টমর্টেম টেমপ্লেট, পরিচালন-প্রস্তুতি চেকলিস্ট, ADR লেখা। **ধাপ ৮ — FinOps:** বাজেট অ্যালার্ম, ট্যাগিং, Cost Explorer পর্যালোচনা, right-sizing। প্রতিটি ধাপে যাচাই: "এই ধাপ শেষে কী পরীক্ষা করবেন?" — উত্তর ছাড়া পরের ধাপে যাবেন না।

বাস্তব শিক্ষা: প্রোডাকশন কোনো একদিনের ঘটনা নয় — এটি অভ্যাস। TechNova-র দল প্রতি সপ্তাহে একটি ছোট উন্নতি করে: একটি অ্যালার্ম, একটি রানবুক, একটি right-sizing। এক বছরে সেটাই প্রোডাকশন-মানের সিস্টেম হয়।

২৪.১১ প্রোডাকশন পরিস্থিতি — TechNova-র লঞ্চ সপ্তাহ

লঞ্চ সপ্তাহে দল চেকলিস্ট ধরে এগোল: লোড টেস্টে দেখা গেল ALB-এর পেছনে টাস্ক ৩টি যথেষ্ট নয় — ৬-এ বাড়ানো হলো; কোটা বাড়ানোর অনুরোধ আগেই করা ছিল; বাজেট অ্যালার্ম ৮০%-এ বেজে উঠলে দেখা গেল একটি পরীক্ষামূলক EKS ক্লাস্টার চলছে — বন্ধ করা হলো। লঞ্চ সফল হলো, কোনো ঘটনা ঘটেনি। শিক্ষা: লঞ্চের সাফল্য লঞ্চের দিনে তৈরি হয় না — আগের সপ্তাহগুলোর প্রস্তুতিতে তৈরি হয়।

২৪.১২ নিরাপত্তা, খরচ, ভুল ও সমস্যা সমাধান

নিরাপত্তা: প্রোডাকশন অ্যাকাউন্টে মানুষের সরাসরি অ্যাক্সেস সীমিত করুন — পরিবর্তন শুধু পাইপলাইন দিয়ে; রুট অ্যাকাউন্ট লক করা; Security Hub স্কোর নিয়মিত পর্যালোচনা। **খরচ:** প্রতি মাসে Cost Explorer পর্যালোচনা, অপ্রয়োজনীয় রিসোর্স বন্ধ, Savings Plans বিবেচনা — AWS Pricing Calculator-এ যাচাই করুন।

সাধারণ ভুল: (১) এক অ্যাকাউন্টে সবকিছু; (২) ট্যাগ ছাড়া রিসোর্স; (৩) DR পরিকল্পনা না পরীক্ষা করা; (৪) ADR না লেখা — ছয় মাস পরে কেউ জানে না কেন; (৫) লঞ্চের দিনে কোটা বাড়াতে চাওয়া; (৬) খরচ শুধু বিল এলে দেখা।

সমস্যা সমাধান: খরচ হঠাৎ বাড়লে → Cost Explorer-এ দল/সেবা অনুযায়ী ভাঙুন, নতুন রিসোর্স খুঁজুন; সেবা ধীর হলে → X-Ray ট্রেস, সোনালি সংকেত, সাম্প্রতিক ডিপ্লয় মেলান; অ্যাকাউন্টে অ্যাক্সেস সমস্যা → Organizations SCP ও IAM নীতি যাচাই।

২৪.১২ক প্রোডাকশন লঞ্চের আগের শেষ পরীক্ষা — প্রি-ফ্লাইট

TechNova-র লঞ্চের আগের রাতে দল একটি **প্রি-ফ্লাইট পর্যালোচনা** চালাল — Well-Architected পর্যালোচনার সংক্ষিপ্ত রূপ। প্রতিটি স্তম্ভে তিনটি প্রশ্ন: পরিচালন উৎকর্ষ — ডিপ্লয় কি স্বয়ংক্রিয়? রোলব্যাক কি পরীক্ষিত? নিরাপত্তা — Security Hub-এ critical finding আছে কি? সিক্রেট কি Secrets Manager-এ? নির্ভরযোগ্যতা — Multi-AZ কি? ব্যাকআপ পুনরুদ্ধার কি পরীক্ষিত? কর্মক্ষমতা — লোড টেস্টের ফল কী? ক্যাশিং আছে কি? খরচ — বাজেট অ্যালার্ম চালু কি? সবচেয়ে বড় তিনটি খরচের উৎস কী? স্থায়িত্ব — অপ্রয়োজনীয় রিসোর্স বন্ধ কি? কোনো প্রশ্নের উত্তর "জানি না" হলে লঞ্চ পিছিয়ে দেওয়া হয় — একটি পিছিয়ে যাওয়া লঞ্চ একটি ব্যর্থ লঞ্চের চেয়ে সস্তা।

লঞ্চের পরের প্রথম সপ্তাহে দল **হাইপারকেয়ার** (hypercare) চালাল: প্রতিদিন সকালে ড্যাশবোর্ড পর্যালোচনা, অন-কলে দুজন, ডিপ্লয় বন্ধ (change freeze) জরুরি সংশোধন ছাড়া। এই সময়ে ধরা পড়ল একটি ছোট সমস্যা — Redis ক্যাশে মিসের হার বেশি, কারণ ক্যাশে কী-তে পরিবেশের নাম ছিল না, স্টেজিং ও প্রোডাকশন একই ক্যাশে ভাগ করছিল। সমাধান এক লাইনের, কিন্তু ধরা পড়ল কারণ কেউ দেখছিল। শিক্ষা: লঞ্চের পরের সপ্তাহে পর্যবেক্ষণই পণ্য — নতুন বৈশিষ্ট্য নয়।

অবশেষে, এই বইয়ের প্রতিটি অধ্যায়ের জ্ঞানকে একটি **চলমান অভ্যাসে** বদলে নিন: প্রতি মাসে একটি Well-Architected প্রশ্ন নিয়ে দল বসুন; প্রতি ত্রৈমাসিকে একটি গেম ডে; প্রতি ছয় মাসে IAM অনুমতি পর্যালোচনা; প্রতি বছর DR পরীক্ষা। প্রোডাকশন মানের সিস্টেম একদিনে বানানো যায় না — কিন্তু প্রতিদিন একটু করে বানানো যায়। TechNova-র যাত্রা এখানেই শেষ নয়; আপনার যাত্রা এখান থেকে শুরু।

২৪.১৩ সারাংশ — পুরো বইয়ের সারাংশ

প্রোডাকশন AWS মানে পাঁচটি অভ্যাস: কোড হিসেবে অবকাঠামো, স্বয়ংক্রিয় ডেলিভারি, স্তরে স্তরে নিরাপত্তা, পর্যবেক্ষণযোগ্যতা, ও খরচ-সচেতনতা। TechNova-র MERN স্থাপত্য দেখায় — CloudFront থেকে ECS, RDS থেকে Lambda, CI/CD থেকে DR — প্রতিটি অংশ কীভাবে একসঙ্গে একটি নির্ভরযোগ্য, নিরাপদ ও সাশ্রয়ী সিস্টেম হয়। বই শেষ, যাত্রা শুরু: ছোট থেকে শুরু করুন, প্রতিটি ধাপ পরীক্ষা করুন, ADR লিখুন, আর প্রতি সপ্তাহে একটি উন্নতি করুন।

শেষ কথা: এই বইয়ের প্রতিটি অধ্যায় একটি করে দক্ষতা শেখায়, কিন্তু প্রোডাকশন প্রকৌশলী হওয়া মানে সেগুলোকে একসঙ্গে ব্যবহার করা — একটি পরিবর্তন লিখে, পরীক্ষা করে, নিরাপদে ডিপ্লয় করে, পর্যবেক্ষণ করে, প্রয়োজনে ফিরিয়ে নিয়ে, আর প্রতিটি ঘটনা থেকে শিখে। TechNova-র গল্পটি কাল্পনিক, কিন্তু পদ্ধতিটি বাস্তব: ছোট শুরু, পরীক্ষিত ধাপ, লিখিত সিদ্ধান্ত, আর থামা না। আপনার পরের ধাপটি ছোট হোক — একটি অ্যালার্ম, একটি ADR, একটি গেম ডে — কিন্তু আজই হোক। শুভকামনা রইল।

২৪.১৪ পর্যালোচনা প্রশ্ন

১. প্রোডাকশন স্থাপত্যের পাঁচটি নীতি কী? ২. মাল্টি-অ্যাকাউন্ট কেন? ৩. FinOps-এর তিনটি ধাপ কী? ৪. ADR কেন লিখবেন? ৫. মাল্টি-Region কখন দরকার, কখন নয়? ৬. পরিচালন-প্রস্তুতি চেকলিস্টে কী কী থাকবে? ৭. TechNova স্থাপত্যে কেন Fargate? ৮. ভারী কাজ কেন SQS+Lambda-তে? ৯. "নীরব" খরচগুলো কী কী? ১০. Well-Architected-এর ছয়টি স্তম্ভ কী?

২৪.১৫ অনুশীলন — ক্যাপস্টোন

১. TechNova স্থাপত্যের একটি ADR লিখুন: "ECS বনাম EKS"। ২. আপনার নিজের অ্যাপের জন্য প্রোডাকশন চেকলিস্ট তৈরি করুন (২৪.৭ অনুযায়ী)। ৩. Cost Explorer দিয়ে একটি পরীক্ষামূলক অ্যাকাউন্টের খরচ ভেঙে একটি সাশ্রয় খুঁজে বের করুন। ৪. ক্যাপস্টোন ওয়াকথ্রুর ধাপ ১–৩ Terraform দিয়ে একটি পরীক্ষামূলক পরিবেশে বাস্তবায়ন করুন, তারপর মুছে ফেলুন। ৫. একটি গেম ডে পরিকল্পনা লিখুন: AZ বিভ্রাটের অনুশীলন।